

从 C++ 到计算系统：第二册

硬件原理、多核并行、高性能计算、同步、无锁结构与分布式计算

CPU Performance Study

本册接续第一册的程序执行基础，系统讲解现代 CPU 硬件、多核心并发、高性能计算、锁与无锁数据结构、并行算法、运行时和分布式计算。AI 模型、算子开发和推理引擎放入第三册。

前言

第一册已经回答了基础问题：一个 C++ 程序如何变成进程、地址空间、二进制接口、汇编指令和可测量的执行现象。它负责打牢源码到二进制、x86-64 汇编、System V ABI、Linux 基础工具、虚拟地址空间和可信 benchmark 的底座。第二册不再把这些内容重新讲一遍，而是把它们当作读者已经掌握的工具：能看汇编，能确认二进制身份，能写基本实验报告，能理解进程和虚拟地址空间的基本边界。

本册继续向下和向外展开，但不提前进入 AI。真正要理解高性能程序，必须先把“计算本身”讲清楚：核心怎样取指、译码、调度和提交，乱序执行为什么能隐藏延迟，分支预测为什么会失败，缓存和 TLB 为什么决定很多程序的上限，多核心之间为什么需要一致性协议，NUMA 为什么让同一块内存存在不同核心看来成本不同。第一册把单个程序放到机器上，第二册追问这台机器怎样扩展为多核心计算系统，再怎样扩展为多进程、多节点和分布式计算系统。

软件层面同样不能跳步。多线程不是简单地把循环切成几份；锁不是只有“慢”一个性质；原子操作不是更快的锁；无锁数据结构也不是不用同步。并行算法要面对数据依赖、负载均衡、局部性、归约顺序和任务粒度。分布式计算更进一步：一旦机器之间只能通过网络通信，延迟、带宽、故障、重试、超时、复制和一致性就会成为计算模型的一部分。

因此第二册的主题是计算系统，而不是某个应用领域。它从硬件原理开始，讲到单机高性能计算、多线程并发、锁与无锁结构、并行算法、运行时系统、异步 I/O 和分布式计算。AI 模型、张量、算子开发和量化推理引擎会放到第三册。这样安排不是延后重点，而是先把地基打牢：没有硬件和并发基础，后续任何 AI 算子优化都会变成经验堆砌。

核心思想

第二册的核心问题是：给定真实硬件、操作系统和网络边界，怎样组织数据、线程、同步、任务和通信，使计算能够正确、可测量、可扩展地运行。

本册仍然沿用第一册的写法：原理先行，代码作为证据；实验必须记录输入、环境、命令和误差；源码阅读抓数据结构、热路径、同步边界和性能瓶颈。不过，本册的证据重心会改变。第一册主要用反汇编、ELF、GDB、`readelf`、`objdump` 和基础 `perfstat` 建立“这段程序真的这样执行”的证据；第二册会更多使用拓扑、PMU、调度事件、锁等待、cache line 迁移、NUMA 放置、队列长度、吞吐曲线和失败注入来解释“系统为什么不能继续扩展”。

读者读完后，应当能解释一个多核程序为什么没有线性加速，能判断锁竞争和 false sharing 的差异，能写出可测试的并行归约、有界队列和环形队列，能读懂 Linux 调度、futex、RCU、内存管理和 perf 相关源码入口，也能把单机计算原则扩展到分布式系统。

怎样阅读本册

本册默认目标平台是本地 Linux 机器。普通 x86-64 Linux 台式机、笔记本或服务器适合完成完整实验；Termux 可以用于阅读、构建小实验和运行基础测试，但 perf、NUMA、线程亲和性、硬件计数器、futex 观察、eBPF 观测和分布式网络实验可能受权限、内核和设备能力限制。所有性能结论都必须写清环境。

阅读本册前，应默认已经完成第一册的最低要求：能把源码编译成二进制并确认产物身份，能读基本 x86-64 汇编，能解释调用约定和栈帧，能写出隔离热路径的 benchmark，能使用 Linux 常见工具观察进程、地址空间和基础性能计数。第二册不会反复解释这些背景；遇到这些内容时，本册只会说明它们怎样服务多核、同步、运行时或分布式问题。

阅读顺序建议严格按章节推进。前四章讲硬件执行模型，这是后面所有内容的解释基础。单机高性能计算部分讲测量、数据布局、SIMD 和典型内核，帮助读者把“程序慢”分解成计算、内存、分支、同步或 I/O 问题。并发部分讲线程、锁、条件变量、原子、内存序和无锁数据结构，重点不是背 API，而是理解共享状态怎样被保护、发布和回收。并行算法和分布式计算部分再把这些原则扩展到任务图、运行时、网络通信、复制和一致性。

本册的贯穿实验叫 Compute Systems Labs。它不追求一开始很大，而是从小而真实的模块开始：顺序归约、并行归约、带锁计数器、原子计数器、有界队列、任务切分和简单 benchmark。后续扩写会继续加入线程池、工作窃取、SPSC 环形队列、MPMC 队列、并行扫描、分布式 shuffle 和故障注入。

心智模型

读本册时始终追问四件事：数据在哪里，谁会读写它，硬件会怎样移动它，失败或竞争发生时系统怎样保持正确。

每章至少做一个实验。硬件章节要配合 `lscpu`、`numactl`、`perfstat`、拓扑图和内存访问 microbenchmark；并发章节要写出可重复触发竞争、阻塞、false sharing、CAS 失败或内存序问题的小程序；运行时章节要记录队列长度、任务粒度、worker 空闲时间和背压；分布式章节即使只在本机多进程模拟，也要记录延迟、重试、超时、重复请求和消息顺序。

常见误区

不要把高性能计算理解成“把线程数开大”。线程数只是资源请求，性能来自数据复用、负载均衡、同步控制和测量证据。没有这些证据，加线程常常只是更快地制造争用。

本册的章节之间有明确承接关系。第一部分回答硬件成本从哪里来，第二部分回答怎样把成本变成可测假设，第三部分回答共享内存里怎样保持正确，第四部分回答怎样把多个线程组织成可扩展运行时，第五部分回答当共享内存消失、网络和故障进入模型后，计算原则怎样变化。不要把某章当成孤立技巧阅读；每次做实验，都要把结论回连到这条链上。

全书结构

本册分为五个部分，围绕“计算系统如何正确且高效地扩展”展开。第一册已经承担源码、进程、虚拟地址、汇编、ABI、工具链和基础测量，本册只在需要建立新成本模型时短暂回扣这些内容，不再重复铺陈。这样处理的目的，是把篇幅留给第一册没有充分展开的主题：多核心硬件、缓存一致性、NUMA、SIMD 和内存带宽、Linux 调度和同步、无锁结构、并行运行时、异步 I/O、分布式计算和故障模型。

第一部分是硬件执行模型。它讲核心流水线、乱序执行、分支预测、寄存器重命名、缓存层级、TLB、页表、预取、NUMA、互连和缓存一致性。读者要在这里建立硬件成本模型：一条指令不只是语法，背后有取指、依赖、执行端口、访存、提交和一致性流量。

第二部分是单机高性能计算。第一册已经讲过 benchmark 纪律，所以本部分不再停留在“怎样计时”这类基础问题，而是进入多核进阶测量、Roofline、PMU 归因、cache 和 TLB 事件、数据布局、局部性、SIMD、指令级并行和典型计算内核。目标是把“优化”从口号变成可验证的分析：到底是算力不够、内存带宽不够、cache miss 太多、分支预测差、TLB 覆盖不足、同步等待太长，还是 benchmark 边界仍然写错。

第三部分是多线程、同步与内存模型。它讲线程生命周期、调度、亲和性、锁、条件变量、信号量、原子操作、内存序、false sharing、无锁队列和内存回收。这里的重点是正确性先于性能：如果可见性、互斥、进度保证和对象生命周期没有说清楚，所谓高性能就是不稳定的。

第四部分是并行算法与运行时。归约、扫描、分区、图遍历、任务图、线程池、工作窃取、异步 I/O 和背压，都是把多核硬件变成可用计算能力的桥梁。读者在这里要学会从算法依赖和数据移动角度设计并程序序。

第五部分是分布式计算。它把单机原则扩展到网络和集群：RPC、序列化、时间、故障、分片、复制、一致性、共识、shuffle、检查点和观测性。分布式不是“多几台机器”，而是把通信和失败纳入计算模型。

核心思想

第二册的主线是一条从硬件到集群的链：硬件决定局部成本，多线程决定共享内存内的协作，并行算法决定可扩展形状，分布式系统决定跨机器的通信和故障边界。

和第一册的边界

第一册已经讲清楚的内容，本册只作为前提使用：源码到目标文件和 ELF 的路径，进程和虚拟地址空间的基本概念，x86-64 汇编阅读，System V AMD64 ABI，C++ 与汇编互操作，基础 Linux 命令，基础 benchmark 设计，基础 **perfstat** 和 Linux 源码入口。第二册遇到这些内容时，重点不是重复定义，而是解释它们在可扩展计算中的新作用。

例如，第一册讲虚拟地址空间时，目标是让读者理解进程地址、映射和缺页；第二册讲 TLB 和页级性能时，目标是解释页大小、page walk、NUMA 放置和首次触页怎样改变吞吐。第一册讲 **perfstat** 时，目标是建立可信测量意识；第二册讲 PMU 时，目标是用事件组合区分前端、后端、内存、同步和调度瓶颈。第一册讲汇编控制流时，目标是读懂分支和跳转；第二册讲分支预测时，目标是理解输入分布、预测器状态和流水线回滚怎样限制单核吞吐。

本册扩展的重点

本册扩展不是简单增加术语，而是围绕可扩展计算的核心矛盾展开。硬件部分要能从 pipeline、ROB、load/store queue、store buffer、cache line、TLB、NUMA 和一致性协议解释成本。同步部分要能从不变量、等待谓词、futex、内存序、线性化点和内存回收解释正确性。运行时部分要能从任务粒度、局部性、工作窃取、背压和关闭语义解释工程稳定性。分布式部分要能从消息、重试、幂等、复制、共识、shuffle、检查点和观测性解释跨机器计算。

每个章节都要有三个出口：能画出机制图，能写出最小可运行例子，能设计一份可复查实验。只

有这样，第二册才不是第一册的复述，而是把第一册的基础能力推进到现代计算系统。

规模目标和章节配额

第二册按正式书籍标准写作，最终正文不少于 50 万单位。统计口径只看 `chapters/` 下的正文内容，以中文字符加英文单词计数；代码块、命令、反汇编、终端输出、配置、前言、目录、附录和 LaTeX 标记不计入。50 万只是最低验收线，不是上限；如果硬件、多核、运行时、无锁结构和分布式计算的知识链需要更大篇幅，就按完整性继续写，不能因为数字达到门槛而停在半成品。这个目标不是排版长度目标，而是知识密度目标：每一章都必须把原理、机制、案例、实验和源码阅读边界讲清楚。

按 18 章估算，平均每章需要 2.7 万到 2.9 万单位。考虑章节难度不同，硬件执行、缓存/TLB/NUMA、测量、锁与原子、无锁结构、任务运行时和分布式计算应承担更高篇幅；实验路线和源码索引只能辅助正文，不能替代正文。后续扩写时，每章都要围绕同一条链展开：先讲模型，再讲硬件或系统实现，再讲 C++ 代码形态，再讲测量证据，再讲反例和工程边界。

常见误区

本册不接受用重复定义、空泛口号、附录堆表、题目清单或模板化实验报告凑字数。若一个段落不能帮助读者理解现代计算系统的机制、判断或实践，就不应进入正文。

章节质量标准

每章必须按“机制、案例、代码、测量、源码、作业”六个维度写作。机制部分要解释底层原理，不只给术语定义；案例部分要从真实问题出发，展示为什么朴素方案不够；代码部分要给 baseline、改进版本和边界处理版本，不能只给最终答案；测量部分要说明实验矩阵、指标、预期现象和失败解释；源码部分要给 Linux 或生产系统的窄入口，并说明读源码要验证什么；作业部分要能训练读者独立复现、修改、测量和写报告。

本册的案例不能停留在“数组求和”“打印线程 ID”这种单点玩具上。简单例子可以作为入口，但必须继续推进到工程问题：线程数增加为什么不线性加速，锁竞争和 false sharing 怎样区分，TLB miss 和 cache miss 怎样分开，任务粒度怎样影响线程池，队列满时背压怎样传导，分布式 shuffle 为什么会被热点 key 拖垮。每个案例都要有问题背景、朴素方案、失败原因、改进方案、代价和实验验证。

贯穿大项目

第二册贯穿项目命名为 Compute Systems Engine。它不是一个孤立实验，而是整本书的工程主线。第一阶段实现顺序和并行归约、测量 harness、拓扑记录和结果校验；第二阶段加入数据布局实验、线程本地 partial、false sharing 对照和 NUMA 初始化策略；第三阶段加入 mutex 队列、条件变量、有界队列、原子计数器和内存序实验；第四阶段加入线程池、任务图、工作窃取和运行时指标；第五阶段加入 SPSC ring buffer、MPMC queue、内存回收实验和压力测试；第六阶段加入异步 I/O 模拟、背压策略、批处理和限流；第七阶段在本机多进程模拟分布式 shuffle、幂等提交、重试、检查点和故障注入。

每章扩写时都要回答两个问题：本章为 Compute Systems Engine 增加了什么能力；这个能力需要哪些测试和测量才能被认为可靠。若某章不能落到项目上，也必须说明它提供的是哪一种模型能力，例如分支预测帮助解释热循环，TLB 帮助解释大工作集，NUMA 帮助解释多 socket 扩展曲线。

目录

第一部分	硬件执行模型：计算为什么会快也会慢	
第 1 章	计算系统全景	2
第 2 章	核心流水线、乱序执行与分支预测	11
第 3 章	缓存、TLB 与页级性能	25
第 4 章	NUMA、互连与缓存一致性	36
第二部分	单机高性能计算：从测量到数据流	
第 5 章	可信测量、Roofline 与性能计数器	47
第 6 章	数据布局、局部性与预取	56
第 7 章	SIMD、指令级并行与向量化	70
第 8 章	典型计算内核模式	82
第三部分	多线程、同步与内存模型	
第 9 章	线程、调度与亲和性	95
第 10 章	锁、条件变量与信号量	105
第 11 章	原子操作与内存序	118
第 12 章	无锁数据结构	131
第四部分	并行算法与运行时	
第 13 章	归约、扫描与分区	143
第 14 章	任务运行时与工作窃取	156
第 15 章	I/O、异步与背压	167
第五部分	分布式计算：把单机原则扩展到集群	
第 16 章	分布式计算模型	179
第 17 章	分片、复制与共识	190
第 18 章	分布式计算工程实践	201
附录 A	实验路线	218
附录 B	源码阅读索引	219

第零部分

硬件执行模型：计算为什么会快也会慢

第 1 章

计算系统全景

第一册已经说明程序如何变成进程、虚拟地址、调用约定和机器指令。第二册继续追问：当一个程序需要利用多个核心、多个内存层级、多个线程甚至多台机器时，计算能力从哪里来，又会在哪里损失。这个问题不能只从代码出发，也不能只从硬件手册出发。真正的计算系统由硬件、操作系统、运行时、算法和通信协议共同构成。

1.1 从第一册到第二册

第一册的学习对象是一段程序如何成为一个可观察的执行实例。读者已经知道源文件、目标文件、链接、ELF、进程、虚拟地址、ABI、汇编指令和基础测量之间的关系。第二册的学习对象不是再把这条路径复述一遍，而是把这个执行实例放进更复杂的资源环境：多个核心同时执行，多个缓存层级同时保存数据，多个线程共享状态，多个任务争用队列，多个节点通过网络交换消息。

这意味着本册每次遇到第一册知识时，都要换一个问题问。看到汇编，不再只问“这条指令做什么”，还要问“它处在依赖链上还是可以并行发射”；看到虚拟地址，不再只问“它属于哪个映射”，还要问“这个访问是否会击穿 TLB、是否发生首次触页、页面是否放在远端 NUMA 节点”；看到 `perfstat`，不再只问“数字如何记录”，还要问“这些事件能否区分前端、后端、内存带宽、锁等待和调度噪声”。

核心思想

第一册把程序执行讲清楚，第二册把可扩展计算讲清楚。前者关心“程序如何被机器执行”，后者关心“多个执行实体怎样在硬件、操作系统和网络边界内协同前进”。

1.2 从一条指令到一个系统

一条加法指令看起来很简单，但它并不是孤立执行的。核心要取指、译码、重命名寄存器、等待操作数、选择执行端口、写回结果并提交状态。如果操作数在寄存器里，成本很低；如果来自 L1 cache，延迟增加；如果来自 L3、远端 NUMA 节点或磁盘，程序看到的时间会跨越几个数量级。并发程序还要考虑另一个核心是否正在写同一条 cache line，分布式程序还要考虑消息是否跨越网络。

因此本册把计算系统分成五层。第一层是硬件执行，包括核心、缓存、TLB、内存控制器和互连。第二层是操作系统，包括调度、页表、系统调用、I/O 和计数器。第三层是运行时，包括线程池、任务队列、同步原语和内存分配器。第四层是算法，包括归约、扫描、分区、图遍历和负载均衡。第五层是分布式协议，包括 RPC、复制、共识、shuffle 和检查点。

核心思想

高性能不是某一层单独决定的。硬件给出能力边界，操作系统分配资源，运行时组织执行，算法决定数据依赖，分布式协议处理通信和故障。

1.3 四种尺度

计算系统至少有四种尺度。第一种是单条指令尺度，讨论延迟、吞吐、依赖、端口和分支预测。第二种是单个核心尺度，讨论流水线、乱序窗口、寄存器重命名、load/store queue 和 SIMD。第三种是单台机器尺度，讨论缓存层级、TLB、NUMA、线程调度、锁、原子、队列和 I/O。第四种是集群尺度，讨论 RPC、分片、复制、共识、shuffle、检查点和故障恢复。

这四种尺度不能混用。单条指令尺度上，减少一条指令可能有效；单机多线程尺度上，减少一条指令可能完全被锁等待淹没；集群尺度上，减少一百万条指令也可能抵不过一次跨机重试。工程判断必须先确认当前瓶颈属于哪个尺度。第二册的核心训练之一，就是把“慢”定位到正确尺度，再选择对应证据。


```

source code
-> instructions and dependencies
-> core pipeline and local cache
-> shared-memory multicore machine
-> runtime queues and tasks
-> networked nodes and distributed state

```

这张文本图不是排版装饰，而是阅读本册的索引。越往右，通信成本越高，故障模式越复杂，确定性越弱。第一册主要覆盖左侧两层，本册从中间向右推进。

1.4 计算成本的基本单位

分析性能时，不能只看“执行了多少行代码”。更稳定的单位是指令数、周期数、访存字节数、cache miss、TLB miss、分支误预测、同步等待时间、上下文切换、系统调用次数和网络往返延迟。不同单位之间不能随便相加，但它们共同解释程序为什么慢。

例如，一个数组求和循环的算术量很小，主要成本是把数据从内存送到核心。一个矩阵乘如果分块得当，会重复使用缓存中的数据，瓶颈可能转向浮点执行端口。一个共享队列如果每次 push 都抢同一把锁，瓶颈不是算术，也不是内存带宽，而是临界区串行化和 cache line 所有权迁移。

1.5 把成本转成证据

第二册不鼓励凭经验猜瓶颈。每个成本单位都应该有可观察证据。依赖链问题可以用汇编、优化报告、IPC 和不同累加器版本比较；分支问题可以用输入分布、branch-misses 和 branchless 版本比较；cache 问题可以用工作集大小、访问步长和 cache 事件比较；TLB 问题可以用页数、大页和 dTLB 事件比较；锁问题可以用等待时间、futex、context switches 和吞吐曲线比较；NUMA 问题可以用 first-touch、绑定策略和远端访问计数比较；分布式问题可以用 trace、队列长度、重试次数和超时分布比较。

证据的作用不是一次性证明真理，而是排除错误方向。若一个归约程序在线程数增加后不再加速，可能原因很多：内存带宽到顶、false sharing、锁竞争、线程迁移、任务粒度太小、NUMA 放置错误、CPU 频率下降或 benchmark 把初始化混入热路径。没有证据时，这些解释都像对；有了证据，才能逐步缩小范围。

1.6 工作负载决定系统形状

同一台机器在不同工作负载下会呈现完全不同的系统形状。一个大数组顺序求和主要考验内存带宽、预取和 TLB 覆盖；一个哈希表查询主要考验随机访问、分支、缓存 miss 和分配器布局；一个日志写入服务主要考验队列、批处理、I/O 延迟和背压；一个并行图遍历主要考验负载均衡、前沿大小、随机邻接访问和同步开销。若不先描述工作负载，就谈不上性能分析。

工作负载至少要写清五件事。第一，输入规模和数据分布。例如数组长度、key 的倾斜程度、图的顶点度分布、请求大小分布。第二，读写比例。例如只读查询、读多写少、写多读少、读写混合。第三，局部性形状。例如顺序扫描、固定步长、随机访问、热点访问、分层访问。第四，依赖形状。例如每个元素独立、前缀依赖、图依赖、任务依赖、跨节点依赖。第五，失败和等待形状。例如是否会阻塞 I/O，是否会超时重试，是否会遇到队列满，是否会有节点失败。

经典教材式写法不应把“程序”当成孤立对象，而应把程序放在 workload 里看。一个函数在小输入上很快，不代表在大输入上快；在均匀 key 上扩展良好，不代表在热点 key 上可靠；在单线程上 cache 命中高，不代表多线程不会互相驱逐；在本机多进程模拟正常，不代表真实网络环境下重试和乱序没有影响。第二册后续所有实验都必须先写 workload，再写实现，再写测量。

1.7 瓶颈会迁移

优化不是把一个程序永久变快，而是把瓶颈从一个位置推到另一个位置。一个标量循环最初可能受长依赖链限制；使用多累加器后，依赖链缓解，瓶颈可能变成加载端口；再使用 SIMD 后，指令数下降，瓶颈可能变成内存带宽；再多线程后，瓶颈可能变成 NUMA、LLC 争用或内存控制器；

再把任务分布到多机后，瓶颈可能变成 shuffle、热点 key 或检查点。每次优化都应该重新判断瓶颈，而不是沿用上一次结论。

瓶颈迁移也是很多性能争论的根源。一个人说“这个优化没用”，另一个人说“这个优化提升很大”，可能他们测的是不同阶段的程序。在内存带宽已经打满的版本上减少几条整数指令，确实不会明显加速；在还受依赖链限制的版本上，多累加器可能非常有效。没有上下文的优化建议通常没有意义。

因此本册要求性能报告写出“优化前瓶颈假设”和“优化后瓶颈是否迁移”。例如优化并行归约时，可以先写：单线程版本 IPC 较低，怀疑依赖链；四累加器版本 IPC 提高但带宽接近上限；多线程版本 8 线程后不再加速，怀疑内存带宽或 NUMA；按 NUMA 节点初始化后带宽改善，说明原先包含远端访问因素。这样的叙述才像工程分析，而不是只列快了多少。

1.8 系统边界和责任边界

计算系统由多个层次组成，但工程修改必须有责任边界。硬件给出指令、缓存、TLB、互连和计数器；操作系统负责调度、地址空间、页表、I/O 和权限；运行时负责任务、队列、线程池、内存分配和同步封装；算法负责依赖、切分、合并和数据结构；业务协议负责语义、重试、幂等和恢复。性能问题经常跨层出现，但修复不能没有边界。

例如，一个服务尾延迟高，可能是锁竞争、页错误、磁盘抖动、网络重试、GC 或业务热点。排查时可以跨层收集证据，但修复时要明确到底改哪一层。如果把业务幂等问题硬塞进线程池，线程池会变得难以复用；如果把 NUMA 亲和性写死在算法库里，库就难以适配不同机器；如果把所有指标都放在业务日志里，底层运行时就无法单独诊断。好的系统设计会让层次之间有清晰接口，同时允许证据跨层关联。

本册后面讲线程池时，会强调 submit、shutdown、wait 的语义；讲无锁结构时，会强调线性化点和内存回收；讲分布式系统时，会强调请求 ID、重试预算和提交点。这些内容看起来分散，其实都在回答同一个问题：系统的责任边界在哪里，失败时谁负责恢复，性能结论应归因到哪一层。

1.9 延迟、吞吐和并发度

系统分析经常混淆三个词：延迟、吞吐和并发度。延迟是单个操作从开始到结束的时间，吞吐是单位时间完成多少操作，并发度是同时在系统中飞行的操作数量。一个内存 load 的延迟可能很高，但如果核心能同时挂起多个 miss，整体吞吐仍然可以接受；一个网络请求延迟很高，但如果客户端保持足够多的 in-flight 请求，吞吐可以接近带宽上限。

这三个量之间有一个朴素但非常有用的关系：要填满一个延迟很高的管道，需要足够并发度。磁盘、网络、内存和 CPU 后端都可以用这个心智模型理解。区别在于，CPU 的并发度来自乱序窗口、load buffer、SIMD lane 和多核心；网络系统的并发度来自连接数、请求队列和异步 I/O；分布式计算的并发度来自分片、任务和流水线。

心智模型

延迟回答“一个操作要等多久”，吞吐回答“系统每秒能做多少”，并发度回答“为了达到吞吐，需要同时容纳多少未完成工作”。

1.10 Amdahl 和 Gustafson

多核性能讨论离不开可并行比例。Amdahl 定律说明，如果程序有一部分必须串行执行，那么线程数增加到一定程度后，加速会被串行部分限制。Gustafson 定律从另一个角度看问题：如果随着核心数增加而扩大问题规模，可并行部分也随之增加，系统仍然能获得更高总吞吐。

这两个定律不是考试公式，而是工程判断工具。一个服务端请求路径中，如果日志锁、全局分配器、单分片元数据或单线程事件循环占比很高，增加 worker 不会线性加速。一个离线计算任务如果可以把输入切成更多分片，让每个节点处理独立数据，再做树形合并，扩大规模时就更接近 Gustafson 的场景。

1.11 观察系统的三张图

学习计算系统时，建议每遇到一个程序都画三张图。第一张是数据流图：输入从哪里来，经过哪些结构，输出到哪里。第二张是执行图：哪些线程、任务或节点执行哪些阶段，哪里并行，哪里等待。第三张是成本图：每个阶段主要消耗 CPU、内存、I/O、锁等待还是网络。

这三张图可以避免把问题混成一团。一个队列积压，可能是消费者计算慢，也可能是生产者太快，也可能是下游 I/O 阻塞，也可能是队列锁竞争。只有先画清数据和执行边界，测量才知道该放在哪里。

数据流图应标出数据的所有权和复制点。数组是只读输入，还是会被多个线程写回；任务队列里保存的是值、指针还是共享对象；网络请求里发送的是完整数据、增量还是引用；文件映射里的页面是否会被多个进程共享。所有权不清，优化很容易破坏正确性。比如把对象从共享指针改成裸指针可能减少引用计数开销，但如果生命周期没有重新证明，就可能引入悬垂访问。

执行图应标出并行区、串行区和等待区。很多程序源码看上去“用了线程”，执行图却显示所有线程都在等待一个队列、一把锁或一个单线程提交阶段。执行图还要标出调度关系：任务由谁提交，由谁执行，是否允许嵌套，是否可能在 worker 内等待 future，是否可能被取消。没有执行图，线程池和异步代码很容易变成看不见的控制流。

成本图应标出主成本和次成本。一个阶段可以同时消耗 CPU 和内存，但报告要判断主导因素。若主成本是内存带宽，减少日志字符串格式化不一定有用；若主成本是锁等待，改 SIMD 不会解决；若主成本是网络重试，优化本地解析函数意义有限。成本图不是精确模型，而是让团队讨论同一个对象。

1.12 正确性先于性能

并发和分布式计算最容易出现的错误，是把速度放在正确性之前。数据竞争可能让程序偶尔错；错误的内存序可能只在某些 CPU 上触发；无锁结构如果没有处理对象回收，可能在压力测试里才出现悬垂访问；分布式系统如果没有处理超时和重试，可能在网络抖动时重复提交请求。

本册每次讨论优化，都要先给出语义边界。共享变量由谁写，谁读，什么时候发布，什么时候回收。任务是否可以重排，结果是否依赖顺序。消息是否幂等，故障后是否可以重试。只有这些问题说清楚，性能数字才有意义。

1.13 从单机到分布式的一条线

很多读者会把并发和分布式分成两门课：前者讲线程，后者讲网络。第二册刻意把它们放在一条线上，是因为它们面对的是同一种基本矛盾：多个执行实体共享某些状态，但共享状态越多，协调成本越高。在单机上，协调成本表现为锁、原子、cache line 迁移、调度和内存序；在集群中，协调成本表现为 RPC、复制、共识、重试、超时和数据迁移。

因此，一个好的单机设计原则常常能迁移到分布式系统。线程本地计数再合并，对应 map-side combine；分片锁对应按 key 分区；树形归约对应分层聚合；有界队列和背压对应限流和排队控制；线性化点对应协议中的提交点；对象生命周期对应消息幂等和状态恢复。反过来，分布式系统也会提醒单机程序：只要协调成本足够高，就必须减少共享、批量通信、明确失败语义。

1.14 贯穿项目：Compute Systems Engine

本册需要一个贯穿全书的大项目，否则知识会散成很多孤立技巧。这个项目命名为 Compute Systems Engine。它的目标不是做一个商业框架，而是用足够小、可测试、可测量的工程，把现代计算系统的核心问题串起来：数据怎样放置，线程怎样执行，任务怎样调度，同步怎样证明，队列怎样背压，失败怎样恢复，分布式阶段怎样保持幂等。

项目的第一条主线是计算内核。最开始只有顺序归约和并行归约，但它们不是为了演示“多线程加速”这么简单。顺序归约用于建立 reference 和测量基线；多累加器版本用于解释依赖链和乱序执行；线程本地 partial 用于解释 false sharing 和共享写；NUMA 分块初始化用于解释 first-touch；树形合并用于连接后面的分布式聚合。一个小小的归约，在第二册里要承担从核心流水线到集群聚合的教学任务。

项目的第二条主线是运行时。先实现一个 mutex 和 condition variable 保护的 bounded queue，用它讲清锁保护的不变量、等待谓词、关闭语义和背压。然后实现固定线程数线程池，用它讲清 submit、

wait、shutdown、异常传播和任务粒度。再加入任务图和工作窃取，用它讲清依赖、ready 队列、本地性和负载均衡。最后把队列和线程池用于分布式模拟，让读者看到单机运行时和分布式执行器之间的相似性。

项目的第三条主线是并发数据结构。先比较 mutex counter、atomic counter 和 sharded counter，解释原子热点和 cache line 所有权迁移。再实现 SPSC ring buffer，解释 head/tail 单写者、acquire/release 和环形容容量边界。之后再讨论 MPMC 队列、ABA、hazard pointer、epoch 和 RCU，不让读者一上来就陷入复杂无锁代码。每一步都要有 reference、有压力测试、有错误注入。

项目的第四条主线是分布式计算模拟。它可以先在本机多进程或多线程中完成，不依赖真实集群。输入按分片读取，worker 本地聚合，shuffle 按 key 重分布，reduce 合并输出。随后加入重试、超时、重复请求、幂等提交、检查点和故障注入。这个阶段把前面所有原则重新组合：局部性变成数据本地调度，锁热点变成热点 key，背压变成限流，线性化点变成提交点，内存回收变成状态恢复。

心智模型

贯穿项目的每一次扩展，都必须同时回答三个问题：语义是否正确，性能瓶颈在哪里，失败时怎样恢复。只回答其中一个，都不是系统工程。

1.15 数据移动守恒

计算系统里最容易被低估的成本，是数据移动。一个整数加法可能只需要很少周期，但把数据从内存、磁盘或网络移动到执行单元，往往比加法昂贵得多。很多优化本质上不是“少算”，而是“少搬”“近处搬”“成批搬”“只搬热数据”。缓存分块减少从内存反复搬同一块数据；map-side combine 减少跨网络发送重复 key；压缩用更多 CPU 换更少 I/O；冷热分离避免把调试字段搬进热循环。这些看似不同的技巧，其实都遵守同一条原则：昂贵移动必须服务足够多的有效工作。

这条原则也能解释为什么某些算法复杂度相同，实际性能差很多。两个 $O(n)$ 算法，一个顺序扫描数组，一个随机追踪指针，它们搬动的数据量和等待形状完全不同。两个 $O(n \log n)$ 算法，一个在连续数组上排序，一个不断分配小节点，它们对 cache、TLB 和分配器的压力也不同。第二册不是替代算法复杂度，而是把复杂度放回真实机器里理解。

数据移动还有尺度差异。L1 到寄存器很快，远端 NUMA 内存慢很多，NVMe 更慢，跨机网络又是另一套延迟和带宽模型。一个设计如果在 L1 尺度上节省几次加法，却在网络尺度上多发一次请求，通常得不偿失。反过来，分布式系统为了减少网络字节，常愿意在 map 端多做本地聚合。成本尺度决定优化优先级。

1.16 延迟阶梯

学习现代计算系统时，建议把常见操作放到延迟阶梯里。寄存器访问、L1 命中、L2 命中、LLC 命中、DRAM、本地 NVMe、网络 RTT、远端存储，它们相差可能是数量级。具体数字会随硬件变化，但阶梯关系长期稳定。高性能代码的很多设计，就是尽量让热路径停留在更低阶梯，把高阶梯操作批量化、异步化或移出关键路径。

延迟阶梯不能单独看，还要看吞吐。一次 DRAM 访问延迟高，但内存系统能同时处理多个请求；一次网络请求延迟高，但多个 in-flight 请求可以填满带宽；一次磁盘写延迟高，但顺序大批写能获得高吞吐。延迟回答单个操作要等多久，吞吐回答系统单位时间能做多少，二者需要并发度连接。这个心智模型会在内存级并行度、异步 I/O 和分布式 RPC 里反复出现。

读者要避免一个常见错误：看到某个操作延迟高，就认为它一定是瓶颈。若高延迟操作很少发生，或者被流水线隐藏，它可能不是主成本；若低延迟操作发生在所有元素上，并且阻塞关键路径，它也可能成为瓶颈。性能分析的对象不是单次操作，而是整个 workload 下的频率、依赖和并发度。

1.17 状态、数据和控制

系统设计可以拆成状态、数据和控制三类对象。状态描述系统当前处于什么阶段，例如队列 open 还是 closed，任务 pending 还是 committed，分片属于哪个 epoch。数据是被计算处理的主体，例如数组元素、日志记录、shuffle block。控制是让状态和数据按规则流动的机制，例如调度器、锁、路由表、提交协议。

很多混乱来自把三者混在一起。把每条大数据记录都写入共识日志，是把数据面塞进控制面；把

线程池的关闭语义隐藏在任务函数里，是把控制状态塞进业务数据；把请求去重只放在日志文本里，是把协议状态变成不可查询的副作用。清晰系统会让控制面小而可靠，让数据面高吞吐，让状态有明确持久化和恢复边界。

单机系统同样如此。环形队列的 head、tail、size 是控制状态，buffer 里的任务是数据，mutex 和条件变量是控制机制。分布式 shuffle 中，manifest 是控制状态，中间文件是数据，driver 的 commit protocol 是控制机制。把这种对应关系看清，后面学习锁、无锁、任务图和分布式提交时会更容易。

1.18 不确定性从哪里来

系统越往右侧尺度推进，不确定性越多。单线程程序的执行顺序大体由代码决定；多线程程序会受到调度、缓存一致性、内存序和锁竞争影响；异步 I/O 会受到设备完成时间、队列长度和取消路径影响；分布式系统会受到网络延迟、消息重复、节点故障和时钟漂移影响。工程设计不能假设这些不确定性不存在，而要把它们纳入语义。

不确定性有两种处理方式。第一种是消除，例如用锁建立互斥，用固定合并树保证浮点归约顺序，用 route epoch 拒绝旧请求。第二种是容忍，例如用幂等处理重复请求，用重试预算处理临时失败，用误差范围处理浮点差异，用背压处理流量波动。优秀系统会明确哪些不确定性必须消除，哪些可以容忍，哪些只能观测和降级。

这也是为什么正确性先于性能。一个不确定性没有被定义的系统，性能数字越漂亮越危险。并发程序偶尔丢任务，分布式作业偶尔重复计数，I/O 管线偶尔在关闭时 use-after-free，这些问题都不能用平均吞吐掩盖。第二册每个章节都会先写语义边界，再写性能优化。

1.19 软件接口如何影响硬件

软件接口会决定硬件能看到什么模式。一个函数如果每次只接受单个对象，硬件只能看到零散访问；一个函数接受连续 span，编译器和预取器才有机会处理批量数据。一个 API 如果允许输入输出重叠，编译器必须保守；一个 API 明确只读输入和独占输出，就更容易向量化和并行化。一个运行时如果把所有任务都塞进全局队列，硬件会看到共享 cache line 争用；一个运行时使用本地队列和分层合并，硬件会看到更多本地访问。

因此，性能不是最后用几行特殊指令补上的。类型、容器、所有权、错误返回、任务粒度、消息格式和提交协议都会影响底层行为。高质量系统教材必须把接口设计和硬件成本放在同一张图里。第一册讲过 C++ 对象和内存，本册继续追问：这些对象如何穿过核心、缓存、线程、队列和网络。

1.20 从小实验到大系统

本册很多实验看起来很小：数组求和、计数器、环形队列、bounded queue、parallel scan。它们之所以重要，是因为每个小实验都代表一个大系统中的核心矛盾。数组求和代表数据移动和依赖链；计数器代表共享写热点；环形队列代表发布和消费；bounded queue 代表背压；parallel scan 代表无冲突输出位置；shuffle 代表跨节点数据重排。

读者做实验时，不要只得到“这个版本快”或“那个版本慢”的结论，而要把实验抽象成系统原则。计数器分片成功后，要想到热点 key；bounded queue 阻塞成功后，要想到服务过载；SPSC ring buffer 正确后，要想到生产者消费者模型的适用范围；manifest 去重正确后，要想到外部效果 exactly-once。小实验是可控环境，大系统是同一原则的组合。

1.21 本册的阅读方式

第二册不适合只顺着文字看。每章至少要做三件事。第一，画图：数据流、执行图和成本图。第二，写 reference：无论是算法还是协议，都先写最简单正确版本。第三，跑对照：改变一个变量，观察结果是否支持假设。只看概念不做实验，容易把系统知识学成口号；只写代码不画图，容易在复杂并发和分布式路径里迷路；只跑 benchmark 不写语义，容易优化错误程序。

本册也不要求读者一开始就拥有大服务器。手机 Termux、普通 Linux 笔记本或云主机都可以完成大部分语义实验。没有 NUMA 设备时，就写清不能验证远端内存；没有硬件计数器权限时，就用输入规模、时间和源码结构做替代；没有多机时，就用本机多进程模拟消息乱序和重试。证据边界要诚实，但学习不必等待完美环境。

1.22 一个请求的全链路视角

为了把本册的层次放在一起，可以跟踪一个日志统计请求。用户提交一个作业，driver 读取作业配置，切分输入文件，创建 map task。Worker 线程从队列取到 task，读取文件页，解析文本，写入列式 batch，做本地 histogram，把 partial 放入 shuffle 队列。Reduce worker 读取 partial，合并 key，写输出临时文件，driver 提交 manifest，最后返回作业完成。

这条链路里，每一步都对应本册一类知识。读取文件涉及 page cache、短读、I/O 背压；解析文本涉及分支、SIMD 字节扫描和数据布局；本地 histogram 涉及 cache line、线程本地 partial 和合并；队列涉及 mutex、条件变量、关闭和背压；线程池涉及任务粒度和调度；shuffle 涉及分片、热点 key 和队列容量；manifest 涉及提交点、幂等和恢复。一本系统书的价值，就在于让读者能把这些层次连成一条链。

当这条链路变慢时，也要沿链路分析，而不是随便挑一个熟悉点优化。若输入读取慢，改 SIMD 没用；若 reduce partition 热点，改 parser 没用；若 manifest fsync 慢，增加 worker 可能让等待更严重；若队列容量太大，吞吐看似高但尾延迟失控。全链路视角能防止局部优化伤害整体。

1.23 成本账本

系统工程可以像记账一样记录成本。每处理一条记录，需要读多少字节，解析多少字符，写多少热字段，更新多少本地桶，发送多少 shuffle 字节，最终写多少输出。每个阶段还要记录固定成本：任务调度、队列入出、批次头部、manifest 条目、checkpoint fsync。可变成本和固定成本共同决定最佳 batch 大小和任务粒度。

成本账本不需要一开始完全精确。先做粗估：每条记录 40 字节文本，解析后热字段 24 字节，本地聚合后每个 key 16 字节，shuffle 压缩后多少字节。再用实际指标校正。若估算和测量差异巨大，说明有隐藏成本，例如字符串分配、写放大、cache miss 或重试。成本账本是把直觉变成可检查假设的方法。

1.24 语义账本

性能账本之外，还要有语义账本。每个数据项要知道来源、所有者、可变性、提交点和恢复方式。输入 split 来自文件 offset，只读；ParsedLogHotBatch 由 parse stage 生成，由 compute stage 读取，batch 生命周期由队列管理；ShuffleBlock 由 map attempt 生成，只有 driver commit 后有效；OutputBlock 由 reduce attempt 生成，manifest 发布后对外可见。语义账本能防止重复处理和悬垂访问。

很多 bug 都能归因到语义账本缺失。一个 batch 被下游异步使用时，上游复用了缓冲；一个 attempt 输出已经失败，却被 reduce 读到；一个队列关闭后 producer 仍然写入；一个配置对象发布后旧版本被释放。把所有权和提交点写清，是性能系统的安全网。

1.25 全书复用的六个问题

遇到任何新系统，都可以问六个问题。第一，数据在哪里，按什么布局存放。第二，谁执行，执行实体是线程、任务、进程还是节点。第三，谁共享，哪些状态被多个执行实体读写。第四，谁等待，等待发生在锁、I/O、队列、网络还是依赖。第五，谁提交，什么时候外部可以认为结果有效。第六，谁恢复，失败后从哪里继续。

这六个问题会在后文反复使用。数组内核回答数据和执行；锁章回答共享和等待；I/O 章回答等待和背压；分布式章回答提交和恢复；贯穿项目把六个问题同时回答。读者如果能把任何系统拆成这六类，就已经掌握第二册的主线。

1.26 学习路线和阶段目标

第一阶段，读者应能解释一个单线程循环为什么慢。这里的关键词是依赖链、分支、cache line、TLB、预取和编译器优化报告。不要急着上线程，先把单核执行模型看清。第二阶段，读者应能把一个循环扩展到多线程，并说明任务切分、partial、false sharing、线程绑定和扩展曲线。第三阶段，读者应能写出 bounded queue、线程池和任务图，说明关闭、等待、异常和取消。第四阶段，读者应能把单机 pipeline 扩展到本机分布式模拟，处理 request id、重试、manifest 和 checkpoint。

每个阶段都有明确交付物。单线程阶段交付一个 reference 和性能报告；多线程阶段交付扩展曲线和 partial 布局说明；运行时阶段交付线程池状态机和测试；分布式阶段交付故障矩阵和恢复演练。这样学习不会变成泛泛读书，而是形成可运行、可测量、可解释的项目。

1.27 常见误区总览

第一，看到 CPU 使用率低就认为要开更多线程，但真实瓶颈可能是 I/O、锁等待或背压。第二，看到 atomic 就认为比 mutex 快，但热点 atomic 仍会造成 cache line 迁移。第三，看到 SIMD 就认为一定更快，但内存带宽和数据布局可能才是瓶颈。第四，看到队列积压就扩大队列，但这可能只是延迟和内存风险后移。第五，看到重试能提高成功率就无限重试，结果放大故障。第六，看到复制就以为安全，却没有备份和恢复演练。

这些误区贯穿全书。每当读者想套用一个简单答案，都应回到证据：当前瓶颈是什么，语义要求是什么，失败路径是什么，成本迁移到哪里。系统工程很少有永远正确的局部技巧，只有在上下文中成立的设计。

1.28 从面试知识到工程知识

很多读者熟悉算法复杂度、操作系统概念和 C++ 语法，但进入工程系统后会发现问题变得模糊。一个 $O(n)$ 扫描可能慢在 TLB，一个线程安全队列可能慢在 false sharing，一个正确 RPC 可能因为重试风暴拖垮服务，一个分布式作业可能因为响应丢失重复计数。第二册要做的事情，就是把这些看似分散的工程问题放回统一框架：数据移动、共享状态、等待关系和提交语义。

面试知识常把问题边界给好：输入在内存里，函数一次调用，失败不存在，线程不存在。工程知识需要自己定义边界：输入来自哪里，能否失败，谁拥有内存，是否并发，结果何时提交，重复执行怎么办。不是面试知识没用，而是它只覆盖系统的一部分。本册希望读者能把算法题里的思维扩展到真实计算系统。

1.29 系统设计审查表

读完本册后，面对一个新模块，可以按审查表提问。数据层面：输入规模多大，字段冷热如何，是否需要列式布局，是否有稳定 ID，是否需要序列化。执行层面：任务粒度多大，线程数如何配置，是否可能阻塞，是否存在嵌套并行。同步层面：哪些状态共享，谁写谁读，锁保护什么不变量，原子是否发布其他数据，是否需要回收协议。I/O 层面：队列是否有界，写入何时持久化，超时后底层完成如何处理。分布式层面：请求如何去重，提交点在哪里，旧路由如何拒绝，恢复从哪个 checkpoint 开始。

这个审查表不是流程负担，而是防止遗漏。系统 bug 常常不是因为工程师不知道某个概念，而是设计时忘了问某个问题。比如写了线程池但没有关闭语义，写了 manifest 但没有校验，写了重试但没有幂等，写了 SIMD 内核但没有 scalar reference。审查表把这些经验变成可重复动作。

1.30 模块成熟度

一个模块可以按成熟度分层。第一层是能跑通 happy path。第二层是有 reference 和边界测试。第三层是有性能指标和对照实验。第四层是有失败注入和恢复。第五层是有文档、审查表和可运维指标。很多教学代码停在第一层，本册要求至少把核心模块推进到第四层。只有这样，项目才像计算系统，而不是演示程序。

成熟度也能帮助安排工作顺序。不要在第一层还不稳定时做复杂优化；不要在没有 reference 时引入多线程；不要在没有幂等提交时开启 speculation；不要在没有指标时调调度器。工程推进需要台阶，每个台阶都要能验证。

1.31 全书实验总表

全书实验可以按能力排列。硬件阶段做单累加器、多累加器、分支预测和链表对照。内存阶段做 AoS/SoA、冷热分离、TLB、mmap 和 page fault。单机高性能阶段做自动向量化、histogram、top-k、stream compaction。同步阶段做 bounded queue、atomic counter、SPSC ring 和 hazard/epoch 设计。运行时阶段做线程池、任务图、work stealing 和 trace。I/O 阶段做批处理、背压、fsync 和 completion。分布式阶段做 request id、重试、manifest、checkpoint 和热点 key。

这张总表能防止学习碎片化。每个实验都不是孤立练习，而是下一阶段的前置。比如 stream compaction 的 scan，会在 shuffle offset 中复用；bounded queue 的 close，会在线程池 shutdown 中复用；request id 的幂等，会在 task attempt commit 中复用。

1.32 如何判断自己学会了

判断是否真正学会第二册，不是看能否复述术语，而是看能否面对一个慢系统提出可验证假设。比如一个日志作业慢，你能否画出数据流和执行图；能否说出是读取、解析、聚合、shuffle、reduce 还是提交慢；能否设计对照实验；能否说明失败后如何恢复；能否写出哪些指标缺失。若能做到这些，就说明你已经从“知道知识点”进入“能做系统分析”。

另一个判断标准是能否拒绝不合适的优化。别人建议“换无锁队列”，你能问生产者消费者模型是什么、关闭语义是什么、瓶颈是否真在队列；别人建议“上 SIMD”，你能问数据是否连续、是否受内存带宽限制、是否有 reference；别人建议“无限重试”，你能指出幂等和重试预算。能拒绝错误优化，是成熟工程能力的一部分。

1.33 本册贯穿实验

第二册的实验项目叫 Compute Systems Labs。第一阶段实现顺序归约、并行归约、带锁计数器、原子计数器和有界队列。后续章节会围绕它继续扩展：加入线程池、任务图、工作窃取、无锁队列、并行扫描、I/O 背压和分布式模拟。

实验一：运行 Compute Systems Labs。记录 CPU 型号、核心数、线程数、缓存层级、系统版本、编译器版本和测试命令。不要只写测试通过，还要解释每个模块分别验证了计算系统的哪一类问题。

第 2 章

核心流水线、乱序执行与分支预测

现代 CPU 核心不是按源码顺序一条一条机械执行。为了提高吞吐，核心会把指令拆成微操作，放入队列，分析依赖，尽量让互不依赖的操作并行执行。理解这一层，是理解单线程性能、SIMD 前的标量优化和并发路径的基础。

2.1 流水线不是免费并行

流水线把取指、译码、执行、访存和提交分成阶段。理想情况下，每个周期都有新指令进入，每个阶段都在工作。但真实程序有数据依赖、控制依赖和资源冲突。后一条指令需要前一条指令结果时，执行端口即使空闲也不能使用；分支目标未知时，前端可能取错指令；访存 miss 时，后端会等待数据。

流水线的关键不是“越深越快”，而是吞吐和延迟的权衡。深流水线可以提高频率，却放大分支预测失败的代价。并发程序中的短临界区也会受流水线影响：一个看似只有几条指令的锁操作，背后可能包含原子读改写、cache line 独占获取和内存序约束。

2.2 前端：取指、译码和微操作

核心前端负责把指令流喂给后端。它要从 instruction cache 取指，预测下一条指令地址，译码复杂指令，把它们转成内部微操作，并放入队列。前端跟不上时，后端即使有空闲执行端口也没有工作可做。大函数、过多分支、间接跳转、指令 cache miss 和译码压力都可能让前端成为瓶颈。

x86-64 指令长度可变，译码比定长指令集复杂。现代核心通常会使用 micro-op cache 或类似结构缓存译码结果，让热循环不用每次重新译码。这个机制解释了为什么热循环体大小、函数内联和代码布局会影响性能。内联可以减少调用开销，但过度内联会膨胀代码，伤害 instruction cache 和前端供给。

第一册已经教读者读汇编。第二册读汇编时要多看一个维度：这段机器码对前端是否友好。短小热循环、稳定分支、连续代码布局和可预测调用目标，通常更容易让前端持续供给后端。反过来，大量模板实例化、过度内联、异常路径混在热路径、虚调用目标多变、解释器式大 switch，都可能让前端成本超过源码直觉。

深入理解

前端瓶颈的一个典型现象是：数据已经在缓存中，算术也不复杂，但 IPC 仍然低，并且改变数据布局帮助不大。这时应检查代码体积、分支目标、间接跳转和译码压力，而不是继续只改数组排列。

2.3 后端：执行端口和调度

后端接收微操作后，要等待操作数准备好，并把它们发射到合适执行端口。整数 ALU、浮点乘加、加载、存储、分支和地址生成都有各自资源。一个循环如果每轮需要太多 load，可能受加载端口限制；如果每轮地址计算复杂，可能受地址生成单元限制；如果依赖链很长，端口空着也不能发射。

因此分析单核性能时，要区分“端口吞吐不够”和“依赖导致发不出去”。多累加器能改善后者；SIMD 能提高每条指令处理的数据量；简化地址表达式和连续访问能减轻加载和地址生成压力。不同 CPU 微架构端口配置不同，严肃优化必须结合具体机器。

后端不是一个抽象黑箱。一个 load 通常需要地址生成、TLB 查询、缓存访问和可能的 miss 处理；一个 store 通常拆成地址和数据路径，并进入 store buffer；一个浮点 FMA 会占用特定向量执行资源；一个整数分支会占用分支执行资源。循环的性能取决于这些微操作怎样分布到端口上，也取决于它们之间是否互等待。

这解释了为什么源码层面的“操作次数”常常误导。两个循环都执行同样数量的加法，一个可能因为单累加器依赖链而慢，另一个可能因为多累加器让后端并行发射而快。两个循环都做同样数

量的 load，一个可能连续访问被预取器支持，另一个可能指针追踪导致每次 load 等上一次 load 的地址。

2.4 乱序执行的边界

乱序执行允许核心在不改变单线程可观察结果的前提下，提前执行已经准备好的指令。寄存器重命名消除了很多假依赖，保留站和重排序缓冲区让多个指令同时等待资源。这样做可以隐藏一部分延迟，例如在一次乘法等待时执行另一个独立加法。

但是乱序执行不能越过所有边界。真实数据依赖不能消除，cache miss 太多时可并行等待的 miss 数量有限，内存屏障会限制重排，原子操作会带来更强的顺序约束。写高性能代码时，增加独立累加器、减少循环携带依赖、让数据访问可预测，都是在帮助乱序核心找到可调度空间。

2.5 重排序缓冲区和执行窗口

乱序执行不是无限的。核心只能观察一个有限窗口内的指令。重排序缓冲区保存尚未提交的指令，load buffer 保存未完成加载，store buffer 保存尚未对外可见的存储。如果窗口被长延迟 load 占住，后续可执行工作又不够，核心会停顿。

链表遍历就是典型例子。每次读取下一个指针之前，必须等当前节点加载完成。乱序核心很难提前知道下一步地址，所以内存级并行度很低。相比之下，遍历多个独立数组或同时处理多条链，可以让多个 load 同时飞行，更容易隐藏延迟。

执行窗口可以被理解为硬件能同时“看见”的未来。窗口越大，越有机会在一个长延迟操作等待时找到其他独立工作。但如果代码中充满循环携带依赖，窗口再大也找不到可提前执行的操作；如果 cache miss 太多，load buffer 被占满，后续加载也发不出去；如果分支预测失败，窗口里的错误路径工作会被丢弃。

软件能做的事情，是把独立工作显式放进窗口。多累加器、分块处理、批量处理多个请求、一次处理多个链表游标、把检查从热循环中外提，都属于给乱序核心提供更多可调度空间。相反，频繁调用不可内联函数、隐藏别名、每步都依赖共享原子结果，会压缩这个空间。

2.6 Load、Store 和内存消歧

内存操作比寄存器操作复杂。核心要判断一个 load 是否可能读取前面尚未完成的 store。如果地址关系不明确，硬件会做内存消歧预测。预测错误时，需要回滚并重新执行相关操作。高性能循环中，减少指针别名、使用清晰的数组边界、避免复杂交叉读写，可以帮助硬件和编译器更大胆地调度。

store buffer 也影响并发语义。普通 store 可以先进入 store buffer，稍后再对其他核心可见。内存屏障、原子操作和锁释放会约束这种行为。C++ 内存模型的 acquire/release 最终要落到这类硬件机制上。

Load/store queue 还是很多微妙性能问题的来源。若一个循环中既写数组又读另一个可能别名的数组，编译器和硬件都必须保守处理读写顺序。若 store 地址很晚才算出来，后面的 load 就难以确认是否可以提前。若 store buffer 堆积，后续 store 可能被阻塞，锁释放和原子操作也可能放大这个问题。

并发程序中，store buffer 让“本线程已经执行了写”不等于“其他线程已经能按你想象的顺序看到写”。这不是说硬件不可靠，而是说可见性需要同步语义。后面讲 release/acquire 时，会把这条硬件直觉变成 C++ 层面的 happens-before 关系。

2.7 分支预测和控制流

分支预测让核心在条件结果出来之前先猜路径。预测正确时，流水线保持饱满；预测失败时，错误路径上的工作要被丢弃。随机分支、数据相关分支和复杂间接跳转都会破坏预测。很多高性能循环会把条件变成掩码、查表或分段处理，就是为了减少不可预测控制流。

并发程序也受分支预测影响。锁的 fast path 通常期望“不竞争”，如果竞争比例上升，分支行为和 cache 行为都会改变。无锁结构的 CAS 循环在低竞争时很短，在高竞争时会反复失败，控制流和内存系统同时恶化。

深入理解

判断一段代码是不是受分支预测影响，可以比较不同输入分布下的 cycles、branches 和 branch-misses。如果随机数据显著慢于有序数据，而内存访问量没有明显变化，分支预测通常就是主要嫌疑。

2.8 SMT 和共享核心资源

SMT 让一个物理核心暴露多个逻辑 CPU。它可以在一个线程等待内存或前端供给不足时，让另一个线程使用空闲资源。但 SMT 不是免费翻倍。两个逻辑线程会共享前端、执行端口、缓存、TLB 和乱序资源的一部分。如果两个线程都在做同样高强度向量计算，它们可能互相抢执行资源；如果一个线程经常等内存，另一个线程可能填补空档。

因此 benchmark 中不能只写“8 个 CPU”。要区分物理核心和 SMT 线程。对于 CPU-bound 数值内核，先扩展到物理核心往往比立刻使用所有逻辑 CPU 更稳定；对于延迟隐藏明显的工作负载，SMT 可能提高吞吐。结论必须来自本机测量，而不是固定经验。

2.9 微架构和 ISA 的边界

第一册强调 ISA 和 ABI：同一条 `add` 在 x86-64 语义上做什么，函数参数怎样传递。第二册强调微架构：同一个 ISA 指令在不同 CPU 上可能拆成不同微操作，使用不同端口，拥有不同延迟和吞吐。ISA 给出程序可见语义，微架构决定实现成本。

这条边界非常重要。写教材时不能把某个 CPU 的端口配置说成 x86-64 的永久性质；写性能报告时也不能只说“x86 上这个指令很快”。更准确的说法是：在某个 CPU 型号、某个编译选项、某个输入分布下，观察到某个指令序列的瓶颈更像前端、端口、依赖、内存或分支。

2.10 典型性能症状

前端瓶颈常表现为 IPC 低、branch miss 或 instruction cache 相关事件高，代码布局和热循环大小值得检查。后端执行瓶颈常表现为数据在缓存中、分支也稳定，但 IPC 仍受端口或依赖限制，适合检查 SIMD、多累加器和指令混合。内存瓶颈常表现为 cache miss、TLB miss 或带宽接近平台上限。同步瓶颈则可能表现为 IPC 低、cache line 迁移、context switch、futex 等待或原子热点。

这些症状不会自动给答案，但能排除方向。不要在内存带宽瓶颈上执着减少几条整数指令，也不要锁竞争瓶颈上只调整循环展开。性能工程的价值在于把“慢”拆成可以证伪的假设。

2.11 从硬件回到代码

核心流水线给软件三个直接启发。第一，减少长依赖链，例如把单个累加器拆成多个局部累加器。第二，减少不可预测分支，让热路径尽量直。第三，减少会强制顺序的操作，例如不必要的原子和屏障。后续章节讨论 SIMD、锁、无锁队列和并行归约时，都会回到这些原则。

2.12 问题案例：为什么一个求和循环也会慢

本章不能只停留在“流水线很深、乱序很强”这种抽象描述上。我们从 Compute Systems Engine 的第一条主线开始：对一个很大的 `std::int32_t` 数组求和。这个问题看起来太简单，甚至像入门练习，但正因为它简单，才适合暴露单核执行的几个基本矛盾：每次循环都有加载、扩展、加法、循环控制和分支；总和变量形成循环携带依赖；数组访问通常可被预取器识别；输入足够大时又会被内存带宽限制。一个求和循环可以同时讲清“依赖、前端、后端、内存和测量”这几条线。

朴素版本通常写成一个累加器。

Listing 2.1 baseline: 单累加器顺序归约

```
1 std::int64_t sum_baseline(std::span<const std::int32_t> values) {
2     std::int64_t sum = 0;
3     for (std::size_t i = 0; i < values.size(); ++i) {
4         sum += values[i];
5     }
```

```

6     return sum;
7 }

```

这个版本的好处是清楚、正确、容易作为 reference。它的弱点也很明显：每一轮的 `sum` 都依赖上一轮的 `sum`。处理器可以提前加载后面的数组元素，也可以预测循环分支，但加法结果本身是一条长链。只要下一次加法必须等上一次加法结果，乱序执行就不能把这些加法任意并行起来。此时瓶颈不一定是加法器数量，而可能是依赖链延迟。

改进版本把一个长依赖链拆成多条短依赖链。

Listing 2.2 改进版本：四个独立累加器

```

1 std::int64_t sum_four_accumulators(std::span<const std::int32_t> values) {
2     std::int64_t sum0 = 0;
3     std::int64_t sum1 = 0;
4     std::int64_t sum2 = 0;
5     std::int64_t sum3 = 0;
6
7     std::size_t i = 0;
8     const std::size_t limit = values.size() / 4 * 4;
9     for (; i < limit; i += 4) {
10         sum0 += values[i];
11         sum1 += values[i + 1];
12         sum2 += values[i + 2];
13         sum3 += values[i + 3];
14     }
15     for (; i < values.size(); ++i) {
16         sum0 += values[i];
17     }
18     return (sum0 + sum1) + (sum2 + sum3);
19 }

```

这个版本不是为了“手动展开看起来高级”，而是给乱序核心更多可调度空间。四个累加器之间没有真实数据依赖，核心可以在一个累加器等待结果时推进另一个累加器。循环体变大，前端压力略增，但依赖链压力下降。若输入在缓存中，这种改法常能提高 IPC；若输入远大于 LLC 并且已经接近内存带宽上限，提升可能很小，因为瓶颈已经从依赖链迁移到数据移动。

还要注意边界处理。第二个循环处理不能被四整除的尾部元素。很多性能示例为了简短直接假设长度是四的倍数，这是教材不应采用的写法。真实系统里，输入规模经常来自文件、网络或上层分片，不能为了微优化丢掉边界正确性。Compute Systems Engine 的 reference 测试必须覆盖空输入、一个元素、非对齐长度、负数和大规模输入，不能只测漂亮输入。

2.13 优化不是越展开越好

既然四个累加器能拆依赖链，是否八个、十六个更好？答案取决于机器、编译器和工作负载。展开过少，依赖链仍然限制后端；展开过多，会增加寄存器压力、代码体积和前端供给压力。寄存器不够时，编译器可能把临时值溢出到栈上，反而引入额外 load/store。代码体积变大时，热循环可能更容易超过微操作缓存或 instruction cache 的舒适范围。

这就是本册反复强调的“瓶颈会迁移”。一个版本慢，是因为单累加器依赖；另一个版本慢，可能因为寄存器压力；再另一个版本慢，可能因为已经受内存带宽限制。优化不能只问“这个技巧有没有用”，要问“在当前证据下，它针对的是哪个瓶颈，改完后瓶颈会迁到哪里”。

教材里常见的坏写法是直接给“最佳版本”。这种写法让读者只会背答案，不会分析。正确写法应保留 baseline，因为 baseline 是 correctness oracle；给出有限改进版本，因为它暴露优化动机；再说明它的边界，因为工程中没有永久最优版本。对求和来说，baseline 用于验证结果，多累加器用于解释依赖链，SIMD 版本用于解释向量执行，线程本地 partial 用于解释共享写和 false sharing，多线程

程版本用于解释带宽和 NUMA。这些版本不是重复代码，而是同一个问题在不同系统层面的投影。

2.14 不要把共享原子当作并行归约

很多初学者把并行求和写成多个线程同时对一个原子变量做加法。这个版本结果可能是正确的，却几乎一定不是好的归约实现。

Listing 2.3 反例：所有 worker 争用同一个原子和同一条 cache line

```

1 std::int64_t sum_with_hot_atomic(std::span<const std::int32_t> values,
2                                 std::int32_t worker_count) {
3     std::atomic<std::int64_t> global_sum{0};
4     std::vector<std::thread> workers;
5     workers.reserve(static_cast<std::size_t>(worker_count));
6
7     for (std::int32_t worker = 0; worker < worker_count; ++worker) {
8         const std::size_t begin =
9             values.size() * static_cast<std::size_t>(worker) /
10             static_cast<std::size_t>(worker_count);
11         const std::size_t end =
12             values.size() * static_cast<std::size_t>(worker + 1) /
13             static_cast<std::size_t>(worker_count);
14         workers.emplace_back([&values, &global_sum, begin, end]() {
15             for (std::size_t i = begin; i < end; ++i) {
16                 global_sum.fetch_add(values[i], std::memory_order_relaxed);
17             }
18         });
19     }
20
21     for (std::thread& worker : workers) {
22         worker.join();
23     }
24     return global_sum.load(std::memory_order_relaxed);
25 }
```

这里使用 `memory_order_relaxed` 并没有错，因为这个原子只用于统计自身数值，不发布其他对象。但 `relaxed` 只能减少顺序约束，不能消除一致性争用。所有线程仍在更新同一个 cache line。每一次 `fetch_add` 都要求对应 cache line 获得独占修改权，多个核心会反复争夺所有权。结果是用户以为自己写了“无锁高性能”，硬件看到的是一个极热的共享写点。

正确方向通常是线程本地 `partial`。

Listing 2.4 工程版本：线程本地 `partial`，再串行合并

```

1 struct alignas(64) PartialSum {
2     std::int64_t value = 0;
3 };
4
5 std::int64_t sum_with_partials(std::span<const std::int32_t> values,
6                                std::int32_t worker_count) {
7     std::vector<PartialSum> partials(static_cast<std::size_t>(worker_count));
8     std::vector<std::thread> workers;
9     workers.reserve(static_cast<std::size_t>(worker_count));
10
11     for (std::int32_t worker = 0; worker < worker_count; ++worker) {
12         const std::size_t begin =
```

```

13         values.size() * static_cast<std::size_t>(worker) /
14         static_cast<std::size_t>(worker_count);
15     const std::size_t end =
16         values.size() * static_cast<std::size_t>(worker + 1) /
17         static_cast<std::size_t>(worker_count);
18     workers.emplace_back([&values, &partials, worker, begin, end]() {
19         std::int64_t local = 0;
20         for (std::size_t i = begin; i < end; ++i) {
21             local += values[i];
22         }
23         partials[static_cast<std::size_t>(worker)].value = local;
24     });
25 }
26
27 for (std::thread& worker : workers) {
28     worker.join();
29 }
30
31 std::int64_t total = 0;
32 for (const PartialSum& partial : partials) {
33     total += partial.value;
34 }
35 return total;
36 }

```

这个版本把热路径写共享变成线程本地写。每个 worker 在自己的寄存器里累加，结束时只写一次 partial。对齐槽位用于避免多个 partial 落在同一条 cache line 上形成 false sharing。它的代价也要写清楚：需要额外 partial 数组，需要合并阶段，线程创建和 join 也有成本。如果输入很小，线程成本可能超过计算成本；如果输入很大，局部顺序扫描加一次合并通常比热原子好得多。

2.15 从源码到汇编应该看什么

第一册已经讲过如何读汇编。第二册读汇编不是为了炫耀指令名，而是为了验证源码是否产生了符合预期的执行形态。看求和循环时，应至少关注四件事。

第一，看编译器是否保留了循环。若结果没有被使用，优化器可能删除整个循环，benchmark 就变成空测。第二，看循环是否形成单累加器依赖，还是被编译器展开成多个累加器。现代编译器在优化级别较高时可能自动做循环展开和向量化，因此手写版本和编译器生成版本要放在同一编译选项下比较。第三，看是否有意外的内存访问。例如寄存器压力过大导致 spill，会在热循环中出现额外栈访问。第四，看是否有分支和调用留在热路径里。例如调试检查、虚调用、异常路径或不可内联函数都可能改变前端压力。

下面是一份实验报告可以采用的观察顺序。先用 **objdump** 或编译器的汇编输出确认热循环；再用 **perfstat** 比较 cycles、instructions 和 IPC；然后改变输入规模，让数据分别处在 L1、L2、LLC 和内存范围；最后再改变版本，从 baseline 到多累加器到线程本地 partial。若只看一个规模，很容易把依赖瓶颈和内存瓶颈混在一起。

深入理解

同一个源码版本，在不同编译器和不同优化级别下可能生成不同机器码。教材里的结论必须写成“分析方法”，不能写成“某段源码永远会生成某条指令”。严肃报告要记录编译器版本、目标架构、优化选项和关键汇编片段。


```

3     std::int64_t count = 0;
4     for (const std::int32_t value : values) {
5         count += value > threshold ? 1 : 0;
6     }
7     return count;
8 }

```

这段源码仍然写了条件表达式，编译器可能生成条件移动、集合指令或向量比较，也可能仍生成分支，具体取决于目标架构和优化器。教材重点不是保证某种机器码，而是教读者提出可验证假设：随机输入下 branch miss 是否更高；无分支版本 instructions 是否增加；总时间是否下降；当输入分布从随机变成有序后，两个版本差距是否消失。如果这些现象吻合，就说明分支预测是主要因素之一。

分支优化也有代价。把复杂 if 改成掩码运算，可能让所有路径都执行，增加算术或内存访问；把数据分组处理，可能增加预处理成本；把虚调用改成分发表，可能降低可读性。不要把“branchless”当作普遍更快。只有当分支不可预测且热路径短，减少分支才经常有收益。

2.18 前端瓶颈案例：热循环里的调用和大 switch

很多性能分析只盯着数据 cache，却忽略了前端供给。前端要取指、预测、译码和提供微操作。若热路径里有过多不可内联调用、异常边、模板膨胀后的大代码体、解释器式大 switch 或间接跳转，后端可能一直等不到足够微操作。

一个典型案例是记录处理器。每条记录包含类型字段，不同类型执行不同处理逻辑。朴素实现可能把所有逻辑放进一个巨大的 switch，每个 case 又调用复杂函数。若输入类型分布随机，间接分支或多路分支预测困难；若每个 case 代码很大，instruction cache 和微操作缓存压力上升。优化方向可能不是“少做几次整数加法”，而是重排数据：先按类型分桶，再批量处理同类记录。这样做改变了控制流形状，让分支更稳定，也让热代码更集中。

这类优化有一个重要边界：分桶会增加一次额外遍历和写入，还可能破坏原始顺序。若语义要求严格保持输入顺序，就需要额外索引或稳定输出；若记录数量很少，分桶成本超过收益；若每个记录处理非常重，分支成本可能不是主导。系统工程的结论永远要回到工作负载。

2.19 内存级并行度和指针追踪

乱序执行能隐藏一部分内存延迟，前提是窗口里有多个独立 load。顺序数组扫描虽然会 cache miss，但预取器能提前请求后续 cache line，多个 miss 可以同时飞行。链表遍历则不同：下一步地址存放在当前节点里，必须等当前节点加载完成后才知道下一次 load 地址。这种指针追踪几乎把内存访问串成依赖链。

下面的代码形态看起来只是遍历节点，硬件看到的是一条地址依赖链。

Listing 2.8 指针追踪：下一次加载地址依赖当前加载结果

```

1 struct Node {
2     std::int32_t value = 0;
3     Node* next = nullptr;
4 };
5
6 std::int64_t sum_list(const Node* head) {
7     std::int64_t sum = 0;
8     const Node* current = head;
9     while (current != nullptr) {
10         sum += current->value;
11         current = current->next;
12     }
13     return sum;
14 }

```

这段代码不适合作为热路径大规模数据扫描的默认结构。它的每个节点可能来自不同页面，cache miss 和 TLB miss 都可能很高；下一节点地址不可提前知道，内存级并行度低；分配器还可能把节点分散到不同 NUMA 节点。若业务语义允许，把节点压缩成连续数组或索引数组，通常更适合现代 CPU。若必须使用链式结构，可以考虑批量处理多条独立链，让核心同时等待多个地址链；也可以使用池化分配，让节点尽量连续；还可以把热字段和冷字段分离，减少每个 cache line 带入的无用数据。

2.20 和编译器优化的边界

有些读者会问：既然编译器能展开、向量化、调度指令，为什么还要学习微架构？答案是，编译器非常强，但它不是业务语义的读心器。编译器不知道某个输入分布是否随机，不知道两个 span 在业务上是否必不重叠，不知道一把锁竞争是否来自热点 key，不知道任务粒度是否会造成线程池空转，也不知道分布式 shuffle 是否会遇到倾斜。它只能在 C++ 语义、目标架构和优化启发式内工作。

软件工程师要做的是把真实语义表达给编译器和硬件。用连续容器表达顺序访问，用只读 span 表达输入不可修改，用线程本地 partial 表达写入独占，用稳定的数据分块表达可预测访问，用测试保证边界输入正确。高性能代码不是处处和编译器对抗，而是提供更清晰的结构，让编译器能安全优化，让硬件能持续执行。

2.21 Linux 与工具源码阅读入口

本章的 Linux 源码阅读不应从调度器全局策略开始，而应从性能证据的来源开始。若使用 `perfstat` 观察 cycles、instructions、branches 和 branch-misses，可以阅读 Linux 内核的 perf event 子系统入口，理解用户态工具如何请求硬件计数器、内核如何复用和调度计数器、权限如何限制某些事件。体系结构相关事件通常在 `arch/x86/events/` 一类目录下，通用框架在 `kernel/events/` 附近。不同内核版本路径会变化，读源码时必须记录版本。

编译器源码阅读可以从优化报告开始，而不是直接钻进后端。GCC 和 Clang 都能输出向量化或优化诊断。读者可以先写两个循环，一个容易向量化，一个因为别名或分支不容易向量化；观察诊断后，再选择性阅读对应编译器文档或源码。目标不是成为编译器开发者，而是理解“为什么这段循环没被优化”的证据链。

2.22 流水线不是简单并行流水

很多入门图把 CPU 流水线画成取指、译码、执行、访存、写回五段，这有助于入门，但现代核心远比五段复杂。前端要取指、预测分支、解码指令、使用微操作缓存；中间要重命名寄存器、分配 reorder buffer、调度微操作；后端要在多个执行端口、load/store queue 和缓存层级之间执行；提交阶段还要按程序顺序退休结果。理解现代性能时，不能停留在五段流水线的静态图。

乱序执行的核心思想是：只要不违反数据依赖和内存依赖，后面的独立操作可以先执行，用来隐藏前面操作的等待。若一个 load 等待内存，核心可以执行窗口中其他独立加法、乘法或 load。若窗口里全是依赖这个 load 的操作，核心就只能等。软件优化常做的多累加器、循环展开、数据连续化，本质上是在给乱序窗口提供更多独立工作。

乱序执行不是无限的。窗口大小有限，load buffer、store buffer、物理寄存器、调度队列和执行端口都有限。若程序有太多未完成 miss，load buffer 会满；若分支预测错，错误路径上的工作会被丢弃；若指令前端供不上微操作，后端再强也会饿。性能分析要问的是哪一部分资源限制了当前循环。

2.23 寄存器重命名和假依赖

寄存器重命名让硬件把架构寄存器映射到更多物理寄存器，从而消除某些假依赖。比如两次写同一个架构寄存器，如果后一次不需要等待前一次的值，硬件可以给它们分配不同物理寄存器。这样写后写和写后读的某些约束被解除，更多指令可以并行执行。

但真正的数据依赖无法靠重命名消除。一个累加循环中，下一次加法需要上一次 sum 的结果，这是真依赖。硬件不能凭空知道最终和可以分成多个 partial 再合并，除非编译器或程序写出多个累

加器。多累加器优化的意义就在这里：把一条长依赖链拆成几条短依赖链，给乱序执行和 SIMD 留空间。

Listing 2.9 多个累加器拆开真依赖链

```
1 std::int64_t sum_four_lanes(std::span<const std::int32_t> values) {
2     std::int64_t a = 0;
3     std::int64_t b = 0;
4     std::int64_t c = 0;
5     std::int64_t d = 0;
6     std::size_t i = 0;
7     for (; i + 3 < values.size(); i += 4) {
8         a += values[i];
9         b += values[i + 1];
10        c += values[i + 2];
11        d += values[i + 3];
12    }
13    for (; i < values.size(); ++i) {
14        a += values[i];
15    }
16    return a + b + c + d;
17 }
```

这段代码不是保证一定比编译器自动优化更快，而是帮助读者看见依赖形状。实验时要检查编译器是否已经自动展开；若自动展开，手写版本可能没有收益。教材强调机制，不鼓励无证据地手写微优化。

2.24 前端：取指、译码和微操作缓存

前端负责持续给后端提供可执行微操作。若热代码太大、分支太乱、间接跳转太多、函数无法内联或模板膨胀严重，前端可能成为瓶颈。此时后端执行端口并未打满，但核心仍然无法退休更多有用指令。解释器、大型状态机、复杂虚调用层和异常路径都可能遇到前端压力。

微操作缓存能缓存已经解码的微操作，减少重复译码成本。但它容量有限，代码布局和热路径大小会影响命中。把冷错误处理、日志格式化和调试路径混在热循环附近，可能增加指令 cache 和微操作缓存压力。冷热分离不仅适用于数据，也适用于代码路径。业务代码中常见的早期返回、错误分支和虚调度，也会影响前端行为。

优化前端瓶颈时，常见方法包括内联真正热的小函数、把冷路径拆出去、按类型分桶减少随机分支、使用表驱动但避免巨大不可预测间接跳转、减少模板生成的大量重复代码。每种方法都有可维护性成本，必须有证据支持。

2.25 后端：端口和执行单元

后端有多个执行端口，不同端口连接整数 ALU、乘法、浮点、加载、存储地址生成等执行单元。一个循环可能总指令数不多，却被某个端口限制。例如每轮需要两次 load 和一次 store，加载端口可能比整数加法更早成为瓶颈。另一个循环乘法很多，乘法吞吐可能限制整体速度。

端口分析通常需要反汇编、架构手册或工具辅助。教材阶段不要求读者手算每条指令端口，但要形成意识：减少指令数不一定减少瓶颈端口压力；增加一条看似便宜的地址计算，可能挤占关键端口；把复杂寻址改成简单指针递增，有时能减轻地址生成压力。SIMD 章节会继续讨论执行端口和向量吞吐。

2.26 Load/store queue 和内存依赖

内存操作也需要调度。Load queue 记录未完成加载，store queue 记录尚未提交或尚未写回的存储。处理器要判断一个 load 是否可能依赖前面的 store。如果地址不确定，硬件可能保守等待；如

果预测错，还要回滚。这就是为什么别名信息会影响性能：编译器和硬件都需要知道读写是否可能指向同一位置。

C++ 中的指针、引用和 span 都可能表达别名。一个函数如果接收两个可写 span，编译器很难证明它们不重叠。即使硬件能做一些内存依赖预测，语言语义仍限制编译器重排。高性能 API 应尽量表达只读输入和独占输出，必要时用调试检查保证不重叠。不要把别名问题完全推给编译器。

2.27 分支预测器的学习对象

分支预测器不是读懂业务逻辑，它根据历史模式、地址和上下文猜测分支方向和目标。循环分支通常容易预测，数据相关随机分支难预测，间接分支目标多时更困难。分支预测器也会被不同分支之间的历史干扰，输入分布变化时可能短暂失效。

一个判断是否需要 branchless 的方法是改变输入分布。若有序输入和随机输入差距巨大，branch miss 也随之变化，分支预测很可能参与瓶颈。若两者差距不大，优化分支可能不是重点。另一个方法是按类型分桶：先把同类记录聚在一起，再批量处理，让分支模式稳定。这个方法用一次数据重排换更稳定控制流，是否划算要看记录数和每条记录处理成本。

2.28 Speculation 的成本

预测执行让核心在分支结果未知时先沿着猜测路径工作。猜对时节省等待，猜错时丢弃错误路径的结果并从正确路径重新开始。错误路径虽然不提交架构状态，但它消耗了取指、译码、执行端口和缓存资源。短循环里频繁错误预测，会让有效工作比例下降。

Speculation 还影响安全边界。现代 CPU 的投机执行曾引发多类侧信道问题。第二册不是安全专著，但要提醒读者：硬件为了性能做的预测和缓存，会让“没有提交的路径”仍可能留下微架构痕迹。高性能系统和安全系统并不是完全独立的领域。

2.29 SMT 的收益和风险

Simultaneous Multithreading 让一个物理核心同时运行多个硬件线程，共享前端、执行端口、缓存和队列资源。它能在一个线程等待内存或分支时，让另一个线程利用空闲资源；但如果两个线程都争用同一资源，例如都在做带宽密集扫描，SMT 可能降低单线程性能，甚至降低总效率。

性能实验要区分物理核心数和逻辑 CPU 数。若 8 物理核心 16 逻辑 CPU 的机器上，线程数从 8 增加到 16 没有提升或变慢，不一定是程序错误，可能是 SMT 资源争用。若任务有大量内存等待，SMT 可能有收益；若任务已经打满执行端口或内存带宽，SMT 可能无益。报告里写“使用 16 线程”时，必须说明这是逻辑线程还是物理核心。

2.30 频率、功耗和温度

CPU 频率不是固定常数。Turbo boost、功耗限制、温度、核心数和负载类型都会影响实际频率。单线程运行时，核心可能提升到较高频率；全核高负载时，频率可能下降；长时间 AVX 或宽向量负载也可能触发频率调整。若 benchmark 没记录频率和运行时间，就可能把频率变化误判为算法变化。

这对手机和笔记本尤其明显。短测试可能跑在高频，长测试因发热降频；后台充电状态、电源模式和温度都会影响结果。严肃实验应预热、重复运行、记录样本分布，并在报告中说明设备限制。不能把一次短跑数字当作稳定吞吐。

2.31 代码布局 and I-cache

数据 cache 经常被讲得很多，instruction cache 却容易被忽略。大型模板代码、过度内联、巨型 switch 和大量错误处理混在热路径里，都会增加 I-cache 压力。若前端 miss 增加，后端会等待指令供应。对解释器、正则引擎、数据库执行器和网络协议解析器，I-cache 可能很重要。

优化代码布局时，可以把冷路径标记或拆分出去，把常见 case 放在紧凑路径，把罕见错误处理放到单独函数。也可以通过 profile-guided optimization 让编译器根据真实运行频率布局代码。教学项目不必一开始使用 PGO，但要让读者知道，代码也是被缓存的对象。

2.32 反汇编阅读方法

阅读汇编时，不要试图逐条记住所有指令。先看循环主体：每轮有几次 load、几次 store、几次算术、几次分支，是否有函数调用，是否使用 SIMD，是否展开，是否有原子指令或锁前缀。再看循环边界：尾部如何处理，是否有额外检查，是否因为别名生成运行时版本分派。最后结合 perf 指标判断瓶颈。

编译器输出不同不代表其中一个一定更好。更短的汇编可能因为少做了必要检查而不正确，也可能因为使用了更强假设。读汇编的目的不是炫技，而是验证源码结构是否被编译成预期形状。若你希望循环向量化，却看到标量 load 和分支，就要回到源码检查别名、对齐、控制流和编译选项。

2.33 案例：哈希查找为什么难快

哈希表查找常被认为平均 $O(1)$ ，但在核心流水线里，它可能很难快。一次查找要计算 hash，访问 bucket，比较 key，可能追踪链表或探测多个槽位，遇到分支和随机内存。若 key 是字符串，还要读取字符串字节。每一步都可能产生 cache miss、分支 miss 或依赖链。平均复杂度没有表达这些硬件成本。

开放寻址哈希表通常比链式哈希更容易利用 cache，因为探测槽位连续；但高负载因子会增加探测长度。链式哈希删除简单，但每个节点可能分散在堆上。若业务 key 可以 intern 成整数，查找会大幅简化。这个案例连接第 2 章的流水线、第 3 章的 cache、第 6 章的数据布局和第 8 章的聚合内核。

2.34 案例：虚函数和间接分支

面向对象设计常用虚函数表达多态。热路径里每个元素调用虚函数，硬件看到的是间接分支和不可内联调用。若对象类型分布随机，分支目标预测困难；若每种类型代码很大，I-cache 压力增加。优化方向可能是按类型分桶，或者把热操作改成数据驱动的 switch 或 function table，并用 profile 证明收益。

这不是说虚函数不好。冷路径、控制面和插件接口中，虚函数的清晰性很有价值。问题是热循环里每元素虚调用可能成为前端和分支瓶颈。系统工程要区分热路径和冷路径，不要把所有代码都按同一个风格写。

2.35 案例：异常路径和热路径

C++ 异常在未抛出时通常不直接执行大量代码，但异常边、栈展开信息和不可见控制流可能影响优化和代码布局。更重要的是，热路径中把错误处理逻辑写得很大，会增加 I-cache 和分支复杂度。高性能代码常把快速路径写得紧凑，把错误处理拆到冷函数。

例如解析日志时，正常行占 99.9%，错误行很少。热循环应快速识别字段并写入 hot batch；错误详情、格式化消息和上下文收集应进入冷路径。这样既保持错误可诊断，也不让冷逻辑污染热指令流。代码冷热分离和数据冷热分离是同一思想。

2.36 微架构实验清单

本章建议增加四个小实验。第一，单累加器与多累加器，观察依赖链。第二，可预测分支与随机分支，观察 branch-misses。第三，虚函数调用与按类型分桶，观察前端和分支影响。第四，链表遍历与数组遍历，观察内存级并行度和 TLB。每个实验都保留相同语义和 correctness check。

实验报告不要只写时间。要写输入分布、循环形状、预期瓶颈、可用计数器和结果解释。微架构知识如果不落到实验，很容易变成背术语。

2.37 案例：解释器循环

解释器或字节码执行器是前端和分支预测的典型压力场景。一个大 switch 根据 opcode 分派，每个 case 执行不同逻辑。若 opcode 分布随机，分支目标预测困难；若每个 case 很小，分派成本占比高；若每个 case 很大，I-cache 压力上升。优化方向包括 direct threading、按 opcode 分组、热点 opcode 专门化、JIT 或把多个简单 opcode 融合成 superinstruction。

这个案例对第二册也有启发。任务运行时、数据处理管线和未来 AI 推理引擎都可能有“解释器

式”控制流：根据算子类型、消息类型或记录类型分派。若热路径每条记录都做复杂分派，前端成本会很高。把控制流批量化，让一批同类数据走同一条路径，是通用优化。

2.38 案例：函数指针表和预测

函数指针表可以替代大 switch，但它不自动更快。间接调用目标如果很多且随机，预测仍然困难；函数无法内联，还会增加调用开销。若目标集合很小且稳定，函数指针表可能足够；若目标频繁变化，按类型分桶或生成专门化代码可能更好。性能报告应比较 switch、函数指针和分桶三种形态，而不是凭风格选择。

这个案例也提醒接口设计。过早把热路径抽象成大量虚接口，会让编译器难以内联；完全去抽象又会让代码难维护。常见折中是控制面使用抽象，数据面热内核使用模板、函数对象或枚举派发，并用 benchmark 验证。

2.39 微架构章节总结

现代核心的性能来自前端供给、乱序窗口、寄存器重命名、执行端口、load/store 队列、分支预测和缓存层级共同作用。源码层面的一个 for 循环，硬件看到的是依赖图、指令流和内存请求。优化时先问依赖是否可拆、控制流是否可预测、数据是否连续、前端是否供得上、后端哪个资源受限。这个问题清单比记住某个 CPU 名词更有价值。

2.40 循环优化决策树

优化一个热循环，可以按决策树走。第一，结果是否正确且有 reference；没有 reference，先补 reference。第二，循环是否真的热；如果 profile 不在这里，先别优化。第三，循环主要是计算、访存、分支还是调用；用源码、输入分布和计数器判断。第四，若是依赖链，考虑多累加器、分块或改变算法；若是访存，考虑布局、连续访问、预取友好和减少字节；若是分支，考虑数据分桶、掩码和输入分布；若是调用，考虑内联、冷热拆分或批量接口。第五，优化后重新测，因为瓶颈可能迁移。

决策树的价值是防止跳步。很多人看到循环就先展开，看到分支就写 branchless，看到数组就写 SIMD。真正的工程优化应先定位受限资源。比如一个循环已经受内存带宽限制，再展开可能只增加代码大小；一个分支高度可预测，branchless 可能更慢；一个函数调用不在热路径，内联只会让代码膨胀。决策树让优化从证据开始，而不是从偏好开始。

2.41 微架构和可维护性

微架构优化常会牺牲可读性，因此必须设边界。标量 reference 必须保留，复杂优化必须有注释说明语义和适用条件，benchmark 要覆盖小输入和大输入，回退路径要可用。若某个优化只在一台机器上提升一点，却让代码难以维护，可能不值得。性能工程不是把每段代码都写到底层，而是把最热、最稳定、最有证据的路径优化到合适程度。

2.42 本章落到贯穿项目

Compute Systems Engine 在本章至少要得到三个能力。第一，顺序 reference: `sequential_sum` 保持最直接写法，用它验证所有优化版本结果。第二，单核优化版本：增加多累加器或向量化实验，但每个版本都要保留 correctness check。第三，测量 harness：能以相同输入、相同编译选项、相同计时边界比较 baseline、多累加器、热原子和线程本地 partial。这里的目标不是立刻写最强归约，而是建立“同一问题跨层演化”的实验骨架。

实验报告建议按下面结构写。先说明输入规模、数据类型和正确性 oracle；再说明 CPU 型号、编译器、优化选项、线程数和是否绑定；然后列出版本：baseline、四累加器、热原子、多线程 partial；最后解释每个版本的预期瓶颈和实际指标。若四累加器没有变快，不能直接说技巧无效，要检查编译器是否已经自动展开、输入是否被内存带宽限制、计时是否包含线程创建、以及结果是否被优化掉。

2.43 本章习题

习题与作业

习题一：为 `sum_baseline`、`sum_four_accumulators` 和当前项目里的 `parallel_sum` 写同一组正确性测试。输入必须包含空数组、单元数组、长度不是四的倍数的数组、包含负数的数组和足够大的随机数组。报告每个测试验证的不是“跑得快”，而是哪一种语义边界。

习题二：构造两个 `count_greater` 输入。第一个输入使分支非常可预测，第二个输入使分支接近随机。用 `perfstat` 记录 `branches` 和 `branch-misses`。若当前设备权限不足，写出你本来要运行的命令、预期现象和无法运行的原因。

习题三：把一个链表求和改写成连续数组求和。要求说明两版在源码复杂度、内存布局、cache line、TLB、预取和对象生命周期上的差异。不要只写“数组更快”，要写清为什么更可能快，以及在哪些业务语义下不能直接替换。

习题四：阅读当前项目的 `books/cpu-volume-2/labs/compute_systems/src/parallel_reduce.cpp`。指出 `parallel_sum` 的 correctness oracle、任务切分方式、线程创建成本、`partial` 写入位置和可能的 false sharing 风险。给出一个改进计划，但不要直接删除 `baseline`。

实验：写两个求和版本。第一个使用单累加器，第二个使用四个独立累加器。用 `perfstat` 比较 `cycles`、`instructions` 和 `IPC`。再构造一个带随机分支的版本，观察 `branch-misses` 对时间的影响。

第 3 章

缓存、TLB 与页级性能

CPU 核心很快，主内存相对很慢。缓存层级的存在，是为了让核心大多数时候从更近、更小、更快的存储中取数据。虚拟内存的存在，是为了让进程拥有独立地址空间、权限保护和按需映射。第一册已经讲过进程地址空间、映射和缺页的基本概念；本章不重复这些基础，而是继续追问：地址翻译、缓存映射、写共享、page walk、首次触页和 NUMA 放置怎样限制多核程序的吞吐。

3.1 缓存层级与 cache line

缓存不是按单个字节移动数据，而是按 cache line 移动。常见 cache line 大小是 64 字节。顺序访问数组时，一次 miss 可以带来后续多个元素；随机访问链表时，每个节点可能引入一次新的 miss。这个差异解释了为什么连续数组常常比指针结构更快。

缓存有容量、相联度和替换策略。容量不足会产生容量 miss，相联度不足会产生冲突 miss，多核心共享写会产生一致性流量。一个算法理论复杂度更低，不代表真实更快；如果它把连续访问变成随机访问，可能输给复杂度略高但局部性更好的算法。

从第二册的角度看，cache line 还是多核协作的最小一致性单元。两个线程哪怕写的是两个不同字段，只要字段落在同一条 cache line 上，硬件就必须在核心之间移动同一个一致性单元。很多并发程序的扩展性问题，不是算法复杂度问题，而是写入布局问题。后面讲锁、原子和无锁队列时，都要回到这个事实。

```
one cache line, 64 bytes in a common x86 machine

| counter[0] | counter[1] | counter[2] | counter[3] | ...
   T0         T1         T2         T3

looks independent in source code
shares one coherence unit in hardware
```

这类图要让读者形成一个直觉：源码里的变量边界，不等于硬件里的传输边界。硬件搬的是 cache line，TLB 记的是页，操作系统调度的是线程，分布式系统发送的是消息。边界错位，是系统性能问题最常见的来源之一。

3.2 缓存映射和冲突

缓存通常按 set associative 组织。一个地址会映射到某个 set，同一个 set 里只有有限个 way。如果多个热点地址恰好映射到同一个 set，而数量超过相联度，就会频繁互相驱逐，即使总工作集小于缓存容量也会慢。这类冲突 miss 很隐蔽，因为从容量角度看“不应该放不下”。

矩阵步长、数组对齐和结构体大小都可能触发冲突。若多个数组按相同大步长访问，并且基地址关系不佳，它们可能持续争用同一组 cache set。改变 padding、改变块大小或调整访问顺序，有时能显著改善性能。

3.3 Inclusive、Exclusive 和共享缓存

不同处理器的缓存层级策略不同。有的层级倾向 inclusive，也就是上层 cache 的内容也在下层保留；有的倾向 exclusive，尽量让不同层级存放不同内容；还有很多混合策略。软件通常不需要依赖某个具体策略，但要知道共享 LLC 不是无限资源。多个线程在同一 socket 上跑不同任务，可能互相驱逐 LLC。

这对并行任务调度有直接影响。把共享数据的任务放在共享缓存附近，可能复用 LLC；把互相干扰的大流式任务放在同一 socket，可能争抢带宽和 LLC。操作系统调度器会做一般性决策，但高

性能程序常需要自己记录拓扑并做亲和性实验。

3.4 写路径和 false sharing

读缓存可以共享，写缓存必须取得 cache line 的独占所有权。两个线程写不同变量，如果这些变量落在同一条 cache line 上，硬件仍然会把它们当成同一块一致性单元反复迁移，这就是 false sharing。它不是锁竞争，却会表现出类似的扩展性下降。

解决 false sharing 的方法包括按线程分配独立槽位、使用对齐和填充、批量合并结果、减少共享写频率。并行归约通常先让每个线程写自己的 partial sum，最后再合并，正是为了避免每次加法都写同一个共享变量。

Listing 3.1 用对齐槽位减少 false sharing

```
1 struct alignas(64) LocalCounter {
2     std::int64_t value = 0;
3 };
4
5 std::vector<LocalCounter> counters(worker_count);
6 for (std::int32_t worker = 0; worker < worker_count; ++worker) {
7     counters[static_cast<std::size_t>(worker)].value += 1;
8 }
```

这段代码不是说所有结构体都要强行 64 字节对齐，而是表达一个设计原则：如果多个线程会高频写不同槽位，这些槽位不要共享同一条 cache line。否则程序表面没有共享变量，硬件层面仍然在共享一致性单元。

false sharing 的危险在于它不像数据竞争那样直接破坏结果。程序可能完全正确，测试也全部通过，只是在核心数增加时吞吐突然停止增长。排查时要看写入频率、字段布局、线程到槽位的映射和 cache line 迁移事件。若没有 `perfctr` 或类似工具，也可以用对齐填充做对照实验：只改变布局，不改变算法。如果对齐版本显著更快，就说明共享写路径值得重点检查。

3.5 读共享和写共享

共享数据不总是坏事。只读共享表、常量配置、不可变索引和代码段可以被多个核心同时缓存，通常扩展性很好。真正昂贵的是写共享，尤其是多个核心高频写同一条 cache line。读多写少的数据可以使用读写锁、RCU、版本化快照或 copy-on-write，目的都是让读路径尽量不进入写共享热路径。

写共享也分层次。最差情况是每次操作都写同一个全局变量，例如所有线程对一个原子计数器做 `fetch_add`。较好的情况是每个线程写本地槽位，周期性合并。更好的情况是让写入天然按数据分片分散，例如按 key 分片的哈希表或按 NUMA 节点分组的内存池。布局、同步和算法在这里是同一个问题的三个面。

3.6 TLB 和页表

虚拟地址访问物理内存前需要地址翻译。TLB 缓存虚拟页到物理页的映射。工作集页数太多或访问模式太随机时，TLB miss 会增加，核心需要走页表。大页可以减少页数，提高 TLB 覆盖范围，但也会影响内存分配、NUMA 放置和碎片。

理解 TLB 对数据结构设计很重要。一个跨很多小对象的图遍历，不只会造成 cache miss，还可能造成 TLB miss。紧凑存储、池化分配和按块遍历，可以同时改善 cache 和 TLB 行为。

TLB 的特殊之处在于，它限制的是“能高效覆盖多少虚拟页”。一个程序的工作集如果只看字节数不大，但分散在大量页面上，仍然可能受到 TLB miss 影响。链表、树、哈希表桶、对象池和图邻接结构都可能出现这个问题。把对象压缩到连续数组里，不只是减少 cache miss，也是在减少活跃页数。

3.7 Page walk 的成本

TLB miss 后，硬件需要按页表层级查找映射。x86-64 常见四级或五级页表，每一级页表项本身也在内存里，虽然页表项可能被缓存，但最坏情况下一个地址翻译会触发多次内存访问。若程序随机访问大量页面，真正拖慢它的不只是数据 load，还有地址翻译。

大页把页大小从 4 KiB 扩大到 2 MiB 或更大，显著增加 TLB 覆盖范围。它适合大数组、数据库缓存和数值计算工作集，但不适合所有场景。大页可能增加内存碎片，NUMA 放置也要更谨慎。使用大页前应先使用计数器确认 TLB 确实是瓶颈。

Page walk 还会和缓存层级交织。页表项本身也要从内存层级读取，也可能命中或错过缓存。一个随机访问大数组的程序，表面上每次只读一个元素，实际上可能同时触发数据 cache miss 和页表访问。若工作集跨越大量页面，TLB miss 的成本会叠加在数据访问成本上。优化时不能只看“每个元素读几个字节”，还要看“每个时间窗口内活跃多少页”。

3.8 缺页和内存分配

分配内存不等于物理页已经准备好。操作系统通常按需分配页面，第一次触碰页面时可能发生 page fault。benchmark 如果把第一次触碰计入核心循环，会把页错误成本混入算法时间。严谨实验通常要预热、固定输入规模，并记录 page-faults。

缺页也分 minor fault 和 major fault。minor fault 不需要从磁盘读数据，可能只是建立页表映射；major fault 需要从存储设备读取，成本高得多。高性能计算通常要避免在热路径出现 major fault，也要避免把大量 minor fault 误判为算法本身慢。

第一册已经要求 benchmark 避免把初始化混进热路径。第二册要进一步要求初始化方式和计算方式一致。NUMA 机器上，如果主线程单独触碰全部页面，再让多个 socket 上的 worker 计算，页面可能集中在主线程所在节点，后续计算变成远端内存访问。正确实验通常让每个 worker 初始化自己之后要处理的分块，这样 page fault、物理页分配和计算拓扑保持一致。

3.9 从虚拟内存到 mmap

mmap 可以把文件或匿名内存映射到进程地址空间。它让文件 I/O 看起来像内存访问，但不意味着没有 I/O 成本。第一次访问未驻留页面时仍然会缺页，顺序扫描大文件时内核预读可能有效，随机访问时可能产生大量 page fault。

数据库、搜索引擎和日志系统常利用 mmap 或 page cache，但必须理解内核何时回收页面、何时写回脏页、何时产生 I/O 抖动。系统工程里，虚拟内存既是便利抽象，也是性能变量。

3.10 Linux 源码阅读入口

第一册的 Linux 源码阅读只要求读者能从用户态现象找到入口。第二册要读出数据结构和性能路径。围绕本章，可以固定一个 Linux 版本，沿下面几条路径做窄范围阅读：页错误入口可从体系结构相关的 fault handler 进入，再走到通用内存管理逻辑；VMA 管理可关注 mm/mmap.c 和相关 maple tree 结构；页表和 fault 处理可关注 mm/memory.c；NUMA 策略可关注 mm/mempolicy.c；page cache 和文件映射可从 mm/filemap.c 开始。

源码阅读报告不要写成“Linux 使用某某机制”这么宽泛。合格报告要写清内核版本、阅读路径、关键结构、用户态实验和结论边界。例如研究首次触页，可以写一个大数组实验，记录 page-faults，再阅读 fault 路径中如何建立匿名页映射。若当前设备权限不足，仍然可以保留源码阅读，但必须承认它不是当前运行内核的直接证据。

3.11 本章的设计原则

缓存和 TLB 优化可以压缩成四条原则。第一，让热路径连续访问，减少随机指针跳转。第二，让共享写分散，避免多个线程争用同一条 cache line。第三，让工作集按块复用，避免超出缓存后仍反复访问。第四，让页面放置和线程执行位置一致，避免首次触页造成远端访问。这四条原则会贯穿后面的数据布局、并行归约、线程池、工作窃取和分布式 shuffle。

常见误区

不要把 `std::vector` 的连续性理解成所有成本都消失了。连续布局改善 cache line 和预取，但如果容量超过缓存，仍然会受内存带宽限制；如果首次访问大量新页，仍然会看到缺页成本。

3.12 从变量边界到硬件边界

本章最重要的模型，是把软件边界和硬件边界分开。C++ 源码里，`counter[0]` 和 `counter[1]` 是两个不同元素；硬件里，如果它们落在同一条 cache line 上，写入时就是同一个一致性单元。C++ 里，两个对象可能由两个 `new` 表达式产生；分配器里，它们可能挨在同一个页上，也可能散落到不同页。进程里，虚拟地址看起来连续；物理页可能分布在不同 NUMA 节点。性能问题经常不是因为某一层“错了”，而是因为层与层的边界没有对齐。

因此分析内存性能时，不要只画类图和对对象关系，还要画四种边界。第一是对象边界，说明字段和容器怎样组织。第二是 cache line 边界，说明哪些字段会一起被搬运和一致性维护。第三是页边界，说明工作集跨越多少页，TLB 是否能覆盖。第四是 NUMA 边界，说明页面在哪个节点，线程在哪个节点执行。只有对象图没有硬件边界，分析会停留在源码层；只有硬件术语没有对象图，优化又会脱离业务语义。

Compute Systems Engine 的 `partial` 数组就是一个很好的例子。源码里每个 worker 写一个 `partial`，逻辑上互不共享；如果每个 `partial` 只是一个 `std::int64_t`，八个 `partial` 可能正好落在同一条 64 字节 cache line 上。八个 worker 写不同元素，却让同一条 cache line 在核心之间迁移。若给每个 `partial` 做 `alignas(64)`，源码上的数组变胖了，内存占用增加了，但写共享路径被拆开。这里没有绝对答案，只有工作负载下的权衡：若 worker 很少或写入很少，padding 可能不值得；若每个 worker 高频写 `partial`，对齐常常值得。

3.13 局部性不是一个词

局部性至少有三种。时间局部性表示刚访问过的数据很可能很快再次访问；空间局部性表示访问一个地址后，附近地址很可能被访问；结构局部性表示程序的数据结构让相关数据天然挨在一起。很多教材只讲时间和空间局部性，但工程中结构局部性更容易被设计改变。

数组顺序扫描空间局部性很好，时间局部性可能不强，因为每个元素只用一次。矩阵分块同时制造时间局部性和空间局部性，因为一个块中的数据会被多次复用。哈希表查询如果 key 随机，空间局部性差；若桶数组连续但节点链表分散，第一跳和后续跳的局部性还不同。图算法更复杂，邻接表可能连续，但顶点状态数组访问可能随机。说“这个算法局部性不好”太粗，应说明是哪一类局部性不好，坏在数据结构的哪个位置。

局部性还要和读写性质一起看。只读随机访问通常只是慢，写随机共享访问可能同时慢且影响其他核心。只读大表可以被多个核心缓存；写热点计数器会触发一致性迁移；写不同字段如果落在同一 cache line 上，会形成 false sharing。后续讲无锁队列时，head 和 tail 的布局就是典型局部性问题：把它们放近可以减少对象体积，但若生产者写 tail、消费者写 head，放太近反而让两端互相干扰。

3.14 案例：三种计数器的内存路径

为了把缓存一致性讲清楚，可以比较三个计数器版本。第一个版本用 `mutex` 保护全局计数器，第二个版本用 `atomic` 全局计数器，第三个版本用分片计数器。三者都能给出正确总数，但内存路径完全不同。

Listing 3.2 baseline: mutex counter 表达最清楚的不变量

```
1 class MutexCounter {
2 public:
3     void add(std::int64_t delta) {
4         std::lock_guard<std::mutex> lock(this->mutex_);
5         this->value_ += delta;
```



```

6     }
7
8     std::int64_t value() const {
9         std::lock_guard<std::mutex> lock(this->mutex_);
10        return this->value_;
11    }
12
13 private:
14     mutable std::mutex mutex_;
15     std::int64_t value_ = 0;
16 };

```

mutex 版本的语义最直接：锁保护 `value_`。低竞争时，它可能只走用户态 fast path；竞争上升后，线程可能进入 futex 等待，调度器和唤醒成本会进入系统。它的瓶颈通常是临界区串行化和锁等待，不只是那一次整数加法。

Listing 3.3 改进但不一定扩展：atomic counter

```

1 class AtomicCounter {
2 public:
3     void add(std::int64_t delta) {
4         this->value_.fetch_add(delta, std::memory_order_relaxed);
5     }
6
7     std::int64_t value() const {
8         return this->value_.load(std::memory_order_relaxed);
9     }
10
11 private:
12     std::atomic<std::int64_t> value_{0};
13 };

```

atomic 版本消除了锁等待和内核睡眠路径，但所有线程仍写同一 cache line。低竞争时它可能比 mutex 快；高竞争时它可能因为 cache line 所有权迁移而扩展性很差。这里的 relaxed 只说明不需要用计数器同步其他数据，不说明这次写没有硬件成本。

Listing 3.4 扩展方向：按 worker 分片计数

```

1 struct alignas(64) CounterShard {
2     std::int64_t value = 0;
3 };
4
5 class ShardedCounter {
6 public:
7     explicit ShardedCounter(std::int32_t shard_count)
8         : shards_(static_cast<std::size_t>(shard_count)) {}
9
10    void add(std::int32_t shard, std::int64_t delta) {
11        this->shards_[static_cast<std::size_t>(shard)].value += delta;
12    }
13
14    std::int64_t value() const {
15        std::int64_t total = 0;
16        for (const CounterShard& shard : this->shards_) {

```

```

17         total += shard.value;
18     }
19     return total;
20 }
21
22 private:
23     std::vector<CounterShard> shards_;
24 };

```

分片版本把高频写入分散到多个 cache line。它牺牲了读取总数的成本和实时性：每次读取总数要扫描所有分片；如果读取和写入并发，还需要额外同步或定义弱一致语义。它适合写多读少的统计路径，不适合每次操作都需要立即得到全局精确值的协议。这个案例说明高性能设计常常不是“换一个更快原语”，而是改变读写比例和共享形状。

3.15 false sharing 的对照实验

false sharing 很适合用对照实验学习，因为它不改变算法结果，只改变布局。实验可以构造两个版本。版本一使用紧凑数组，每个线程反复写自己的槽位；版本二使用 `alignas(64)` 的槽位。两个版本的循环次数、线程数和写入次数完全相同，只改变槽位是否共享 cache line。如果对齐版本在多线程下显著更快，而单线程差异不大，就说明写共享边界是关键变量。

实验报告要避免两个误区。第一，不要只跑一个线程数。false sharing 的危害随线程数和核心拓扑变化，至少应从 1 线程跑到物理核心数。第二，不要只看耗时。若工具允许，应观察 cache line 迁移或相关事件；若没有权限，也应写出对照设计为什么能隔离变量。第三，不要把 padding 当成默认答案。对齐会增加内存占用，可能降低缓存容量利用率。只有高频并发写的槽位才优先考虑 padding。

3.16 TLB 覆盖范围和数据结构

很多人把 TLB miss 当成“虚拟内存细节”，但它会直接改变数据结构选择。假设页面大小是 4 KiB，一个数据结构每个节点只有几十字节，但节点随机分布在几百万个页面上。每次访问节点不仅可能 cache miss，还可能 TLB miss。TLB 覆盖范围是有限的，活跃页数超过覆盖范围后，地址翻译本身就会变成瓶颈。

紧凑数组同时改善两件事：一条 cache line 带来多个相邻元素，一个 TLB 项覆盖同一页内多个元素。对象池也有类似作用：即使保留指针结构，至少让节点来自较少的连续页面。图算法中的 CSR 格式把邻接边压成连续数组，不只是为了少分配，也是为了提高 cache 和 TLB 友好性。哈希表开放寻址常常比链地址法更 cache 友好，也部分因为它减少指针跳转和页面分散。

大页是另一种提高 TLB 覆盖范围的方法。把 4 KiB 页换成 2 MiB 页，同样数量的 TLB 项能覆盖更大地址范围。它适合大数组和长时间驻留的大工作集，但要考虑碎片、权限、部署和 NUMA 放置。不要一看到 TLB miss 就无脑开大页。正确顺序是先证明活跃页数和 TLB miss 是瓶颈，再评估大页对分配、首次触页和系统配置的影响。

3.17 首次触页和 NUMA 的隐藏变量

NUMA 机器上，页面通常在首次写入时分配到触碰它的线程所在节点。若主线程分配并初始化整个数组，然后把计算分给其他 socket 上的线程，其他线程可能大量访问远端内存。程序源码里看不到这个问题，因为数组地址连续、线程切分也正确；硬件拓扑里却出现远端访问。

这就是为什么 benchmark 的初始化也属于实验设计。若计算阶段按 worker 分块，那么初始化阶段也应按同样分块让 worker 触碰自己的页面。这样 page fault、物理页分配和计算线程位置更一致。若为了方便只让主线程初始化，测到的可能不是算法性能，而是错误页面放置后的远端带宽。

Compute Systems Engine 后续要把输入生成拆成两种模式。第一种是 single-touch，由主线程初始化，用于展示错误实验设计可能带来的远端访问。第二种是 parallel first-touch，由每个 worker 初始化自己负责的块，用于展示正确放置。两者结果如果在单 socket 机器上接近，在多 socket 机器上可能差异明显。教材不能假设每个读者都有 NUMA 设备，但要让读者知道该怎样设计实验。

3.18 矩阵访问：从行列顺序到分块

矩阵是讲缓存最经典的案例。行优先存储的二维数组按行扫描，连续访问；按列扫描，步长等于一行长度，若矩阵很大，每次访问都可能落到新的 cache line 甚至新的页。这个例子不能只停在“行优先快”上，还要继续推进到分块。

矩阵乘法中，朴素三重循环会反复访问矩阵块。如果循环顺序不合适，某些数据刚被加载就被驱逐，下次又要从更慢层级取回。分块的思想是让一小块数据在缓存中被多次复用，提高算术强度。块太小，循环开销和复用不足；块太大，超过缓存后又失效。块大小还与 cache 容量、相联度、TLB、SIMD 宽度和数据类型有关。

这个案例的意义不是要求第二册变成矩阵计算专著，而是让读者学会一类推理：先估算工作集大小，再估算每个块的数据复用，再根据缓存层级选择实验矩阵。后面讲分布式 shuffle 时，思想相同：先本地聚合减少网络数据，再按分区批量发送，避免每条记录都独立跨网络移动。cache block 和 network batch 在不同尺度上解决同一个问题：让昂贵移动服务更多有用计算。

3.19 分配器和对象布局

C++ 程序常把性能问题归咎于容器，却忽略分配器。频繁小对象分配会让对象分散、增加元数据开销、破坏 cache 和 TLB 局部性，还可能引入锁竞争。多线程分配器通常有线程本地缓存、arena 或 tcache 机制，用于减少全局锁，但也会影响对象在哪个线程、哪个 NUMA 节点上实际分配。

对象池不是万能答案。它可以改善局部性和分配成本，但也会引入生命周期管理、碎片复用、调试困难和峰值内存占用问题。如果对象跨线程释放，池的所有权和回收路径又会变复杂。无锁数据结构尤其要小心：节点从结构中摘除，不等于可以立即释放；分配器复用同一地址还可能放大 ABA 问题。缓存、TLB、分配器和内存回收最终会在无锁章节汇合。

3.20 写回、脏页和 I/O 误判

并不是所有内存成本都来自 DRAM。文件映射、page cache 和脏页写回会把内存访问和 I/O 混在一起。一个程序用 `mmap` 扫描文件，第一次访问可能触发缺页和磁盘读取；写入映射文件后，脏页可能稍后由内核写回，导致延迟在看似无关的时刻出现。若 benchmark 只测用户态循环，不记录 page faults 和 I/O，就可能把系统行为误判为算法波动。

日志系统和数据库特别容易遇到这个问题。用户写入内存缓冲并不等于数据已经安全落盘；调用 `fsync` 又会把持久化成本集中暴露出来。第二册虽然重点是计算系统，但分布式计算章节会讨论检查点和输出提交，那里必须区分“写入内存”“写入 page cache”“持久化到存储”和“对外宣布提交”。单机虚拟内存知识会直接影响分布式正确性。

3.21 Cache line 是共享和移动单位

CPU cache 不是按单个字节搬运，而是按 cache line 搬运。常见 cache line 大小是 64 字节，具体平台要查询。程序读取一个 `std::int32_t`，硬件通常会把它所在的整条 cache line 带入缓存。顺序扫描数组时，这很好，因为一条 cache line 包含多个相邻元素；随机访问时，可能每条 cache line 只用一个元素，大部分带宽被浪费。

Cache line 也是一致性协议的基本单位。两个线程写同一条 cache line 上的不同变量，也会互相争夺所有权，这就是 false sharing。软件层面看它们没有共享变量，硬件层面它们共享移动单位。理解这一点后，`alignas(64)`、结构体 padding、线程本地 partial 和 per-worker stats 的意义就很清楚：它们不是为了美观，而是为了让高频写不落在同一条 line 上。

但 padding 也有成本。一个 8 字节计数器对齐到 64 字节，内存占用变成 8 倍。若有百万个计数器，padding 会浪费大量 cache 容量和内存。只有高频跨线程写的槽位值得优先隔离。只读共享或低频写共享不一定需要 padding。布局优化始终是成本交换。

3.22 缓存相联度和冲突 miss

Cache 容量不是唯一限制。多数 cache 是组相联结构，一个地址会映射到某个 set，set 中只有有限 way。如果多个热地址映射到同一个 set，超过相联度后会互相驱逐，即使总工作集小于 cache 容量，也可能 miss 很多。这叫 conflict miss。矩阵步长、哈希表大小和数组对齐都可能触发冲突。

这也是为什么实验要改变输入大小和步长。若某个规模突然变慢，可能不是算法复杂度变化，而是 cache set 冲突、TLB 覆盖范围或页边界效应。通过改变数组 padding、起始偏移、步长和容量，可以验证是否存在冲突。不要把所有拐点都简单归因于 L1、L2、L3 容量。

3.23 写分配和写回

写入内存也会触发 cache 行为。普通写 miss 常采用 write-allocate：先把目标 cache line 读入缓存，再修改，之后再写回内存。对将来不会再读取的大块输出，这可能浪费带宽，因为读入旧内容没有价值。某些架构提供 non-temporal store，用于流式写出，减少缓存污染和写分配。但它也有边界：如果后续马上读取这些数据，绕过缓存可能变慢。

并行算法写输出时要考虑写分配。Stream compaction 的输出可能是连续写，后续 reduce 也许马上读；此时普通写可能有益。文件 I/O 或网络发送缓冲如果只是写完交给设备，缓存污染可能更明显。教材不要求读者手写 non-temporal 指令，但要知道写也会产生内存流量。

3.24 硬件预取器能看懂什么

硬件预取器擅长顺序访问和某些固定步长访问。它不理解复杂指针结构，也不理解哈希表随机访问。若程序顺序扫描 `std::vector`，预取器能提前请求后续 cache line；若程序遍历链表，下一地址必须等当前节点加载出来，预取器很难帮忙。若程序按随机索引访问数组，即使数组本身连续，访问序列仍然不可预测。

这说明“连续存储”和“连续访问”是两件事。一个 vector 按随机索引访问，仍可能 miss 很多；一个链表如果由对象池按顺序分配，也许比完全随机链表好，但仍受下一指针依赖限制。优化访问模式比手写 prefetch 更可靠。软件预取只有在地址能提前算出、预取距离合适、不会污染缓存时才可能有效。

3.25 TLB 的多级结构

地址翻译也有缓存。TLB 保存虚拟页到物理页的映射，常有 L1 dTLB、iTLB 和更大的二级 TLB。TLB miss 时，硬件或内核需要遍历页表，成本可能很高。一个程序如果访问很多页面且每页只用很少数据，TLB miss 会成为瓶颈。链表、巨大哈希表和随机图遍历都容易遇到这个问题。

TLB 覆盖范围等于 TLB 项数乘以页面大小。使用大页可以扩大覆盖范围，但会带来分配、碎片、权限和 NUMA 放置问题。透明大页可能自动启用，也可能因为内存碎片或访问模式没有生效。性能报告中若讨论 TLB，应记录页面大小、是否启用透明大页、工作集跨越多少页，以及 dTLB 事件是否可用。

3.26 Page fault 的几类含义

Page fault 不一定表示错误。Minor fault 可能表示虚拟页第一次映射到物理页、按需分配零页、建立页表项；Major fault 通常涉及磁盘 I/O，例如文件页不在 page cache 中。还有写时复制 fault：fork 后父子共享页面，某一方写入时内核复制页面。不同 fault 的成本差别很大。

Benchmark 如果没有预热和初始化，第一次访问大数组会包含大量 minor fault；第一次 mmap 文件会包含 page cache miss 和 major fault；fork 后写内存可能包含 COW 成本。若目标是热循环性能，就应把这些成本分离；若目标是冷启动或批处理端到端，就应保留并单独报告。不要让 page fault 悄悄混进“算法时间”。

3.27 虚拟内存和安全边界

虚拟内存不仅是性能机制，也是隔离机制。每个进程看到自己的虚拟地址空间，页表控制哪些页可读、可写、可执行。缺页异常让内核有机会按需分配、加载文件、处理 COW 或拒绝非法访问。理解这些机制有助于解释为什么越界访问可能崩溃，为什么 mmap 可以把文件映射为内存，为什么进程间共享内存需要明确映射。

高性能系统有时会使用共享内存绕过网络或减少拷贝，但共享内存会重新引入并发同步和生命周期问题。两个进程共享一块内存，不代表它们自动有一致的协议。仍然需要 ring buffer、sequence、futex、eventfd 或其他同步机制。虚拟内存提供地址映射，不提供业务语义。

3.28 Page cache 和持久化

Linux 的 page cache 会缓存文件数据。读取文件时，数据可能来自内存中的 page cache，而不是磁盘；写文件时，写入通常先进入 page cache，稍后由内核写回。调用 write 成功，不等于数据已经安全落盘。调用 fsync 或 fdatasync 才把持久化成本显式拉到当前路径，但具体语义还受文件系统和设备影响。

这对检查点非常重要。一个分布式作业写 checkpoint 文件后，如果没有正确 fsync，进程崩溃或机器掉电后文件可能不存在或内容不完整。写 manifest 时还要考虑目录项持久化，rename 后是否 fsync 目录。教学项目可以简化持久化，但文字上必须说明“写文件”和“可恢复提交”不是同一件事。

3.29 mmap 的优点和陷阱

mmap 可以把文件映射到地址空间，让程序像访问内存一样访问文件。它适合随机读取、由内核按需分页、避免显式 read 缓冲管理。缺点是 page fault 出现在访问指令处，延迟可能分散；错误处理也不同，访问损坏或截断文件可能触发信号；预取和释放需要 madvise 等机制辅助；写映射文件还涉及脏页和持久化。

顺序扫描大文件时，普通 read 加显式缓冲有时更容易控制 I/O 节奏和错误处理；mmap 更简洁，但性能不一定总更好。系统教材应避免把 mmap 神化。选择 read 还是 mmap，要看访问模式、错误处理、文件大小、内存压力和持久化要求。

3.30 内存分配和页粒度

用户态分配器向内核申请大块内存，再切给小对象。小对象释放后，内存可能回到分配器缓存，不一定立即还给内核。进程 RSS、虚拟内存大小和业务对象大小之间可能差很多。并发分配器还可能为每个线程保留缓存，线程退出或跨线程释放会影响内存回收和局部性。

这会影响性能实验。一个测试反复分配释放小对象，第二轮可能比第一轮快，因为分配器缓存已经热；也可能 RSS 不下降，让人误以为泄漏。对象池和 arena 可以改善可预测性，但要报告峰值内存和释放策略。内存优化不是只看 malloc 次数，还要看页、TLB、cache 和生命周期。

3.31 缓存一致性和虚拟内存的连接

Cache coherence 处理多个核心对物理内存中同一 cache line 的一致视图，虚拟内存处理虚拟地址到物理页的映射。两个不同虚拟地址可能映射到同一物理页，例如共享内存或文件映射；两个进程通过不同地址写同一物理页，仍会触发 cache line 一致性。理解这个连接，有助于分析 mmap 共享文件、进程间队列和 page cache。

分布式系统中，跨机器没有硬件 cache coherence，必须用协议维护一致性。单机共享内存的 cache line 迁移，在集群中变成 RPC、复制和共识。第二册先讲虚拟内存和 cache coherence，再讲分布式复制，就是为了让读者看到“一致视图”的成本如何随尺度上升。

3.32 内存实验的设计

本章实验应覆盖三类访问。第一，顺序扫描，观察带宽和预取。第二，固定步长访问，观察 cache line 利用率、set 冲突和 TLB 覆盖。第三，随机访问或指针追踪，观察预取失效和内存级并行度下降。每类实验都要保持逻辑工作量一致，例如访问相同数量的元素，返回相同类型的校验结果。

实验还要控制数据初始化。先分配再初始化，记录初始化是否计时；如果比较 NUMA，初始化线程和计算线程要明确；如果比较 page fault，冷启动和热启动要分开；如果比较 mmap，page cache 状态要说明。内存实验最容易被隐藏状态影响，因此报告必须详细。

3.33 案例：对象数组和指针数组

一个常见布局选择是保存对象数组，还是保存指针数组。对象数组 `std::vector<Record>` 中对象连续，遍历时 cache 友好；指针数组 `std::vector<Record*>` 本身连续，但每个指针指向的对象可能分散。若热路径需要访问每个对象字段，指针数组会多一次间接访问，并可能导致随机 cache miss 和 TLB miss。

指针数组也有价值。对象很大且需要稳定地址，或者多个索引共享同一对象，或者对象由不同生命周期管理时，指针更灵活。高性能系统常用“稳定 ID 加对象池”折中：外部持有 ID，内部对象池尽量连续。这个案例提醒读者：局部性和稳定引用经常冲突，需要设计层面的取舍。

3.34 案例：内存映射日志文件

使用 mmap 扫描日志文件很方便。程序可以把文件映射成字节 span，然后查找换行和分隔符。但第一次访问页面可能触发 page fault，文件被截断时访问可能出错，映射区域的生命周期必须覆盖解析过程。若解析阶段把 `raw_offsets` 指向 mmap 区域，后续错误报告必须保证映射仍然存在。

普通 read 版本则显式管理缓冲。它可能多一次复制，但错误处理和背压更直接：读取一个 batch，解析完释放或复用缓冲。mmap 和 read 的选择取决于访问模式、错误处理和生命周期。项目可以同时保留两种输入后端作为实验，而不是假设 mmap 永远更快。

3.35 案例：Page fault 进入计时区

假设 benchmark 分配一个 1 GiB vector，然后直接开始计时求和，但初始化没有触碰所有页面。求和第一次读到页面时可能触发缺页，计时结果包含页表建立和物理页分配。另一个版本提前初始化数组，求和时没有这些 fault。两者时间差不一定来自求和算法。这个案例非常常见。

正确实验应明确分配、初始化、预热和计时。若要测端到端，就报告 page faults；若要测热循环，就在计时前触碰页面。没有这个区分，内存实验经常得出错误结论。

3.36 内存安全和性能

越界访问、use-after-free 和数据竞争不仅是正确性问题，也会破坏性能分析。未定义行为可能让编译器做出激进优化，导致 benchmark 结果没有意义。释放后访问可能偶尔命中旧内存，看似正常，压力下崩溃。性能代码更需要 sanitizer 和边界测试，因为它常使用手写布局、对象池和指针。

项目应在 debug/asan/tsan 构建中跑 correctness tests，在 release 构建中跑性能。不要用 sanitizer 构建得出性能结论，也不要再用 release-only 测试证明内存安全。正确性工具和性能工具各有位置。

3.37 案例：Page cache 和 checkpoint

Checkpoint 写入常被误测。程序把 checkpoint 写到文件后立即返回，测试显示很快；但真正崩溃恢复时，文件可能还在 page cache，没有持久化。若下一章分布式项目把这个时间当作提交点，就会得到错误可靠性。正确实验要区分 write 时间、fsync 时间和 rename 提交时间。

可以设计一个小实验：写固定大小 checkpoint，分别只 write、write 后 fsync、临时文件 fsync 后 rename 并 fsync 目录。比较时间，并说明每种语义。这个实验会让读者理解为什么可靠输出比普通文件写入复杂。性能和持久性是交换关系，不能只看最快路径。

3.38 案例：共享内存队列

两个进程通过 mmap 共享内存实现队列时，cache、TLB、虚拟内存和同步会同时出现。共享内存提供同一物理页映射，但 head/tail 的发布仍需要原子和内存序；进程退出时，队列状态可能停在中间；不同进程地址不同，不能在共享结构中保存普通指针，应保存 offset 或索引。

这个案例非常适合作为本章到无锁章节的桥梁。虚拟内存解决映射，cache coherence 解决同一机器上的可见性，C++ 原子解决语言级同步，状态机解决关闭和恢复。任何一层缺失，队列都不完整。

3.39 缓存章节总结

缓存、TLB 和虚拟内存共同决定“访问一个地址”的真实成本。连续访问利用 cache line 和预取，随机访问浪费带宽并增加 TLB 压力；page fault 和 page cache 会把内存访问与 I/O 混在一起；mmap 提供方便但不消除状态机；写入不等于持久化。理解内存层级后，数据布局、并行算法和分布式 checkpoint 才有坚实基础。

3.40 内存问题排查路径

遇到疑似内存瓶颈，可以按路径排查。第一，看工作集大小是否超过缓存层级，输入规模曲线是否有拐点。第二，看访问模式，是顺序、步长、随机还是指针追踪。第三，看每个元素实际使用多少字节，cache line 里有多少无用数据。第四，看页面数量和 TLB 覆盖范围，是否存在大量随机跨页访问。第五，看是否有 page fault、mmap 冷启动或 page cache 干扰。第六，看写入是否产生写分配、脏页和 fsync 成本。

这个路径可以应用到很多问题。哈希聚合慢，可能是随机访问和分配；图遍历慢，可能是邻接数据分散和 TLB；日志扫描慢，可能是 page cache 冷启动；checkpoint 慢，可能是 fsync 和写回。把“内存慢”拆成具体层次，优化方向才清楚。

3.41 内存章节项目要求

项目中每个大数据结构都应写明三件事：内存布局、生命周期、访问模式。ParsedLogHotBatch 是列式热字段，生命周期为一个 batch，访问模式为顺序扫描；ShuffleBlock 是序列化后的中间数据，生命周期到 manifest 清理，访问模式为写一次读一次；TaskGraphStorage 是连续节点和边，生命周期为一个 stage，访问模式为随机 ready 更新和顺序 successor 遍历。这样写文档能让后续优化有依据。

3.42 本章落到贯穿项目

Compute Systems Engine 在本章至少要增加四类实验。第一，partial 对齐实验：比较紧凑 partial 和 `alignas(64)` partial，观察多线程写入扩展曲线。第二，访问模式实验：比较顺序数组、固定步长、随机索引和链表遍历。第三，页级实验：改变工作集跨越页数，记录 page faults 和可能的 dTLB 事件。第四，first-touch 实验：在支持 NUMA 的机器上比较主线程初始化和 worker 分块初始化。

这些实验都必须有 reference。访问模式版本必须返回同样的逻辑结果；partial 对齐版本必须只改变布局，不改变算法；first-touch 版本必须分离初始化时间和计算时间，同时也可以单独报告初始化成本。若实验一次改变多个变量，结论就不可靠。

3.43 本章习题

习题与作业

习题一：在当前项目中为 partial 数组设计两个布局，一个紧凑，一个按 cache line 对齐。说明每个字段的大小、对齐、每个 cache line 可能容纳几个 partial，以及为什么这个设计能隔离 false sharing。

习题二：写三个遍历版本：连续数组遍历、随机索引数组遍历、链表遍历。要求三者处理相同数量的逻辑元素。报告中分别分析 cache line、TLB、预取器和内存级并行度，不允许只写“随机慢”。

习题三：设计一个首次触页实验。说明如何分离分配、初始化、预热和计算；如何记录 page-faults；在没有 NUMA 机器时，哪些结论不能验证，哪些结论仍然可以通过 page fault 观察。

习题四：阅读 `mm/memory.c`、`mm/mmap.c` 或对应内核版本中的 fault 路径入口。写一份不超过两页的源码阅读报告，要求把用户态实验现象和内核路径联系起来，不要罗列无关函数。

实验：比较行优先访问矩阵和列优先访问矩阵。记录 cache-misses、dTLB 相关事件和时间。再把矩阵规模从 L1、L2、L3 可容纳范围逐步扩大，观察拐点。

第 4 章

NUMA、互连与缓存一致性

多核心 CPU 不是把多个核心简单放在一起。核心之间通过片上互连通信，共享某些缓存层级，连接内存控制器，并用缓存一致性协议维护共享内存语义。多 socket 机器还会引入 NUMA：不同核心访问不同内存节点的成本不同。

4.1 共享内存的硬件代价

在 C++ 程序里，多个线程似乎可以直接读写同一地址。但硬件上，写一个共享 cache line 需要让其他核心的副本失效或更新。一个原子自增不是普通加法，它需要获得 cache line 独占权，并按内存序约束发布结果。争用越激烈，cache line 在核心之间来回迁移越频繁。

这解释了为什么全局计数器扩展性差。四个线程分别做本地计数，最后合并，通常比四个线程同时 `fetch_add` 同一个原子变量快。前者减少共享写，后者让每次更新都进入一致性协议热路径。

4.2 MESI 的直觉

很多一致性协议都可以用 MESI 直觉理解：Modified 表示本核心有被修改且尚未写回的独占副本，Exclusive 表示本核心有未修改的独占副本，Shared 表示多个核心可以读，Invalid 表示副本无效。当一个核心要写 Shared 行时，需要让其他核心副本失效；当其他核心随后要读，又要重新获取数据。

软件看不到这些状态名，但能看到它们的后果。一个只读共享表可以被多个核心同时缓存，扩展性很好；一个频繁更新的共享 head 指针会让 cache line 在核心之间跳来跳去；一个包含多个线程本地字段的结构体如果没有 padding，也会因为 false sharing 触发同样的迁移。

真实处理器可能使用 MESIF、MOESI 或其他变体，但教学时先抓住一个核心事实：读共享容易扩展，写共享需要取得所有权。Modified 或 Owned 这类状态的具体名称不重要，重要的是写入会触发状态迁移，状态迁移会占用互连和缓存层级资源。高性能并发设计的第一反应，不应该是“换一条更快的原子指令”，而应该是“能不能减少共享写”。

```
read mostly table:
  core0 Shared
  core1 Shared
  core2 Shared

hot atomic counter:
  core0 Modified -> core1 Modified -> core2 Modified -> core0 Modified
```

上面的文本图表达两种完全不同的共享。前者多个核心一起读，硬件可以复制；后者多个核心轮流写，所有权不断迁移。很多扩展性问题就藏在这条差异里。

4.3 一致性和内存模型的分工

缓存一致性解决的是“同一内存位置的多个副本如何保持一致”，内存模型解决的是“不同内存操作之间能被哪些线程以什么顺序观察”。硬件一致性不等于程序自动无数据竞争。没有同步的普通读写仍然可能在 C++ 层面形成未定义行为。

一致性协议通常围绕状态机工作，例如 Modified、Exclusive、Shared、Invalid 这类状态。软件不需要手写这些状态，但必须理解状态迁移的成本：共享读便宜，共享写贵，频繁写同一行最贵。

C++ 内存模型和硬件一致性的边界也要分清。硬件一致性通常保证同一 cache line 或同一地址的副本最终按协议协调，但 C++ 程序如果对普通变量并发读写而没有同步，语言层面就是数据竞争。语言规则先决定程序是否有定义，硬件规则再决定有定义的同步怎样落地。不要用“我的 CPU

有缓存一致性”来为未同步普通变量辩护。

4.4 Store buffer 和可见性

核心执行 store 后，写入可能先进入 store buffer。对本线程来说，后续 load 可能通过 store forwarding 看到自己的写；对其他核心来说，这个写何时可见还受缓存一致性和内存序约束影响。锁释放、release store 和内存屏障会把这种可见性变成可依赖的同步关系。

这就是为什么“我在写 flag 前已经写了 data”在并发中不够。编译器和硬件都可能在允许范围内重排或延迟可见性。正确发布对象需要用 release/acquire、mutex 或其他同步机制建立 happens-before。

Store buffer 还会影响锁的性能。释放锁时，持锁线程在临界区内的写入必须以满足同步语义的方式对后续拿锁线程可见。获取锁时，线程可能需要等待锁所在 cache line 的最新状态。低竞争时这条路径很短；高竞争时，锁变量所在 cache line 会成为所有核心争夺的热点。锁的成本不是一个固定常数，而是竞争、拓扑和写入路径共同决定的结果。

4.5 NUMA 放置

NUMA 机器上，内存离某些核心更近。线程在节点 A 上运行，却频繁访问节点 B 分配的内存，会产生远端访问。Linux 的 first-touch 策略让首次触碰页面的线程影响页面放置，因此并行初始化和线程绑定会影响后续计算性能。

NUMA 优化的基本原则是让计算靠近数据。大数据组按块分给线程时，初始化也应按相同块并行完成。跨节点共享队列和全局分配器要谨慎，因为它们可能把远端访问和同步争用叠加在一起。

NUMA 不只出现在多 socket 服务器上。有些桌面和移动平台也有非均匀的缓存、内存或芯片 let 拓扑，只是操作系统暴露方式不同。教材里使用 NUMA 这个词，是为了让读者形成拓扑意识：核心不是一个无差别集合，内存也不是一个无差别池。线程、数据和 I/O 设备之间的位置关系会影响成本。

4.6 First-touch 和并行初始化

Linux 常见 first-touch 策略意味着页面通常分配到第一次写入它的线程所在 NUMA 节点。如果主线程单线程初始化一个大数组，然后工作线程分布在多个节点上处理这个数组，很多工作线程可能访问远端内存。更好的方式是让每个工作线程初始化自己后续要处理的分块。

这条规则在 benchmark 中尤其重要。若初始化方式和计算方式不一致，测到的可能是内存放置错误，而不是算法本身。实验报告必须记录初始化策略、线程绑定和 NUMA 策略。

4.7 互连不是无限带宽

核心、缓存、内存控制器和 I/O 设备之间的互连带宽有限。一个程序即使单线程局部性很好，多线程扩展后也可能撞上内存带宽或互连带宽上限。此时继续增加线程，只会让每个线程分到更少带宽，吞吐不再提高。

互连瓶颈常常表现为“每个线程单独跑都快，一起跑就都慢”。原因可能是多个核心争用同一内存控制器，多个 socket 之间远端访问过多，或者共享写导致 cache line 在互连上来回移动。排查时要把线程数、绑定位置、数据放置和访问模式作为实验矩阵，而不是只改算法代码。

4.8 拓扑感知设计

拓扑感知不是过早优化，而是避免把硬件当作平面资源池。线程池可以按 NUMA 节点分组，内存池可以按节点分配，队列可以按生产者或消费者分片，归约可以先在 socket 内合并再跨 socket 合并。这样的层次化设计和分布式系统中的本地聚合很像：先减少近处流量，再把少量结果发到远处。

4.9 Linux 观察入口

在 Linux 上，拓扑可以从 `lscpu`、`/sys/devices/system/cpu/`、`/sys/devices/system/node/` 和 `numactl--hardware` 开始观察。线程绑定可以用 `taskset`、`numactl--physcpubind` 或程序内亲和性 API。内存策略可以用 `numactl--membind`、`numactl--interleave` 和 `first-`

touch 实验观察。若权限允许，`perf` 可以帮助识别 cache-to-cache 流量，`perfstat` 的 cache、LLC、context-switch 和迁核指标也能提供间接证据。

源码阅读可以从调度拓扑、NUMA 策略和内存分配路径切入。不要试图一次看完整调度器或完整内存管理。更好的问题是：线程迁移时哪些状态会变化，first-touch 影响页面放置的路径在哪里，NUMA 策略怎样被记录和查询，某个 benchmark 的现象能不能从这些路径得到合理解释。

4.10 层次化归约案例

并行归约是理解拓扑的好例子。最差设计是所有线程对同一个原子变量累加。改进设计是每个线程写自己的 partial，最后一个线程合并。进一步的拓扑感知设计是每个核心或每个 worker 先本地累加，每个 NUMA 节点内再合并，最后跨节点合并。这样做减少共享写，也减少远端通信。

这个结构和分布式计算中的树形聚合完全一致。不同只是成本单位变化：单机里远处是另一个 NUMA 节点，分布式里远处是另一台机器。理解这条相似性，可以帮助读者把第二册前后内容串起来。

4.11 一致性流量的三个等级

共享内存程序里的数据共享可以分成三个等级。第一个等级是只读共享，例如配置表、只读索引、常量权重、不可变输入数组。多个核心可以同时缓存这些数据，扩展性通常很好。第二个等级是低频写共享，例如阶段结束时写一次完成标志、每秒汇总一次指标、偶尔替换一份配置快照。它需要同步，但不一定成为瓶颈。第三个等级是高频写共享，例如所有请求更新同一个原子计数器、所有 worker 争用同一个队列 tail、多个线程反复写同一 cache line 上的不同字段。这个等级最危险，往往表现为核心数越多越慢。

优化时要先判断共享等级，而不是先决定使用 mutex、atomic 还是无锁结构。mutex 保护低频复杂状态可能完全合理；atomic 保护高频热点计数器仍然可能很慢；无锁队列如果所有生产者都争用同一个 tail，也逃不开一致性流量。把共享写变成分片写、批量写、线程本地写或按 NUMA 节点写，通常比替换同步原语更根本。

这也是第二册区分“正确性同步”和“扩展性设计”的原因。正确性同步回答的是数据竞争、可见性和不变量；扩展性设计回答的是共享写频率、cache line 迁移和拓扑距离。一个程序可以同步正确但扩展性很差，也可以局部很快但同步语义错误。高质量系统必须同时满足二者。

4.12 目录协议和窥探协议的直觉

一致性协议在小规模系统中可以用广播窥探思想理解：一个核心想写某条 cache line，就向其他核心广播或通过互连通知，让其他副本失效。核心数量增加后，纯广播会变贵，因此很多系统使用目录式信息，记录某条 cache line 可能在哪些核心或节点有副本，再有针对性地发送消息。具体实现非常复杂，但软件层面的直觉很简单：共享范围越大，维护一致性的通信越难便宜。

这条直觉能解释为什么“全局共享结构”在小机器上看不出问题，在大机器上突然变差。四核心机器上，一个全局队列的 cache line 迁移还勉强能承受；多 socket 服务器上，迁移可能跨互连和 NUMA 节点，延迟和带宽成本都更高。工程设计不能只在开发机上看正确性测试通过，还要在目标拓扑上看扩展曲线。

目录和窥探的差别不需要读者背协议细节，但要形成层次化思维。让数据只在一个核心私有，成本最低；让数据在一个 socket 内共享，成本较低；让数据跨 socket 高频写共享，成本高；让数据跨机器共享，成本更高。线程本地 partial、socket 内合并、跨 socket 合并、跨机器 reduce，就是把把这个成本阶梯显式写进算法。

4.13 内存控制器和带宽饱和

NUMA 节点通常连接自己的内存控制器。一个节点上的线程大量访问本地内存，会消耗本地控制器带宽；远端访问则需要经过互连到另一个节点的内存控制器。若所有线程都访问一个节点上的内存，即使 CPU 核心分布在多个节点上，带宽也可能集中压在一个控制器上。此时线程数增加，吞吐未必提高，因为瓶颈不是核心数量，而是某个内存节点的带宽。

first-touch 实验的价值就在这里。主线程初始化全部页面，可能让所有页面落在一个 NUMA 节点；worker 分布到多个节点计算时，远端访问集中出现。按 worker 分块初始化，可以让页面分散到

各自节点，使多个内存控制器共同工作。若实验显示 parallel first-touch 版本带宽更高，就说明优化来自页面放置和控制器并行，而不是加法本身改变。

带宽饱和也有反直觉现象。使用更多线程可能让总吞吐略增，但每线程吞吐下降；使用 SMT 可能让内存延迟隐藏更好，也可能只增加争用；把线程平均分到两个节点可能提高总带宽，也可能因为跨节点合并阶段变慢。结论必须来自实验矩阵，不应写成固定经验。

4.14 拓扑感知队列

队列是互连成本的集中点。一个全局 MPMC 队列很容易成为共享写热点：所有生产者争用 tail，所有消费者争用 head 或槽位状态。低线程数下它简单好用，高线程数下它可能把所有 worker 绑在一组 cache line 上。拓扑感知设计通常会拆成多级队列：每个 worker 或每个 NUMA 节点有本地队列，只有负载不均时才访问其他队列。

线程池的工作窃取就是这个思想。worker 优先使用本地 deque，空闲时才偷别人的任务。NUMA 感知线程池还可以先在同节点 worker 中窃取，再跨节点窃取。这样做不是为了复杂而复杂，而是为了让高频路径局部化、低频路径承担远端成本。

有界队列的背压也要考虑拓扑。若所有生产者把任务推到单个队列，队列满时所有生产者都等待同一个条件变量，唤醒和竞争集中；若每个分片有自己的队列，热点分片仍会背压，但无关分片可以继续执行。分片设计会增加任务路由和负载均衡复杂度，因此要先测出全局队列确实是瓶颈，再拆分。

4.15 Linux NUMA 策略和迁移

Linux 提供多种 NUMA 相关策略：默认 first-touch、按节点绑定、交错分配、自动 NUMA balancing 等。自动机制可以在一般场景改善局部性，但它不是性能实验的替代品。自动迁移页面本身有成本，统计窗口和迁移决策也可能与 benchmark 生命周期不匹配。短时间 benchmark 尤其容易受到初始化策略影响。

用户态实验应记录三类信息。第一，线程位置：线程是否绑定，绑定到哪些 CPU，是否发生迁移。第二，页面位置：是否通过 first-touch、mbind 或 interleaved 控制，是否能观测各节点内存使用。第三，访问形状：线程是否主要访问本地分块，是否有跨节点共享写，合并阶段是否集中到一个节点。只记录 `numactl--hardware` 不够，必须把拓扑和程序行为联系起来。

源码阅读可以围绕 `mm/mempolicy.c`、NUMA balancing 相关路径和调度拓扑展开。不要试图一次读懂所有内存管理。合格阅读报告应选择一个问题，例如“为什么主线程初始化会影响页面节点”，然后追踪 VMA 策略、缺页处理和页面分配之间的关系。

4.16 C++ 接口如何表达拓扑

C++ 标准库不直接暴露 NUMA 拓扑，但工程代码可以在接口层表达位置意识。例如线程池可以接收 worker 拓扑描述；内存池可以按节点创建 arena；任务可以带有 preferred node；归约可以返回分层 partial；队列可以按 shard id 提交。接口不需要一开始绑定某个 Linux API，但要给后续拓扑策略留下位置。

下面是一个概念性的任务描述。

Listing 4.1 任务元数据中保留位置偏好

```
1 struct TaskPlacement {
2     std::int32_t preferred_worker = -1;
3     std::int32_t preferred_numa_node = -1;
4 };
5
6 struct ComputeTask {
7     std::size_t begin = 0;
8     std::size_t end = 0;
9     TaskPlacement placement;
10 };
```


这段代码不是要读者立刻实现完整 NUMA runtime，而是提醒接口设计不要把任务看成无位置的函数。对数组块任务，`begin/end` 暗含数据位置；对分布式 shuffle，partition id 暗含目标节点；对线程池，本地队列暗含 worker 位置。位置语义一旦在接口中完全丢失，后面很难再补回来。

4.17 拓扑不是核心数列表

很多程序只读取“有多少 CPU”，然后创建同样数量线程。这不足以描述现代机器。CPU 拓扑包含 socket、NUMA node、core、SMT sibling、cache 共享层级、内存控制器和 I/O 设备位置。两个逻辑 CPU 可能是同一个物理核心的 SMT sibling，也可能在同一个 socket 的不同核心，也可能跨 socket。它们之间共享资源和通信成本完全不同。

性能报告应至少记录 `lscpu` 中的核心数、线程数、socket 数、NUMA 节点和 cache 信息。在多 socket 机器上，还应记录每个 NUMA node 的 CPU 列表。线程池设计也要区分 worker id 和 CPU id。worker id 是运行时内部编号，CPU id 是操作系统调度对象，NUMA node 是内存位置。把它们混为一谈，会让绑定和 first-touch 实验失去意义。

4.18 互连成本如何出现

多核心之间通过片上网络、环、mesh 或 socket 间互连通信。软件不会直接调用“互连”，但互连成本会出现在 cache line 迁移、远端内存访问、跨 socket 锁竞争和共享 LLC 争用中。一个全局原子计数器在单 socket 小机器上看起来还能接受，在多 socket 上可能因为 cache line 在 socket 间来回迁移而急剧变慢。

互连成本也会出现在读多写少结构中。只读共享通常可扩展，因为多个核心可以缓存副本；写入会让其他副本失效。若一个配置对象每秒更新一次，问题不大；若一个统计字段每次请求都写，问题很大。软件层面要区分只读共享、低频写共享和高频写共享。这比盲目选择 atomic 或 mutex 更重要。

4.19 目录协议的工程影响

目录协议维护某条 cache line 的副本位置或所有权信息。软件工程师不需要实现目录协议，但要理解它的结果：共享范围越大，一致性维护越贵；写者越分散，所有权迁移越频繁；数据越能保持本地私有，扩展性越好。分片、线程本地、per-NUMA partial 和分层合并，都是把共享范围缩小。

这也解释为什么“每个线程写自己的数组下标”不一定安全高效。如果这些下标落在同一 cache line，硬件仍然认为它们共享同一个一致性单位。软件的逻辑独立必须映射成硬件的物理独立，才真正减少一致性流量。

4.20 First-touch 的细节

First-touch 通常发生在页面第一次被写入或实际分配物理页时。只调用 `reserve` 或 `malloc` 可能只获得虚拟地址，不一定分配物理页。第一次写入触发缺页，内核选择物理页所在 NUMA 节点。若主线程初始化全部数组，页面可能集中在主线程所在节点；后续其他节点线程访问就变成远端访问。

因此 NUMA 实验要把分配、初始化和计算分开。分配阶段只申请地址；初始化阶段按目标线程分块写入，建立页面位置；计算阶段使用同样分块读取。若初始化也计入总时间，要单独报告，因为 parallel first-touch 可能把初始化变快或变慢。若只关心计算阶段，就要在计时前完成初始化并预热。

```
NUMA experiment phases:
  allocate virtual memory
  initialize pages by owner worker
  optionally warm up
  compute with the same worker-to-range mapping
  merge partials by node then globally
```


4.21 线程绑定的收益和风险

线程绑定可以减少调度迁移，让 worker 更稳定地访问本地 cache 和本地 NUMA 内存。它也可能降低操作系统调度灵活性。如果绑定策略错误，把多个 heavy worker 绑到同一个核心或同一个 NUMA 节点，就会变慢。绑定还要考虑系统上其他进程、容器配额和可用 CPU 集合。

实验中，绑定的价值是减少变量。未绑定实验可能因为操作系统迁移而样本波动；绑定后更容易解释线程和数据位置。但生产系统是否绑定，要看服务类型和部署环境。批处理计算通常更适合显式绑定；通用服务可能需要给调度器更多空间。教材应要求读者记录绑定策略，而不是强制所有程序绑定。

4.22 分层归约

NUMA 友好的归约通常分层。每个 worker 计算本地 partial；同一 NUMA 节点内先合并成 node partial；最后跨节点合并。这样高频读写发生在本地，跨节点通信只在较小 partial 上发生。这个结构也对应分布式系统中的 local combine、node-level reduce 和 global reduce。

```
hierarchical reduce:
  worker partials: no sharing during hot loop
  node partials: merge within NUMA node
  global result: merge node partials
```

分层合并的代价是代码复杂度和额外阶段。若数据很小或机器只有单 NUMA 节点，收益不明显。实验要比较平坦合并和分层合并，并记录每阶段耗时。不要把 NUMA 策略写成不可关闭的复杂路径。

4.23 跨节点队列的代价

全局队列在 NUMA 机器上常成为跨节点热点。生产者和消费者分布在不同节点，队列锁、head/tail 和槽位状态会在节点间迁移。分片队列或每节点队列可以减少这种迁移。任务优先进入本节点队列；本节点空闲时先偷同节点任务，再跨节点偷。这个策略和分布式调度中的数据本地性一致。

有界队列的容量也可以按节点分配。若每个 NUMA 节点有自己的输入队列，上游可以根据数据位置提交任务；某个节点过载时，其他节点不必被同一个全局队列阻塞。缺点是负载均衡更复杂，任务路由需要知道位置。设计要看工作负载是否有明显数据本地性。

4.24 远端释放和内存池

NUMA 不只影响分配，也影响释放。一个对象在节点 A 分配，由节点 B 释放，可能导致分配器元数据和 cache line 在节点间移动。高性能分配器常使用线程本地或节点本地缓存，但跨线程释放仍然需要特殊路径。对象池设计应考虑 owner：对象最好由分配它的线程或节点回收，跨节点释放可以先放入 remote free 队列，再由 owner 批量回收。

无锁数据结构的回收协议也会遇到这个问题。Hazard pointer 或 epoch 延迟释放后，最终由哪个线程 delete 节点？如果所有删除都集中到一个线程，可能造成远端释放和内存热点。内存管理和拓扑不能完全分开。

4.25 I/O 设备和 NUMA

网卡、NVMe 和 GPU 等设备通常连接到某个 NUMA 节点附近。线程处理设备 I/O 时，如果运行在远端节点，可能增加 PCIe 和内存访问成本。高性能网络和存储系统常把中断、队列、缓冲区和处理线程放在同一 NUMA 节点。虽然本册第二册不深入设备驱动，但要让读者知道 I/O 也有拓扑。

分布式 shuffle 如果在真实服务器上运行，网络接收线程、解压线程和 reduce worker 的位置会影响吞吐。若网卡在节点 0，接收缓冲在节点 0，但 reduce worker 在节点 1 读取，远端内存流量会增加。拓扑意识不只属于计算内核，也属于 I/O 管线。

4.26 自动 NUMA balancing

Linux 自动 NUMA balancing 会采样页面访问，并可能迁移页面到更常访问它的节点。它能改善一些长期运行程序，但也会引入额外 page fault、迁移成本和实验不确定性。短 benchmark 可能还没等自动机制收敛就结束；长服务可能在负载变化时发生页面迁移。

实验报告应记录是否启用自动 NUMA balancing，至少要知道它可能影响结果。若要做可控实验，可以使用 numactl、线程绑定和明确 first-touch。若要评估生产行为，则应在接近真实配置下测量，并记录自动迁移指标。不要让内核自动策略成为看不见的变量。

4.27 容器和 cgroup 的影响

现代部署常在容器里运行。容器可能限制 CPU 集合、内存节点、CPU 配额和内存上限。程序看到的 `std::thread::hardware_concurrency` 未必等于实际可用核心数；cgroup 配额可能让线程周期性被 throttled；cpuset 可能只允许访问部分 CPU。性能报告若忽略容器限制，就可能误判扩展曲线。

在 Termux、手机或云环境中，也要承认设备限制。手机通常没有多 socket NUMA，但有大小核、温度和频率限制；云主机可能共享物理资源；容器可能禁止读取某些性能事件。教材项目应记录环境，而不是假设每个读者都有理想服务器。

4.28 拓扑感知不等于硬编码

接口应表达拓扑信息，但不应把某台机器的拓扑硬编码进算法。正确做法是运行时发现或配置拓扑，然后把任务、内存池和队列映射到抽象 node。算法只知道 preferred node 或 shard id，不直接写死 CPU 编号。这样同一代码可以在单 socket、双 socket、容器和手机上运行，只是策略不同。

Compute Systems Engine 可以从简单配置开始：`worker_count`、是否绑定、每个 worker 的 node id。没有 NUMA 信息时，所有 worker 属于 node 0。后续在支持的 Linux 机器上再读取真实拓扑。这样项目不会因为缺少硬件而无法运行，也不会放弃拓扑扩展。

4.29 NUMA 实验的结论边界

没有 NUMA 机器时，不能声称验证了远端内存优化；但仍可以验证 false sharing、线程绑定、任务粒度和内存带宽曲线。单 socket 上 parallel first-touch 和主线程初始化可能差不多，这是正常结果，不是实验失败。报告要写清：本环境只能验证某些机制，不能验证跨 socket 页面放置。

有 NUMA 机器时，也不能只跑一次。需要比较绑定与未绑定、主线程初始化与 worker 初始化、平坦合并与分层合并、单节点内存与 interleave 内存。只有多组对照一致，才能说 NUMA 放置参与了性能结果。

4.30 案例：双 socket 归约

在双 socket 机器上，大数据组归约可以展示 NUMA 的完整链条。版本一由主线程初始化数组，然后所有 worker 分块求和。版本二由每个 worker 初始化自己的块，再求和。版本三按 socket 分组：每个 socket 的 worker 求本地 partial，socket 内合并，再跨 socket 合并。若版本二和三明显更好，说明页面放置和分层合并参与了性能。

实验要记录线程绑定。若 worker 在计算阶段被迁移到其他 socket，first-touch 的页面位置和线程位置又不匹配。还要记录数组大小，太小可能全部被 cache 影响，太大才体现内存控制器。这个案例把第 3 章的 page fault、第 4 章的 NUMA、第 13 章的分层 reduce 连在一起。

4.31 案例：跨 socket 队列

另一个案例是全局任务队列。所有 worker 从同一个队列取任务，队列锁和 head/tail 位于某个内存节点。跨 socket worker 频繁访问，会让队列状态在 socket 间迁移。本地队列加工作窃取可以减少高频跨 socket 访问：worker 优先本地 pop，空闲时才偷取。

实验可以构造大量短任务，让队列成本显著；再构造少量长任务，让队列成本被计算掩盖。这样读者能看到：同一个队列设计在不同任务粒度下表现不同。NUMA 优化不能脱离任务粒度。

4.32 分层资源管理

NUMA 机器适合分层资源管理。每个节点有本地 worker、本地内存池、本地队列、本地 partial。跨节点只交换压缩后的 partial 或被窃取的任务。这个结构和分布式系统的节点本地聚合非常像。若在单机上已经学会分层，扩展到集群时会更自然。

分层也会增加复杂度。每个层级都需要指标：本地队列长度、跨节点 steal 次数、节点 partial 大小、远端访问比例。没有指标，分层策略可能只是让代码复杂。项目可以先用单层实现，再在实验分支中加入分层，对比收益。

4.33 大小核和 NUMA 的区别

手机常见大小核，服务器常见 NUMA。二者都意味着执行资源不均匀，但原因不同。大小核是核心性能和能耗不同，同一内存访问拓扑未必像多 socket 那样分节点；NUMA 是内存位置和访问延迟不同，核心本身可能同构。调度策略也不同：大小核更关注把重计算放大核、后台任务放小核；NUMA 更关注线程和内存同节点。

在 Termux 环境中，读者更可能遇到大小核而不是多 socket NUMA。实验报告应避免把大小核现象误称为 NUMA。若线程数增加后性能下降，可能是小核加入、降频、温度或调度迁移，不一定是远端内存。记录环境是第一步。

4.34 拓扑抽象接口

项目可以定义一个抽象拓扑接口，而不是直接依赖某个 Linux API。接口返回 `worker_count`、`node_count`、`worker_to_node`、是否支持绑定。单 socket 或手机环境返回一个 node；双 socket 服务器返回多个 node。算法根据 `node_count` 决定是否启用分层 partial。这样代码能在不同设备上运行。

Listing 4.2 拓扑描述的接口形态

```
1 struct WorkerTopology {
2     std::int32_t worker_count = 1;
3     std::int32_t node_count = 1;
4     std::vector<std::int32_t> worker_to_node;
5 };
```

这个结构不直接包含 CPU id，是为了让教学项目先表达策略，再逐步接入真实 affinity。若绑定不可用，仍然可以使用 `worker_to_node` 做逻辑分层实验。接口应允许能力缺失，而不是让项目只能在服务器上运行。

4.35 拓扑指标

拓感知优化需要指标验证。每个 node 的任务数、处理字节、partial 大小、队列长度、跨节点 steal 次数、远端调度次数都应记录。若启用分层后跨节点 steal 很多，说明负载不均或任务划分不好；若某个 node 长期空闲，说明调度没有利用资源；若 node partial 合并很慢，说明最终合并成为瓶颈。

没有这些指标，拓扑策略容易变成不可解释的复杂代码。项目应先输出简单文本摘要，后续再画图。

4.36 拓扑和故障域

拓扑不只影响性能，也影响故障域。单机里，一个 NUMA 节点过载可能拖慢本地 worker；集群里，一个 rack、zone 或机房故障会影响一组节点。分布式副本放置通常要跨故障域，避免一处故障丢失多数副本。单机拓扑意识可以自然扩展到集群故障域意识：不要把所有关键副本、任务或队列集中在同一个风险位置。

Compute Systems Engine 的本机模拟可以先用 node id 表示逻辑故障域。后续多进程或多机版本中，node id 可以映射到主机、rack 或 zone。调度器在追求本地性的同时，也要避免把所有副本

放同一域。性能和可靠性有时会冲突，系统设计要显式权衡。

4.37 拓扑章节总结

本章的核心不是让读者背 NUMA API，而是建立位置意识。线程有位置，内存有位置，队列有位置，I/O 设备有位置，副本也有位置。位置决定通信成本和故障相关性。没有位置意识的程序在小机器上可能正常，在大机器上会遇到扩展性和可靠性问题。

4.38 拓扑决策表

做拓扑相关设计时，先问机器是否真的有多多个内存节点，是否能设置亲和性，数据是否大到超过 cache，任务是否访问固定数据块，是否有跨节点共享写，是否需要跨故障域放置副本。若没有 NUMA 或数据很小，复杂拓扑策略可能无收益；若数据大且任务固定访问某块，first-touch 和绑定值得实验；若共享写频繁，先分片再谈绑定；若是分布式副本，优先考虑故障域而不是只考虑延迟。

4.39 拓扑策略的降级设计

拓扑感知代码必须能降级。很多读者会在手机、容器、云虚拟机或普通单 socket 机器上运行实验，程序不一定能读取完整 NUMA 信息，也不一定有权限设置 affinity。如果代码把真实 NUMA 当成必需条件，就会让教材项目难以运行。更好的策略是把拓扑能力分层：能读取真实 node 时使用真实 node；不能读取时退化成一个逻辑 node；能绑定线程时执行绑定；绑定失败时记录失败并继续运行；能控制内存策略时做 first-touch 实验；不能控制时只做普通分块。

降级不等于忽略拓扑。即使只有一个逻辑 node，代码仍然可以保留 worker 到 node 的映射、分层 partial 的接口和队列分片的抽象。这样程序在小机器上正确运行，在大机器上能启用更多策略。教材项目尤其适合这种设计：先让抽象存在，再逐步接入真实硬件能力。读者不会因为没双 socket 服务器就学不了拓扑意识。

拓扑策略还要写入输出。若报告只写“启用 NUMA 优化”，但实际上 affinity 设置失败，读者会得到错误结论。程序应输出能力检测结果、降级原因和最终策略。例如 `node_count` 是 1，`binding_enabled` 是 false，`binding_error` 是 permission denied，`memory_policy` 是 default。这样实验结果才诚实。高性能系统的第一步不是让所有优化都打开，而是知道哪些优化真的生效。

降级设计还有一个好处：它迫使接口表达意图，而不是表达某个平台的偶然细节。算法真正需要的是“这个任务最好靠近哪一组数据”和“这个 worker 属于哪个执行域”，不一定需要知道具体 CPU 编号。平台能力强时，执行域可以映射到真实 NUMA 节点；平台能力弱时，执行域只是逻辑分组。把意图和机制分开，代码才不会因为换设备而到处改。

这种抽象也能保护学习路径。读者先在普通设备上理解任务、数据和执行域的关系，再到服务器上验证真实远端访问成本。先学清楚问题，再打开平台能力，实验结论会稳得多。

4.40 本章落到贯穿项目

Compute Systems Engine 在本章应加入三个设计点。第一，拓扑记录：benchmark 输出可见 CPU 数、用户指定 worker 数、是否启用绑定、是否能读取 NUMA 信息。第二，分层 partial：即使当前只实现每 worker partial，也在设计文档中说明未来可以扩展成每 NUMA 节点 partial。第三，队列分片路线：线程池初版可以用全局队列，但要记录全局队列可能成为共享热点，后续工作窃取章节再拆成本地队列。

本章的验收不是一定在手机或单 socket 机器上测出 NUMA 差异，而是实验设计正确。没有 NUMA 设备时，报告应写明限制：可以验证 false sharing、线程绑定和内存带宽曲线，但不能验证远端内存放置。承认证据边界，比编造一个漂亮结论更专业。

4.41 本章习题

习题与作业

习题一：把一个全局原子计数器、每线程计数器、每 NUMA 节点计数器画成三张数据共享图。分别标出哪些 cache line 被谁写，哪些阶段需要合并，读取全局值的成本在哪里。

习题二：设计一个 NUMA first-touch 实验。要求写清初始化线程、计算线程、绑定策略、输入规模、测量指标和预期现象。若当前设备没有 NUMA，写出哪些部分只能作为计划保留。

习题三：为线程池设计一个两级队列方案：worker 本地队列加全局溢出队列。说明高频路径、低频路径、锁保护对象、可能的窃取顺序和关闭语义。

习题四：阅读 Linux NUMA 策略或调度拓扑的一个窄入口。报告必须从一个用户态实验问题出发，不允许只复制函数名。

实验：在支持 NUMA 的机器上，用 `numactl--hardware` 查看节点。写一个大数组初始化和求和程序，比较单线程初始化、多线程初始化、线程绑定和未绑定情况下的带宽。如果没有 NUMA 机器，也要用 `lscpu` 记录拓扑并解释限制。

第零部分

单机高性能计算：从测量到数据流

第 5 章

可信测量、Roofline 与性能计数器

性能工程必须先建立测量纪律。第一册已经讲过输入规模、环境、编译选项、计时方式、热路径隔离、正确性校验和基础 `perfstat`。本章不重复这些基本规则，而是讨论第二册真正需要的进阶测量：如何把多核程序分解成计算、内存、分支、TLB、同步、调度和 I/O 假设；如何用 Roofline 判断上限；如何用性能计数器和对照实验排除错误解释。

5.1 测量对象

测量前先定义对象。是测单次操作延迟，还是测批量吞吐；是包含内存分配，还是只测热循环；是冷缓存，还是热缓存；是单线程，还是多线程；是平均值，还是尾延迟；是关心总吞吐，还是关心每个 worker 是否均衡。不同对象需要不同方法，不能混用结论。

常见错误包括：把初始化时间算进内核，把编译器优化掉的循环当作结果，把首次缺页当作算法成本，把输出打印放进计时区，把不同 CPU 频率状态下的结果直接比较。

第二册还要特别避免一种错误：只测总时间，不看扩展曲线。多线程程序的核心问题不是“某一次运行快不快”，而是从 1 个线程到多个线程时，吞吐如何变化。一个程序在 2 个线程时加速，在 8 个线程时停滞，在 16 个线程时变慢，这条曲线本身就是证据。它提示瓶颈可能从单核执行转移到了内存带宽、锁竞争、调度、NUMA 或 I/O。

5.2 计时器和统计方法

计时器应使用单调时钟，避免系统时间调整影响。短任务要重复执行足够多次，并防止编译器消除结果。报告结果时不要只给一次运行时间，应至少给出多次运行的最小值、中位数和波动范围。最小值常代表干扰较少的情况，中位数更接近日常情况，尾部异常则提示系统噪声或资源竞争。

预热同样重要。第一次运行可能包含页错误、动态链接、分支预测冷启动、缓存冷启动和 CPU 频率爬升。若目标是冷启动延迟，就应该保留这些成本；若目标是热循环吞吐，就应该把它们排除。测量对象不同，预热策略不同。

多核 benchmark 还要记录线程绑定和运行顺序。若每轮测试先跑单线程再跑多线程，CPU 温度、频率和缓存状态可能系统性变化。更稳妥的做法是随机化版本顺序，保留原始样本，并记录每轮线程数、绑定策略和输入位置。若差异小于系统波动，就不要写强结论。

5.3 Roofline 模型

Roofline 用算术强度连接计算峰值和内存带宽。算术强度是操作数与数据移动字节数的比值。低算术强度程序通常受内存带宽限制，高算术强度程序才可能接近计算峰值。这个模型不是精确预测器，而是帮助判断优化方向。

数组求和每读取一个整数只做一次加法，算术强度低；矩阵乘如果分块得当，每个元素可以被多次复用，算术强度高。若一个内核已经接近内存带宽上限，继续手写更复杂的算术指令可能没有意义；若内核远低于计算上限且数据复用充分，就要检查 SIMD、指令级并行和执行端口。

Roofline 的价值在于先给出“最多可能多快”的粗上限。假设一个内核每处理一个元素读取 8 字节、写入 8 字节，只做少量整数运算，那么它的算术强度很低。若平台可持续内存带宽是 50 GiB/s，那么不管循环写得漂亮，只要每个元素必须从内存读写 16 字节，吞吐上限就被数据移动限制。反过来，若内核通过分块让同一批数据被重复使用，算术强度提高，才有资格讨论 FMA、SIMD 宽度和执行端口。

心智模型

Roofline 不是为了画漂亮图，而是为了阻止错误优化：内存带宽瓶颈不要只改算术指令，计算端口瓶颈不要只改数据结构，同步瓶颈不要只改循环展开。

5.4 从字节数推导瓶颈

Roofline 的关键是先估算字节数。以 `std::int32_t` 数组求和为例，每个元素读取 4 字节，做一次整数加法。如果数组远大于缓存，主要流量来自内存读取。若机器可持续内存带宽是 40 GiB/s，那么理想情况下每秒最多读取约 100 亿个整数。实际还要扣除循环开销、预取效率、TLB 和其他系统干扰。

这个估算能提前判断优化方向。如果实测已经接近带宽上限，就不该期待换一种加法写法获得数量级提升。若实测只有带宽上限的一小部分，就要检查访问是否连续、是否触发 TLB miss、是否有分支、是否被编译器向量化、是否被线程调度干扰。

字节数估算要区分“算法逻辑字节”和“实际流量”。逻辑上，一个原子计数器每次只更新 8 字节；硬件上，它可能让一整条 cache line 在核心之间来回迁移。逻辑上，一个对象只读取一个字段；硬件上，cache line 可能带入多个冷字段。逻辑上，`mmap` 文件像数组；系统上，第一次访问可能触发缺页和 page cache 读取。第二册的实验报告必须说明这些差异，否则字节数估算会过于乐观。

5.5 性能计数器

Linux 上常用 `perfstat` 查看 cycles、instructions、branches、branch-misses、cache-misses、page-faults、context-switches 等指标。更深入时可以用 `perfrecord` 和 `perfreport` 定位热点。不同 CPU 的事件名称不同，实验报告要记录命令和平台。

计数器要组合解释。IPC 低可能因为 cache miss，也可能因为分支失败或同步等待。cache-misses 高不一定说明程序差，流式扫描大数组本来就会不断 miss，但可能充分利用带宽。context-switches 多可能说明线程过多、阻塞频繁或系统有干扰。

进阶测量更重要的是“假设驱动”。如果怀疑前端瓶颈，就比较指令缓存、分支、uops 供给和代码布局；如果怀疑内存带宽，就比较工作集大小、线程数、带宽工具和 LLC miss；如果怀疑 TLB，就改变页大小或访问页数；如果怀疑锁竞争，就看吞吐曲线、等待时间、futex 路径和临界区长度；如果怀疑 NUMA，就改变 first-touch 和绑定策略。不要一次性收集一大堆事件再事后找故事。

5.6 Top-Down 思维

现代性能分析常把 CPU 周期粗分为前端受限、错误预测受限、后端受限和有效退休。不同平台工具名称不同，但思路相同：先判断核心为什么没有持续退休有用指令，再进入更细分类。前端受限时检查取指、译码、代码布局和间接跳转；错误预测受限时检查分支输入分布；后端受限时继续区分执行端口、内存延迟、内存带宽和同步等待；有效退休比例高时，说明核心大部分时间确实在做有用工作，优化要回到算法或工作量本身。

这套思维可以避免把 IPC 低简单等同于“CPU 不行”。同样是 IPC 低，链表随机访问、锁等待、page fault、系统调用阻塞和分支乱跳的解决方向完全不同。报告里应该写“当前证据更支持哪一类受限”，而不是只列一个 IPC 数字。

5.7 同步和调度的测量

多线程程序经常慢在不退休用户态指令的地方。线程可能睡在 futex 上，可能被调度器切走，可能自旋等待 cache line，可能阻塞在 I/O，也可能因为 cgroup 配额用完而被节流。只看 cycles 和 instructions，容易忽略这些等待。

测同步要记录至少三类事实。第一，吞吐随线程数怎样变化；第二，等待在哪里发生，是用户态自旋、内核 futex、条件变量、I/O 还是队列为空；第三，竞争对象是什么，是一把锁、一个原子变量、一条 cache line、一个队列还是一个分片。若能使用 `perfsched`、`perflock`、`perfc2c` 或 eBPF 工具，应记录工具版本和权限；若不能使用，也要用对照实验替代，例如拆分计数器、增加 padding、改变线程绑定和改变队列容量。

5.8 多核扩展曲线

一个高质量性能报告不只给“优化前后快了多少”，还要给扩展曲线。横轴是线程数或 worker 数，纵轴可以是吞吐、耗时、加速比、效率或尾延迟。理想线性加速很少出现，重要的是解释拐点。若 1 到 4 线程接近线性，8 线程开始变慢，可能是共享缓存、内存带宽、NUMA 或同步热点；若从

2 线程开始就没有提升，可能是任务粒度太小、串行部分太大或全局锁太重。

扩展曲线还要配合正确性校验。并行程序在高线程数下更快但偶尔错误，不是优化成功，而是暴露了竞争。每轮性能样本都应关联校验结果，校验失败的样本不能混入性能统计后继续分析速度。

5.9 实验记录模板

一份合格性能记录至少包含：机器型号、核心和 NUMA 拓扑、内存容量、编译器和编译选项、输入规模、数据类型、线程数、绑定策略、运行次数、计时范围、正确性校验、perf 命令和原始输出摘要。缺少这些信息，别人无法复现实验，你自己几周后也很难判断数字是否可靠。

性能结论要写成果因果假设，而不是口号。例如“并行版本在线程数超过 8 后不再加速，perf 显示 LLC miss 和内存带宽接近平台上限，因此瓶颈更像内存带宽，而不是线程创建成本”。这样的结论可以继续被验证或推翻。

本册建议把报告分为五段。第一段写被测问题和正确性 oracle。第二段写机器、系统、编译和绑定环境。第三段写实验矩阵，包括输入规模、线程数、版本、运行轮次和顺序。第四段写原始结果摘要，不只给平均值，也给中位数、最小值和异常。第五段写归因假设和下一步验证。若证据不足，要明确写“当前只能排除某些解释，不能确认最终瓶颈”。

5.10 从问题到实验矩阵

真正的性能实验不是随手跑一个命令，而是把问题拆成可证伪的假设。以 Compute Systems Engine 的归约为例，用户观察到“并行版本没有线性加速”。这句话本身不能直接指导修改，因为可能原因太多：线程创建成本太高，任务粒度太小，单线程 baseline 已经被编译器向量化，输入超过内存带宽上限，partial 发生 false sharing，worker 被调度到同一个物理核心的 SMT sibling，主线程首次触及导致远端内存，或者 benchmark 把输入生成混入了计时区。

实验矩阵要把这些解释逐步分开。第一组只测单线程：baseline、四累加器、可能的 SIMD 版本，输入规模从 L1 可容纳、L2 可容纳、LLC 可容纳一直扩到内存。若小规模下四累加器有效、大规模下无效，说明瓶颈可能从依赖链迁移到内存。第二组固定算法，改变线程数，从 1 到物理核心数，再到逻辑 CPU 数。若到物理核心数后增长变慢，再使用 SMT 变差，说明资源共享或带宽上限值得检查。第三组固定线程数，改变 partial 布局，比较紧凑和对齐。若对齐版本明显改善，说明 false sharing 是关键证据。第四组改变初始化方式，比较主线程初始化和 worker first-touch。若 NUMA 机器上差异显著，说明页面放置参与了结果。

这样的矩阵看起来比“优化前后跑一下”繁琐，但它能防止错误归因。性能工程最怕的是改了十个变量后得到一个数字，然后把提升归功于自己最喜欢的解释。严肃实验宁愿一次只改变一个变量，哪怕多跑几组，也不要同时把线程数、布局、输入、编译选项和绑定策略同时改变。

5.11 正确性 oracle 和防优化

性能实验必须有正确性 oracle。归约的 oracle 可以是顺序 reference；过滤计数的 oracle 可以是简单版本；并发队列的 oracle 不能只看最终数量，还要验证 FIFO 或线性化语义；分布式 shuffle 的 oracle 需要验证每个 key 的最终聚合值和重复请求处理。没有 oracle 的 benchmark 很危险，因为最快的程序可能只是算错了，或者被编译器优化掉了。

防止优化器删除计算也要讲清。常见错误是循环计算了结果但从未使用，优化器发现结果对程序可观察行为没有影响，于是删除整个循环。为了保留计算，可以把结果参与校验、打印摘要、写入 `volatile` sink，或者通过 benchmark 框架的接口告知编译器。更好的方式是把性能样本和 correctness check 绑定：每轮运行都返回结果，结果不对则丢弃样本并报告错误，而不是继续统计耗时。

但防优化不能破坏测量对象。如果把每轮结果都打印到终端，测到的就不是计算时间，而是 I/O 时间。如果把结果写入同一个全局原子，可能引入新的共享写瓶颈。如果把校验放进热循环，校验成本会污染内核。正确做法通常是：热循环只做被测工作，循环外做轻量校验，输出只写汇总，不在计时区打印每个元素。

5.12 预热、冷启动和稳态

预热不是形式主义。第一次运行可能包含动态链接、页面分配、minor fault、缓存冷启动、分支预测器冷启动和 CPU 频率变化。若目标是服务冷启动延迟，这些成本应保留；若目标是稳定吞吐，就应先预热再测。实验报告必须说明测的是冷启动、热循环还是混合流程。

稳态也不是永远稳定。CPU 温度升高可能降频，后台任务可能抢占，容器 CPU 配额可能周期性限流，内核写回线程可能突然工作，透明大页可能在后台整理内存。一个严肃报告应保留多次样本，而不是只给最漂亮的一次。若最小值和中位数差距很大，说明系统噪声或尾部事件值得解释。若样本呈周期性波动，可能要检查 cgroup、频率、热限制或 I/O 写回。

本册建议在报告里至少列出三类值：最小值、中位数和最大值或尾部百分位。最小值常代表干扰较少的理想近似；中位数代表常规情况；尾部代表稳定性风险。高性能系统不只追求平均吞吐，还要理解波动来源。一个吞吐高但尾延迟不可控的队列，可能不适合作为服务请求路径。

5.13 计数器不是事实本身

性能计数器非常有用，但它们不是无条件真理。硬件事件可能受平台支持、内核权限、计数器复用、采样误差和事件定义影响。同一个事件名在不同微架构上含义可能不同；某些高层事件由工具组合推导，不是硬件直接计数；同时请求太多事件时，内核可能 multiplex，导致估计值精度下降。

因此报告中应写出命令、事件列表、是否出现 multiplex、运行时间是否足够长、是否固定 CPU、是否只测当前进程。若工具输出里有权限制或不支持事件，不能忽略。特别是在手机、容器或受限 Linux 环境中，很多硬件事件无法读取，这时可以退而使用时间、上下文切换、page faults、输入规模对照和源码结构分析，但要明确证据等级降低。

计数器解释也要组合。IPC 高不一定代表程序好，可能只是做了很多无用指令但都能退休；IPC 低不一定代表前端差，可能是内存等待、锁等待或分支错误。cache-misses 高也不一定坏，流式扫描大数组天然会 miss，但如果带宽接近上限，miss 可能是有效数据移动。branch-misses 高才需要结合输入分布、分支数量和耗时变化解释。单个数字不能代替因果链。

5.14 Roofline 的实际计算步骤

Roofline 很容易被画成漂亮图，却没有指导实验。实际使用时先做三步。第一步，估算每个元素的逻辑数据移动和算术操作。例如 `std::int32_t` 求和每个元素读取 4 字节，做一次整数加法，最终写一个总和。第二步，估算实际数据移动的额外成本：cache line 带入未使用数据，写分配带来额外读，原子写导致 cache line 迁移，TLB miss 导致页表访问。第三步，用平台可持续带宽和计算吞吐给出上限，再和实测比较。

例如一个简单拷贝内核读取 4 字节并写入 4 字节，逻辑上每元素 8 字节。但如果写入使用普通 write-allocate 路径，写 miss 可能先把目标 cache line 读入，再修改，实际内存流量高于 8 字节每元素。若使用非临时写入，可能减少写分配，但也改变缓存污染和后续复用。教材不能把字节数估算写成固定公式，要让读者知道实际流量取决于写策略、缓存命中和硬件行为。

Roofline 还要区分单线程和多线程。单线程可能受单核加载端口、预取能力或乱序窗口限制；多线程可能逐步接近内存控制器带宽。若单线程远低于平台总带宽，增加线程可能提升；若多线程已经接近可持续带宽，继续加线程只会增加争用。性能报告应把单线程曲线和多线程曲线放在同一个解释里。

5.15 Amdahl、Gustafson 和扩展曲线

多线程加速经常用 Amdahl 定律解释：如果程序中有不可并行的串行部分，那么线程数再多也会受串行部分限制。这个模型有用，但容易被误用。真实程序的串行部分不是固定不变的：任务切分、合并、内存分配、队列调度、锁竞争和 NUMA 远端访问都可能随线程数变化。换句话说，线程数增加后，程序结构本身也在变。

Gustafson 的视角强调问题规模随资源增加而增长。在很多计算任务中，固定输入规模时加速有限，但输入规模扩大后，并行资源可以处理更多工作。教材要同时讲这两种视角。面试刷题常把复杂度看成固定输入下的函数；系统工程还要看“给更多核心后，能否处理更大批量、更高吞吐或更高并发”。固定规模加速和弱扩展是两个不同实验。

扩展曲线至少有三种画法。强扩展固定总工作量，观察线程数增加后时间下降；弱扩展让每个

线程工作量固定，观察线程数增加后总吞吐是否保持；尾延迟扩展关注并发请求增加后延迟分布怎样变化。Compute Systems Engine 的归约适合强扩展和弱扩展；有界队列和线程池更适合吞吐和尾延迟；分布式 shuffle 则要同时看吞吐、长尾任务和重试成本。

5.16 同步路径的测量陷阱

锁和原子很容易被微基准误导。一个锁在空循环里竞争，测到的是最坏的锁争用形态；在真实业务里，临界区可能保护较大的工作，锁成本占比反而不高。反过来，一个队列在单生产者单消费者测试里表现很好，不代表多生产者多消费者下仍能扩展。同步结构必须按目标竞争模式测。

测 mutex 时应区分无竞争、短竞争和阻塞路径。无竞争 fast path 通常很快；短竞争可能自旋；阻塞路径会进入 futex，涉及内核调度和唤醒。测 atomic 时应区分单线程原子成本和多线程 cache line 争用。测无锁队列时应记录 CAS 失败率或至少记录重试行为，否则只看吞吐无法解释尾延迟。测条件变量时应记录等待谓词、队列长度和唤醒策略，否则不知道线程是在等数据、等空间还是被错误唤醒。

5.17 案例：归约实验报告示范

下面给出一段报告式叙述，展示本册希望读者达到的分析密度。

深入理解

问题：比较顺序归约、四累加器归约、热原子并行归约和线程本地 partial 归约。输入为 256 MiB 的 `std::int32_t` 数组，结果用顺序 reference 校验。机器为某型号 8 物理核心 16 逻辑 CPU，编译器使用固定版本和 -O3。每个版本预热一次，正式运行 10 次，报告中位数和最小值。线程数取 1、2、4、8、16。

预期：顺序 baseline 可能受依赖链和内存流量共同影响；四累加器在小输入下可能改善 IPC，但在大输入下可能接近带宽上限；热原子版本在多线程下会因为共享写扩展性差；线程本地 partial 应显著减少共享写，但 16 逻辑 CPU 可能因 SMT 和带宽争用收益下降。若观察结果与预期不同，下一步分别检查编译器是否已自动向量化、partial 是否 false sharing、初始化是否 first-touch、线程是否迁移。

这段报告没有给固定数字，因为数字属于具体机器；它给的是实验结构和解释路径。一本教材如果只给“某机器快了 3 倍”，读者换设备后很难迁移。更重要的是教读者怎样提出假设、怎样排除错误、怎样承认结论边界。

5.18 观测性进入项目设计

很多学生把观测性当成上线后再加的日志，这在系统工程里太晚。Compute Systems Engine 从第一天就应该记录版本、输入规模、线程数、运行时间、校验结果、队列长度、任务数和错误计数。后续加入线程池时，要记录每个 worker 执行任务数、空闲时间和偷取次数；加入有界队列时，要记录 push 等待次数、pop 空队列次数和最大队列长度；加入分布式模拟时，要记录消息数、重试数、重复请求数、提交延迟和检查点耗时。

观测性也要控制成本。热路径中每个元素都写日志会摧毁性能；每个任务都拿全局锁记录指标会引入新的瓶颈。常见做法是线程本地计数、批量汇总、采样、分级日志和只在实验模式下打开详细指标。观测系统本身也是计算系统，仍然受本册所有原则约束。

5.19 统计方法：不要只报平均值

性能数据有波动。平均值容易被少数异常样本拉偏，最小值容易过于乐观，最大值容易受偶然干扰影响。报告应至少给出中位数、最小值和尾部，必要时给出四分位或百分位。对于服务延迟，P95、P99 往往比平均值更重要；对于批处理吞吐，中位数和最小值能帮助区分稳定能力和偶发干扰。

样本数量也要足够。一次运行不能代表稳定性能。短任务应重复更多轮，长任务可以少一些但要记录环境。若样本方差很大，不应简单取平均，而要调查噪声来源：CPU 频率、后台任务、page fault、I/O 写回、GC、热限制、cgroup throttling、网络抖动。统计不是装饰，而是发现不稳定性的工具。

5.20 计时边界

计时边界决定实验测的是什么。一个归约实验如果把输入生成、随机数填充、线程创建、任务提交、计算和校验都放进同一个计时区，结果就不能解释计算内核。端到端时间有价值，但要单独命名。通常应同时报告两类时间：端到端时间和核心阶段时间。端到端时间回答用户体验，核心阶段时间回答优化方向。

线程池实验也类似。若每轮都创建和销毁线程，测到的是线程生命周期成本；若线程池预先创建，测到的是任务调度和执行成本。两者都可能有用，但不能混淆。分布式模拟中，是否把 checkpoint 写入、故障重试、输出提交放进计时，也要明确。

5.21 输入生成和随机性

随机输入要可复现。使用固定 seed，记录分布参数。不要使用运行时随机设备生成不可复现输入，否则性能异常无法重放。对于分支预测实验，要明确输入分布：全小于阈值、全大于阈值、排序后半段大于阈值、随机一半大于阈值。对于热点 key 实验，要明确 Zipf 参数或热点比例。输入分布是 workload 的一部分。

输入生成本身可能很慢，也可能污染 cache。若目标是测计算内核，生成后应在计时外完成，并决定是否预热 cache。若目标是端到端系统，输入生成可以进入计时，但报告要说明。性能实验的每个阶段都要有名字。

5.22 编译选项和二进制一致性

编译选项会大幅影响性能。`-O0` 适合调试，不适合性能结论；`-O2`、`-O3`、`-march=native`、LTO、PGO、`fast-math` 都会改变生成代码和语义边界。报告必须记录编译器版本和选项。若使用 `-march=native`，二进制可能只适合当前机器，换机器结果不可比。

浮点实验尤其要谨慎。`fast-math` 可能允许重排、忽略 NaN 或改变舍入语义。并行归约如果要求确定性，不能随意打开破坏语义的选项。性能优化不能偷偷改变正确性合同。每个编译选项都应有理由。

5.23 基准污染

Benchmark 可能被日志、断言、内存分配、锁、原子 sink、系统调用和调试构建污染。比如在热循环里输出日志，测到的是终端 I/O；在每个元素上调用虚函数，测到的是调度而不是算法；在每轮都分配大 vector，测到的是分配器和 page fault；在并行循环里更新全局统计，测到的是原子热点。

这并不意味着这些成本不重要，而是要命名清楚。如果目标是测完整业务路径，日志和分配可能应保留；如果目标是测内核算子，它们应移出计时区。一个好 benchmark 应能拆分阶段，让读者看到每个成本分别是多少。

5.24 对照实验的单变量原则

优化前后只改变一个变量，结论才清楚。若同时把容器从 list 改成 vector、把算法从单线程改成多线程、把编译选项从 `O2` 改成 `O3`、把输入规模改大，就算变快也不知道原因。实际工程有时需要多改，但实验应尽量拆成多个对照：先只换布局，再只换线程，再只换编译选项。

单变量原则也适用于失败实验。如果某个优化没有效果，不要马上丢弃。先检查变量是否真的变化：编译器是否已自动做了同类优化，输入是否太小，瓶颈是否迁移，测量噪声是否太大。失败结果也是知识，只要记录清楚。

5.25 Benchmark harness 的数据结构

项目可以定义一个简单结果结构，统一记录实验。它不需要复杂框架，但要避免每个实验手写不同输出格式。

Listing 5.1 Benchmark 结果的基本字段

```
1 struct BenchmarkResult {
2     std::string name;
```



```

3   std::uint64_t input_items = 0;
4   std::int32_t worker_count = 1;
5   double milliseconds = 0.0;
6   bool correct = false;
7   std::uint64_t bytes_processed = 0;
8 };

```

生产代码应避免在热路径频繁构造字符串；这里的结构用于实验汇总。后续可以输出 CSV 或 JSON，方便画图。关键是每个样本都带 `correct` 字段，错误结果不能进入性能结论。

5.26 Linux perf 的使用层次

`perfstat` 适合整体计数，例如 `cycles`、`instructions`、`branches`、`branch-misses`、`cache-misses`、`context-switches`、`page-faults`。`perfrecord` 和 `perfreport` 适合采样热点函数。`perftop` 适合实时观察。调度问题可以看 `perfsched`，锁问题在支持环境中可以看 `lock` 相关工具，`cache line` 争用可以研究 `c2c`。不同发行版和权限支持不同，报告要写清不可用项。

使用 `perf` 时要避免测错进程。若程序启动子进程或线程，命令行要确认是否跟踪到目标。若运行时间很短，计数器误差会大。若事件 `multiplex`，工具会给出缩放估计，精度下降。若 CPU 迁移频繁，某些 per-CPU 事件解释更难。`perf` 是强工具，但不是免思考按钮。

5.27 Off-CPU 时间

很多程序慢在 off-CPU 时间，也就是线程没有在 CPU 上运行。原因可能是等待锁、等待 I/O、等待条件变量、被调度器抢占、cgroup 节流或睡眠。CPU profile 只看 on-CPU 热点，可能完全看不到等待原因。服务端尾延迟分析经常需要 off-CPU 视角。

简单替代方法是应用内部记录等待时间。例如 `bounded queue` 的 `push` 等待时间、`pop` 等待时间；线程池 `worker` 空闲时间；RPC `in-flight` 时间；I/O `completion` 等待时间。即使没有 `eBPF` 工具，也可以通过状态机指标构造 off-CPU 证据。第二册强调把观测性放进项目，就是为了在工具受限时仍能分析。

5.28 内存带宽测量

内存带宽不是硬件规格表上的峰值，而是当前工作负载下的可持续带宽。可以用顺序读、写、`copy`、`triad` 等简单内核估算平台上限，再和实际算法比较。若算法吞吐接近可持续带宽，继续减少几条算术指令通常收益小；若远低于带宽，可能受依赖、随机访问、TLB 或前端限制。

带宽也受线程数、NUMA、频率和写策略影响。单线程带宽远低于多线程总带宽很正常；跨 NUMA 访问可能降低带宽；写分配会增加实际流量。报告中写“内存带宽瓶颈”时，应说明依据：字节估算、线程扩展曲线、`cache miss`、平台带宽对照，而不是只凭感觉。

5.29 Profiling 和 Tracing

Profiling 聚合“时间花在哪里”，tracing 保留“事件按什么顺序发生”。计算内核常用 profiling；运行时、队列、异步 I/O 和分布式系统更需要 tracing。一个线程池平均函数耗时不高，但 `trace` 可能显示 `worker` 大量空闲或等待队列；一个分布式作业总耗时高，`trace` 可能显示最后一个 `reduce partition` 长尾。

Compute Systems Engine 应支持轻量 `trace`：任务开始、任务结束、队列满、消息发送、消息完成、重试、提交。`Trace` 要有开关，默认可以只记录摘要。详细 `trace` 会产生大量数据，适合调试而不是每次性能运行。

5.30 性能回归

项目变大后，需要防止性能回归。不是每次提交都跑完整 `benchmark`，但至少要有小规模 `smoke benchmark` 和关键路径指标。比如顺序归约、并行归约、有界队列吞吐、线程池任务调度和分布式模拟正确性。若某次修改让归约慢 30%，应能及时发现。

性能回归判断要允许噪声。可以设置相对阈值、重复运行取中位数，并保留历史环境信息。手

机和共享云环境噪声大，阈值要更宽；专用机器可以更严格。回归测试的目标是发现明显退化，不是追求实验室级精确。

5.31 报告中的诚实边界

高质量报告会明确说哪些结论已经验证，哪些只是推测。比如“当前环境无法读取 dTLB 事件，因此 TLB 解释来自页数和访问模式对照，证据弱于直接计数”；“本机没有 NUMA，因此 first-touch 只作为设计保留”；“EPYC 服务器上结果不能直接外推到手机大小核”。承认证据边界不是减分，而是工程严谨。

相反，糟糕报告会把一次运行数字写成普遍结论，把某台机器的结果推广到所有 CPU，把不可复现随机输入当作事实，把没有正确性校验的最快版本当作成功。第二册要求读者避免这种写法。

5.32 案例：一次性能回归排查

假设某次提交后，parallel histogram 慢了 25%。第一步不是猜，而是确认正确性是否仍通过，输入和编译选项是否相同。第二步看阶段指标：本地统计慢了，还是合并慢了，还是队列等待增加。第三步看 diff：是否改变了桶布局、增加了全局指标、把本地 vector 改成 `unordered_map`、或在热循环中加入日志。第四步跑对照：关闭新指标、恢复旧布局、固定线程数。

这样的排查流程比“感觉是 cache 问题”可靠。性能回归常来自小改动：一个统计计数器从线程本地变成全局原子，一个调试日志进入热路径，一个结构体字段改变导致 padding 变大，一个 vector reserve 被删除导致频繁分配。性能工程需要把代码 diff、指标和实验联系起来。

5.33 案例：手机环境测量

在手机 Termux 上测性能，要特别谨慎。CPU 可能有大小核，温度和电量会影响频率，后台应用可能抢资源，某些 perf 事件不可用，存储 I/O 受系统策略影响。报告应记录设备型号、系统版本、是否充电、是否长时间运行、可见 CPU、是否能设置 affinity、哪些 perf 事件可用。不能把手机一次短跑结果当成服务器结论。

手机环境也有价值。它能训练受限环境下的证据意识。无法读取硬件计数器时，可以用输入规模曲线、线程数曲线、正确性校验、时间分布和源码分析。无法验证 NUMA 时，可以写明限制。工程严谨不要求工具完美，而要求诚实说明证据等级。

5.34 实验数据归档

性能实验应归档原始数据，而不是只在报告里写结论。至少保存命令、配置、输入摘要、运行时间、样本、git commit、机器信息和输出校验。CSV 或 JSON 都可以。图表可以从原始数据生成，报告引用图表。这样几周后发现问题，可以重新分析，而不是重新跑一切。

项目可以在 `results/` 下按日期或 commit 保存实验摘要。不要把巨大临时文件提交，但小的 CSV、报告和脚本可以保留。性能工程是可重复研究的一种形式。

5.35 Benchmark 伦理

不要只挑最好结果，不要删除失败样本后不说明，不要在不同机器结果之间做不公平比较，不要把 debug 构建和 release 构建混在一起，不要把不正确输出的时间当成成功，不要把一次偶然提升宣传成普遍优化。工程团队依赖性能报告做决策，报告必须可信。

这听起来像规范，但它直接影响技术质量。错误的 benchmark 会把团队带向错误优化，甚至牺牲正确性。第二册强调测量，就是为了建立这种专业习惯。

5.36 性能报告示例结构

本册建议每份性能报告采用固定结构。标题说明被测模块和 commit。环境说明机器、系统、编译器、编译选项、CPU 拓扑和限制。工作负载说明输入规模、分布、seed 和正确性 oracle。实验矩阵说明版本、线程数、batch 大小、队列容量和运行次数。结果部分给出样本统计、关键指标和图表。分析部分写瓶颈假设、支持证据、反证和下一步。限制部分说明哪些结论不能外推。

固定结构能提高比较效率。读者以后回看报告，可以直接找到环境和输入；别人审查报告，可

以快速发现缺失信息。性能报告不是文学作品，稳定格式比花哨表达更有价值。

5.37 测量章节总结

测量的目标是建立证据链。计时告诉现象，计数器提供线索，trace 显示顺序，correctness oracle 保证没有优化错对象，对照实验排除错误解释。没有测量，优化是猜；没有语义，测量可能奖励错误程序。本章的方法会贯穿后面所有实验。

5.38 测量决策表

选择测量方法时，先问问题类型。单核循环慢，用计时、汇编、IPC、branch 和 cache 事件；多线程不扩展，用线程数曲线、context switches、锁等待、false sharing 对照；I/O 慢，用队列长度、等待时间、page faults、fsync 时间；分布式慢，用 trace、partition 指标、重试和队列。工具随问题变化，不要用一个 perf 命令解释所有系统。

5.39 本章落到贯穿项目

本章要求项目拥有一个最小但严肃的 benchmark harness。它至少要支持：选择输入规模，选择版本，选择线程数，重复运行多轮，校验输出，输出机器和编译信息，记录最小值和中位数。后续可以逐步加入 CPU 绑定、NUMA 初始化、perf 命令模板、JSON 输出和图表脚本。不要一开始就做复杂框架，但也不能继续依赖手工改代码和肉眼读终端。

项目里的每个优化提交都应附带实验说明：优化目标是什么，影响哪条路径，正确性如何验证，基线版本是什么，在哪些输入和线程数下有效，在哪些情况下无效。这样做看似增加工作量，但能避免“改完之后不知道为什么快，也不知道什么时候会慢”的维护风险。

5.40 本章习题

习题与作业

习题一：为顺序归约、多累加器归约和线程本地 partial 归约设计一份实验矩阵。要求列出输入规模、线程数、运行次数、预热策略、计时范围、正确性校验和需要记录的计数器。每个变量都要说明用于排除哪一种错误解释。

习题二：写一段报告，解释为什么热原子归约即使使用 relaxed 内存序也可能扩展性很差。报告必须同时使用 C++ 语义、cache line 所有权和测量指标三个层面的语言。

习题三：设计一个队列 benchmark。要求区分 SPSC、MPSC 和 MPMC 三种竞争模式，记录吞吐、队列长度和等待次数。说明为什么只测单生产者单消费者不能证明一个 MPMC 队列适合线程池全局队列。

习题四：阅读当前系统的 `perfstat` 可用事件。记录哪些事件可用，哪些事件因权限或平台不可用。对不可用事件，写出替代实验方案，而不是直接放弃分析。

核心思想

测量不是为了得到一个数字，而是为了排除错误解释，缩小瓶颈范围，并验证优化是否真的改变了瓶颈。

实验：对 Compute Systems Labs 的顺序归约和并行归约运行 `perfstat`。记录时间、cycles、instructions、IPC、cache-misses 和 context-switches。解释并行版本是否真的更快，如果没有，瓶颈可能在哪里。

第 6 章

数据布局、局部性与预取

数据布局决定程序怎样使用缓存、TLB 和内存带宽。很多高性能优化并不是改变算法复杂度，而是把数据摆成硬件更容易消费的形状。理解布局，是理解数组、结构体、队列、图和矩阵内核的共同基础。

6.1 AoS 与 SoA

Array of Structures 把一个对象的字段放在一起，适合经常同时访问完整对象的场景。Structure of Arrays 把同类字段连续存放，适合批量处理某个字段的场景。例如粒子模拟中，如果每轮只更新所有粒子的 x 坐标，SoA 更容易形成连续访问和 SIMD；如果每次处理一个粒子的全部属性，AoS 更直观。

选择布局不能只看代码好不好写，要看热路径读写哪些字段、是否需要向量化、是否有 false sharing、是否跨页跳转。工程中常见折中是 AoSoA，把数据按小块组织，既保留局部对象关系，又适合 SIMD 和缓存块。

Listing 6.1 AoS 与 SoA 的访问差异

```
1 struct Particle {
2     float x;
3     float y;
4     float z;
5 };
6
7 struct ParticleSoA {
8     std::vector<float> x;
9     std::vector<float> y;
10    std::vector<float> z;
11 };
```

如果热路径只更新所有粒子的 x ，SoA 版本会连续读取和写入 x 数组；AoS 版本每次还把 y 和 z 附近的数据带入 cache line。若热路径总是同时使用 x 、 y 、 z ，AoS 的空间局部性反而更自然。

布局选择要从访问矩阵出发，而不是从结构体是否“面向对象”出发。可以把字段列成列，把热路径列成行，在每个格子里标出读、写、只读配置、冷路径调试。若一个热路径只读少数字段，SoA 或冷热分离更有优势；若多个字段总是一起使用，AoS 或 AoSoA 更清晰；若需要 SIMD，字段连续性和对齐会更重要；若需要并发写，线程到字段和槽位的映射必须避免 false sharing。

```
hot path: update position
  reads:  x, y, z, vx, vy, vz
  writes: x, y, z

hot path: compute bounding box
  reads:  x, y, z
  writes: thread local min and max

cold path: debug print
  reads:  name, id, material, history
```

这张访问矩阵会自然推出布局：调试字段不应混在高频数值字段里；只在统计阶段使用的字段

不应污染每帧更新；线程本地输出应分开存放，最后合并。

6.2 预取

硬件预取器擅长识别顺序访问和固定步长访问。连续数组扫描通常能被预取器提前加载，随机指针追踪则很难。软件预取可以在少数场景有效，但如果预取太早会挤出有用缓存，太晚则来不及隐藏延迟。

最好的预取优化通常不是手写预取指令，而是改变访问模式：把随机访问排序，按块处理，压缩节点，减少指针跳转，让硬件预取器能看懂程序。

软件预取只有在地址能提前算出、未来确实会访问、当前有足够独立工作、预取距离合适时才可能有效。指针追踪中，如果下一步地址必须等当前节点加载完成，预取很难发挥作用；批量处理多个游标时，才有机会提前为另一个游标发起预取。预取错误还可能污染缓存，把真正热的数据挤出去。

因此本书把预取看成最后一步，而不是第一步。先让数据连续，先分块，先减少对象分散，先测出内存延迟或带宽瓶颈，再考虑预取指令。很多工程代码的性能来自“让硬件预取器自然工作”，而不是手写大量 `prefetch`。

6.3 冷热数据分离

对象里不是所有字段都同样热。把热字段和冷字段混在一起，会让缓存带入很少使用的数据。冷热分离把热路径频繁访问的字段放在紧凑结构中，把调试信息、统计字段、名字、配置等冷数据放到旁边结构或间接存储。数据库执行器、游戏引擎和网络服务都常使用这种思路。

冷热分离也有代价：代码复杂度增加，对象关系不如原来直观。是否值得做，要看热路径字段访问比例和缓存压力。设计数据结构时，先写清哪个路径是热路径，哪个字段在热路径必读或必写。

6.4 内存分配和对象布局

小对象频繁分配会带来分配器开销、碎片、TLB 压力和缓存局部性问题。高性能系统常用 arena、对象池、批量分配或结构化生命周期来减少热路径分配。并发分配器还要避免全局锁和跨线程释放导致的远端缓存流量。

常见误区

不要把“使用堆”简单等同于慢。真正的问题是生命周期是否清晰、分配是否在热路径、对象是否分散、是否跨线程释放、是否破坏局部性。

对象池和 arena 的核心收益不是“分配函数少调用几次”这么简单。它们把生命周期相近的对象放在相近内存中，减少元数据开销和碎片，提高遍历局部性，也让释放可以批量发生。并发环境下，还可以按线程或 NUMA 节点维护本地池，减少全局分配器锁和远端释放。

代价同样要写清楚。Arena 适合批量释放，不适合单个对象精确析构；对象池可能保留大量空闲内存；跨线程归还对象可能重新引入同步；自定义分配器如果没有测试，可能比通用分配器更难维护。高质量教材不能只说“用内存池更快”，必须说明适用的生命周期形状。

6.5 布局与并发

多线程程序的数据布局还要考虑写共享。线程本地数据应尽量分开，跨线程只读数据可以共享，跨线程写数据要减少同一 cache line 上的冲突。并行归约、线程池队列和日志缓冲区都常常需要为每个线程准备独立槽位。

6.6 结构体大小和 ABI

结构体字段顺序影响 padding 和总大小。较大的结构体会降低缓存密度，也会增加拷贝成本。公共 ABI 中随意调整字段顺序可能破坏二进制兼容，因此高性能库常把内部热结构和外部稳定接口分开。内部结构按性能布局，外部接口保持稳定语义。

6.7 布局重构流程

数据布局优化应按流程进行。第一步，确认热路径和访问矩阵。第二步，估算每轮实际需要读写的字节数。第三步，检查对象是否连续、是否跨页、是否存在共享写。第四步，设计一个只改变布局、不改变算法语义的对照版本。第五步，用相同输入和相同绑定策略测量时间、cache、TLB 和扩展曲线。第六步，评估代码复杂度是否值得。

很多布局优化失败，是因为同时改了太多东西：换容器、改算法、改线程数、改内存分配、改输入规模。这样即使变快，也不知道原因。教材实验必须强调单变量对照，让读者学会建立因果证据。

6.8 AoSoA：折中不是妥协

AoS 和 SoA 常被讲成二选一，但真实系统经常使用 AoSoA。它把数据按小块分组，每个块内部采用 SoA，块与块之间保持对象批次关系。这样可以让一个块适配 SIMD 宽度或缓存块，同时又不会把整个系统改造成完全分离的字段数组。粒子系统、物理模拟和向量数据库里都能看到类似结构。

Listing 6.2 AoSoA 的基本形态

```
1 constexpr std::int32_t PARTICLE_BLOCK_WIDTH = 8;
2
3 struct ParticleBlock {
4     std::array<float, PARTICLE_BLOCK_WIDTH> x;
5     std::array<float, PARTICLE_BLOCK_WIDTH> y;
6     std::array<float, PARTICLE_BLOCK_WIDTH> z;
7     std::array<float, PARTICLE_BLOCK_WIDTH> vx;
8     std::array<float, PARTICLE_BLOCK_WIDTH> vy;
9     std::array<float, PARTICLE_BLOCK_WIDTH> vz;
10 };
11
12 struct ParticleBlocks {
13     std::vector<ParticleBlock> blocks;
14     std::int32_t size = 0;
15 };
```

这里的块宽度不是魔法数字。它可能来自 SIMD lane 数、cache line 大小、对齐要求、尾部处理成本和代码复杂度。块太小，循环层次变多但复用不足；块太大，尾部浪费和缓存压力上升。AoSoA 的价值是提供一个调节旋钮，让布局能同时服务向量化和对象组织。

AoSoA 也有成本。随机访问第 *i* 个对象需要计算块号和块内偏移；插入删除比 `std::vector<Particle>` 更复杂；调试打印不如 AoS 直观；接口要隐藏布局细节，否则业务代码会到处写块索引。工程中通常把 AoSoA 封装在专用容器或视图后面，让热路径拿到高效访问，普通路径仍保持清晰 API。

6.9 访问矩阵如何落到代码

访问矩阵不是文档装饰，它应该直接影响类型设计。假设日志统计系统的每条记录有时间戳、用户 ID、事件类型、原始文本、解析状态和调试信息。热路径只需要用户 ID 和事件类型做聚合，原始文本只在错误报告时使用。如果把所有字段放在一个大结构里，聚合时每次都把冷字段附近的数据带入缓存。更好的设计是把热字段压成连续数组，把冷字段放在旁边，通过索引关联。

Listing 6.3 冷热分离的记录批次

```
1 struct HotLogBatch {
2     std::vector<std::uint64_t> user_ids;
3     std::vector<std::int32_t> event_types;
4 };
5
```

```

6 struct ColdLogBatch {
7     std::vector<std::string> raw_lines;
8     std::vector<std::int32_t> parse_status;
9 };
10
11 struct LogBatch {
12     HotLogBatch hot;
13     ColdLogBatch cold;
14 };

```

这个设计牺牲了“一个对象包含所有字段”的直观性，换来热路径更少的数据移动。它还让并发更容易：聚合 worker 可以只读 **hot**，错误处理线程再查看 **cold**。若业务经常需要完整记录，冷热分离可能不值得；若绝大多数请求只走热路径，它就很自然。布局设计必须从访问频率和访问组合出发。

6.10 索引优于指针的场景

指针直接表达对象关系，但在大规模数据结构中，索引常常更适合性能和序列化。索引可以是 `std::uint32_t` 或 `std::uint64_t`，指向连续数组中的位置。它比指针更紧凑，容易持久化和跨进程传递，也让对象搬迁和压缩更容易。图的邻接表、编译器 IR、游戏 ECS 和列式存储都大量使用索引。

索引也有代价。每次访问需要多一步数组寻址；删除对象后要处理空洞、代际版本或稳定 ID；越界和悬空索引需要调试检查。若系统需要稳定引用，可以使用“索引加 generation”的句柄，避免旧索引误指向新对象。这个思想和无锁结构里的 ABA 类似：同一个位置再次被复用时，需要版本信息区分“看起来一样”和“语义上还是同一个对象”。

Listing 6.4 索引句柄表达稳定引用

```

1 struct EntityId {
2     std::uint32_t index = 0;
3     std::uint32_t generation = 0;
4 };
5
6 struct EntitySlot {
7     std::uint32_t generation = 0;
8     bool alive = false;
9 };

```

这里使用明确位宽类型，是为了让内存布局和序列化边界清楚。系统教材中不能随手使用 `longlong` 这类平台相关宽度。布局设计和数据类型选择是一件事：类型宽度会影响结构体大小、cache 密度、文件格式和网络协议。

6.11 压缩、编码和计算成本

有时减少内存流量比减少计算更重要。列式数据库和搜索系统常把整数压缩成变长编码、差分编码或位图。这样每个元素读取的字节数减少，但解码需要额外 CPU。是否划算取决于瓶颈：若系统受内存带宽限制，压缩可能提高吞吐；若系统受计算限制，压缩可能变慢。

这个权衡也出现在分布式 shuffle 中。压缩中间数据可以减少网络 and 磁盘 I/O，但会增加 CPU 时间和延迟。单机 cache 层面的“少搬字节，多做一点计算”和分布式层面的“少发网络，多做一点压缩”是同一个思想。教材应让读者从底层内存开始建立这种成本交换模型。

6.12 并发写布局

并发数据布局要先标出写者。字段是否共享，不看它是否在同一个结构体里，而看哪些线程写它、写入频率如何、是否落在同一 cache line 上。一个常见错误是把多个线程的统计字段放进同一个 `Stats` 数组，觉得每个线程写自己的下标就安全。结果每个字段很小，多个线程的字段落在同一条 cache line，形成 false sharing。

更好的设计是把写热字段按写者分开，并且让每个写者独占 cache line 或至少独占写热点。读者汇总时再扫描这些槽位。若读取频率很高，可以维护分层汇总；若读取只是指标展示，可以接受延迟。并发表局本质上是在写入吞吐、读取成本和内存占用之间取舍。

Listing 6.5 线程本地统计槽位

```
1 struct alignas(64) WorkerStats {
2     std::uint64_t tasks_executed = 0;
3     std::uint64_t steal_attempts = 0;
4     std::uint64_t empty_polls = 0;
5 };
6
7 std::vector<WorkerStats> stats(static_cast<std::size_t>(worker_count));
```

这类结构适合 worker 高频写自己的统计。它不适合每个请求都读取全局精确值。如果业务需要实时限流，统计槽位还要配合周期性汇总或原子发布。布局优化总是要回到语义。

6.13 迁移旧代码的策略

把成熟代码从 AoS 改成 SoA 或冷热分离，风险很高。直接大改会破坏接口、测试和调试习惯。稳妥策略是分阶段迁移。第一阶段写访问矩阵和性能证据，证明布局确实是瓶颈。第二阶段增加新布局的内部表示，但保留旧接口，通过 adapter 暴露兼容访问。第三阶段把热路径迁到新布局，保留 reference 对照。第四阶段逐步清理旧路径。

迁移时要防止“双写不一致”。如果旧结构和新结构同时存在，更新必须保证二者一致，或者明确一个是权威数据，另一个是缓存。缓存需要失效策略；双写需要事务边界；延迟同步需要版本号。布局重构不是单纯换容器，而是数据所有权迁移。

6.14 标准容器的内存形状

数据布局不只发生在自己定义的结构体里，也发生在每一次选择标准容器时。`std::vector` 的元素连续存放，适合顺序扫描、批量复制、SIMD 读取和按索引访问。它的扩容会搬迁元素，因此外部保存元素指针或引用时必须小心。`std::deque` 不是一个大连续数组，而是由多个固定大小块组成，头尾插入更稳定，但长距离顺序扫描的局部性通常不如 `std::vector`。`std::list` 每个节点独立分配，插入删除不移动其他节点，但遍历时会产生指针追踪、cache miss、TLB miss 和分配器开销。很多初学者只记住复杂度表，却没有把复杂度和硬件成本合起来看。

例如一个任务运行时维护 ready task。如果任务总数在构图后基本稳定，`std::vector` 加索引队列或环形缓冲通常更适合。如果任务需要频繁从中间删除，但遍历又很热，链表也未必正确，因为删除的理论优势可能被遍历成本吞掉。若真的需要稳定句柄，可以用索引加 generation，而不是直接把每个任务做成堆上节点。容器选择不是 API 风格问题，而是访问模式、生命周期和硬件局部性的共同结果。

Listing 6.6 索引队列比链表更容易保持局部性

```
1 struct ReadyTask {
2     std::uint32_t task_id = 0;
3     std::uint32_t generation = 0;
4 };
5
6 class ReadyRing {
```



```

7 public:
8     explicit ReadyRing(std::uint32_t capacity)
9         : buffer_(capacity), head_(0), tail_(0), size_(0) {}
10
11     bool push(ReadyTask task) {
12         if (this->size_ == this->buffer_.size()) {
13             return false;
14         }
15         this->buffer_[this->tail_] = task;
16         this->tail_ = (this->tail_ + 1) % this->buffer_.size();
17         ++this->size_;
18         return true;
19     }
20
21     std::optional<ReadyTask> pop() {
22         if (this->size_ == 0) {
23             return std::nullopt;
24         }
25         const ReadyTask task = this->buffer_[this->head_];
26         this->head_ = (this->head_ + 1) % this->buffer_.size();
27         --this->size_;
28         return task;
29     }
30
31 private:
32     std::vector<ReadyTask> buffer_;
33     std::size_t head_;
34     std::size_t tail_;
35     std::size_t size_;
36 };

```

这个例子还不是并发队列，因为它没有锁和内存序。它要说明的是另一个层次：在考虑同步之前，先让数据本身连续、容量有界、访问路径可预测。后续给它加互斥或 SPSC 语义时，至少不会再被链式节点和频繁分配拖住。很多高性能结构的第一步不是“无锁”，而是“固定容量、连续存储、清楚所有权”。

6.15 分配器不是只影响分配速度

分配器常被理解成“申请内存快不快”，但它对布局的影响更深。通用分配器需要支持不同大小、不同生命周期、不同线程的分配和释放，因此会维护元数据、空闲链表、线程本地缓存和大块映射。小对象如果被零散分配，遍历时访问的是分配器历史留下的空间分布，而不是业务逻辑希望的顺序。一个图算法使用链式邻接节点，即使每个节点只有十几个字节，也可能因为节点分散而消耗大量 cache 和 TLB 资源。

Arena 的优势来自生命周期整齐。解析一个输入文件时产生的临时节点，可以在解析结束后一次性释放；构建一轮任务图时产生的任务对象，可以在图执行结束后批量销毁。这样做减少了单个对象释放的成本，也让相近生命周期的数据更容易靠近。缺点是对象不能随意提前释放，析构顺序要清楚，长期存活对象不能混进短生命周期 arena。否则 arena 会保留大量已经无用但不能回收的空间。

对象池则适合固定大小、频繁复用的对象，例如网络请求上下文、日志批次缓冲、任务描述符。对象池可以把对象初始化和内存申请分开，减少热路径分配；也可以按线程或 NUMA 节点分池，减少跨线程竞争。对象池的风险是复用对象时忘记清理旧状态，或者跨线程归还导致远端 cache line 流量。高质量实现要把 reset、owner thread、debug generation 和峰值容量都写进设计。

Listing 6.7 固定大小对象池的基本接口形状

```

1 struct RequestContext {
2     std::uint64_t request_id = 0;
3     std::uint32_t attempt = 0;
4     std::uint32_t worker_id = 0;
5 };
6
7 class RequestPool {
8 public:
9     explicit RequestPool(std::size_t capacity) : storage_(capacity) {
10         for (std::uint32_t index = 0;
11             index < static_cast<std::uint32_t>(storage_.size());
12             ++index) {
13             this->free_.push_back(index);
14         }
15     }
16
17     std::optional<std::uint32_t> acquire() {
18         if (this->free_.empty()) {
19             return std::nullopt;
20         }
21         const std::uint32_t index = this->free_.back();
22         this->free_.pop_back();
23         this->storage_[index] = RequestContext{};
24         return index;
25     }
26
27     void release(std::uint32_t index) {
28         this->storage_[index] = RequestContext{};
29         this->free_.push_back(index);
30     }
31
32 private:
33     std::vector<RequestContext> storage_;
34     std::vector<std::uint32_t> free_;
35 };

```

这个池子仍然只是单线程教学版本。并发版本必须决定 free list 由谁保护，是否每个 worker 一个本地池，跨线程释放是否进入 remote free 队列，是否要用 generation 防止旧句柄复用。不要因为代码短就忽略这些语义。内存池的工程难点不在“写一个 vector”，而在生命周期、并发所有权和调试可见性。

6.16 面向数据的接口设计

布局优化失败的常见原因，是内部换了 SoA，外部接口仍然要求每次返回一个完整对象。这样热路径又不得不临时拼对象，或者频繁跨数组取字段，收益被接口吃掉。面向数据的接口应该让调用者表达批量意图。例如不是提供 `get_particle(i)` 返回完整粒子，而是提供 `x_span()`、`velocity_span()` 或批处理函数，让热路径直接操作连续字段。

接口也要避免泄漏实现细节。完全暴露 `std::vector<float>&` 会让外部随意 `resize`，破坏布局不变量；完全隐藏成单元素 `getter` 又丢掉批量优化机会。折中做法是暴露只读或可写 `std::span`，并用类型说明生命周期和访问权限。只读输入、可写输出、临时 `scratch`、线程本地 `partial` 应该在函数签名中区分开。

Listing 6.8 批量接口比单元元素接口更能表达热路径

```

1 struct PositionView {
2     std::span<float> x;
3     std::span<float> y;
4     std::span<float> z;
5 };
6
7 void integrate(PositionView position,
8               PositionView velocity,
9               float delta_time) {
10     const std::size_t count = position.x.size();
11     for (std::size_t i = 0; i < count; ++i) {
12         position.x[i] += velocity.x[i] * delta_time;
13         position.y[i] += velocity.y[i] * delta_time;
14         position.z[i] += velocity.z[i] * delta_time;
15     }
16 }

```

这个函数把布局意图表达给编译器和读者：三个坐标数组连续，循环边界统一，写入位置清楚。实际生产代码还要检查 span 长度一致、别名关系和对齐条件。接口设计的目标不是把所有优化都写进函数，而是不要一开始就把优化可能性封死。

6.17 ECS 和列式布局

Entity Component System 常见于游戏引擎，但它背后的思想适用于任何大量对象、少数字段热访问的系统。实体只是 ID，组件按类型连续存放，系统按组件批量处理。渲染系统关心位置和网格，物理系统关心位置和速度，AI 系统关心状态和目标。每个系统只扫描自己需要的组件，避免把完整对象的冷字段一起带入缓存。

日志分析和数据库执行器也使用类似思想。列式存储把同一列连续存放，过滤某个字段时只读取相关列；投影阶段再把需要的列组合输出。若查询只需要用户 ID 和事件类型，就不应该读取完整 JSON 文本。若后续需要原始文本，再通过行号或 offset 访问冷数据。列式布局的核心不是“数据库专属格式”，而是按访问组合移动最少字节。

列式布局还有压缩优势。同一列的数据类型相同、分布相近，更容易做字典编码、差分编码、位图和运行长度编码。压缩后减少内存和 I/O 流量，但增加解码成本。若过滤条件能直接在压缩表示上执行，收益很大；若每次都要完整解码，收益可能下降。布局、编码和算法必须一起设计。

6.18 CSR：图结构的局部性案例

图算法最容易暴露指针结构的问题。一个朴素邻接表如果每条边都是独立节点，遍历邻居时会不断指针跳转。Compressed Sparse Row 把所有边目标压到一个连续数组里，再用 offsets 数组标出每个顶点的邻居范围。这样遍历顶点 `v` 的邻居时，只需要扫描 `edges[offsets[v]]` 到 `edges[offsets[v+1]]` 之间的连续区间。

Listing 6.9 CSR 图的核心布局

```

1 struct CsrGraph {
2     std::vector<std::uint32_t> offsets;
3     std::vector<std::uint32_t> edges;
4 };
5
6 std::uint64_t degree_sum(const CsrGraph& graph) {
7     std::uint64_t total = 0;
8     for (std::size_t vertex = 0; vertex + 1 < graph.offsets.size(); ++vertex) {
9         const std::uint32_t begin = graph.offsets[vertex];

```

```

10     const std::uint32_t end = graph.offsets[vertex + 1];
11     total += static_cast<std::uint64_t>(end - begin);
12 }
13 return total;
14 }

```

CSR 的代价也要讲清楚。它适合静态图或批量更新后的图，不适合高频单边插入删除。插入一条边可能需要移动后续大量边，或者维护增量结构再周期性重建。它还把顶点 ID 映射和边排序变成预处理问题。很多系统会使用双层结构：主图用 CSR 保持扫描性能，增量更新放在小的补丁区，定期合并。这个设计和 LSM-tree、列式数据库的 delta store 很像：热更新和冷扫描分层处理。

6.19 块大小的选择方法

分块是布局优化的核心技巧，但块大小不能拍脑袋。选择块大小时至少要考虑五个因素。第一，块内工作集是否能放进目标 cache 层级。第二，块内循环是否能形成足够 SIMD 和指令级并行。第三，块大小是否造成尾部处理过多。第四，块是否方便按线程或 NUMA 节点分配。第五，块元数据是否过多。

以矩阵乘法为例，一个块需要同时容纳 A 的子块、B 的子块和 C 的子块。若三者合起来超过 L1 或 L2，块内复用就会下降。以日志聚合为例，一个批次需要容纳 key、value、hash、输出缓冲和临时计数。批次太小，函数调用和调度成本高；批次太大，cache 复用差，背压粒度粗。块大小应通过候选集实验选择，例如从 1 KiB、4 KiB、16 KiB、64 KiB、256 KiB 开始，观察吞吐、cache miss、TLB miss 和尾延迟。

块大小还影响错误恢复。分布式计算中，任务块越大，调度开销越低，但失败重做成本越高；块越小，负载均衡更好，但调度和元数据成本更高。单机 cache block、线程池 task grain、分布式 partition chunk 其实是同一个问题在不同尺度上的版本。

6.20 可测量的布局报告

布局优化的报告至少应包含四类数据。第一，结构体大小和对齐：使用 `sizeof`、`alignof`，列出字段顺序和 padding。第二，访问字节数估算：每处理一个元素理论上读取和写入多少字节，哪些字节是冷字段。第三，硬件指标：时间、带宽、cache miss、TLB miss、分支和 IPC。第四，语义影响：接口是否改变，生命周期是否改变，是否影响稳定引用和并发写入。

只有耗时数字的报告不够。假设 SoA 比 AoS 快了 1.8 倍，你还要说明原因是否来自少读冷字段、SIMD 更好、预取更稳定、TLB 覆盖更高，还是因为新版本顺便少做了别的工作。若无法解释，就应该补实验。可以构造一个只访问全部字段的场景，如果 AoS 反而更快，说明 SoA 的优势确实来自字段子集访问，而不是神秘加速。

下面是一份布局报告的文字模板，但写报告时要填真实数据。

```

layout report:
workload: aggregate event type from N records
baseline: vector<Record>, hot fields mixed with raw line
candidate: HotLogBatch plus ColdLogBatch
semantic change: external record id remains stable
bytes per hot record: baseline estimated 80, candidate 12
metrics: time, cache misses, dTLB misses, bandwidth
result: candidate improves hot aggregation, cold error path unchanged

```

6.21 布局优化的反例

布局优化也会失败。第一类失败是工作集很小，本来就全部在 cache 里，换布局只增加代码复杂度。第二类失败是热路径需要完整对象，SoA 反而让字段分散在多个数组中，增加地址计算和 cache line 数。第三类失败是业务频繁插入删除，CSR 或紧凑数组重建成本太高。第四类失败是并发访问模式改变，新布局把原来分散的写入集中到同一 cache line。第五类失败是接口没有同步迁移，内部

布局高效，外部仍然单元素调用。

因此本章不是要求读者把所有对象都改成 SoA，而是要求读者学会问问题：热路径读哪些字段，写哪些字段，数据能否批量处理，生命周期是否成批，是否有稳定 ID 需求，是否多线程写，是否需要序列化，是否有恢复和调试要求。布局是系统设计，不是局部技巧。

6.22 日志系统的布局案例

贯穿项目的日志统计可以用一个具体布局案例串起来。原始日志行是冷数据，解析后的 event type、user id、timestamp 和 value 是热数据，解析错误信息是低频冷数据。若每条记录都保存成一个包含字符串和多个字段的大结构，聚合 event type 时会把大量原始文本和调试字段带入 cache。更好的设计是先把输入按 batch 解析，热字段放入列式数组，原始行或错误上下文按 offset 另存。

这种设计还方便错误处理。正常路径只扫描热字段；遇到解析错误时，把错误行号和原始 offset 写入冷错误列表；需要输出错误报告时再回到原始文本。热路径和冷路径分开后，性能和调试都更清楚。很多系统性能差，不是因为算法复杂，而是每次处理热字段都拖着一堆冷调试信息走。

Listing 6.10 日志批次的热冷布局

```
1 struct ParsedLogHotBatch {
2     std::vector<std::uint64_t> timestamps;
3     std::vector<std::uint64_t> user_ids;
4     std::vector<std::uint32_t> event_types;
5     std::vector<std::int64_t> values;
6 };
7
8 struct ParsedLogColdBatch {
9     std::vector<std::uint64_t> raw_offsets;
10    std::vector<std::uint32_t> error_rows;
11 };
```

这个布局不要求原始输入立即复制。`raw_offsets` 可以指向 mmap 文件或输入缓冲中的位置，但生命周期要清楚：如果输入缓冲释放，offset 就不能再用于错误报告。布局和生命周期永远绑定在一起。

6.23 ECS 变体：稀疏集合

ECS 常用 sparse set 维护实体到组件位置的映射。Dense 数组保存实际组件，sparse 数组从 entity id 映射到 dense index。遍历组件时扫描 dense，局部性好；查询某个 entity 是否有组件时，用 sparse 快速定位。删除时可以用 swap-remove，把最后一个元素移到删除位置，再更新 sparse。这个结构牺牲稳定顺序，换取紧凑遍历。

如果业务需要稳定顺序，swap-remove 就不合适；如果 entity id 非常稀疏，sparse 数组可能很大；如果组件跨线程频繁增删，需要同步。再次说明，数据结构没有脱离语义的绝对优劣。Sparse set 适合大量实体、组件遍历热、增删可接受重排的场景。

6.24 Columnar batch 和 vectorized execution

数据库的 vectorized execution 通常以 batch 为单位处理列式数据。一个 batch 可能包含几千行，每个算子处理一个 batch 后交给下一个算子。Batch 太小，函数调用和调度成本高；batch 太大，cache 压力和延迟增加。Batch 模型是 row-at-a-time 和全量材料化之间的折中。

日志统计项目可以借鉴这个模型。输入解析成 batch，filter 处理 batch，histogram 处理 batch，shuffle 按 batch 发送。每个 batch 有热字段列、选择向量、错误列表和统计摘要。这样项目既能展示数据布局，也能展示 I/O 背压：下游慢时，上游 batch 队列满，读取暂停。

6.25 布局和序列化的差异

内存布局和磁盘/网络格式不必相同。内存中为了计算使用 SoA，网络上为了兼容和压缩使用长度前缀或 protobuf 类格式，磁盘上为了恢复使用稳定 manifest。直接把内存结构 dump 到磁盘，短期方便，长期会被 ABI、padding、字节序和版本拖垮。高质量系统会显式转换格式。

转换本身有成本，因此需要批处理和缓冲。一次转换一条记录，函数调用和分配成本高；一次转换一个 batch，可以顺序读取列、批量编码、批量校验。布局优化和 I/O 章节在这里相遇：好的内存布局也要服务序列化。

6.26 缓存友好的哈希表

哈希表是系统里常见的性能热点。链地址法每个 bucket 指向链表节点，容易指针追踪；开放寻址把元素放在连续数组中，探测时更 cache 友好，但删除、扩容和高负载因子要小心。Robin Hood hashing、SwissTable 类结构通过控制探测长度和元数据字节改善局部性。教材不要求实现这些成熟结构，但要让读者知道 `std::unordered_map` 不是唯一选择。

日志聚合如果使用 `std::unordered_map<std::string, std::int64_t>`，每个 key 可能分配字符串和节点，局部性较差。若先把 event type intern 成 `std::uint32_t`，再用数组或开放寻址表，热路径会更清楚。数据布局优化常常从“把复杂 key 转成紧凑 id”开始。

6.27 内存布局的调试工具

布局优化需要工具。可以用 `sizeof` 和 `alignof` 检查结构体大小；用编译器警告或工具查看 padding；用地址打印检查数组连续性；用 `perfstat` 观察 cache 和 TLB；用 heap profiler 看分配热点；用 sanitizer 检查越界和 use-after-free。布局优化如果没有调试工具，很容易变成猜测。

项目可以增加一个 layout report 命令，打印关键结构体大小、对齐、字段数量、batch 容量和估算字节数。这个命令不直接提升性能，但能让读者看到数据结构的物理形状。系统教材应让抽象类型落到字节。

6.28 布局迁移的测试策略

从 AoS 迁移到 SoA 时，测试要覆盖读写一致性。可以保留旧 AoS reference，随机生成记录，分别填充 AoS 和 SoA，执行同一聚合，比较结果。还要测试边界：空 batch、单元素、非对齐长度、错误行、超长字符串、冷热字段缺失。迁移期间如果保留双表示，要测试更新一个字段后另一个表示是否同步或明确无效。

性能测试则比较至少三条路径：完整记录访问、只访问热字段、错误报告访问。SoA 可能在热字段聚合上快，但完整记录访问变慢。报告要承认这种取舍，而不是只展示最好场景。

6.29 对齐分配器和实际收益

有些内核希望输入按 32 字节或 64 字节对齐。可以使用对齐分配器或平台 API 分配内存，也可以让容器内部保证对齐。对齐能减少跨 cache line 访问和满足某些 SIMD 要求，但它不是万能优化。若访问本来连续且 CPU 对非对齐访问支持良好，收益可能很小。若切片从对齐数组中间开始，起始地址仍可能不对齐。

项目中可以把对齐作为实验参数。生成同样数据，分别使用普通 vector、对齐缓冲、故意偏移一个元素的视图。比较自动向量化和手写 chunked 版本。报告中写清对齐如何保证、如何验证、对尾部 and 偏移如何处理。

6.30 Scratch buffer 管理

高性能算法常需要 scratch buffer，例如 scan 的 block sums、partition 的 counts、top-k 的候选、解析的临时位置列表。每次函数调用都分配 scratch，会引入分配器成本和内存波动。更好的做法是由运行时或调用者提供 scratch，或者每个 worker 复用本地 scratch。

Scratch 复用要清楚生命周期和大小。一个 worker 处理过大 batch 后，scratch 可能扩容到很大，后续小任务继续占用内存。可以设置最大保留容量，超过后释放或归还池。Scratch buffer 是数据布局和资源管理的交叉点。

6.31 Layout API 的封装

内部使用 SoA 或 AoSoA，不代表外部调用者要知道所有数组。可以提供 batch view，让热路径拿 span，冷路径通过 record id 获取完整记录。封装的目标是同时保护布局不变量和提供批量访问能力。单元素 getter 方便但可能慢；完全暴露 vector 容易破坏不变量。View 是折中。

Listing 6.11 批量视图封装布局

```
1 struct EventColumnsView {
2     std::span<const std::uint32_t> event_types;
3     std::span<const std::int64_t> values;
4 };
5
6 EventColumnsView event_columns(const ParsedLogHotBatch& batch) {
7     return EventColumnsView{batch.event_types, batch.values};
8 }
```

这个接口让聚合内核只看到自己需要的列，不依赖完整 batch 结构。后续如果 batch 从 vector 改成 mmap 或压缩列，接口仍有机会保持稳定。

6.32 布局章节总结

本章的主线是让数据形状服务访问形状。顺序热访问需要连续，字段子集热访问需要列式或冷热分离，高频并发写需要分片和对齐，稳定引用需要索引和 generation，静态图需要 CSR，动态更新需要增量结构。布局优化不是把所有东西改成一种格式，而是让每条热路径少搬无用字节、少追随机指针、少共享写。

设计布局时，先写访问矩阵，再写生命周期，再写并发写者，再写序列化边界，最后才写代码。这个顺序能避免“为了性能换结构，最后语义乱了”的常见错误。

6.33 布局决策表

选择布局时，可以先填表。字段是否总是一起访问？若是，AoS 可能清晰；若只访问少数字段，SoA 或冷热分离更好。对象是否需要稳定地址？若需要，考虑索引句柄或对象池；若不需要，连续数组更简单。是否高频插入删除？若是，CSR 这类紧凑静态结构可能不合适。是否要 SIMD？字段连续和对齐更重要。是否要序列化？内存布局和 wire format 要分开。是否多线程写？写者要分片或对齐。

这张表能避免布局争论变成风格争论。面向对象、面向数据、列式、池化都只是工具。真正决定工具的是访问矩阵、生命周期和并发语义。

6.34 案例：任务图的布局

任务图也需要布局。最直观表示是每个节点一个对象，每个对象有 `std::vector` successors。小图这样写清楚，大图会有很多小 vector 分配，遍历后继时指针跳转多。CSR 式任务图把节点和边分别放入连续数组，ready count 在节点数组中，successor id 在边数组中。执行时，worker 完成节点后顺序扫描后继范围，局部性更好。

缺点是动态修改困难。如果任务图在执行中不断新增边，CSR 重建成本高。可以使用两层结构：静态主图用 CSR，动态新增任务用小的 pending edge 列表，stage 结束后合并。这个设计和静态图加增量边、列式主图加 delta store 是同一模式。布局思维可以迁移到运行时元数据。

6.35 案例：模型权重的预告

虽然 AI 在第三册展开，但模型权重布局可以作为预告。权重通常是大块只读数据，推理时被反复扫描；量化权重可能按 block 存放 scale、zero point 和 int8 数据；算子希望连续读取并适配 SIMD。这个场景和本章的列式、AoSoA、对齐、冷热分离完全相关。第二册学习日志 batch 和矩阵 block，不是只为日志系统，而是为后续权重和 tensor 布局打基础。

这也说明布局知识具有迁移性。字段从 event type 换成 tensor value，原则不变：热数据连续，元数据分离，block 大小匹配 cache 和 SIMD，序列化格式与内存计算格式分开。

6.36 本章落到贯穿项目

Compute Systems Engine 可以在本章加入两个实验模块。第一个是记录批处理模块：用 AoS、SoA 和冷热分离三种布局统计 key，比较热字段扫描、错误记录访问和内存占用。第二个是 worker stats 模块：紧凑统计槽位与 `alignas(64)` 槽位对照，观察多线程写入扩展性。这两个模块分别训练“读局部性”和“写共享”两种布局能力。

项目代码不必一口气实现复杂 AoSoA 容器，但要把布局实验和 benchmark harness 接起来。每个布局版本必须使用同一份输入、同一个 reference 结果和同一组线程策略。否则布局差异会和算法差异混在一起。

6.37 从业务记录推导列式批次

很多布局优化失败，是因为一开始就讨论“要不要 SoA”，却没有先讨论业务记录到底怎样被使用。正确推导应从一个具体记录开始。假设日志记录包含时间戳、用户编号、事件类型、数值、原始行、解析错误、调试标签和来源文件。解析阶段需要原始行、来源文件和错误字段；聚合阶段只需要用户编号、事件类型和数值；错误报告阶段需要原始行和错误字段；调试查询阶段才需要标签。把这些字段全部放在一个大对象里，聚合阶段每处理一条记录都会把大量冷字段一起带进缓存。聚合代码看起来只读三个字段，硬件实际搬动的字节却远多于三个字段。

推导列式批次时，可以先保留最直观的 AoS reference。它的价值是语义清晰，便于测试新布局。然后为热路径建立列式批次：用户编号是一列，事件类型是一列，数值是一列，解析状态可以是一列，原始行和调试信息放入冷批次。热聚合只接收热批次视图，错误报告才接收冷批次视图。这样做并不是为了追求某个抽象名称，而是为了让热路径少搬无用字节，让编译器更容易生成顺序扫描，让硬件预取器更容易工作。

迁移时要特别注意记录编号。列式批次里第 i 个用户编号、第 i 个事件类型、第 i 个数值和冷批次第 i 个原始行必须表示同一条逻辑记录。这个关系可以通过统一长度、统一追加、统一过滤掩码和稳定 record id 保持。如果某个阶段删除错误记录，不能只从一列删除而忘记其他列。更稳的做法是先生成 selection vector，表示哪些记录有效，后续聚合按 selection vector 读取；等到批次稳定后，再做 compact。这样能避免列之间长度和顺序不一致。

列式批次还要考虑异常和边界。解析一行失败时，热列是否填默认值？错误列如何记录原因？聚合阶段是否跳过失败记录？如果错误记录很多，selection vector 会不会成为新的热点？如果原始行很长，冷批次是否保存完整字符串，还是保存文件偏移和长度？这些问题都属于布局设计，而不是外围细节。真正的工程布局必须同时说明成功路径、失败路径、调试路径和序列化路径。

6.38 稀疏结构和图布局

图和稀疏矩阵是布局思想最集中的场景。最直观的图表示是每个节点一个对象，每个对象保存一个后继列表。这种写法适合教学和小图，但在大图上会产生大量小分配。遍历一个节点的邻居时，程序先读节点对象，再读 vector 元数据，再跳到邻居数组；如果每个邻居数组都来自不同分配位置，cache 和 TLB 压力会很高。CSR 表示把所有边放到一条连续边数组里，再用 offset 数组记录每个节点的边范围。遍历节点 v 的邻居时，只需要顺序扫描 edges 从 `offsets[v]` 到 `offsets[v + 1]` 的区间。

CSR 的核心收益是把指针追踪变成连续区间扫描。连续扫描让硬件预取器能提前加载，也让多线程分片更容易。若节点范围按编号切分，每个 worker 扫描一段节点和对应边，读路径相对稳定。代价是动态更新困难。插入一条边可能需要移动后续大量边，删除也可能留下空洞。因此 CSR 更适合静态图、阶段性重建图或批处理图。动态图可以使用主 CSR 加 delta 边列表：主图负责绝大多数只读遍历，新增边暂存在小结构中，阶段边界再合并。

稀疏矩阵也类似。按行压缩适合行遍历和矩阵向量乘；按列压缩适合列遍历；块稀疏适合局部密集区域和 SIMD。选择格式时，不能只说“稀疏矩阵用 CSR”。要看内核按行访问还是按列访问，是否需要转置，是否需要并行写输出，非零分布是否倾斜，行长度差异是否很大。行长度极不均匀时，按行分配线程可能负载不均；可以按非零数量切分，也可以把长行拆成多个任务，但拆长行又会引入输出累加同步。布局、算法和并行调度在这里不能分开。

Compute Systems Engine 后续做 shuffle、join 和任务图时，都可以借用稀疏结构思维。任务图的 successors 数组就是边数组，任务的 successor range 就是 offset。分区后的 key 到记录位置，也可以看成倒排索引或稀疏结构。读者如果只把 CSR 当成图算法专用格式，就错过了更重要的思想：当大量对象之间的关系可以静态收集时，把关系压成连续区间，通常比每个对象挂一串指针更适合现代硬件。

6.39 布局实验报告怎样写

布局实验报告必须说明输入规模、记录大小、字段访问比例、线程策略、内存分配方式和测量指标。只报告“快了两倍”没有意义，因为读者不知道快在哪里。一个合格报告至少要写三组结果：完整对象访问、热字段访问、冷路径访问。SoA 可能让热字段聚合更快，但完整对象调试变慢；冷热分离可能减少缓存流量，但增加索引和封装复杂度。报告要把收益和代价都列出来。

指标上，时间只是起点。还应观察 cache miss、TLB miss、内存带宽、分配次数、峰值内存、输出是否一致。多线程布局实验还要看扩展曲线和 false sharing。若紧凑统计槽位在单线程下更快，对齐槽位在多线程下更快，这不是矛盾，而是写共享形状改变了瓶颈。单线程结论不能直接推到多线程，多线程结论也不能忽略内存占用。

报告还要防止把算法变化伪装成布局变化。AoS 版本如果使用普通循环，SoA 版本同时换成 SIMD 和分块，就无法判断收益来自哪里。更严谨的方式是逐步改变：先只改布局，算法保持一致；再加分块；再加 SIMD；再加线程。每一步都有 reference 结果和性能指标。这样得到的不是一组漂亮数字，而是一条可复查的因果链。

6.40 本章习题

习题与作业

习题一：为一个日志统计系统写访问矩阵。字段至少包括时间戳、用户 ID、事件类型、原始文本、解析错误、调试标签。热路径至少包括解析、按用户聚合、错误输出和调试查询。根据矩阵给出布局方案。

习题二：把一个指针链表设计改成索引数组设计。说明索引宽度、generation、删除策略、遍历局部性和序列化影响。不要只写“索引更快”，要写清生命周期如何保证。

习题三：设计一个冷热分离迁移计划。要求保留旧接口、建立 reference 测试、说明双写或缓存一致性策略，并给出何时删除旧布局的条件。

习题四：实现或设计 worker stats 的紧凑版和对齐版。报告每个版本的结构体大小、对齐、cache line 分布、写入线程和汇总成本。

实验：实现 AoS 和 SoA 两个版本的字段求和。比较顺序访问和随机访问。再构造两个线程分别写相邻字段和分离字段的场景，观察 false sharing。

第 7 章

SIMD、指令级并行与向量化

SIMD 让一条指令处理多个数据 lane，是现代 CPU 单核吞吐的重要来源。指令级并行则让多个互不依赖的操作同时在不同执行资源上推进。二者共同决定很多数值循环的上限。

7.1 从标量语义到向量语义

向量化不是把代码表面改成更复杂的形式，而是确认多个元素之间没有依赖，可以被同一条向量指令处理。数组逐元素加法容易向量化，因为每个元素独立；前缀和不容易直接向量化，因为后一个结果依赖前一个结果。

编译器自动向量化需要知道别名关系、对齐、循环边界和数据依赖。使用 `std::span` 表达连续区间有助于接口清晰，但编译器仍可能因为别名保守而放弃向量化。查看优化报告和汇编，比猜测更可靠。

判断循环能否向量化，要先写出每次迭代之间的关系。如果第 `i` 次迭代只读写第 `i` 个位置，通常容易向量化；如果第 `i` 次迭代读取第 `i-1` 次写出的结果，就存在循环携带依赖；如果每次迭代根据数据写入不同位置，就可能有冲突写；如果循环内部调用不可内联函数，编译器可能无法证明副作用边界。向量化首先是语义问题，其次才是指令问题。

7.2 别名和 restrict 思维

两个指针可能指向同一片内存时，编译器必须保守。例如 `a[i]=b[i]+c[i]` 中，如果 `a`、`b`、`c` 可能重叠，编译器要保证重叠情况下结果仍正确，这可能阻碍向量化。C++ 没有标准 `restrict` 关键字，但接口设计可以减少别名不确定性，例如使用不重叠容器、清晰文档和局部拷贝。

手写 SIMD 前，应该先让标量代码容易被编译器理解。清晰的循环边界、连续数据、少分支、少别名，是自动向量化的基本条件。

7.3 多累加器和延迟隐藏

即使不用 SIMD，多个独立累加器也能提高吞吐。单累加器形成长依赖链，每次加法依赖上一次结果；多个累加器把依赖链拆开，让乱序执行有更多可调度操作。SIMD 内核中也常见多个向量累加器，目的相同。

这就是并行归约在单线程内部也有优化空间的原因。先分成多个局部和，再合并，不仅适用于多线程，也适用于单核流水线。

Listing 7.1 多个累加器拆开依赖链

```
1 std::int64_t s0 = 0;
2 std::int64_t s1 = 0;
3 for (std::size_t i = 0; i + 1 < values.size(); i += 2) {
4     s0 += values[i];
5     s1 += values[i + 1];
6 }
7 const std::int64_t sum = s0 + s1;
```

这段代码的意义不是只用两个累加器，而是展示依赖链拆分。真实内核会根据执行端口、寄存器数量和向量宽度选择更多累加器。累加器太少隐藏不了延迟，太多会增加寄存器压力。

7.4 尾部处理和对齐

向量宽度通常不能整除数组长度，因此要处理尾部元素。常见方法是标量尾循环、掩码加载和补齐填充。对齐可以减少某些加载成本，但现代 CPU 对非对齐连续访问已经比较友好；真正要避免的是跨 cache line、跨页和随机访问。

尾部处理是性能代码里常见的正确性漏洞。主循环只处理完整向量宽度，剩余元素必须保证不丢、不重复、不越界。手写 intrinsics 时，尾部可以用标量循环、mask load/store 或提前 padding。选择哪种方式要看 ISA 支持、数据类型、越界读是否合法、输出是否允许覆盖 padding。教材示例宁愿保守清晰，也不能为了展示技巧隐藏边界条件。

常见误区

不要为了让 SIMD 主循环漂亮而忽略尾部。真实库里的很多错误，不发生在主循环，而发生在长度为 0、1、向量宽度减 1、刚好跨页或输入输出重叠的边界。

7.5 SIMD 与线程的关系

SIMD 和多线程不是替代关系。SIMD 提高单核吞吐，多线程使用多个核心。高性能程序通常先让单线程内核足够高效，再扩展到多线程。否则多线程只会复制低效内核，并更快撞上内存带宽。

7.6 数值语义

向量化可能改变浮点运算顺序，从而改变舍入误差。整数加法在不溢出的数学模型下更容易重排，浮点加法不严格满足结合律。编译器的 fast-math 选项会放宽数值语义，可能提高性能，也可能改变边界行为。工程中必须明确是否允许这种变化。

7.7 自动向量化到手写 SIMD

优化顺序通常是先写清晰标量代码，再查看编译器是否自动向量化，再决定是否手写 intrinsics。自动向量化失败时，先看原因：别名不明确、循环边界复杂、函数调用副作用、数据依赖、类型转换、未对齐访问还是成本模型认为不值得。只有当标量语义已经清楚、数据布局稳定、热点足够重要、自动向量化不足时，手写 SIMD 才值得。

手写 SIMD 的维护成本很高。它引入 ISA 分发、不同宽度实现、尾部处理、测试矩阵和可移植性问题。生产库通常会把通用标量 reference、自动向量化版本和手写 ISA 版本分开，并用同一组 correctness tests 验证。第二册讲 SIMD 的目标不是让读者到处写 intrinsics，而是让读者知道什么时候应该追求向量吞吐，什么时候瓶颈根本不在这里。

7.8 向量化和内存带宽

SIMD 提高的是每条指令处理的数据量，但它不能凭空增加内存带宽。对于流式数组求和，如果单线程已经被内存带宽限制，更宽的 SIMD 可能只减少指令数，不显著提高总时间。对于算术强度高、数据能复用的内核，SIMD 才可能接近计算峰值。判断前必须回到 Roofline：这个内核到底缺算力，还是缺数据。

7.9 向量化前先定义语义

SIMD 教材最常见的错误，是上来就讲寄存器宽度和指令名，却没有先定义循环语义。向量化的前提不是“循环长得像数组”，而是多个迭代之间可以被重排、合并或并行执行。一个循环能否向量化，首先由数据依赖、异常语义、别名关系、写入冲突和数值模型决定。

以数组加法为例，每个输出位置只依赖同一位置的两个输入，且不同位置互不影响。只要输入输出不发生破坏性重叠，这个循环天然适合向量化。前缀和则不同，位置 i 的输出依赖位置 $i-1$ 的结果，不能直接把所有元素当成独立 lane。直方图更复杂，每个元素会写入由数据决定的桶，不同 lane 可能写同一个桶，形成冲突写。图遍历则同时有随机读、分支和不均匀工作量。把这些循环都叫“处理数组”，对性能分析没有帮助。

因此本章的学习顺序是：先写出串行语义，再判断迭代独立性，再考虑编译器自动向量化，再考虑手写 SIMD。若语义不允许重排，强行使用 SIMD 只会写出错误程序。若语义允许重排但编译

器没能向量化，应先找出失败原因，而不是直接责怪编译器。

7.10 baseline：标量数组变换

看一个最简单的变换：把两个数组相加写到输出。这个例子很朴素，但它包含自动向量化需要的几乎所有条件：连续内存、线性索引、无循环携带依赖、固定类型和简单表达式。

Listing 7.2 baseline：标量数组加法

```
1 void add_arrays_scalar(std::span<const float> left,
2                       std::span<const float> right,
3                       std::span<float> output) {
4     const std::size_t count =
5         std::min(left.size(), std::min(right.size(), output.size()));
6     for (std::size_t i = 0; i < count; ++i) {
7         output[i] = left[i] + right[i];
8     }
9 }
```

这段代码适合作为 reference，但它没有明确说明三个 span 是否重叠。若 `output` 和 `left` 指向同一数组，语义仍然可能正确，因为每个位置先读后写同一位置；若 `output` 是 `left` 向后偏移一个元素的重叠视图，写 `output[i]` 可能影响后续 `left[i+1]` 的读取。编译器如果不能排除这种情况，就可能生成运行时别名检查，或者放弃某些向量化策略。

工程接口要尽量表达不重叠语义。可以在文档中声明三个区间不得破坏性重叠，可以在 debug 版本检查地址范围，也可以把输入和输出放在不同所有者对象中。C++ 标准没有可移植的 `restrict` 关键字，因此高质量 API 设计比某个编译器扩展更重要。

7.11 自动向量化报告

自动向量化不应该靠猜。Clang 和 GCC 都能输出优化报告，告诉你某个循环是否向量化、为什么失败、使用了多宽向量、是否做了运行时别名检查。不同版本选项略有差异，报告中应记录编译器版本和命令。一个报告可以包含如下问题：循环是否被 vectorized；vector width 是多少；interleave count 是多少；失败原因是依赖、调用、副作用、成本模型还是不支持的数据类型。

如果报告说循环未向量化，下一步不是立刻手写 intrinsics，而是修改源码让语义更清楚。把复杂函数调用移出循环，把热字段变成连续数组，把输出位置改成唯一写入，把分支拆成内部区域和边界区域，把可能别名的输入输出拆开。很多时候，自动向量化失败不是编译器弱，而是源码没有给足证明条件。

优化报告还要和汇编对照。报告说向量化，不代表生成了你预期的高效代码；报告说未向量化，也可能编译器通过其他方式优化。查看汇编时要关注是否出现向量 load/store、向量 add/mul/fma、循环展开、尾部处理和运行时别名检查。读汇编的目标不是背指令名，而是验证执行形态。

7.12 循环携带依赖和重写

循环携带依赖是向量化的主要障碍。最典型的是单累加器求和：每次迭代都依赖上一次 sum。编译器可以使用向量归约，把多个 lane 局部累加后水平合并，但这要求它确认重排语义允许。整数在有符号溢出语义、浮点舍入语义和 fast-math 选项下会有不同结果。

很多循环可以通过改写暴露独立性。例如计算差分数组：

Listing 7.3 可向量化的相邻差分

```
1 void adjacent_difference_kernel(std::span<const std::int32_t> input,
2                               std::span<std::int32_t> output) {
3     if (input.size() < 2 || output.empty()) {
4         return;
5     }
```



```

6   const std::size_t count = std::min(input.size() - 1, output.size());
7   for (std::size_t i = 0; i < count; ++i) {
8       output[i] = input[i + 1] - input[i];
9   }
10 }

```

这个循环读相邻元素，但每个输出位置独立，通常可以向量化。相反，前缀和每个输出依赖前一个输出，不能直接按同样方式处理。判断时不要只看“是否读相邻元素”，要看“是否读本轮之前写出的结果”。

7.13 尾部处理的三种策略

尾部处理有三种常见策略。第一是标量尾循环：主循环处理完整向量宽度，剩余元素用普通循环。它清晰、可移植、适合教学。第二是 mask load/store：使用掩码只处理有效 lane，适合 AVX-512、SVE 等支持较好的 ISA。第三是 padding：让数组末尾多出安全空间，主循环可以越过逻辑长度但不越过分配边界。这适合内部数据结构，不适合任意用户传入 span。

尾部策略还影响异常和内存安全。对输入做越界读即使结果被掩掉，在 C++ 语义上也可能是未定义行为；跨页越界读还可能触发 page fault。生产库不能随便假设尾部后面有安全字节，除非容器明确保证 padding。教材示例应优先使用标量尾循环，等语义清楚后再讨论 mask。

Listing 7.4 标量尾循环的结构

```

1 constexpr std::size_t SIMD_WIDTH = 8;
2
3 void add_arrays_chunked(std::span<const float> left,
4                         std::span<const float> right,
5                         std::span<float> output) {
6     const std::size_t count =
7         std::min(left.size(), std::min(right.size(), output.size()));
8     std::size_t i = 0;
9     const std::size_t vector_limit = count / SIMD_WIDTH * SIMD_WIDTH;
10    for (; i < vector_limit; i += SIMD_WIDTH) {
11        for (std::size_t lane = 0; lane < SIMD_WIDTH; ++lane) {
12            output[i + lane] = left[i + lane] + right[i + lane];
13        }
14    }
15    for (; i < count; ++i) {
16        output[i] = left[i] + right[i];
17    }
18 }

```

这段代码仍是标量 C++，不是 intrinsics，但它展示了向量主循环和尾循环的结构。编译器可能把内部 lane 循环向量化，也可能完全展开。教学中先写这种结构，有助于读者理解手写 SIMD 版本会怎样组织。

7.14 多版本派发和可移植性

手写 SIMD 不能只为一台机器写。x86 上有 SSE、AVX、AVX2、AVX-512；ARM 上有 NEON 和 SVE；不同机器支持的指令集不同。生产库通常需要多版本派发：保留标量 reference，再提供一个或多个 ISA 优化版本，运行时根据 CPU 能力选择。编译期只为当前机器生成一种版本，部署到其他机器可能无法运行。

多版本派发也有成本。每个版本都要测试，尾部处理要一致，数值结果要在允许误差内，构建系统要管理不同编译选项，调试时要知道实际跑的是哪个版本。对于不够热的循环，手写多版本不值得。自动向量化和标准库算法可能已经足够。

Compute Systems Engine 的第一阶段不需要手写所有 ISA 版本，但应保留接口层次：reference 标量版本、自动向量化友好版本、未来可能的 ISA 版本。这样项目不会因为一个手写内核绑死在某台 CPU 上。

7.15 Gather、scatter 和压缩写

SIMD 最喜欢连续 load/store。gather 从多个不连续地址加载，scatter 向多个不连续地址写入，通常比连续访问贵得多。它们能表达图遍历、哈希表查询和稀疏计算的一部分并行性，但不能把随机访问变成顺序访问。很多时候，重排数据、排序索引、分块处理比直接使用 gather 更有效。

压缩写常见于过滤：只把满足条件的元素写到输出。朴素分支版本会对每个元素判断并 push；并行版本需要先统计每个块保留多少，再扫描偏移，再写输出。某些 SIMD ISA 支持 compress store，但依然需要处理输出位置和尾部。过滤类问题把 SIMD、scan 和分区连接起来，因此第 13 章会专门展开。

7.16 分支、掩码和数据分布

向量化常把分支变成掩码。对于简单条件选择，例如 `output[i]=value>threshold?a:b`，掩码很自然。对于复杂分支，如果两个分支都需要大量计算，把它们都执行再用掩码选择可能更慢。优化要看数据分布：如果分支高度可预测，标量分支可能很快；如果分支随机且分支体短，掩码更可能有收益。

高性能内核常把数据预先分组，让同一批数据走同一条路径。日志解析可以按记录类型分桶，图算法可以按顶点度数分层，数据库执行器可以把 selection vector 传给后续算子。这样做的目的不是为了形式上“向量化”，而是让控制流和数据流更规则。

7.17 数值稳定性和重排

浮点 SIMD 需要更严肃的数值语义。浮点加法不满足数学结合律，改变合并顺序会改变舍入。向量化归约通常会改变加法顺序，因此结果可能与串行逐项求和不同。对于科学计算，这可能需要误差界、Kahan 求和、pairwise sum 或固定合并树；对于图形和机器学习某些场景，误差容忍度可能更高。教材不能只说“结果有一点不同”，要让读者决定业务是否允许。

整数也不是完全无忧。C++ 有符号整数溢出是未定义行为，编译器可以基于“不会溢出”做优化。若业务需要模 32 位或模 64 位行为，应使用无符号类型或明确的溢出处理。系统代码中的位宽类型和溢出语义要写清楚，否则优化和正确性会互相踩踏。

7.18 本章落到贯穿项目

Compute Systems Engine 在本章应加入三个版本的数组内核。第一，reference 标量版本，语义最清楚，用于校验。第二，自动向量化友好版本，数据连续、边界清晰、少别名、少分支，并输出编译器优化报告。第三，实验性 chunked 版本，显式展示主循环和尾部处理。项目不必马上手写 AVX 或 NEON，但要把 correctness tests 和 benchmark harness 准备好。

实验报告要记录：编译器版本、优化选项、目标 CPU、是否启用 fast-math、循环是否向量化、输入规模、对齐方式、尾部长度、结果校验和性能指标。若向量化版本没有更快，要分析是内存带宽、尾部、别名检查、前端压力还是数据太小。不要只写“SIMD 没用”。

7.19 Linux 和工具入口

本章更适合读编译器报告和反汇编，而不是直接读 Linux 内核源码。但 Linux 工具仍有用：`perfstat` 可以观察 instructions、cycles、IPC、cache miss；`perfrecord` 可以确认热点是否目标循环；`lscpu` 可以记录 CPU flags；`objdump` 或编译器汇编输出可以确认指令形态。报告中应把工具输出和源码结构对应起来。

如果要读源码，可以从成熟库开始，例如 C++ 标准库实现中的算法派发、数学库中的多版本内核、数据库或向量搜索库的 SIMD 路径。源码阅读重点不是复制 intrinsics，而是看它如何组织 reference、ISA dispatch、尾部、对齐、测试和 fallback。

7.20 本章习题

习题与作业

习题一：选择三个循环：数组加法、前缀和、直方图。分别写出它们的迭代依赖、写入位置和向量化难点。说明哪个适合直接向量化，哪个需要算法重写，哪个需要分阶段处理。

习题二：为 `add_arrays_scalar` 设计 debug 版别名检查。说明哪些重叠是安全的，哪些重叠会破坏语义。不要依赖非标准 `restrict` 作为唯一方案。

习题三：运行或设计一次自动向量化报告实验。记录编译器命令、报告内容、是否发生 vectorization、失败原因和汇编证据。若当前环境不支持某个报告选项，写出替代观察方式。

习题四：为浮点归约写误差说明。比较串行求和、四累加器求和、pairwise 求和和 fast-math 下的可能差异。说明你的业务场景是否允许改变合并顺序。

7.21 手写 SIMD 的接口边界

若最终确实需要手写 SIMD，第一步不是写 intrinsics，而是设计接口边界。一个好的 SIMD 内核通常有四层。第一层是公开 API，接收 span、配置和语义选项。第二层是 portable reference，用标准 C++ 实现，作为 correctness oracle。第三层是 dispatch，根据 CPU 特性选择实现。第四层是 ISA-specific kernel，只处理已经检查过的输入和尾部策略。

这种分层能把复杂性关在局部。公开 API 不暴露 AVX 或 NEON 类型；reference 不依赖平台；dispatch 只处理选择；具体内核只处理性能关键路径。若把 intrinsics 直接散落在业务代码里，后续测试、移植、调试和回退都会变得困难。

Listing 7.5 SIMD 内核的分层接口草图

```
1 enum class KernelBackend : std::int32_t {
2     Scalar,
3     AutoVectorized,
4     NativeSimd,
5 };
6
7 struct KernelOptions {
8     KernelBackend backend = KernelBackend::AutoVectorized;
9     bool allow_fast_math = false;
10 };
11
12 void transform_batch(std::span<const float> input,
13                     std::span<float> output,
14                     const KernelOptions& options);
```

这里没有把某个指令集写进公开接口，是为了让调用者表达语义需求，而不是绑定实现。运行时可以根据平台选择 AVX2、AVX-512、NEON 或标量 fallback。测试可以强制选择 Scalar 和 AutoVectorized，比较结果。若某个 ISA 版本出问题，可以临时关闭，而不影响业务 API。

7.22 运行时派发的测试矩阵

多版本派发的测试不只是“默认路径跑通”。必须强制每个 backend 跑同一组 correctness tests。输入要覆盖长度为 0、1、向量宽度减一、正好向量宽度、向量宽度加一、大数组、非对齐起点、输出和输入不同数组、允许的重叠或禁止重叠场景。浮点内核还要覆盖 NaN、Inf、负零、极大值和极小值，除非 API 明确不支持这些值。

性能测试则要区分小输入和大输入。小输入时 dispatch 成本、函数调用和尾部处理可能占主导；大输入时内存带宽或计算吞吐主导。若一个 SIMD 版本只在超大输入上快，API 可以设置阈值，小输入走标量路径。很多成熟库都有这种 size threshold，因为 SIMD 启动成本不是零。

7.23 对齐策略的工程现实

早期 SIMD 教材常强调对齐加载。现代 CPU 对连续非对齐访问已经更友好，但对齐仍然有价值：减少跨 cache line 或跨页访问，便于某些 ISA，简化内部块处理。工程上常见做法是让容器或分配器提供对齐保证，但公开 API 仍要能处理普通 span。内部可以先处理到对齐边界，再进入对齐主循环，最后处理尾部。

不过，对齐优化要防止过度复杂化。若内核主要受内存带宽限制，复杂的对齐前缀可能收益很小；若输入来自用户任意切片，对齐主循环比例可能不高；若需要跨平台，某些 ISA 的对齐要求不同。测量应覆盖对齐和非对齐输入，而不是只在理想对齐数组上报告性能。

7.24 水平归约和 lane 间通信

SIMD lane 之间并非完全免费通信。逐元素 map 每个 lane 独立，最适合 SIMD；归约需要把多个 lane 的 partial 合并成一个标量，涉及 horizontal add、shuffle 或树形 lane 合并。不同 ISA 对水平操作支持不同，成本也不同。因此向量归约通常会使用多个向量累加器，先在向量寄存器里积累很多元素，最后再做一次水平合并。

这个模式与线程本地 partial 类似。lane 内先局部累加，最后水平合并；线程内先本地 partial，最后线程间合并；节点内先 map-side combine，最后跨节点 reduce。层次化归约不是某一层技巧，而是贯穿硬件 lane、线程、NUMA 和集群的通用结构。

7.25 Vector-friendly 数据结构

不是所有数据结构都适合 SIMD。链表、树节点和虚函数对象通常难以向量化，因为地址不连续、分支多、类型不统一。若热路径需要 SIMD，应把数据组织成 vector-friendly 形状：连续数组、SoA、AoSoA、紧凑索引、固定宽度字段和分离冷数据。第 6 章的数据布局不是 SIMD 之前的附属内容，而是 SIMD 能否发挥作用的前提。

以日志解析为例，原始文本是变长字符串，很难直接 SIMD 化完整业务逻辑；但某些子问题可以向量化，例如扫描分隔符、统计字符类别、批量检查 ASCII 范围。工程中常把复杂解析拆成多个阶段：先用 SIMD 找结构位置，再用标量状态机处理复杂字段。不要要求 SIMD 解决整个业务函数，而要找出规则、重复、数据并行的子内核。

7.26 本章的反例清单

判断 SIMD 是否值得时，可以先看反例。第一，输入很小，dispatch 和尾部成本超过收益。第二，内核受内存带宽限制，SIMD 只减少指令数不减少字节。第三，数据结构是随机指针追踪，gather 成本高且预取困难。第四，分支复杂且各分支工作量大，掩码执行会做大量无用工作。第五，数值语义要求逐位一致，重排空间受限。第六，代码需要跨很多平台维护，但热点不够热。

这些反例不是让读者不用 SIMD，而是让优化更有目标。真正应该优先 SIMD 化的，是热、规则、数据连续、迭代独立、算术强度足够或函数调用成本可摊薄的内核。Compute Systems Engine 的 map、normalize、简单解析扫描、局部归约都可以成为候选；全局队列、锁等待和分布式重试不是 SIMD 能解决的问题。

7.27 自动向量化的前置条件

自动向量化不是编译器魔法，它需要源码表达出足够清楚的循环形状。典型前置条件包括：循环边界可分析，迭代之间没有无法证明的依赖，内存访问尽量连续或固定步长，分支能转成掩码或被分离，函数调用能内联或被证明无副作用，输入输出别名关系清楚。任何一个条件不满足，编译器可能保守放弃。

许多向量化失败不是因为编译器弱，而是接口太含糊。一个循环写 `output[i]`，同时读 `input[i+1]`，如果 input 和 output 可能重叠，编译器必须考虑写入影响后续读取。一个循环内部调用虚函数，编译器很难知道函数是否访问全局状态或抛异常。一个循环每轮可能 push 到 vector，输出位置不是简单函数，也不容易向量化。软件工程师要做的是把数据依赖变成编译器能看懂的结构。

向量化报告要成为开发流程的一部分。GCC、Clang 和 MSVC 都有不同形式的优化报告。读

报告时要关注 “not vectorized” 的原因，而不是只看成功。常见原因如 unsafe dependent memory operations、cannot identify array bounds、call in loop、control flow not understood。把这些原因翻译回源码，就能指导接口和布局改造。

7.28 Selection vector

数据库和批处理系统常使用 selection vector 表示哪些行通过过滤。它不是立刻把数据压缩到新数组，而是保存通过过滤的行号或位图，后续算子根据 selection 处理。这样可以避免每个 filter 都材料化输出，也能让多个过滤条件组合。Selection vector 连接了 SIMD 掩码、filter、compaction 和数据库执行。

若选择率很高，位图或掩码可能更紧凑；若选择率很低，索引数组只保存命中位置更高效。若后续算子需要随机访问原数组，索引数组会引入 gather；若后续先 compaction，再连续处理，可能更适合 SIMD。选择哪种表示，要看后续管线，而不只是当前 filter。

Listing 7.6 选择向量的接口形态

```
1 struct SelectionVector {
2     std::vector<std::uint32_t> selected_rows;
3 };
4
5 SelectionVector select_positive(std::span<const std::int32_t> values) {
6     SelectionVector selection;
7     selection.selected_rows.reserve(values.size());
8     for (std::uint32_t i = 0; i < static_cast<std::uint32_t>(values.size()); ++i) {
9         if (values[i] > 0) {
10             selection.selected_rows.push_back(i);
11         }
12     }
13     return selection;
14 }
```

这个版本是标量 reference。优化版可以用 SIMD mask 找出命中 lane，再把命中位置追加到 selection。无论实现如何，语义是稳定的：selection 中的行号按输入顺序排列。这个稳定性会影响后续 join、聚合和测试。

7.29 SIMD 和解析

文本解析看起来不适合 SIMD，因为字段变长、状态复杂、错误路径多。但解析中有一些子任务很适合向量化：查找换行符、逗号、冒号、引号；判断字节是否在 ASCII 数字范围；批量寻找非法字符；扫描固定格式字段。成熟 JSON、CSV 和日志解析器常先用 SIMD 找结构字符，再用标量状态机处理语义。

这种分层很重要。不要试图把完整解析器一次性 SIMD 化。先找出规则、重复、无依赖的子内核，例如扫描一块 64 字节里的分隔符位置。输出可以是 bit mask 或 position list，后续标量代码消费这些结构位置。这样 SIMD 处理的是字节分类，业务语义仍由清晰状态机负责。

7.30 Saturating、rounding 和溢出语义

某些 SIMD 指令提供饱和加法、舍入转换、最小最大、绝对值和打包。这些语义和普通 C++ 运算不一定相同。例如图像处理常需要 8 位饱和加法，超过 255 时保持 255；普通无符号 8 位加法则按模 256 回绕。若手写 SIMD 时不明确溢出和饱和语义，结果会和 reference 不一致。

量化和 AI 推理中，这个问题更重要。虽然 AI 放在第三册，本册仍要先建立概念：向量指令不仅影响速度，也可能携带特定数值语义。任何使用特殊指令的内核，都必须有 reference 和边界测试，覆盖最大值、最小值、负数、舍入边界和溢出边界。

7.31 混合精度和类型转换

SIMD 内核经常要在不同类型之间转换，例如 `std::int8_t` 到 `std::int32_t` 累加，`float` 到 `std::int32_t` 量化，`std::uint16_t` 到 `float` 计算。类型转换可能成为瓶颈，也可能改变精度和舍入。内核设计要把 `load`、`extend`、`compute`、`narrow`、`store` 的步骤写清楚。

混合精度优化必须有误差或溢出说明。若把 32 位累加改成 16 位累加，可能溢出；若把 `double` 改成 `float`，可能误差扩大；若把 `float` 转 `int`，舍入模式要定义。高性能不是牺牲语义的借口。

7.32 内核融合和寄存器压力

把多个循环融合成一个循环，可以减少内存读写。例如先 `map` 再 `filter`，若分开执行，需要写中间数组；融合后可以直接处理并写输出。但融合会增加循环体复杂度、分支、寄存器占用和代码大小。寄存器压力过高时，编译器会 `spill` 到栈，反而变慢。前端压力也可能增加。

因此内核融合要测量。适合融合的阶段通常是简单、连续、没有复杂依赖的 `map/filter`。`Reduce`、`sort`、`join`、`shuffle` 这类 `pipeline breaker` 不容易完全融合。数据库执行器中的 `vectorized batch` 模型就是折中：每个 `batch` 内尽量局部融合，但在需要重排或聚合的地方材料化。

7.33 SIMD 与线程并行的顺序

优化时常问：先做 SIMD 还是先多线程？稳妥顺序通常是先写标量 `reference`，再让单线程内核数据布局和自动向量化友好，然后再多线程。原因是多线程会引入调度、NUMA、同步和带宽变量，容易掩盖单核问题。单线程内核若已经受内存带宽限制，多线程能提高总带宽直到平台上限；单线程内核若受依赖链限制，多线程也许掩盖问题但不解决单核效率。

当然，某些场景先多线程更实用，例如业务任务天然独立且单个任务不热。工程选择取决于热点和成本。教材建议读者至少保留单线程优化报告，因为它是理解多线程扩展曲线的基础。

7.34 本章新增验收

本章在 `Compute Systems Engine` 中的验收应包括：每个内核有 `scalar reference`；自动向量化报告保存到实验记录；强制标量和自动向量化版本输出一致；尾部长度覆盖从 0 到向量宽度减一；对齐和非对齐输入都测试；浮点内核写明误差策略；小输入和大输入分别 `benchmark`；默认路径可回退到 `scalar`。

这些验收比手写一段漂亮 `intrinsics` 更重要。没有 `reference` 和回退，SIMD 内核很难维护；没有尾部测试，边界 `bug` 很常见；没有误差策略，浮点结果争议无法解决。

7.35 案例：ASCII 数字扫描

日志解析里常见子问题是判断一段字节是否全是 ASCII 数字。标量版本逐字节检查，遇到非数字退出。SIMD 版本可以一次加载多个字节，分别比较是否大于等于字符 0 且小于等于字符 9，再把结果形成 `mask`。若 `mask` 表示所有 `lane` 都满足，就整块通过；否则找到第一个失败位置。

这个案例适合讲 SIMD，因为它规则、数据连续、每个字节独立，输出可以是一个位置或布尔值。它也展示了边界：输入长度不是向量宽度倍数时要处理尾部；跨页加载要小心；错误位置需要从 `mask` 转成索引；若字符串很短，标量可能更快；若输入大多第一字节就失败，标量早退出可能更好。SIMD 优化要看数据分布。

7.36 案例：分隔符位置列表

CSV 或日志解析还需要找逗号、空格、制表符或换行。SIMD 可以批量比较一块字节是否等于分隔符，生成 `bit mask`，再把 `set bit` 转成位置列表。位置列表就是一种 `selection vector`。后续标量解析器根据位置切分字段。这种结构把“找结构”与“解释语义”分开，常见于高性能解析器。

位置列表也有容量问题。若一块中分隔符很多，输出位置多；若分隔符很少，输出少。可以先用固定小数组保存一个 `block` 的位置，再追加到 `batch-level vector`。追加时要避免每个位置都抢全局锁，通常每个 `worker` 有本地位置缓冲，最后合并。

7.37 案例：向量化归一化

另一个适合 SIMD 的内核是数组归一化：`output[i]=(input[i]-mean)*inv_std`。它是纯 map，输入输出连续，迭代独立。若 `mean` 和 `inv_std` 已经计算好，循环很适合自动向量化。若 `mean` 也要从 input 求出，则整个算法分成 reduce 求和、计算 `mean`、map 归一化三阶段。这里可以看到 map 和 reduce 的组合。

归一化还展示浮点语义。求 `mean` 的归约顺序会影响结果，map 阶段的 fused multiply-add 也可能改变舍入。若业务要求严格复现，需要固定归约顺序和编译选项；若允许误差，需要写明 `tolerance`。一个看似简单的 SIMD map，也可能被前置 reduce 的数值语义影响。

7.38 SIMD 性能报告模板

SIMD 实验报告可以按固定模板写。第一，内核语义和 reference。第二，数据布局和对齐。第三，循环依赖和向量化理由。第四，编译器报告或汇编证据。第五，输入规模和分布。第六，正确性和误差。第七，性能结果和瓶颈归因。第八，反例和回退策略。

例如 ASCII 扫描报告应写：输入是随机 ASCII、全数字、首字节错误、末字节错误四种；输出是第一个非数字位置；reference 是标量逐字节；SIMD 版本对尾部使用标量；小于某个长度走标量；结果完全一致。这样报告才像工程文档，而不是只贴一段 `intrinsics`。

7.39 批大小和 SIMD

SIMD 喜欢批量数据。若每次函数只处理一条记录，向量化很难摊薄固定成本；若把几千条记录放入 batch，很多字段可以列式处理，SIMD 才有空间。Batch 大小影响 SIMD、cache 和延迟。太小，向量利用率低；太大，cache 压力和尾延迟高。第 6 章的 columnar batch 是第 7 章 SIMD 的前提。

日志解析可以先按字节块扫描分隔符，再按记录 batch 解析字段，再按列做聚合。每一层 batch 大小可能不同。字节扫描适合较大连续块，字段解析适合记录 batch，聚合适合 event type 列。不要强求所有阶段同一个 batch 大小。

7.40 SIMD 和错误处理

错误处理会影响 SIMD 内核设计。若 SIMD 扫描发现某个 lane 非法，是立即返回错误，还是记录位置继续扫描？立即返回适合验证函数；记录位置适合批处理解析，因为一个 batch 里可能有多条错误记录，但仍希望处理其他记录。选择不同，输出结构也不同。

批处理系统通常把错误记录到 side buffer，不让少数错误打断整个向量内核。这样热路径保持规则，错误路径后处理。但如果错误表示输入损坏到无法确定边界，例如长度字段非法，必须停止当前消息。错误语义决定 SIMD 控制流。

7.41 跨平台 SIMD 的教学策略

本书不把某个 ISA 作为唯一答案，是因为读者设备可能是 x86、ARM 手机或云服务器。教学策略是先讲循环依赖和数据布局，再讲自动向量化，再讲抽象的 lane、mask、gather、horizontal reduce，最后才读具体 ISA。这样读者即使不写 AVX，也能理解为什么某个循环适合或不适合向量化。

第三册 AI 推理会更深入具体指令，但第二册先建立可迁移模型。现代系统工程师不应只会背某个指令名，而要能判断数据流是否值得向量化，如何设计 reference 和 fallback。

7.42 SIMD 章节总结

SIMD 的核心是把同一种操作作用到多个 lane 上。它要求数据规则、依赖清楚、输出位置明确、数值语义可接受。它经常和数据布局、batch、selection vector、scan、partition 一起出现，而不是孤立出现。能自动向量化就先让编译器做；需要手写时，先设计 reference、dispatch、tail、对齐和测试。SIMD 是系统优化的一层，不是替代测量和算法设计的捷径。

7.43 SIMD 决策表

判断一个内核是否值得 SIMD 化，可以问：输入是否连续，迭代是否独立，输出位置是否固定，分支是否简单，数据量是否足够大，数值重排是否允许，目标平台是否稳定，是否已有自动向量化，是否有 scalar fallback。如果多个答案是否定，先改布局或算法。若答案大多肯定，再考虑编译器报告和手写优化。

这个决策表会在第三册 AI 算子中继续使用。矩阵乘、卷积、量化反量化适合 SIMD；复杂控制流和随机图遍历不一定适合。先判断形状，再选指令。

7.44 案例：向量化失败后的重构

假设编译器报告一个循环因为函数调用无法向量化。循环每轮调用 `classify(record)`，函数内部读取多个字段并返回类别。重构可以分两步：先把 `classify` 拆成只依赖热字段的纯函数，并确保可内联；再把记录字段从 AoS 改成列式，让 `classify` 处理连续数组。若仍有复杂分支，可以先按类型分桶。这个过程展示了 SIMD 优化不是最后一行指令，而是接口、布局和控制流共同重构。

重构后仍要保留原 reference。若 `classify` 涉及错误处理、字符串或时间解析，不应为了向量化改变语义。可以只向量化其中的简单子条件，把复杂记录留给标量慢路径。高质量 SIMD 优化常常是混合路径，而不是全量替换。

7.45 从依赖图看向量化

判断一个循环能否向量化，最直接的方法是画出一轮迭代依赖下一轮迭代的地方。若第 i 次迭代只读 `input[i]` 并写 `output[i]`，各次迭代相互独立，向量化自然。若第 i 次迭代要读上一轮写出的 `output[i-1]`，循环就有跨迭代依赖，普通 SIMD 很难直接处理。若依赖只是归约，例如所有元素累加到一个总和，可以把单一依赖链改成多累加器或树形归约。若依赖是状态机，例如字符串转义、括号嵌套或变长编码，通常要把能规则化的子任务拆出来。

依赖图还能解释为什么有些循环看起来简单却不能向量化。指针别名会让编译器不确定输入和输出是否重叠；函数调用会让编译器不知道它是否有副作用；异常和边界检查会让控制流变复杂；写入位置由数据决定时，多个 lane 可能写到同一位置。解决方法不是盲目加编译选项，而是改变接口：使用 `span` 表达连续范围，拆出纯函数，明确输入输出不重叠，把数据相关写入改成 `count`、`scan`、`scatter` 三阶段。

这也说明 SIMD 和并行算法是同一思想在不同粒度上的表现。SIMD lane 需要独立，线程任务也需要独立；SIMD 需要 mask 处理条件，线程并行需要 selection 和 partition 处理分支；SIMD 的 horizontal reduce 和线程归约都要面对结合性、顺序和误差。学 SIMD 时不要只盯着指令，应把它看成最细粒度的数据并行模型。

7.46 Gather 和 scatter 的真实代价

SIMD 并不要求所有数据都连续。有些 ISA 支持 gather 从多个地址收集 lane，也支持 scatter 把 lane 写到多个地址。但 gather 和 scatter 通常比连续 load 和 store 更贵，而且更容易受 cache miss、TLB miss 和地址生成限制影响。它们解决了表达能力，不等于解决了性能问题。一个随机索引数组上的 gather，可能只是把多个标量 miss 捆在一起，不能让内存系统变成连续访问。

因此，遇到随机访问时要先问能不能重排数据。若可以按 key 排序、按 bucket partition、把热点字段压成连续数组，通常比直接 gather 更可靠。若随机访问不可避免，可以批量处理多个请求，增加内存级并行，或者把索引按页和 cache line 分组，降低完全随机程度。只有在数据形状确实无法改变、且 gather 版本经过测量有收益时，才应把 gather 当成核心优化。

Scatter 更要小心写冲突。多个 lane 可能写到同一输出位置，语义是最后一个赢、累加、取最大还是报错？若是累加，就需要冲突检测、原子或分阶段归约；若是写位置唯一，最好用 scan 或 partition 先分配不重叠区间。数据相关写入是 SIMD 和线程并行共同的难点，不能因为用了向量指令就忽略输出语义。

7.47 选择标量回退的工程判断

高质量 SIMD 代码一定要有标量回退。回退不是失败，而是工程边界。短输入、未对齐尾部、罕见错误路径、平台不支持目标指令、调试模式和语义复杂路径，都可能走标量。标量 reference 让测试清楚，也让平台覆盖更稳。没有回退的 SIMD 内核容易把一个优化点变成全系统移植风险。

回退策略要写进接口。可以在运行时根据 CPU 能力选择 AVX、SSE、NEON 或 scalar；也可以在编译期生成不同目标；还可以让小输入直接走 scalar，避免向量准备成本。选择逻辑必须可测试，不能只在开发机器上跑最快路径。报告应包含 scalar、自动向量化、手写 SIMD 三组结果，而不是只展示最好版本。

回退还帮助定位 bug。若 SIMD 输出和 scalar 不一致，先缩小输入，固定随机 seed，记录 lane mask、尾部长度和第一个不同位置。很多 SIMD bug 出在尾部、符号扩展、饱和和舍入。标量 reference 是这些 bug 的锚点。性能工程不能脱离正确性锚点，否则优化越多，系统越难信任。

实验：用编译器优化报告观察一个数组加法循环是否向量化。再写单累加器和四累加器求和，比较性能。记录编译选项和 CPU 指令集。

第 8 章

典型计算内核模式

高性能计算里反复出现一些内核模式：map、reduce、scan、stencil、矩阵块计算、直方图、排序、图遍历和队列驱动任务。掌握这些模式，比记住某个库函数更重要。

8.1 Map 与 Reduce

Map 对每个元素独立变换，通常容易并行和向量化。Reduce 把多个元素合成一个结果，存在合并顺序问题。整数求和通常结合律明确，浮点求和则会因为舍入误差导致不同合并顺序产生不同结果。

并行 reduce 的基本形态是每个线程处理一段输入，写入线程本地 partial，然后合并。这个结构减少共享写，也让每个线程顺序扫描自己的数据块。Compute Systems Labs 的 `parallel_sum` 就是这个最小模型。

Reduce 的关键不是“把加法拆给多个线程”这么简单，而是确定合并操作是否满足结合律、是否允许重排、partial 放在哪里、合并树长什么样。整数求和如果可能溢出，重排也可能改变 C++ 语义；浮点求和即使不溢出，也会改变舍入；字符串拼接满足语义结合时，内存分配可能成为主要成本。因此每个 reduce 都要同时写正确性模型和成本模型。

8.2 Map 的融合

多个 map 连在一起时，如果每一步都写出中间数组，会增加内存流量。融合把多个逐元素变换合在一次循环里，减少中间读写。融合不是总是好事：融合后循环体变大，寄存器压力可能增加，分支也可能更复杂。是否融合，要看减少的内存流量是否超过新增的执行压力。

8.3 Stencil 与邻域计算

Stencil 计算每个输出元素时读取邻域元素，例如图像卷积、有限差分 and 网格模拟。它的关键是复用邻域数据，避免重复从内存加载。分块、滑动窗口和边界处理是主要问题。

Stencil 的边界处理会影响代码形状。可以把内部区域和边界区域分开，让内部热循环没有复杂分支；也可以用 padding 或 ghost cell 让边界访问统一。前者代码路径多但热循环干净，后者内存多一点但逻辑规律。并行 stencil 还要处理块之间的 halo 数据，单机上表现为相邻块共享边界，分布式上表现为节点之间交换 halo 消息。

8.4 矩阵和块算法

矩阵计算展示了数据复用的重要性。朴素矩阵乘的数学公式很简单，但访问顺序会决定是否复用缓存。分块算法把矩阵切成能放入缓存的小块，让一个块内的数据被多次使用。这个思想不仅用于矩阵，也用于排序、join、图处理和文件扫描。

块大小不是越大越好。太小会增加循环和调度开销，太大又放不进缓存。实际工程中常用 benchmark 或自动调优选择块大小，并针对不同 CPU 做参数化。

8.5 直方图和冲突更新

直方图看似简单：读一个元素，更新一个桶。但多个线程可能更新同一个桶，导致锁竞争或原子争用。常见优化是线程本地直方图，最后合并。这个模式体现了一个原则：把高频共享写变成低频合并。

直方图还会遇到数据倾斜。如果 key 分布均匀，线程本地桶很好；如果大量输入集中到少数桶，全局原子会退化，分片后合并也可能在少数桶上集中。工程实现常按线程、NUMA 节点或缓存块建立局部桶，并在合并阶段使用连续扫描。若桶数量很大，本地直方图会占用大量内存，这时要在内存占用和冲突减少之间折中。

8.6 图遍历

图遍历通常局部性差、分支多、负载不均。压缩邻接表、按层处理、前沿队列和方向优化，都是为了改善访问模式和任务分配。图算法提醒我们，不是所有问题都能靠 SIMD 和连续数组轻松解决。

图遍历的性能问题往往同时包含三件事：邻接表访问随机，顶点度数不均，前沿大小变化剧烈。BFS 在某些层只有少量顶点，线程不够忙；在某些层前沿巨大，访问又非常分散。方向优化 BFS 会在 top-down 和 bottom-up 之间切换，目的就是根据前沿大小改变访问模式。这个例子说明，高性能算法不是固定套一个并行模板，而是随数据形状选择执行策略。

8.7 排序和分区

排序是数据系统的核心内核。快速排序、归并排序、基数排序和样本排序在不同场景下表现不同。并行排序的难点不只是比较次数，还包括分区均衡、临时空间、cache 行为和跨线程合并。分布式排序还会引入 shuffle 和热点分区。

8.8 内核选择清单

遇到一个新计算内核，可以按清单分析。第一，输入输出是否连续，是否能按块切分。第二，每个元素工作量是否接近，是否存在数据倾斜。第三，是否有跨元素依赖，依赖能不能分层处理。第四，写入位置是否唯一，是否会冲突。第五，是否能把全局共享写改成本地写加合并。第六，算术强度大概是多少，瓶颈更像计算还是内存。第七，单机算法扩展到分布式时，哪一步会变成 shuffle 或远程通信。

这份清单会在后面的并行算法和分布式计算里反复出现。它让读者从“我知道几个算法名字”转向“我能根据数据和依赖形状选择执行方式”。

核心思想

分析计算内核时先问四个问题：每个元素读写多少字节，是否存在跨元素依赖，写入是否冲突，工作量是否均衡。

8.9 内核模式不是算法名字

本章讲“内核模式”，不是为了让读者背 map、reduce、scan 这些词，而是训练一种分析方式。一个计算任务进入系统后，先不要急着选择线程池、SIMD 或分布式框架，而要先判断它的数据形状：输入是否连续，输出是否连续，读写比例如何，是否有跨元素依赖，是否存在冲突写，工作量是否均匀，能否按块处理，能否先局部处理再合并。

同一个业务问题可能拆成多个内核。日志统计可能先是 map：解析每行日志；再是 filter：丢掉无效记录；再是 partition：按 key 或时间分桶；再是 reduce：对每个 key 计数；最后是 sort：输出 top key。数据库执行器、搜索引擎、图计算和机器学习数据预处理都类似。理解内核模式，可以把一个看似复杂的系统拆成可优化、可测量、可验证的阶段。

Compute Systems Engine 后续的分布式 shuffle 也会复用这些模式。map 端解析和本地聚合，shuffle 前按 key 分区，reduce 端合并，输出前排序或 top-k。单机内核和分布式执行不是两本书的内容，而是同一组模式在不同尺度上的实现。

8.10 Map：最容易并行，也最容易被内存带宽限制

Map 的特征是每个输入元素独立产生输出。它通常最容易并行、向量化和分块，因为迭代之间没有依赖。朴素实现是一层循环。

Listing 8.1 baseline: 逐元素 map

```
1 void scale_and_shift(std::span<const float> input,
2                     std::span<float> output,
3                     float scale,
4                     float shift) {
5     const std::size_t count = std::min(input.size(), output.size());
```

```

6   for (std::size_t i = 0; i < count; ++i) {
7       output[i] = input[i] * scale + shift;
8   }
9 }

```

这个内核的正确性很简单：每个输出位置只依赖同一位置输入。性能上却要看字节数。如果每个元素读 4 字节写 4 字节，只做一次乘法和一次加法，算术强度很低，输入很大时往往受内存带宽限制。此时 SIMD 可以减少指令数，但未必带来数量级提升。若 map 表达式很重，例如包含多项式、三角函数或复杂解析，计算可能成为瓶颈，向量化和函数近似才更重要。

Map 的常见优化是融合。若先对数组做 scale，再做 clamp，再做转换，每一步都写出中间数组，就会产生多轮内存读写。融合成一次循环可以减少内存流量。

Listing 8.2 融合多个 map，减少中间数组

```

1 void normalize_clamp(std::span<const float> input,
2                     std::span<float> output,
3                     float mean,
4                     float inv_stddev) {
5     const std::size_t count = std::min(input.size(), output.size());
6     for (std::size_t i = 0; i < count; ++i) {
7         const float normalized = (input[i] - mean) * inv_stddev;
8         output[i] = std::clamp(normalized, -1.0F, 1.0F);
9     }
10 }

```

融合也有代价。循环体变大后，寄存器压力可能增加；若把多个不相关路径硬塞进一个循环，前端和分支预测可能变差；若中间结果可复用，融合反而丢掉复用机会。原则是：融合主要用于减少一次性中间读写，不是为了让代码看起来“更优化”。

8.11 Reduce：合并语义决定一切

Reduce 的重点是合并语义。一个 reduce 操作至少要说明单位元、结合律、交换律和重排是否允许。整数最大值很简单，单位元可以是最小可表示值，合并顺序不影响结果。浮点求和不严格满足结合律，合并树会影响舍入。字符串拼接满足结合律时仍可能被分配成本主导。哈希表聚合还要处理 key 冲突和内存分配。

朴素并行 reduce 的错误写法，是所有线程直接更新一个共享结果。

Listing 8.3 反例：reduce 中的共享写热点

```

1 void count_positive_bad(std::span<const std::int32_t> values,
2                         std::atomic<std::int64_t>& total) {
3     for (const std::int32_t value : values) {
4         if (value > 0) {
5             total.fetch_add(1, std::memory_order_relaxed);
6         }
7     }
8 }

```

这个版本的结果可以正确，但高频共享写会让 cache line 所有权不断迁移。更好的结构是每个 worker 本地计数，最后合并。

Listing 8.4 线程本地 reduce，最后合并

```

1 struct alignas(64) CountPartial {
2     std::int64_t positive = 0;

```



```

3 };
4
5 void count_positive_range(std::span<const std::int32_t> values,
6                           CountPartial& partial) {
7     std::int64_t local = 0;
8     for (const std::int32_t value : values) {
9         if (value > 0) {
10             ++local;
11         }
12     }
13     partial.positive = local;
14 }

```

这段代码还不是完整线程池实现，但表达了核心原则：热路径写寄存器和本地 `partial`，不写全局共享对象。它把问题从“每个元素一次同步”变成“每个 worker 一次合并”。当输入很大时，这个差异通常是数量级的。

8.12 Scan：把顺序依赖拆成两层

Scan 看起来比 reduce 更串行，因为每个输出都是前缀结果。但 scan 可以拆成块内扫描、块摘要扫描、回填三阶段。以整数前缀和为例，块内扫描可以并行；每个块产生一个 block sum；对 block sum 做小规模 scan；最后每个块把前面所有块的和加回自己的输出。

Listing 8.5 单块局部 scan

```

1 std::int64_t scan_block(std::span<const std::int32_t> input,
2                          std::span<std::int64_t> output) {
3     const std::size_t count = std::min(input.size(), output.size());
4     std::int64_t prefix = 0;
5     for (std::size_t i = 0; i < count; ++i) {
6         prefix += input[i];
7         output[i] = prefix;
8     }
9     return prefix;
10 }

```

这段局部 scan 本身是串行的，但多个块之间可以并行。并行 scan 的关键是把“所有元素依赖前面所有元素”改写成“块内局部依赖 + 块间摘要依赖”。这个思想非常重要：很多系统问题并不是完全无依赖，而是依赖可以摘要化。分布式计算中的 map-side combine、数据库中的局部聚合、图计算中的 frontier 层处理，都有类似结构。

Scan 的测试要比 reduce 更细。reduce 只需要最终结果，scan 每个位置都要正确。测试要覆盖空输入、单元素、刚好一个块、多个块、最后一块不满、负数和可能溢出。若使用无符号类型，还要说明模运算语义；若使用有符号类型，要避免未定义溢出。

8.13 Filter 和 stream compaction

Filter 选择满足谓词的元素。如果串行写，可以直接 `push_back`；并行写就不能让所有线程争用一个 vector。标准做法是三阶段：每个块统计保留数量，对数量做 scan 得到输出偏移，最后每个块写入自己的连续输出区间。

Listing 8.6 过滤前先统计每个块输出数量

```

1 std::int64_t count_selected(std::span<const std::int32_t> values,
2                             std::int32_t threshold) {
3     std::int64_t count = 0;

```

```

4   for (const std::int32_t value : values) {
5       if (value >= threshold) {
6           ++count;
7       }
8   }
9   return count;
10 }

```

第一阶段只统计，不写输出，因此没有冲突写。第二阶段扫描每个块的 count，得到每个块输出起点。第三阶段每个块再次扫描输入，把满足条件的元素写到自己负责的输出区间。它多读了一遍输入，却消除了共享 push 的锁或原子热点。是否划算取决于选择率、元素大小、谓词成本和内存带宽。

如果选择率很高，输出接近输入大小，两遍读的成本较高；如果选择率很低，输出少，三阶段方法仍然可能更好，因为并发写冲突少且输出紧凑。GPU、数据库和列式执行器都大量使用 selection vector 或 prefix sum 来组织过滤输出。

8.14 Stencil：边界和内部区域分离

Stencil 的核心是邻域复用。以一维三点 stencil 为例，输出位置 *i* 读取 *i-1*、*i* 和 *i+1*。内部区域很规则，边界位置需要特殊处理。若在每次迭代里判断边界，热循环会有分支；若把边界单独处理，内部循环更干净。

Listing 8.7 内部区域和边界分离的一维 stencil

```

1 void smooth_three_point(std::span<const float> input,
2                          std::span<float> output) {
3     const std::size_t count = std::min(input.size(), output.size());
4     if (count == 0) {
5         return;
6     }
7     output[0] = input[0];
8     if (count == 1) {
9         return;
10    }
11    for (std::size_t i = 1; i + 1 < count; ++i) {
12        output[i] = (input[i - 1] + input[i] + input[i + 1]) / 3.0F;
13    }
14    output[count - 1] = input[count - 1];
15 }

```

二维 stencil 更强调分块。一个像素或网格点会被相邻输出复用，按行扫描能利用一部分局部性；按 tile 处理可以把一块输入和 halo 留在缓存中。分布式 stencil 则要交换 halo 数据。单机的 cache block 和分布式的 halo exchange 是同一思想在不同层级上的表现。

8.15 直方图：桶数量决定策略

直方图的策略取决于桶数量、输入分布和线程数。桶很少时，全局原子会产生严重热点；每线程本地桶可以减少冲突，但合并成本低。桶很多时，每线程完整桶数组可能占用大量内存，TLB 和 cache 压力上升。输入分布倾斜时，少数桶成为热点；输入均匀时，分片效果更稳定。

一个合理实现通常从三版开始：全局锁或原子版本作为 baseline；每线程本地直方图版本作为优化；按 NUMA 节点或分块汇总版本作为扩展。每版都要说明内存占用。例如桶数是一百万、线程数是 64，每线程完整 `std::uint64_t` 桶数组需要 512 MiB，仅 partial 就可能过大。这时可以按块处理、稀疏本地 map、分桶分片或两级聚合。

8.16 排序：比较、移动和分区

排序不是只有比较次数。比较排序的成本包括比较函数、数据移动、分支预测、cache locality 和临时空间。若元素很大，排序索引可能比移动对象更好；若 key 是固定宽度整数，基数排序可能比比较排序更适合；若数据已经部分有序，某些算法能利用已有顺序；若需要稳定性，算法选择又不同。

并行排序通常先分区，再局部排序，再合并。分区是否均衡决定负载；局部排序是否 cache 友好决定单核效率；合并是否形成长尾决定扩展性。分布式排序进一步把分区变成 shuffle：样本选择不准会造成某些 reducer 数据过多，形成长尾。排序是连接单机算法和分布式系统的典型内核。

8.17 图遍历：不规则内核的代表

图遍历是规则数组内核的反面。邻接表访问可能随机，顶点度数差异巨大，前沿大小每层变化，写 visited 状态可能有竞争。BFS 的 top-down 策略从当前前沿扩展邻居；bottom-up 策略从未访问顶点检查是否有邻居在前沿。当前沿很小时 top-down 更好，当前沿很大时 bottom-up 可能减少边访问。方向优化的本质，是根据数据形状切换内核。

图算法还暴露负载均衡问题。若一个顶点有百万邻居，而其他顶点只有几个邻居，按顶点静态分配会造成长尾。可以按边切分、按度数分层、把高阶顶点拆成多个任务，或者使用工作窃取。每种方案都改变局部性和同步成本。图遍历提醒我们：高性能计算不只是 SIMD 和矩阵，真实系统有大量不规则数据。

8.18 Top-k 和重 hitter

很多系统不需要完整排序，只需要 top-k 或 heavy hitters。朴素做法是统计所有 key 后全量排序；更好的做法是每个分片维护局部 top-k 或候选集合，最后合并。若 key 空间很大且数据流很长，可以使用近似算法，例如 count-min sketch 或 space-saving，牺牲精确性换取内存和吞吐。

近似算法必须写清误差语义。系统教材不能只说“近似更快”，要说明误差上界、误报/漏报、是否能用于计费或安全策略。对于日志趋势和监控，近似可能足够；对于金融结算，近似不可接受。算法选择受业务语义约束。

8.19 内核组合：日志统计流水线

把本章模式组合起来，可以得到一个日志统计流水线。第一阶段 map：解析每行日志，提取 timestamp、user、event。第二阶段 filter：丢弃无效记录。第三阶段 partition：按 event 或 user 分桶。第四阶段 local reduce：每个 worker 对本地桶计数。第五阶段 merge：合并 partial。第六阶段 top-k：输出最热门事件。

这个流水线可以作为 Compute Systems Engine 的中型案例。每个阶段都有 reference，每个阶段都有可测成本，每个阶段都能映射到后续分布式版本。单机时 partition 是内存写入偏移；分布式时 partition 是 shuffle 目标。单机时 backpressure 是有界队列；分布式时 backpressure 是网络 and 远端 reducer 的限流。这样一本书的前后内容就能真正贯通。

8.20 本章落到贯穿项目

Compute Systems Engine 在本章应增加一个 kernel suite。至少包含：map 变换、count positive reduce、parallel count-if、stream compaction、thread-local histogram、one-dimensional stencil 和 top-k baseline。每个内核都保留串行 reference 和一个并行版本，benchmark harness 记录输入规模、线程数、输出校验和核心指标。

项目不要一开始追求所有内核极致优化。更重要的是让每个内核展示一种系统问题：map 展示内存带宽；reduce 展示共享写消除；scan 展示依赖分层；filter 展示输出位置分配；histogram 展示冲突更新；stencil 展示邻域复用；graph 或 top-k 展示不规则负载和数据倾斜。

8.21 本章习题

习题与作业

习题一：把一个日志统计任务拆成 map、filter、partition、reduce、top-k 五个阶段。每个阶段写出输入、输出、是否可并行、是否可能向量化、是否有共享写和主要瓶颈。

习题二：实现或设计 parallel count-if。要求先写全局原子反例，再写线程本地 partial 版本。报告中解释为什么 relaxed 原子不能解决共享写扩展性问题。

习题三：设计 stream compaction。写出统计、扫描偏移、写输出三阶段，并说明每阶段的正确性测试。要求覆盖选择率为 0、选择率为 100

习题四：为直方图选择三种策略：全局原子、本地桶、稀疏本地 map。分别说明适用的桶数量、输入分布、线程数和内存占用。

8.22 稠密 key 和稀疏 key

聚合和直方图经常取决于 key 空间形状。若 key 是 0 到 1023 的事件类型，数组桶非常合适：连续、无哈希、cache 友好、合并简单。若 key 是 64 位用户 ID，实际出现的 key 很稀疏，直接分配巨大数组不现实，哈希表或排序聚合更合适。若 key 分布高度倾斜，普通哈希表也可能在少数 key 上形成热点或长链。

稠密 key 的本地直方图可以写成数组。

Listing 8.8 稠密 key 的本地直方图

```
1 constexpr std::int32_t EVENT_BUCKET_COUNT = 1024;
2
3 struct LocalHistogram {
4     std::array<std::int64_t, EVENT_BUCKET_COUNT> buckets{};
5 };
6
7 void histogram_dense(std::span<const std::int32_t> events,
8                     LocalHistogram& histogram) {
9     for (const std::int32_t event : events) {
10         if (0 <= event && event < EVENT_BUCKET_COUNT) {
11             ++histogram.buckets[static_cast<std::size_t>(event)];
12         }
13     }
14 }
```

这个版本速度可能很高，但它把 key 范围写进了结构。若 event id 不是小整数，或者需要动态 schema，就不能直接使用。稀疏 key 可能使用 `std::unordered_map`，但本地哈希表会引入分配和随机访问。工程中常见折中是先对输入按块排序或使用 robin-hood/open-addressing 哈希表，减少分配和指针跳转。

8.23 排序聚合和哈希聚合

Group-by 聚合通常有两条路线。哈希聚合边读边更新哈希表，平均复杂度好，适合无序输入和点更新；排序聚合先按 key 排序，再线性扫描相同 key，适合需要排序输出、key 可比较、内存局部性重要或哈希表冲突严重的场景。排序聚合多了排序成本，却可能减少随机写和分配。

选择时要看 key 数量、输入大小、输出是否需要有序、内存预算和倾斜程度。若 key 很少，哈希聚合或数组桶好；若 key 很多且输出要排序，排序聚合可能更合适；若输入已经按 key 近似有序，排序聚合收益更大。数据库执行器通常同时实现多种聚合，并根据统计信息选择。

8.24 Top-k 的两阶段设计

Top-k 不必全量排序。每个 worker 可以维护一个大小为 k 的小堆或有序数组，得到本地 top-k；最后把所有本地 top-k 合并，再取全局 top-k。若 worker 数是 p，最终合并只处理 p 乘 k 个候选，而不是全部元素。这个结构和 map-side combine 一样，先局部减少数据量，再全局合并。

Listing 8.9 本地 top-k 候选的接口形态

```
1 struct TopCandidate {
2     std::int32_t key = 0;
3     std::int64_t score = 0;
4 };
5
6 using TopList = std::vector<TopCandidate>;
7
8 TopList merge_top_candidates(std::span<const TopCandidate> candidates,
9                             std::int32_t limit);
```

Top-k 的难点是 tie-breaking 和确定性。若两个 key 分数相同，输出顺序是否按 key 排序，还是任意？分布式系统中，不确定顺序会导致测试和缓存难以复现。接口应定义 tie-break 规则，例如先按 score 降序，再按 key 升序。

8.25 Join 是组合内核

Join 是数据库和数据处理系统中的核心操作。Hash join 先构建较小表的哈希表，再扫描较大表查找匹配；sort-merge join 先按 join key 排序，再线性合并；nested-loop join 朴素但只适合极小输入或有索引的情况。Join 的成本包括构建、探测、输出放大、内存占用和数据倾斜。

Join 也是多个内核的组合：构建哈希表是聚合/索引构造，探测是 map 加随机访问，输出是 compaction 或 append，分布式 join 还需要 partition/shuffle。若某个 key 一对多匹配很多行，输出可能爆炸，成为真正瓶颈。优化 join 不能只看输入大小，还要估算输出基数。

8.26 Graph frontier 的数据结构

图遍历常用 frontier 表示当前层活跃顶点。frontier 可以是向量、队列、bitmap 或稀疏集合。向量适合前沿较小，遍历活跃顶点；bitmap 适合前沿很大，快速判断某个顶点是否在前沿；稀疏集合适合需要快速去重和遍历。方向优化 BFS 的核心就是根据 frontier 大小和边数量切换数据结构和遍历方向。

这说明数据结构不是静态选择。一个算法的不同阶段可能需要不同表示。高质量系统会测量前沿大小、边访问量和重复访问，再切换策略。单一结构在所有阶段都“还可以”，往往不如在关键阶段选择合适表示。

8.27 内核流水线和材料化

多个内核连接时，要决定中间结果是否材料化。材料化把中间结果写成数组或文件，便于调试、复用和容错，但增加内存或 I/O。流水线把上游输出直接送给下游，减少中间写入，但背压、错误恢复和调试更复杂。数据库执行器中的 volcano、vectorized execution、pipeline breaker 都围绕这个取舍。

Reduce、sort、join、shuffle 常常是 pipeline breaker，因为它们需要看到一批数据或按 key 重排才能继续。Map/filter 通常容易流水线。理解哪些阶段必须材料化，哪些阶段可以融合，是计算系统设计的核心能力。

8.28 Map 内核的细节

Map 是最简单的计算内核：每个输入元素独立产生一个输出元素。正因为简单，它适合作为性能分析起点。Map 的主要问题通常是内存带宽、函数调用、分支和类型转换。若每个元素只做一次

加法，瓶颈很可能是读写字节；若每个元素调用复杂函数，瓶颈可能转向计算或前端；若每个元素有随机分支，分支预测会进入成本。

Map 内核的输出所有权很清晰：第 i 个输入写第 i 个输出。并行切分时，每个 worker 写自己的连续输出区间，不需要锁。这种结构应成为读者心中的“理想并行形状”。后续 filter、partition、join 的复杂性，很多都来自输出位置不再天然等于输入位置。

Map 还有一个常见优化：就地更新。若输入不再需要，可以直接写回原数组，减少输出内存。但就地更新改变了别名和生命周期。若后续还需要原始输入，就不能就地；若多线程中其他阶段仍在读取输入，就地会造成数据竞争或语义错误。内存节省不能脱离数据流图。

8.29 Reduce 内核的细节

Reduce 把多个输入合成一个或少数几个输出。它的难点是依赖和合并顺序。单线程 reduce 有一条累加依赖链；多累加器和分块 reduce 可以拆链；多线程 reduce 需要 partial；分布式 reduce 需要 map-side combine 和 shuffle。一个 reduce 的形态会随着尺度变化，但核心问题始终是：局部怎样合并，全局怎样合并，合并顺序是否影响语义。

不同 reduce 的单位元和结合性不同。计数求和简单，最小值最大值也简单；浮点求和有误差；集合合并可能需要去重；字符串拼接内存成本高；哈希表合并受 key 分布影响。教材不能把 reduce 简化成一个加法例子。每个 reduce 都要写清操作、单位元、是否结合、是否交换、是否确定性。

8.30 Scan 内核的角色

Scan 的价值在于把局部数量变成全局位置。Stream compaction、partition、基数排序、稀疏矩阵构建、GPU kernel 输出分配都需要 scan。它不是只用于前缀和题目，而是并行写输出的基础设施。

Scan 通常是 pipeline breaker，因为后续写位置依赖前面所有块的数量。优化 scan 时，常把它分层：块内 scan，块总和 scan，块内回填。这个结构在单机多线程和 GPU 中都常见。分布式系统里，对每个 partition 的输出大小做前缀求和，分配文件偏移，也是一种 scan 思想。

8.31 Filter 的选择率

Filter 的性能高度依赖选择率。选择率为 0 时，输出很小，但仍要读全部输入；选择率为 100% 时，filter 接近 copy；选择率中等且随机时，分支和输出位置都复杂。选择率还影响数据结构选择：低选择率适合 selected index list，高选择率适合 bitmap 或直接保留原数组加 mask，中等选择率可能需要 compaction。

测试 filter 时必须覆盖不同选择率。只测 50% 随机选择，无法代表真实业务；只测全通过，也看不到输出位置分配成本。日志过滤、权限筛选、数据库谓词和图 frontier 都会有不同选择率分布。算法实现要能处理这些变化。

8.32 Partition 的两种目标

Partition 有两类目标。第一类是为了并行写输出，例如把元素按 bucket 放到连续区间，便于后续每个 bucket 独立处理。第二类是为了改善局部性或控制流，例如按记录类型分桶，让后续同类记录走同一分支。前者关注无冲突写和输出布局，后者关注后续计算效率。很多系统同时使用二者。

Partition 的成本包括两遍扫描、counts 矩阵、输出写入和可能的稳定性维护。如果后续收益不大，partition 可能不划算。一个小输入上为了减少分支而先排序分桶，可能比直接处理更慢。判断 partition 是否值得，要估算后续阶段减少了多少分支、随机访问或网络数据。

8.33 Histogram 的冲突维度

Histogram 的冲突来自多个输入更新同一 bucket。冲突维度有三种：线程之间冲突、cache line 冲突、数据倾斜冲突。线程之间冲突可以用本地桶解决；cache line 冲突要注意相邻 bucket 是否被不同线程频繁写；数据倾斜冲突会让某些 bucket 合并阶段很重。桶数量越少，线程本地数组越小，但热点越强；桶数量越大，本地数组越大，合并成本也高。

稠密 histogram 的合并可以按 bucket 并行，也可以按 worker 并行。按 bucket 并行适合 bucket 很多，每个 bucket 合并所有 worker；按 worker 并行适合 worker 多但 bucket 少，先局部汇总再串行写。合并阶段也可能成为瓶颈，不能只优化本地统计。

8.34 Stencil 的边界处理

Stencil 使用邻域数据计算输出，例如一维平滑、二维卷积、网格 PDE。它的局部性比随机访问好，但边界处理和块间 halo 会带来复杂性。单线程中，边界可以单独处理；多线程中，每个块需要读取邻近块的边界数据。若只读输入写输出，halo 读取没有数据竞争；若就地更新，邻域依赖会破坏并行语义。

Stencil 优化常用 blocking，让一块数据在 cache 中被多次使用。二维 stencil 还要考虑行主序、行跨度、边界分支和向量化。GPU 和图像处理里 stencil 很常见，但 CPU 上同样重要。它训练的是“邻域复用”和“边界语义”。

8.35 Sort 的系统角色

排序不仅是算法题，也是系统工具。排序可以把随机 key 聚成连续区间，便于聚合、join、压缩和写文件。外部排序能处理超过内存的数据：先生成有序 run，再多路归并。分布式排序则先采样选择分区边界，再 shuffle 到 reducer。排序是典型的 pipeline breaker，但它换来后续顺序访问和确定性输出。

排序的稳定性也要声明。稳定排序保持相等 key 的原顺序，适合需要保留时间顺序或二级语义的场景；非稳定排序更快或内存更少。很多系统用复合 key 代替稳定排序，例如先按主 key，再按原始位置。稳定性和确定性是测试和恢复的重要边界。

8.36 Join 的输出放大

Join 最容易低估的是输出放大。两个输入各一百万行，如果某个 key 在左表出现一千次，在右表出现一千次，该 key 的 join 输出就是一百万行。即使输入不大，输出也可能爆炸。系统必须估算输出基数，并设置内存和背压策略。否则 join 不是慢，而是直接 OOM。

Hash join 构建阶段也要选择 build side。通常把较小表放入哈希表，较大表做 probe。但如果小表 key 极端倾斜，某些 bucket 很长；如果大表过滤后很小，先过滤再 join 更好；如果输出需要按 key 排序，sort-merge join 可能减少后续成本。Join 是优化器思维的入口：单个内核选择受上下游影响。

8.37 Sketch 和近似计算

在大规模系统中，有时无法保存所有 key 的精确状态。Sketch 类算法用固定内存近似频率、基数或分位数。Count-min sketch 估计频率，HyperLogLog 估计基数，t-digest 或 KLL 估计分位数。它们适合监控、推荐特征、流量分析和预估，不适合必须精确的结算。

近似算法的教材写法必须包含误差语义、内存大小、合并方式和适用边界。分布式系统喜欢可合并 sketch，因为每个 worker 可以本地构建，最后合并。它们和 reduce 结构天然匹配。若 sketch 不能合并，分布式使用会困难。

8.38 Kernel suite 的代码组织

Compute Systems Engine 的 kernel suite 应把每个内核按相同结构组织：reference、parallel、benchmark、tests。Reference 定义语义，parallel 做优化，benchmark 记录性能，tests 覆盖边界。不要把所有内核写进一个文件。每个内核都要有输入生成器和校验器，便于构造均匀、倾斜、小规模、大规模和边界输入。

接口应使用 span 和明确输出对象。Map 接收 input 和 output span；reduce 返回标量或 partial；histogram 接收 key span 和 bucket span；top-k 返回候选列表；join 返回输出范围或写入 sink。明确输出所有权，可以减少并发写冲突和隐藏分配。

8.39 阶段融合的验收

若要融合两个内核，例如 filter 后 map，应先有未融合 reference。融合版本必须输出完全一致或在声明误差内一致。性能报告要比较三项：未融合耗时、融合耗时、中间数据大小。若融合变快，要解释是少写中间数组、减少分配、改善 cache，还是减少函数调用。若融合变慢，要看寄存器压力、分支复杂度和代码大小。

融合还会影响调试和恢复。分布式作业中，materialized 中间结果可以作为 checkpoint；融合流水线失败后可能要从更早阶段重做。高性能和容错有时冲突。批处理系统常在 stage 边界材料化，就是为了恢复和重试。单机内核融合到分布式系统时，要重新考虑失败语义。

8.40 本章验收清单

本章完成后，读者应能为一个新计算任务做模式识别。它是 map、reduce、scan、filter、partition、sort、join、stencil、graph frontier、top-k 还是 sketch？它的输入输出是什么？输出位置是否天然确定？是否有共享写？是否需要稳定顺序？是否能局部聚合？是否有热点 key？是否有 pipeline breaker？是否能用 reference 校验？这些问题比背诵某个优化技巧更重要。

项目验收则要求至少三个内核串成一个流水线，并保留每阶段指标。比如日志记录先 filter 有效行，再 map 到 event type，再 histogram 聚合，再 top-k 输出。每阶段都要能单独关掉或替换，以便做对照实验。

8.41 案例：日志统计的完整内核链

一个完整日志统计链可以按 batch 处理。第一步，parse map 把原始文本变成热字段列。第二步，valid filter 产生选择向量或错误列表。第三步，event histogram 对有效记录按 event type 本地计数。第四步，partition 把本地计数按 reduce partition 分组。第五步，reduce merge 合并所有 worker 的 partial。第六步，top-k 输出热门事件。

每一步都有独立 reference。Parse reference 可以用简单字符串处理；filter reference 检查每行有效性；histogram reference 用单线程 map；partition reference 检查每个 key 去向；reduce reference 合并所有 partial；top-k reference 全量排序。链路正确性来自每个内核正确加上接口语义正确。这个案例可以作为第 18 章项目的单机子集。

8.42 案例：近似 heavy hitter

如果 event type 很多，精确统计所有 key 成本高，可以使用 heavy hitter 近似。每个 worker 维护固定大小候选集合，记录可能热门 key；最后合并候选，再对候选做精确二次统计或输出近似。这个方法减少内存，但可能漏掉接近阈值的 key。它适合监控和预警，不适合结算。

近似案例的重点是误差合同。报告必须写：最多保留多少候选，什么情况下可能漏报，是否做二次精确验证，输出是否标记近似。系统工程中，近似不是偷懒，而是用明确误差换资源。

8.43 案例：Pipeline breaker 的选择

假设 parse、filter、map 都可以流水线，但 histogram 需要聚合，top-k 需要看到完整结果。Histogram 和 top-k 就是 pipeline breaker。系统可以在 histogram 前保持 batch 流动，在 histogram 后材料化 partial，再进入 top-k。若为了极致低延迟把所有阶段都强行流水线，恢复和调试会复杂；若每一步都材料化，I/O 和内存成本高。

选择 pipeline breaker 时，要问三个问题：后续是否需要全局重排或聚合，失败后是否需要从这里恢复，中间数据是否值得保存。这个判断会贯穿单机执行器和分布式 shuffle。

8.44 内核选择的业务语义

同一个输入可以有多种内核选择。要统计所有 key 的精确计数，用 histogram 或 hash aggregate；只要热门 key，用 top-k 或 heavy hitter；要按 key 连接两个数据集，用 join；要按时间窗口输出趋势，用 partition 加窗口 reduce；要快速估计基数，用 sketch。选择不是只看速度，而是看业务语义：是否精确，是否确定，是否可恢复，是否需要排序，是否允许旧数据。

日志系统里，计费日志必须精确，监控趋势可以近似；审计输出需要稳定顺序，调试指标可以无序；最终报表需要持久化，临时 dashboard 可以丢弃。内核模式和业务语义相连，系统设计不能把所有数据都当成同一种计算。

8.45 计算内核章节总结

计算内核是数据流里的可复用形状。Map 保持位置, filter 选择元素, scan 分配位置, partition 重排数据, reduce 合并状态, sort 建立顺序, join 组合关系, top-k 缩小结果, sketch 用误差换空间。识别这些形状, 就能把复杂业务拆成可测试、可测量、可替换的阶段。

8.46 内核模式决策表

选择内核模式时, 先问输出和输入的关系。一对一是 map, 一对零或一是 filter, 一对多可能是 flat-map 或 join, 多个输入到一个输出是 reduce, 需要位置就是 scan, 需要按 key 分组就是 partition 或 sort, 需要最热少数就是 top-k, 需要固定内存估计就是 sketch。模式识别后, 再讨论并行、SIMD 和 I/O。不要一开始就从实现细节出发。

8.47 内核验收报告

本章每个内核都应该有一份验收报告。报告至少包含: 串行 reference、输入分布、输出校验、数据结构、内存流量估算、并行切分、共享写分析、负载均衡、异常边界和性能指标。没有这些内容的“优化”只是代码片段。

以直方图为例, 验收报告要写: 桶数量、key 分布、线程数、本地桶内存占用、合并策略、是否对齐、是否有 false sharing、输入是否随机、输出是否和 reference 一致、在均匀和倾斜输入下差异如何。这样读者才会把算法、数据和系统放在一起看。

8.48 本章扩展实验

习题与作业

实验一: 实现稠密 key 直方图和稀疏 key 哈希聚合两个版本。构造小 key 空间、大 key 空间、均匀分布和倾斜分布四组输入, 比较内存占用和吞吐。

实验二: 实现本地 top-k 加全局合并。要求定义 tie-break 规则, 并验证多线程和单线程输出顺序一致。

实验三: 设计一个 hash join 和 sort-merge join 的成本比较, 不要求完整实现大系统, 但要估算构建、排序、探测、输出和内存占用。

实验四: 为 BFS frontier 选择 vector 和 bitmap 两种表示。说明当前沿从小变大时, 为什么表示应该切换。

实验: 在 Compute Systems Labs 中扩展一个线程本地直方图。先写全局 mutex 版本, 再写线程本地版本, 比较扩展性, 并解释差异来自锁竞争还是 cache line 迁移。

第零部分

多线程、同步与内存模型

第 9 章

线程、调度与亲和性

线程是操作系统调度的执行实体。多线程程序能利用多核心，但也引入调度、同步、内存可见性和资源竞争。理解线程，必须同时看用户态代码和内核调度。

9.1 线程不是核心

创建八个线程不等于拥有八个物理核心。硬件可能有 SMT，一个物理核心暴露两个逻辑 CPU；系统里还运行着其他进程；线程可能被迁移到不同核心。线程迁移会破坏缓存热度，过多线程会增加上下文切换。

CPU-bound 程序通常不应远多于核心数，I/O-bound 程序可以有更多并发等待。判断依据不是经验数字，而是运行时是否在计算、等待锁、等待 I/O 或被调度器抢占。

9.2 用户线程和内核线程

C++ 的 `std::thread` 通常映射到操作系统线程。操作系统调度的是内核可见的执行实体，它拥有寄存器上下文、栈、调度状态和优先级。用户态协程则不同，协程切换通常不进入内核，它把等待点显式暴露给运行时。协程适合组织大量 I/O 等待，不会自动让 CPU 密集代码并行。

这一区分很重要。把 CPU 密集任务放进单线程协程事件循环，不会使用多个核心；把大量 I/O 等待都变成内核线程，可能造成过多栈内存、调度成本和上下文切换。运行时设计要把计算并行和等待并发分开。

9.3 调度和上下文切换

上下文切换需要保存和恢复执行状态，可能破坏 cache 和 TLB 局部性。频繁阻塞、线程过多、锁竞争和系统调用都可能增加切换。性能计数器里的 context-switches 和 migrations 可以帮助定位这类问题。

Linux CFS 调度器大体追求公平，它会根据虚拟运行时间选择任务。实时调度策略和普通调度策略不同，容器和 cgroup 也会改变 CPU 配额。benchmark 时若机器上有其他负载，或者进程受 cgroup 限制，结果会明显波动。严谨实验应记录负载、CPU governor、容器限制和是否独占机器。

调度器不是只在“线程很多”时才重要。即使线程数等于核心数，线程也可能因为缺页、锁等待、I/O、信号、中断、抢占和 cgroup 配额离开 CPU。对于低延迟系统，一次迁核可能让热缓存失效；对于吞吐系统，频繁上下文切换会减少有效计算时间。性能报告如果只写线程数，不写线程是否真正持续运行在 CPU 上，就缺少关键证据。

Linux 调度源码阅读不建议一开始追完整策略。更好的入口是从一个用户态现象出发：为什么 `perfstat` 里 context-switches 增加，为什么线程会 migration，为什么绑定 CPU 后波动变小，为什么 cgroup 限额下吞吐呈周期性抖动。围绕这些问题，再去看调度实体、runqueue、唤醒路径和上下文切换路径，读出的内容才有边界。

9.4 阻塞、睡眠和自旋

线程等待锁时，可以自旋，也可以睡眠。自旋避免进入内核，适合等待时间极短的场景；睡眠释放 CPU，适合等待时间较长的场景。很多锁实现会先短暂自旋，再进入 `futex` 等内核等待路径。这样低竞争时快，高竞争时不至于烧满 CPU。

选择自旋还是阻塞，本质是在 CPU 时间和唤醒延迟之间取舍。如果临界区很短且核心充足，自旋可能好；如果锁持有者被抢占或等待 I/O，自旋会浪费大量周期。不要在没有测量的情况下把自旋当成高性能。

`futex` 是理解 Linux 用户态锁的重要入口。常见 `mutex` 在无竞争时尽量只走用户态原子操作；竞争严重时，线程通过 `futex` 进入内核睡眠，锁释放时再唤醒等待者。这样做把低竞争 fast path 和

高竞争 blocking path 分开。读锁性能时，应区分用户态自旋成本、futex 睡眠成本、唤醒成本和被唤醒后重新竞争锁的成本。

9.5 线程亲和性

线程亲和性让线程倾向或固定在某些 CPU 上运行。它可以改善缓存局部性和 NUMA 放置，也可能降低调度器自由度。使用亲和性前要理解拓扑：物理核心、SMT 线程、NUMA 节点和共享缓存层级。

亲和性实验要有对照。只绑定线程不绑定内存，可能仍然访问远端页面；只绑定内存不绑定线程，线程迁移后也可能变成远端访问。合理实验至少比较未绑定、只绑线程、只绑内存、线程和内存都按节点绑定几种情况。若平台没有 NUMA，也应记录物理核心和 SMT 拓扑，避免把两个重负载线程绑定到同一个物理核心的两个逻辑 CPU 上后误判算法慢。

9.6 线程生命周期

频繁创建销毁线程成本高，也让任务调度不可控。线程池把线程生命周期和任务生命周期分离，适合大量短任务。但线程池不是万能的，如果任务太小，调度成本会超过计算成本；如果任务阻塞，线程池可能被耗尽。

9.7 线程局部存储

线程局部存储让每个线程拥有独立对象，适合缓存临时缓冲、统计计数和避免共享写。它可以减少锁竞争，但也可能增加内存占用，并让跨线程汇总更复杂。线程池里使用线程局部对象时，还要注意任务迁移：同一个逻辑请求的不同阶段可能被不同 worker 执行，不能把请求状态误放到线程局部。

9.8 线程池里的调度语义

线程池把固定 worker 暴露成任务执行资源，但它不会自动解决调度语义。CPU 密集任务应该限制并发度，避免超过核心数太多；阻塞 I/O 任务如果占住 worker，可能让计算任务饥饿；任务内部再提交子任务并等待，可能造成线程池死锁。设计线程池时，要明确任务是否允许阻塞、是否允许嵌套提交、是否支持 work helping、是否需要独立 I/O 池。

线程池还会改变线程局部数据的含义。线程局部缓冲属于 worker，不属于请求。请求如果跨多个任务执行，就不能假设后续阶段还能访问同一个线程局部对象。运行时应把“worker 本地缓存”和“请求上下文”分成两个概念。

9.9 从任务到线程的映射

并发程序里最容易混淆的是任务和线程。任务是逻辑工作单元，例如处理一段数组、解析一个文件块、执行一个 reduce 分区、发送一个网络请求。线程是操作系统调度实体，它拥有栈和寄存器上下文，会被放入 runqueue，由调度器分配 CPU 时间。一个任务可以在线程上执行，一个线程可以执行很多任务；二者不是同一概念。

Compute Systems Engine 最初的 `parallel_sum` 每次调用都创建一组线程，任务和线程几乎一一对应。这种写法适合教学，因为边界清楚；但它不适合大量短任务，因为线程创建、栈分配、调度注册和 join 都有成本。线程池把线程生命周期延长，让任务通过队列进入 worker。这样任务变轻了，但又引入队列、等待、关闭和异常传播问题。每前进一步，性能和语义都会同时变化。

设计运行时，要先回答三个问题。第一，任务是否 CPU 密集。如果是，worker 数通常不应远超物理核心，除非需要隐藏内存或 I/O 等待。第二，任务是否可能阻塞。如果任务会等待磁盘、网络、锁或 future，把它放进 CPU 池可能耗尽 worker。第三，任务之间是否有依赖。如果一个任务提交子任务后等待，而线程池没有 work helping 或足够 worker，可能死锁。线程池不是“并发万能容器”，它只是执行资源的组织方式。

9.10 线程创建成本和任务粒度

任务粒度是并行运行时最基本的参数。粒度太大，负载不均衡，一个慢任务拖住整体；粒度太小，队列操作、调度、同步和计数器开销超过有效计算。归约任务如果每个 worker 处理上千万元素，线程创建成本相对很小；如果每个任务只处理几十个元素，线程池也可能被调度开销吞掉。

选择粒度不能只靠固定常数。不同机器核心数不同，缓存大小不同，内存带宽不同，任务内核成本也不同。一个合理策略是先用 coarse chunk 保证每个任务有足够计算量，再根据负载不均衡逐步拆小。对均匀数组求和，按连续大块切分足够；对图遍历或 key 倾斜聚合，任务成本不均匀，可能需要动态调度、工作窃取或分片负载估计。

实验上要画任务粒度曲线。固定总输入和线程数，改变 chunk size，记录吞吐、任务数、队列等待、worker 空闲时间和尾部完成时间。曲线通常会出现一个中间区域：粒度太小，调度成本高；粒度太大，负载不均衡；中间区域才是合理范围。这个实验比背“任务数等于线程数几倍”更可靠。

9.11 阻塞任务和线程池耗尽

考虑一个线程池有四个 worker。每个任务启动后提交一个子任务，然后等待子任务完成。如果四个 worker 都在执行父任务并等待子任务，而子任务还在队列里没有 worker 执行，线程池就死锁了。这个问题不是 mutex 死锁，却同样来自等待关系形成环。

解决方向有几种。第一，禁止 worker 内同步等待同一线程池的子任务，把依赖交给任务图调度。第二，支持 work helping：worker 等待时不睡眠，而是继续执行队列里的其他任务。第三，分离 CPU 池和阻塞池，让阻塞 I/O 不占满计算 worker。第四，增加结构化并发语义，让任务的生命周期和父子关系明确。选择哪种方式取决于运行时目标，但必须在接口文档中说明，不能让调用者猜。

本章先建立问题，后续任务运行时章节会实现更完整的任务图。这里要记住：线程池接口不只是 submit，还包括等待、取消、关闭、异常和嵌套提交语义。一个只会把函数塞进队列的线程池，很容易在真实系统中出现停不下来、等不到、吞异常或死锁的问题。

9.12 亲和性和拓扑记录

亲和性优化的前提是拓扑记录。报告里至少要写清逻辑 CPU 数、物理核心数、SMT 关系、NUMA 节点、共享 L1/L2/L3 的范围和内存节点。Linux 上可以用 `lscpu`、`numactl--hardware`、`/sys/devices/system/cpu/` 下的拓扑文件和工具输出记录。没有这些信息，绑定 CPU 的结论很难解释。

绑定策略要和工作负载匹配。CPU 密集向量计算通常先使用每个物理核心一个线程，再测试 SMT；内存延迟敏感但单线程经常等待的任务可能从 SMT 获益；共享数据强的任务可能受益于放在共享 LLC 内；NUMA 分片任务应让线程和内存位于同一节点；跨分片通信强的任务要考虑互连成本。亲和性不是为了把线程“钉死”就结束，而是为了把执行位置、数据位置和共享层级对齐。

亲和性也可能伤害性能。过度绑定会减少调度器处理系统噪声的自由度；把两个重负载线程绑到同一物理核心的 SMT sibling 会互抢执行资源；把线程绑到一个 NUMA 节点但数据在另一个节点，会稳定地产生远端访问；在手机或容器环境里，系统可能限制可用 CPU，强行绑定到不可用 CPU 会失败。实验报告必须记录绑定是否成功。

9.13 上下文切换的真实代价

上下文切换不是一个固定纳秒数。直接成本包括保存和恢复寄存器状态、切换内核调度结构和更新运行队列；间接成本包括 cache 热度下降、TLB 上下文影响、分支预测状态变化和被迁移到不同核心后的数据重新加载。若切换发生在锁持有者身上，其他线程可能自旋或睡眠等待；若切换发生在低延迟请求路径上，尾延迟可能被放大。

测上下文切换要区分自愿和非自愿。自愿切换通常来自线程主动阻塞，例如等待条件变量、I/O 或 futex；非自愿切换通常来自时间片耗尽、抢占或更高优先级任务。两者解决方向不同。自愿切换多，可能要减少阻塞、批处理或改变等待策略；非自愿切换多，可能线程数过多、CPU 竞争严重或系统负载干扰。

在 `perfstat` 或系统工具中看到 context-switches 增加时，不要立刻下结论。一个 I/O 服务有上下文切换可能是正常的；一个 CPU-bound 热循环频繁切换就值得怀疑。要结合 CPU 利用率、线程数、锁等待、runqueue 长度和任务类型分析。

9.14 Linux 调度源码阅读路线

Linux 调度器很复杂，不适合一开始从所有策略全局阅读。本册建议按现象驱动。若你想理解普通线程为什么被调度，可以从核心调度入口、调度类、运行队列和上下文切换路径读起；若想理解唤醒延迟，可以沿 `futex` 或条件变量唤醒路径看到任务如何重新进入 `runqueue`；若想理解迁移，可以阅读负载均衡和 CPU affinity 相关路径；若想理解 `cgroup` 限额，可以阅读 CPU controller 对可运行时间的限制。

报告要写具体问题。例如“在 16 个线程测试中，migrations 明显增加，绑定 CPU 后波动下降”，然后阅读调度迁移和亲和性相关代码。不要写“Linux 使用 CFS 调度器，所以性能如何”这种大而空的句子。源码阅读的价值在于把用户态现象和内核数据结构连接起来：`task`、`runqueue`、调度类、唤醒、抢占、迁移和上下文切换。

9.15 线程局部缓存和内存生命周期

线程局部缓存可以减少分配和共享写，但它改变了对对象生命周期。线程池中的 `worker` 会长期存在，线程局部对象也会长期存在。如果线程局部缓冲按最大请求扩容，处理一次大请求后可能长期占用大量内存。若任务之间复用线程局部状态，还要保证不会泄漏上一个请求的数据。高性能系统里，缓存和生命周期总是绑在一起。

线程局部统计同样有读取语义问题。每个 `worker` 写自己的计数很快，但读取总数需要遍历所有 `worker` 状态。若 `worker` 正在写，读者是否需要精确值？如果只用于指标展示，可以接受近似或延迟汇总；如果用于协议决策，例如限流额度，就需要更强同步。把所有统计都改成线程局部之前，必须说明读取方的语义需求。

9.16 案例：从每次创建线程到固定线程池

下面用归约说明线程生命周期的演化。`baseline` 是每次调用 `parallel_sum` 都创建 `worker_count` 个线程。它适合少量大任务，但不适合频繁短任务。改进方向是固定线程池：程序启动时创建 `worker`，归约请求被拆成多个 `chunk`，提交到队列，`worker` 执行后写 `partial`，主线程等待所有 `chunk` 完成。

这个改进引入了新问题。队列需要同步，提交任务需要分配或复用任务对象，等待所有 `chunk` 完成需要计数器或 `latch`，异常需要传回调用者，线程池关闭时不能丢任务。如果允许多个归约请求同时进入线程池，`partial` 的所有权和生命周期要隔离。一个高质量线程池章节不能只给一段 `while(true)` `worker` 循环，还要讲清这些语义。

本章只要求读者理解演化方向。后续运行时章节会把线程池拆成接口、队列、`worker`、任务状态、关闭协议和指标。这里先建立判断：当任务足够大且调用次数少，每次创建线程可以接受；当任务短且频繁，线程池值得引入；当任务之间有依赖，普通 FIFO 线程池不够，需要任务图或 `work stealing`。

9.17 本章落到贯穿项目

Compute Systems Engine 在本章要增加运行环境记录和线程策略实验。第一，程序启动时记录硬件拓扑和可用线程数，至少输出逻辑 CPU 数和用户指定 `worker` 数。第二，归约 `benchmark` 要支持从 1 到多倍逻辑 CPU 的线程扫描。第三，实验报告要记录 `context-switches` 和 `migrations`。第四，设计线程池前先保留每次创建线程版本作为对照，避免引入线程池后不知道收益来自哪里。

后续项目可以逐步加入亲和性选项，例如不绑定、按物理核心绑定、按 NUMA 节点绑定。每种绑定都必须能失败并报告原因，不能在不支持的平台上默默假装成功。手机或 Termux 环境尤其要谨慎，因为权限、CPU 拓扑和系统调度策略可能和服务器不同。

9.18 线程状态和等待图

线程在系统里并不只有“运行”和“结束”。它可能 `runnable` 但还没拿到 CPU，可能 `running`，可能 `blocked` 在 `futex`，可能 `sleeping` 等 I/O，可能 `stopped`，可能被 `cgroup throttled`。分析并发程序时，要把线程状态画进等待图。一个任务慢，可能不是它计算慢，而是它一直没被调度；一个线程池空闲，可能是上游没有提交，也可能是所有任务阻塞在下游队列。

等待图比线程列表更有用。节点可以是线程、队列、锁、I/O 设备、future、远端服务；边表示等待关系。若线程 A 等锁 L，锁 L 被线程 B 持有，线程 B 等队列 Q，队列 Q 等消费者 A，就形成等待环。很多死锁和线程池耗尽都能用等待图解释。无论是 mutex 死锁、future 死锁，还是分布式请求互相等待，本质都是等待关系没有退出路径。

9.19 Runnable 不等于 Running

一个线程处于 runnable 状态，说明它可以运行，但不代表它正在运行。如果 runnable 线程数长期大于可用 CPU 数，调度器必须轮流运行它们，context switch 增加，每个线程获得的 CPU 时间下降。CPU-bound 程序开远多于核心数的线程，通常不会更快，反而增加调度开销和 cache 污染。

I/O-bound 程序可以有更多线程，因为很多线程在等待 I/O，不同时占用 CPU。但这也有限度：线程栈、调度结构、唤醒风暴和内存占用都会增加。异步 I/O 和事件循环的价值，就是减少“每个等待一个线程”的成本。选择线程数要看任务是 CPU-bound、memory-bound、I/O-bound 还是混合。

9.20 大小核和移动设备

手机和某些笔记本使用大小核架构，不同核心性能和能效不同。线程绑定到大核和小核，性能可能差很多；系统还会根据温度、电源和前台状态迁移线程。Termux 环境下，用户不一定能完全控制调度。报告中如果在手机上测性能，应记录设备型号、电源状态、温度大致情况、可见 CPU 和是否允许 affinity。

大小核让“线程数等于核心数”更复杂。八个核心不代表八个等价执行资源。CPU 密集任务可能更适合使用大核数量作为主并发度；后台任务可以放小核；长时间满载可能降频，短时间 benchmark 不代表持续性能。第二册的实验要尊重设备现实，而不是假设所有机器都是服务器。

9.21 Oversubscription 的后果

Oversubscription 指可运行线程数超过可用硬件执行资源。适度 oversubscription 可以隐藏 I/O 等待；过度 oversubscription 会带来上下文切换、cache 失效、锁持有者被抢占、尾延迟增加和内存压力。线程池默认大小不应盲目等于逻辑 CPU 数乘很多倍，除非任务大部分时间阻塞。

更糟的是嵌套并行。上层已经用 8 个线程处理请求，每个请求内部又调用一个默认开 8 线程的库，瞬间产生 64 个 runnable 线程。矩阵库、压缩库、并行 STL 和业务线程池都可能叠加。工程系统应限制嵌套并行，或者让库共享同一个运行时。性能报告要记录是否调用了内部多线程库。

9.22 调度公平和吞吐

公平调度让每个可运行任务获得合理 CPU 时间，但高吞吐系统有时希望减少迁移和切换。公平、局部性和优先级之间有张力。一个后台压缩线程如果和前台请求线程公平竞争 CPU，可能增加用户请求延迟；完全饿死后台线程，又可能导致 checkpoint 或日志堆积。调度策略要反映业务优先级。

用户态线程池也有类似问题。FIFO 队列公平但可能让长任务阻塞短任务；优先级队列改善前台延迟但可能饿死后台；工作窃取提高负载均衡但可能牺牲数据本地性。操作系统调度和用户态调度叠加，最终行为取决于两者共同作用。

9.23 Run queue 和唤醒延迟

当一个阻塞线程被唤醒，它不是立即运行，而是进入某个 CPU 的 run queue，等待调度器选择。若系统负载高，唤醒延迟可能明显。条件变量 `notify_one` 只是让等待线程有机会运行，不保证它马上持有锁。被唤醒线程还要和其他线程竞争 mutex，可能再次睡眠。

这解释了为什么高竞争条件变量和锁会产生尾延迟。一个生产者通知消费者后，消费者可能晚些才运行；消费者醒来后发现队列又被其他消费者取空；再睡回去。正确性由谓词循环保证，性能则需要减少惊群、控制队列容量、降低竞争和让任务粒度合适。

9.24 优先级反转的系统版

优先级反转不只存在实时系统。一个低优先级后台任务持有全局锁，高优先级请求线程等待这把锁，而中优先级 CPU 任务不断运行，使后台任务迟迟无法释放锁，这就是普通服务中的优先级反转形态。解决方法包括缩短临界区、避免后台任务持有前台锁、资源隔离、优先级继承或使用无锁/分片结构。

在分布式系统中也有类似现象：低优先级 checkpoint 占满 I/O，导致高优先级提交延迟；后台 compaction 占满 CPU，导致 RPC 超时。线程调度的公平性问题会在资源调度层面反复出现。第二册把这些主题放在同一册，是为了让读者看到共性。

9.25 线程栈和内存占用

每个线程都有栈。线程数很多时，栈虚拟地址和实际提交内存都可能成为问题。递归、大对象局部变量和深调用链会增加栈压力。C++ 并发代码中，任务对象如果捕获大 vector 按值复制到线程栈或闭包里，也会造成内存和复制成本。线程池减少线程数量，也能减少栈开销。

用户态 fiber 或 coroutine 可以用更轻量的栈或无栈状态机，但它们引入自己的调度和生命周期语义。第二册不展开 coroutine 细节，但要让读者知道：用更多并发单位隐藏等待时，线程不是唯一选择。线程适合 CPU 执行资源，异步任务适合等待很多 I/O。

9.26 信号和异步打断

Linux 线程还可能收到信号。信号处理函数会在异步位置执行，能做的事情非常有限。高性能和并发代码通常避免在信号处理函数里做复杂逻辑，而是写入 pipe、eventfd 或设置原子标志，让主循环在安全位置处理。信号也可能打断系统调用，导致返回 EINTR。I/O 代码必须考虑这种情况。

这部分不是本册重点，但它提醒读者：真实系统中的控制流不只来自函数调用和线程调度，还可能来自信号、取消、超时和内核回调。状态机要能处理被打断和重试。

9.27 ThreadSanitizer 和调度测试

ThreadSanitizer 能发现许多数据竞争，但它会改变性能，不能用于性能测量。调度压力测试可以通过增加线程数、插入 yield、随机 sleep、缩小队列容量和重复运行来暴露时序 bug。不要在正式性能报告中开启 TSan，也不要因为 TSan 通过就认为没有并发逻辑错误。

线程调度 bug 常常低概率出现。测试应固定随机 seed，记录失败场景，并尽量构造小模型复现。例如有界队列容量设为 1，比容量很大更容易暴露等待和唤醒错误；线程池 worker 数设为 1 或 2，比 16 更容易暴露嵌套等待死锁。小规模极端测试是并发正确性的朋友。

9.28 线程池大小的配置

线程池大小应该是配置，不应写死。默认值可以来自硬件并发度，但要允许用户覆盖。CPU-bound 池、I/O 池和后台池应有不同默认策略。若运行在容器中，还要尊重 cpuset 和 CPU 配额。读取 `std::thread::hardware_concurrency` 失败或返回不准时，应有保守 fallback。

配置还要进入实验记录。一个 benchmark 只写“默认线程池”，读者不知道默认是多少，也不知道是否与机器有关。项目输出应显示 `worker_count`、`hardware_concurrency`、绑定策略和是否使用 SMT。这样结果才可解释。

9.29 线程设计的反例

常见反例包括：每个请求创建一个线程导致高并发时内存爆炸；CPU-bound 任务开数倍于核心的线程导致调度开销增加；线程池 worker 内阻塞等待同池子任务导致死锁；把 I/O 阻塞任务放入计算池导致计算饥饿；在线程局部缓存里保存请求状态导致任务迁移后读不到；全局队列在多 socket 下成为共享热点；忽略关闭语义导致析构时悬挂。

这些反例不是孤立 bug，而是线程、任务、资源和状态边界没有分清。学习线程调度的目标，是在设计阶段就能问出这些问题。

9.30 案例：日志解析线程数

日志解析看起来 CPU 密集，但它可能同时受 I/O、分支和内存带宽影响。若输入来自 SSD，单线程解析可能跟不上读取；增加线程能提高解析吞吐。若输入来自慢存储，增加解析线程只会让线程等待 I/O。若解析产生大量错误日志，输出 I/O 又可能成为瓶颈。线程数选择必须和全链路一起看。

实验可以设置三种输入源：内存中已加载字符串、page cache 热文件、冷文件或模拟慢读取。对每种输入扫描 worker 数。内存输入主要测解析和内存带宽；热文件混合 page cache；慢读取则测 I/O。若三组结果差异大，说明线程数不能脱离输入源讨论。这个案例能训练读者避免“一律开满核心”的经验主义。

9.31 案例：后台任务干扰

假设 Compute Systems Engine 在 reduce 阶段同时做 checkpoint。Checkpoint 线程写大文件并 fsync，可能占用 I/O；如果它还做压缩，可能占用 CPU；如果它持有 driver 元数据锁，可能阻塞 commit。前台 reduce 变慢时，不能只看 reduce 代码，还要看后台任务是否干扰。

解决方法包括资源隔离、限速、独立线程池、优先级和调度窗口。Checkpoint 可以在 stage 边界执行，也可以后台增量写；压缩可以限制线程数；元数据锁应只保护状态切换，不覆盖大文件写入。线程调度和 I/O 背压在这里交汇。

9.32 线程池配置实验

项目应提供线程池配置实验。参数包括 compute worker 数、I/O worker 数、是否允许阻塞任务进入 compute 池、队列容量和任务粒度。实验输入包括 CPU-bound map、I/O-bound read、混合 pipeline。指标包括吞吐、队列长度、context switches、worker idle time、任务延迟和超时。

通过实验，读者可以看到：CPU-bound worker 过多会增加调度成本；I/O-bound 任务放 compute 池会让计算饥饿；队列容量太小会频繁背压，太大会增加延迟；任务粒度太细会增加调度开销。线程池配置不是一次性常量，而是 workload 参数。

9.33 用户态调度和内核调度的边界

用户态线程池决定哪个任务交给哪个 worker，内核调度器决定哪个线程在哪个 CPU 上运行。用户态运行时看不到所有系统负载，内核调度器不知道用户任务的业务优先级。两者之间存在信息差。绑定 CPU、设置线程优先级、分离线程池、使用 cgroup，都在试图协调这两个调度层。

项目文档应说明哪些由用户态控制，哪些由内核控制。比如 worker id 到 CPU id 的映射是亲和性策略；task 到 worker 的映射是运行时策略；worker 是否被抢占是内核调度；页面在哪个 NUMA 节点是内存策略。把边界说清，排查问题时才知道该看哪一层。

9.34 线程池和请求上下文

请求上下文不应该隐式绑定到线程。在线程池中，一个请求的 parse、compute、write 可能由不同 worker 执行；即使当前实现碰巧同一线程，后续 work stealing 或异步 I/O 也可能改变。上下文应作为显式对象传递，或者由 task id 在状态表中查找。线程局部只适合 worker 缓冲、统计和临时 scratch。

这个原则能避免很多隐蔽 bug。把 request id 放在线程局部，日志可能串到别的请求；把取消标志放线程局部，后续任务看不到；把输出缓冲放线程局部并在异步写完成前复用，会产生数据损坏。线程局部是性能工具，不是业务上下文容器。

9.35 调度章节验收清单

本章项目验收包括：能记录硬件并发度和用户 worker 数；能扫描线程数并输出扩展曲线；能记录 context switches 和 migrations；能区分 CPU-bound 和 I/O-bound 任务；线程池配置可调；任务粒度可调；请求上下文显式传递；阻塞任务不会默认进入计算池；报告说明当前环境的 CPU 拓扑限制。满足这些条件，后续线程池和任务图才有可靠基础。

9.36 调度章节总结

线程是执行资源，任务是逻辑工作，二者不能混淆。调度优化要同时考虑任务粒度、阻塞、亲和性、上下文切换、线程局部缓存和内核调度。线程池能减少线程创建成本，但引入队列、等待、关闭和嵌套提交语义。正确的线程设计不是“开几个线程”，而是让执行资源、数据位置、等待关系和业务上下文都有清楚边界。

9.37 调度决策表

配置线程和任务时，先问任务是否 CPU-bound、是否阻塞、是否有依赖、是否需要优先级、是否访问大块本地数据、是否会嵌套提交。CPU-bound 任务按核心控制并发；阻塞任务隔离到 I/O 池；有依赖任务进入任务图；有优先级任务需要多队列或配额；访问大块数据要考虑亲和性；嵌套提交要禁止阻塞等待或支持 work helping。这个表能防止线程池被当成万能容器。

9.38 案例：任务粒度自适应的观察

自适应任务粒度不一定要一开始自动实现，但可以先观察。记录每个任务耗时、队列等待、worker 空闲时间和任务数。如果任务耗时远小于队列操作成本，说明粒度太细；如果 worker 空闲多且少数任务很长，说明粒度太粗或负载不均。运行时可以先把这些指标输出给报告，让人手动调整 cutoff。自动调节只是把这个反馈回路程序化。

这个案例提醒读者：很多“智能调度”可以从简单观测开始。没有观测，自动策略只是猜测；有了观测，即使手动调参，也比固定常数可靠。

9.39 案例：线程池里的阻塞隔离

假设 compute pool 有 8 个 worker，其中 8 个任务都在等待磁盘写完成，那么新的计算任务无法执行，CPU 可能空闲但队列积压。解决方法不是简单增加 compute worker，而是把阻塞写入交给 I/O pool 或异步写出。Compute worker 只生成输出 block，然后返回执行资源。I/O pool 完成后通过 completion 通知 driver。

这个案例说明线程池大小不是唯一问题，任务类型也必须分类。CPU-bound、I/O-bound、control-plane、background maintenance 应有不同资源策略。否则一个慢资源会拖住所有执行资源。

9.40 运行队列视角下的线程池

用户态线程池的队列和内核 run queue 是两层不同队列。任务先排在用户态 ready queue 里，worker 线程取到任务后进入执行；worker 自己又作为内核任务排在某个 CPU 的 run queue 里。用户态队列空，不代表系统没有可运行线程；内核 run queue 繁忙，也不代表线程池有任务。分析吞吐和延迟时，必须把这两层分开。

例如一个服务的用户态任务队列持续增长，但 CPU 利用率很低。可能原因不是 worker 数不足，而是所有 worker 阻塞在 I/O 或锁上，内核看来它们不可运行。再例如 CPU 利用率很高，任务队列也增长。可能是 worker 正在运行但每个任务太慢，也可能是系统里其他进程占用 CPU，让 worker runnable 但很少 running。只看一个队列会误判。正确报告应同时记录用户态队列长度、worker 状态、CPU 利用率、context switch、run queue 压力和阻塞原因。

Linux 的调度器不会理解用户态任务优先级。若线程池只有一个 FIFO 队列，短任务和长任务混在一起，内核只看到 worker 都在运行，不知道某个 worker 正在执行低优先级长任务。若业务需要前台请求优先，就要在用户态运行时设计优先级队列、时间片、任务拆分或隔离线程池。操作系统提供 CPU 时间，业务调度要由程序自己表达。

这个视角也解释了 work stealing 的边界。工作窃取能缓解用户态任务分配不均，让空闲 worker 从其他队列拿任务；但如果某些 worker 在内核层面长期得不到 CPU，或者被 cgroup 限额节流，窃取策略无法凭空创造执行资源。运行时优化要先确认瓶颈在用户态队列、内核调度、I/O 还是硬件资源。

9.41 尾延迟和调度抖动

吞吐系统关注总任务数，低延迟系统还关注尾部任务。一次迁核、一次锁持有者被抢占、一次缺页、一次 cgroup throttling，都可能只影响少数请求，却显著拉高 p99 或 p999。平均耗时变好，不代表用户体验变好。线程调度章节必须把尾延迟作为一等指标，而不是只看总时间。

尾延迟分析要按路径拆开。请求进入队列前等待了多久，排队多久，被哪个 worker 取走，执行中是否阻塞，是否迁移，是否等待锁，是否等待下游 I/O，是否被后台任务干扰。每一段都应有时间戳或事件记录。只有总耗时一个数，就无法知道尾部来自排队、执行、同步还是 I/O。Compute Systems Engine 的任务记录可以逐步加入 submit time、start time、finish time、worker id 和重试次数，这些字段比复杂优化更早产生价值。

调度抖动还有环境因素。手机设备会降频，笔记本会进入省电模式，云服务器可能有邻居噪声，容器可能被 CPU quota 周期性限制。实验报告若不记录环境，尾延迟结论很难复现。对于教学项目，可以先要求读者写出“本次实验不保证独占机器”这样的限制，再逐步学习如何隔离变量。

9.42 亲和性不是固定答案

亲和性常被误解成“绑定一定更快”。实际上，绑定只是减少调度自由度，让执行位置更可控。若任务的数据也在相同 NUMA 节点，绑定可能提高局部性；若数据在远端节点，绑定会稳定地产生远端访问；若系统有其他干扰，适度不绑定反而让调度器能避开忙 CPU。亲和性是一种实验变量，不是默认优化。

一个合理实验应至少比较四组。第一组完全不绑定，作为系统默认行为。第二组只绑定线程，看缓存热度和迁移是否改善。第三组只控制内存放置，观察线程迁移后远端访问。第四组同时绑定线程和内存，测试最佳局部性。若平台不支持 NUMA，也可以比较不绑定、绑定到物理核心、绑定到 SMT sibling、绑定到一组核心。每组都要记录绑定是否成功，因为容器和手机环境可能拒绝或修改 affinity。

亲和性还要和任务分片配合。若 worker 0 固定在 CPU 0，却不断处理来自所有分片的数据，绑定价值有限。更好的策略是 worker、数据分片和队列分片对齐：某个 NUMA 节点的 worker 优先处理本节点数据，跨节点窃取作为兜底。这样能把亲和性从“钉线程”提升为“数据和执行位置共同设计”。后续分布式章节也会看到同样思想：计算应尽量靠近数据。

9.43 线程池指标的最小闭环

线程池想要可调，必须先可观测。最小指标包括提交任务数、完成任务数、队列长度、worker 空闲时间、任务排队时间、任务执行时间、阻塞任务数、拒绝任务数、关闭状态和异常任务数。再进一步，可以记录每个 worker 的任务数、最大连续忙碌时间、窃取次数、空轮询次数和最后一次任务类型。没有这些指标，线程池参数只能凭感觉调整。

指标本身不能破坏性能。高频写指标应使用 worker 本地槽位或 relaxed 原子，周期性汇总；低频控制状态可以用锁保护。不要为了监控每个任务而在热路径写全局锁日志。更好的方式是采样、分片、批量写入和可配置开关。监控代码也是并发系统的一部分，它会改变被监控对象。

指标解释也要谨慎。队列长度高可能是任务太多，也可能是 worker 阻塞；worker 空闲高可能是上游供给不足，也可能是任务粒度太粗导致尾部不均；context switch 高可能是 I/O 服务正常现象，也可能是 CPU-bound 程序过度线程化。指标不是结论，而是提出下一步问题的证据。

9.44 本章习题

习题与作业

习题一：给当前 `parallel_sum` 写一份线程生命周期分析。指出每次调用创建多少线程，线程做多少实际工作，join 等待在哪里发生，哪些成本会在输入很小时占主导。

习题二：设计一个任务粒度实验。固定总输入大小和 worker 数，改变 chunk size。报告要包含任务数、总时间、每个 worker 任务数和尾部完成时间。解释粒度太小和太大分别会出现什么现象。

习题三：构造线程池死锁案例：worker 任务提交子任务并等待。画出等待图，说明为什么增加队列容量不能解决这个死锁。再提出两种运行时语义上的修复方案。

习题四：记录本机或手机上的 CPU 拓扑。写清逻辑 CPU、物理核心或可见核心、是否能看到 SMT、是否有 NUMA 信息、是否允许设置 affinity。不要把无法获取的信息编造成确定结论。

核心思想

线程设计的核心不是“开几个线程”，而是决定哪些状态线程本地，哪些状态共享，哪些共享状态需要同步，哪些等待应该阻塞或异步化。

实验：把并行归约的 worker 数从 1 增加到逻辑 CPU 数的两倍。记录时间、context-switches 和 migrations。解释从哪个点开始扩展性变差。

第 10 章

锁、条件变量与信号量

锁的作用是保护共享状态，让临界区在同一时间只被一个线程执行。锁不是性能敌人，也不是正确性的全部。正确使用锁，需要明确保护对象、锁顺序、临界区长度和等待条件。

10.1 互斥锁保护什么

每把锁都应该有清晰的保护范围。一个队列锁保护 head、tail、size 和 buffer 状态；一个计数器锁保护 value；一个对象锁保护对象不变量。锁如果没有对应的不变量，就容易出现“看起来加锁了但仍然错”的情况。

临界区应尽量短，但不能为了短而破坏不变量。把状态检查放在锁外，可能引入竞态；把耗时 I/O 放在锁内，会扩大阻塞范围。工程上要在正确性和粒度之间找到边界。

Listing 10.1 锁保护队列不变量

```
1 class QueueState {
2 public:
3     void push(std::int32_t value) {
4         std::lock_guard<std::mutex> lock(this->mutex_);
5         this->values_.push_back(value);
6     }
7
8 private:
9     std::mutex mutex_;
10    std::vector<std::int32_t> values_;
11 };
```

这段代码的关键不是 `push_back` 这一行，而是锁和不变量的关系：`values_` 的内部结构不能被多个线程同时修改。如果未来再加入 `size_`、`head_` 或统计字段，它们也必须在同一把锁保护下保持一致。

写锁代码时，应先写“锁保护的不变量”，再写代码。比如有界队列的不变量可以是：`0<=size<=capacity`，`head` 和 `tail` 总是落在环形数组范围内，`size==0` 表示不能 pop，`size==capacity` 表示不能 push。互斥锁保护的是这些关系，不只是保护某一个变量。若 `size` 在锁内改，`head` 在锁外改，这个不变量就被拆碎了。

10.2 锁粒度

粗粒度锁容易证明正确，但并发度低。细粒度锁提高并发度，却增加锁顺序、死锁和状态拆分复杂度。分片锁是一种常见折中：按 key、队列分段或线程组拆成多把锁，让无关操作并行，热点操作仍受保护。

锁粒度应从访问模式出发。如果所有请求都访问同一个热点 key，分片没有帮助；如果请求天然分散，分片锁可以显著减少竞争。锁优化前要先测量等待时间和竞争比例。

10.3 条件变量

条件变量用于等待某个状态变真。它必须和互斥锁配合，并在循环里检查谓词。原因是线程可能被虚假唤醒，也可能在被唤醒后发现条件已经被其他线程消费。正确模式是“持锁检查条件，不满足则等待，醒来继续检查”。

Listing 10.2 条件变量等待谓词

```

1 std::unique_lock<std::mutex> lock(this->mutex_);
2 this->not_empty_.wait(lock, [this]() {
3     return !this->values_.empty();
4 });
5 const std::int32_t value = this->values_.front();

```

这里谓词属于受锁保护的状态。通知本身不携带数据，数据在共享状态里。若不用循环检查谓词，就可能在虚假唤醒或竞争消费后读取空队列。

条件变量最重要的句子是：等待的是谓词，不是通知。通知可能在等待前发生，也可能唤醒多个线程，也可能出现虚假唤醒。只要谓词在锁保护下检查，程序就正确；只依赖“我收到过一次 notify”，程序就脆弱。生产者消费者队列里，`not_empty` 的谓词是队列非空，`not_full` 的谓词是队列未滿。两个谓词对应同一组受锁保护状态。

10.4 信号量和资源计数

信号量表示可用资源数量。它适合限制并发度、实现 bounded buffer 或控制连接池。信号量和互斥锁不同：互斥锁保护状态的一致性，信号量表达资源额度。混用时要小心锁顺序，避免死锁。

10.5 死锁和锁顺序

死锁通常需要互斥、持有并等待、不可抢占和循环等待四个条件。破坏循环等待的常见方法是规定全局锁顺序。复杂系统里，锁顺序文档和锁级别检查很重要，因为死锁往往在低概率路径中出现。

10.6 优先级反转和 convoy

优先级反转发生在低优先级线程持有锁，高优先级线程等待它，而中等优先级线程持续占用 CPU，导致高优先级线程无法前进。锁 convoy 则是多个线程排队等待同一锁，锁释放后唤醒和抢占成本让系统吞吐下降。这些问题说明锁不仅是用户态对象，也和调度器行为相互作用。

工程系统要避免在锁内做不可控工作，例如磁盘 I/O、网络调用、复杂回调和用户插件。锁内执行时间越不可预测，越容易出现尾延迟问题。

10.7 锁的性能路径

mutex 通常有三条路径。第一条是无竞争 fast path，用户态原子操作成功，成本较低。第二条是短暂竞争路径，线程可能自旋几轮，等待锁持有者很快释放。第三条是阻塞路径，线程进入内核等待，涉及 futex、调度和唤醒。性能分析要判断当前主要走哪条路径。一个低竞争 mutex 没必要改成复杂无锁结构；一个长期进入阻塞路径的热点锁，则需要拆分状态、缩短临界区或改变算法。

临界区长度不只由代码行数决定。锁内分配内存、写日志、调用用户回调、访问远端 NUMA 内存、发生 page fault、触发异常路径，都会让持锁时间变长。锁优化报告应列出锁内可能的慢操作，而不只是说“这个锁很粗”。

10.8 读写锁和 RCU 直觉

读多写少场景中，读写锁允许多个读者并发，但写者需要独占。它适合读临界区不太长、写入不太频繁的结构。若读者很多且读路径必须极轻，RCU 的思想更进一步：读者在不阻塞写者的情况下读取旧版本，写者发布新版本后，等所有旧读者离开再回收旧对象。

RCU 的难点不在“读很快”这个口号，而在对象生命周期。旧对象什么时候没人再读，什么时候可以释放，如何处理写者之间的同步，都是必须证明的问题。后面讲无锁内存回收时，会看到相同主题。

10.9 有界队列从不变量开始

Compute Systems Engine 的第一个同步结构应是有界队列，因为它把互斥、条件变量、背压和关闭语义放在同一个小系统里。有界队列不是一个简单容器，它表达了生产者和消费者之间的容量契约：队列未满时生产者可以放入，队列非空时消费者可以取出，队列满时生产者必须等待或失败，队列关闭后等待者必须能醒来并退出。

先写不变量，再写代码。一个环形有界队列至少有这些不变量：容量大于零；`head` 和 `tail` 总在 $[0, \text{capacity})$ 范围内；`size` 总在 $[0, \text{capacity}]$ 范围内；`size==0` 时没有元素可取；`size==capacity` 时没有空槽可写；`head` 指向下一个可读槽；`tail` 指向下一个可写槽。互斥锁保护的是这组关系，而不是某一个整数。

只要写代码时先写不变量，很多错误会提前暴露。例如 `push` 修改 `tail` 但忘了增加 `size`，`try_pop` 读取元素后忘了通知 `not_full`，`size` 用无符号类型导致下溢，析构时仍有线程阻塞，关闭后生产者继续写入，这些都不是语法问题，而是不变量和状态转换没有定义清楚。

10.10 从 try 到 blocking 的接口分层

队列接口不应只有一种操作。常见接口至少分成三类：非阻塞尝试、阻塞等待和关闭。非阻塞 `try_push` 或 `try_pop` 立即返回，适合事件循环或自定义调度；阻塞 `push` 或 `pop` 等待谓词变真，适合普通生产者消费者；关闭 `close` 唤醒等待者，告诉它们系统不再接受新任务或不再产生新元素。

如果队列没有关闭语义，线程池很难优雅退出。worker 可能永远阻塞在空队列上，主线程析构线程池时只能强行终止进程或依赖外部哨兵值。哨兵值在泛型队列中也不可靠，因为每种元素类型未必都有安全哨兵。正确做法是把关闭作为队列状态的一部分，受同一把锁保护，并在关闭时通知所有等待者。

Listing 10.3 有关闭语义的有界队列接口草图

```
1 class BoundedQueue {
2 public:
3     explicit BoundedQueue(std::int32_t capacity);
4
5     bool try_push(std::int32_t value);
6     bool push(std::int32_t value);
7     std::optional<std::int32_t> pop();
8     void close();
9
10 private:
11     std::vector<std::int32_t> buffer_;
12     std::int32_t head_ = 0;
13     std::int32_t tail_ = 0;
14     std::int32_t size_ = 0;
15     bool closed_ = false;
16     std::mutex mutex_;
17     std::condition_variable not_empty_;
18     std::condition_variable not_full_;
19 };
```

这里 `push` 返回 `bool`，是为了表达关闭后不能再写入；`pop` 返回 `optional`，是为了表达关闭且队列为空时消费者应退出。这个接口比单纯抛异常更适合作为线程池内部队列，因为关闭不是异常，而是正常生命周期事件。当然，业务层接口也可以选择抛异常，但内部状态机仍要明确。

10.11 等待谓词的完整写法

生产者等待的谓词不是“收到 not full 通知”，而是“队列未满或已经关闭”。消费者等待的谓词不是“收到 not empty 通知”，而是“队列非空或已经关闭”。关闭必须出现在谓词里，否则关闭时即使 `notify_all`，线程醒来后发现队列仍满或仍空，又会继续睡回去，线程池无法退出。

Listing 10.4 push 的状态转换

```

1 bool BoundedQueue::push(std::int32_t value) {
2     std::unique_lock<std::mutex> lock(this->mutex_);
3     this->not_full_.wait(lock, [this]() {
4         return this->closed_ ||
5             this->size_ < static_cast<std::int32_t>(this->buffer_.size());
6     });
7     if (this->closed_) {
8         return false;
9     }
10
11     this->buffer_[static_cast<std::size_t>(this->tail_)] = value;
12     this->tail_ = (this->tail_ + 1) %
13         static_cast<std::int32_t>(this->buffer_.size());
14     ++this->size_;
15     this->not_empty_.notify_one();
16     return true;
17 }

```

Listing 10.5 pop 的状态转换

```

1 std::optional<std::int32_t> BoundedQueue::pop() {
2     std::unique_lock<std::mutex> lock(this->mutex_);
3     this->not_empty_.wait(lock, [this]() {
4         return this->closed_ || this->size_ > 0;
5     });
6     if (this->size_ == 0) {
7         return std::nullopt;
8     }
9
10    const std::int32_t value =
11        this->buffer_[static_cast<std::size_t>(this->head_)];
12    this->head_ = (this->head_ + 1) %
13        static_cast<std::int32_t>(this->buffer_.size());
14    --this->size_;
15    this->not_full_.notify_one();
16    return value;
17 }

```

这两个片段体现了条件变量的关键原则：状态改变和谓词检查都在同一把锁保护下；通知只用于让等待者重新检查谓词；关闭是状态机的一部分。注意通知可以在持锁时调用，也可以在释放锁后调用，不同写法有不同唤醒和竞争细节。教材阶段更重要的是先保证谓词正确，再讨论微小性能差异。

10.12 锁内代码的边界

有界队列的锁内代码应只做状态转换，不应执行用户回调、复杂分配、日志格式化或长时间 I/O。若 **push** 在锁内调用用户提供的处理函数，队列锁就保护了不可控代码，任何慢操作都会阻塞所有生产者和消费者。若 **pop** 在锁内执行任务，线程池全局队列会退化串行执行器。

这条原则可以推广到所有锁。锁内只维护不变量，锁外做耗时工作。若必须在锁内移动复杂对象，应考虑预分配、移动语义、异常安全和对象析构成本。C++ 中对象析构也可能执行代码，如果最后一个引用在锁内释放，析构函数可能做不可控工作。高质量并发代码会尽量让重析构发生在锁

外。

10.13 异常安全和状态回滚

C++ 队列如果存储复杂对象，`push` 可能在移动或拷贝元素时抛异常。若异常发生在 `tail` 和 `size` 已经修改之后，不变量就会损坏。教学示例用 `std::int32_t` 可以避开这个问题，但正式教材必须说明边界：泛型队列需要把可能抛异常的操作和状态提交点分开，或者要求元素移动不抛异常，或者使用预构造槽位和 `placement new` 小心管理生命周期。

状态机里要有提交点。对于 `push`，元素成功放入槽位并且 `tail/size` 更新后，操作才对消费者可见；对于 `pop`，元素被取出并且 `head/size` 更新后，槽位才重新可用。若中间步骤可能失败，就要保证失败前队列仍保持旧状态，或失败后能回滚。锁只能提供互斥，不能自动提供异常安全。

10.14 信号量实现 bounded buffer 的视角

有界队列也可以用两个信号量表达：一个信号量记录空槽数量，一个信号量记录已有元素数量，再用互斥保护环形索引。生产者先获取空槽，再写入；消费者先获取元素，再读取。这个模型把“资源数量”和“状态互斥”分开，适合帮助读者理解信号量。

但信号量版本同样需要关闭语义。若消费者阻塞在元素信号量上，关闭时如何唤醒？若生产者阻塞在空槽信号量上，关闭时如何退出？标准信号量本身不携带“关闭”状态，需要额外状态和唤醒策略。条件变量版本把谓词写在代码里，更适合教学；信号量版本更接近资源额度控制。两者没有绝对优劣，关键是状态机是否完整。

10.15 锁顺序和组合操作

单个队列容易证明，多个锁组合就会出现锁顺序问题。假设线程池有全局队列锁、worker 本地队列锁和指标锁。如果 worker 持有本地队列锁时尝试拿全局队列锁，而另一个线程持有全局队列锁时尝试拿本地队列锁，就可能死锁。工程系统应定义锁级别：例如先全局配置锁，再队列锁，再指标锁；或者避免同时持有多把锁，用消息和局部复制拆开组合操作。

组合操作还会诱导“为了省事加一把大锁”。大锁让正确性简单，却可能把整个运行时串行化。合理做法是先用粗锁实现 reference，写清不变量和测试；再根据测量拆分锁，而不是一开始就写复杂细锁。拆锁时，每拆一把都要重新写不变量：哪些状态仍由旧锁保护，哪些状态由新锁保护，跨锁关系如何保持。

10.16 futex 慢路径和用户态 fast path

Linux 上常见 mutex 和 condition variable 都会尽量把无竞争路径留在用户态。无竞争加锁可能只是一次原子状态改变；竞争时才进入 futex 让线程睡眠。条件变量等待也会释放 mutex 并进入等待，唤醒后重新竞争 mutex。这个设计说明锁不是一上来就系统调用，系统调用通常是竞争或等待的慢路径。

阅读 futex 相关内容时，要抓住三个点。第一，用户态变量保存同步状态，内核只在需要睡眠和唤醒时参与。第二，等待必须和用户态状态检查配合，避免丢失唤醒。第三，唤醒不等于立即执行，被唤醒线程还要进入调度器、被安排到 CPU，并重新竞争锁。因此高竞争条件变量的成本不只是 `notify_one`，还包括调度和再次竞争。

10.17 互斥锁保护的是不变量

很多人把 mutex 理解成“保护变量”，这句话不够精确。mutex 保护的是一组变量之间的不变量。以环形队列为例，`head`、`tail`、`size` 和 `buffer` 不是四个孤立变量，而是共同满足容量边界、满空状态、元素位置和可用槽位这些关系。只把 `size` 改成原子并不能让队列线程安全，因为 `size` 和 `head/tail` 的关系仍然会被并发修改破坏。

写并发代码时，第一步应写不变量表。哪些字段属于同一个状态机，哪些字段只用于指标，哪些字段可以近似，哪些字段必须精确。锁的粒度也由不变量决定。如果两个字段永远一起变化，拆成两把锁可能反而制造跨锁一致性问题；如果两个字段从语义上独立，一把大锁可能只是偷懒。锁设计不是“粗锁慢、细锁快”的简单比较，而是不变量边界和竞争形状的平衡。

```
bounded queue invariant:
0 <= size <= capacity
head and tail are valid positions
size == 0 means no readable slot
size == capacity means no writable slot
readable slots form one circular interval from head
writable slots are the complement interval
```

有了这张表，代码审查就更具体。每个会修改状态的函数，都必须在持锁状态下从一个合法状态转换到另一个合法状态。每个等待谓词，都必须对应一个状态条件，而不是对应一次通知。每个关闭路径，都必须说明关闭后哪些操作仍允许，哪些操作返回失败，哪些等待者必须被唤醒。

10.18 条件变量不是事件

条件变量最常见的误解，是把它当成事件通知系统。正确理解是：条件变量让线程在某个谓词不满足时睡眠，并在可能满足时醒来重新检查。通知本身不保存状态；如果通知发生时没有等待者，通知不会留在条件变量里等后来者。状态必须保存在受 mutex 保护的变量中。

这条原则可以解释为什么等待必须写成循环或谓词版本。线程可能被虚假唤醒；也可能被唤醒后别的线程先抢到锁并消费了资源；也可能关闭状态改变但队列仍空。醒来不代表条件成立，只代表“应该重新检查”。把 `if` 写成 `while` 不是形式主义，而是条件变量语义的一部分。

Listing 10.6 条件变量等待的是谓词，不是通知次数

```
1 std::optional<std::int32_t> pop_wait() {
2     std::unique_lock<std::mutex> lock(this->mutex_);
3     this->not_empty_.wait(lock, [this]() {
4         return this->closed_ || this->size_ > 0;
5     });
6
7     if (this->size_ == 0) {
8         return std::nullopt;
9     }
10
11     const std::int32_t value =
12         this->buffer_[static_cast<std::size_t>(this->head_)];
13     this->head_ = (this->head_ + 1) %
14         static_cast<std::int32_t>(this->buffer_.size());
15     --this->size_;
16     this->not_full_.notify_one();
17     return value;
18 }
```

这段代码里，`closed_` 和 `size_` 都在同一把锁下读写。若关闭函数只设置 `closed_` 却不 `notify_all`，等待线程可能永远睡眠；若关闭函数通知但不把 `closed_` 放进谓词，线程醒来后又睡回去。状态和通知必须配套。

10.19 关闭语义是并发容器的一等接口

很多教学队列只写 `push` 和 `pop`，但线程池真正需要的是关闭语义。关闭不是析构的附属动作，而是容器状态机的一部分。一个队列关闭后，生产者是否还能写入？消费者是否能继续取出已经存在的元素？空队列关闭后消费者返回什么？满队列关闭后生产者如何退出？这些问题决定 worker 能否优雅停止。

常见策略是：关闭后拒绝新 `push`；已经在队列中的元素仍允许被 `pop` 取出；当队列为空且关

闭时，`pop` 返回空值；关闭函数唤醒所有生产者和消费者。这种策略适合线程池 `drain`：不再接收新任务，但已提交任务可以执行完。另一种策略是 `cancel`：关闭后直接丢弃队列中剩余任务。它适合进程退出或错误恢复，但必须明确通知调用者哪些任务未执行。没有声明的关闭策略，会让上层业务在退出、异常和超时时出现悬挂。

```
queue close policy:
  open + not full: push succeeds
  open + full: producer waits
  closed: push fails
  closed + nonempty: pop drains existing item
  closed + empty: pop returns null
```

线程池的 `shutdown`、`shutdown_now`、`wait` 和析构函数应围绕这套语义设计。析构函数里悄悄阻塞等待所有任务完成，可能让用户在持锁析构时死锁；析构函数直接丢任务，又可能破坏业务语义。教材应鼓励读者把关闭写成显式 API，而不是把复杂语义藏在析构里。

10.20 锁粒度的演化

一个复杂系统最稳妥的路线，通常是先写粗锁 `reference`。粗锁版本把不变量集中在一处，容易测试，容易与串行 `reference` 对比。等到测量证明锁确实是瓶颈，再拆成细锁、分片锁、读写锁或无锁结构。直接从细锁开始，很容易得到一个既不快也不正确的系统。

拆锁时要写迁移计划。第一步，确认锁保护的全部状态和不变量。第二步，把状态按独立性分组，设计新锁的保护范围。第三步，找出跨组操作，例如移动任务、关闭队列、汇总指标。第四步，为跨组操作定义锁顺序或避免同时持锁。第五步，保留粗锁版本作为 `reference` 或测试 `oracle`。拆锁不是把一个 `mutex` 机械替换成多个 `mutex`，而是重新设计状态机边界。

读写锁也不是默认更快。读多写少且读临界区较长时，它可能有收益；读临界区很短时，读写锁本身的元数据和公平策略可能比普通 `mutex` 更重；写者频繁时，读写锁还可能造成写者饥饿或读者抖动。若读路径只需要不可变快照，RCU 或 `copy-on-write` 可能比读写锁更适合。同步原语必须和访问模式匹配。

10.21 锁竞争的观测

锁竞争不能只靠感觉。应用指标可以记录等待次数、等待总时间、最长等待、持锁时间、队列长度和唤醒次数。系统工具可以观察 `futex` 调用、调度切换、CPU 利用率和 `off-CPU` 时间。若一个程序 CPU 使用率很低但延迟很高，可能大量时间在等待锁或 I/O；若 CPU 使用率很高但吞吐不升，可能在自旋或 CAS 失败。

持锁时间和等待时间要分开。持锁时间长，说明临界区里工作太多；等待时间长，可能因为竞争高，也可能因为持锁线程被调度走。若临界区很短但等待时间长，要看线程数是否远超核心、是否有优先级反转、是否发生 NUMA 迁移。锁问题是执行图问题，不是单行代码问题。

在 Linux 上，可以结合 `perf`、`strace-efutex`、调度 `trace` 和应用内部统计。教材不要求每个读者都能开启所有权限，但要让读者知道证据链：用户态等待指标说明哪里堵，`futex` 说明是否进入内核睡眠，调度指标说明线程是否被抢占，CPU 拓扑说明 `cache line` 迁移是否昂贵。

10.22 优先级反转和公平性

锁还涉及调度语义。优先级反转指高优先级线程等待低优先级线程持有的锁，而低优先级线程又迟迟得不到 CPU。实时系统中，这会造成严重延迟。某些系统通过优先级继承缓解：持锁的低优先级线程临时继承等待者的高优先级，尽快释放锁。普通服务端虽然不一定使用实时优先级，也会遇到类似现象：关键请求等待后台任务持有的锁，尾延迟被放大。

公平锁按等待顺序唤醒线程，可以减少饥饿，但可能降低吞吐，因为它限制了“刚好在 CPU 上的线程再次获得锁”的机会。非公平锁吞吐可能更高，但某些线程可能长期等待。条件变量唤醒一个还是唤醒全部，也有公平和惊群权衡。高性能系统要根据业务目标选择：批处理更关心总吞吐，交互服务更关心尾延迟和饥饿。

10.23 RAII 和异常边界

C++ 中锁必须用 RAII 管理。手写 `lock` 和 `unlock` 容易在异常、早返回和多分支中漏解锁。`std::lock_guard` 适合简单临界区，`std::unique_lock` 适合条件变量、延迟加锁和手动解锁。教材示例应让读者形成习惯：锁对象的作用域就是临界区，作用域越小越好，锁外做耗时工作。

RAII 也不能替你设计不变量。一个函数里拿了锁，但把引用返回给外部，让外部在锁释放后修改受保护状态，仍然错误。一个类内部用 `mutex`，但复制构造没有定义好，也可能把锁和状态复制出奇怪语义。并发类通常应禁用复制，或者显式定义移动和共享语义。锁只是同步工具，不是封装边界的替代品。

10.24 生产者消费者的完整测试

生产者消费者测试不能只检查几个值是否按顺序出来。完整测试至少包含容量为一、容量为多、单生产者单消费者、多生产者多消费者、生产者快消费者慢、消费者快生产者慢、关闭空队列、关闭非空队列、满队列关闭、重复关闭、析构前关闭、压力运行和 `ThreadSanitizer`。每个测试对应一个状态机边界。

还要测试没有丢失和没有重复。可以让每个生产者生成带 `producer id` 和 `sequence` 的任务，消费者记录所有任务，最终检查总数、唯一性和每个生产者内部顺序。如果队列声明全局 FIFO，就检查全局顺序；如果只声明每生产者顺序或无序，就不要测试不存在的保证。接口保证必须和测试一致。

```
queue stress test data:
  producer id: identifies writer
  sequence: monotonically increasing per producer
  payload: optional checksum
  consumer log: records every consumed item
  final check: no loss, no duplicate, declared order holds
```

性能测试也要有边界。线程创建成本不应混进队列操作；输入生成不应混进消费；日志输出不应放在热循环；关闭时间和正常处理时间要分开。否则队列性能报告很容易测到无关成本。

10.25 读写锁的适用边界

读写锁允许多个读者并发进入，写者独占进入。它看起来很适合“读多写少”，但真实收益取决于读临界区长度、写入频率、实现公平性和缓存行为。若读临界区很短，读写锁内部维护读者计数和状态的成本可能超过普通 `mutex`；若写者频繁，读者并发优势会被写者阻塞抵消；若实现偏向读者，写者可能饥饿；若实现偏向写者，读者尾延迟可能变差。

读写锁特别适合读操作较长、读者之间不修改共享状态、写操作低频且能接受等待的结构，例如较大的配置表、路由快照或只读索引。它不适合每次请求都要读写的小状态，也不适合读路径本身会更新统计字段的场景。很多“读操作”其实包含隐藏写，例如更新 LRU、增加引用计数、记录访问时间，这会破坏读者并发。

若读多写少结构可以使用不可变快照，RCU 或 `copy-on-write` 可能比读写锁更好。读者读取一个稳定指针，不加锁；写者复制一份新结构，修改后发布；旧结构延迟回收。这样读路径更轻，但写路径复制成本和回收复杂度更高。读写锁、RCU、`copy-on-write` 都是同一问题的不同取舍：读者成本、写者成本、一致性和回收。

10.26 锁分解案例：统计服务

假设一个统计服务维护三类状态：全局配置、每个 `worker` 的计数、最近错误列表。朴素实现可以用一把大锁保护所有状态，正确性最简单，但每次更新计数都和读取配置、写错误列表互相阻塞。拆锁前，应先写状态关系。配置很少更新，读多；计数高频写，读少；错误列表低频写，偶尔查询。它们的访问模式不同，适合不同同步策略。

一种改进是配置使用快照或读写锁，计数使用每 `worker` 分片，错误列表使用单独 `mutex`。这样高频计数不再抢配置锁，错误查询也不阻塞计数热路径。拆锁后要检查跨状态操作。例如“更新配

置并清空统计”是否需要同时触碰配置和计数？如果需要，就要定义锁顺序或改成两阶段操作。拆锁的难点不在把 mutex 数量变多，而在找出跨状态不变量。

Listing 10.7 按访问模式拆分状态的结构形态

```

1 struct alignas(64) WorkerCounter {
2     std::uint64_t accepted = 0;
3     std::uint64_t rejected = 0;
4 };
5
6 class RuntimeStats {
7 public:
8     explicit RuntimeStats(std::int32_t worker_count)
9         : counters_(static_cast<std::size_t>(worker_count)) {}
10
11     void record_accept(std::int32_t worker_id) {
12         this->counters_[static_cast<std::size_t>(worker_id)].accepted += 1;
13     }
14
15 private:
16     std::vector<WorkerCounter> counters_;
17     mutable std::mutex error_mutex_;
18     std::vector<std::string> recent_errors_;
19 };

```

这段代码仍然省略了并发读取总数的同步语义。若只有 worker 写、实验结束后汇总，可以不加锁；若运行中监控线程读取，就要定义近似还是精确。如果要求精确实时快照，需要额外同步，分片计数的写入优势会被读取语义部分抵消。同步设计总是从读写语义出发。

10.27 锁顺序表

多个锁共存后，项目需要锁顺序表。锁顺序表不是形式主义，它让代码审查能发现潜在死锁。例如规定：配置锁先于任务图锁，任务图锁先于队列锁，队列锁先于指标锁。任何代码如果反向获取，就必须重构或证明不会并发。没有锁顺序表，死锁常常在很久后由一次看似无关的功能触发。

锁顺序表还要说明哪些锁不能在持有时调用用户回调、不能做 I/O、不能等待 future。最危险的不是持有两把锁，而是持锁进入未知代码。用户回调可能再进入运行时，拿另一把锁，或者阻塞等待当前锁保护的状态。高质量库通常在锁内只修改内部状态，复制出需要处理的数据，然后锁外调用回调。

10.28 条件变量和多谓词

真实队列常有多个等待条件：不空、不满、关闭、取消、优先级任务到达、队列水位下降。把所有条件混成一个布尔变量会让状态机混乱。更好的方式是用明确状态字段和谓词函数。例如生产者等待“有空间或关闭”，消费者等待“有任务或关闭”，drain 等待“未完成计数为零”。每个谓词都应能用当前状态直接判断。

多个条件变量可以减少无关唤醒，但也增加复杂度。一个 bounded queue 常用 `not_empty` 和 `not_full` 两个条件变量；线程池可能还有 `idle_cv` 用于 wait；关闭时通常要 `notify_all` 所有相关条件变量。通知遗漏会导致悬挂，通知过多会导致惊群。先保证状态机正确，再用指标判断是否需要优化唤醒策略。

10.29 Semaphore 和资源额度

信号量适合表达资源额度。数据库连接池、并发请求上限、I/O in-flight 限额、令牌桶都可以用信号量思想理解。它和 mutex 的区别是：mutex 保护临界区不变量，semaphore 控制有多少个许可。把二者混淆会导致设计不清。一个线程拿到许可后，仍可能需要 mutex 修改共享队列。

信号量还要处理取消和关闭。线程等待许可时，如果上层请求超时，等待能否被取消？系统关闭时，等待者如何醒来？许可已经拿到但任务失败，是否归还？这些都是资源状态机的一部分。信号量不是“比条件变量简单”的替代品，它只是把数量资源表达得更直接。

10.30 锁保护对象的文档格式

并发类应在代码附近写清每个锁保护哪些字段，以及哪些字段是原子或线程本地。注释不需要冗长，但要能帮助维护者判断修改是否安全。比如：“`mutex_` protects head, tail, size, buffer, closed”，“`stats_` is worker-owned and read after join”，“`closed_` is only read while holding mutex”。这种注释比泛泛写“thread-safe”有用得多。

接口文档也要写清线程安全级别。某个对象是否允许多个线程同时调用 `push`？是否允许一个线程析构时其他线程仍在调用？`close` 是否幂等？`wait` 是否可以和 `submit` 并发？这些问题如果不写，调用者会按自己的理解使用，最终把内部不变量打破。

10.31 案例：关闭线程池

关闭线程池是锁和条件变量的综合案例。线程池有任务队列、未完成计数、worker 列表和状态。调用 `shutdown` 后，`submit` 应拒绝新任务，worker 应继续取出已接受任务，队列为空且状态为 shutting down 时退出。`wait_idle` 应等待未完成计数为零，但不一定关闭线程池。析构应确保 worker 已 join。

错误实现常见于三个地方。第一，只设置 `closed` 但没有 `notify_all`，睡眠 worker 不醒。第二，队列空就让 worker 退出，但 `submit` 仍可能并发加入任务。第三，任务抛异常后未完成计数没有减少，`wait` 永久阻塞。正确实现需要一张状态机，而不是几处布尔判断。

10.32 案例：有界队列和内存预算

有界队列的容量可以按任务数，也可以按字节。线程池任务通常按数量限制；I/O batch 队列更需要按字节限制。若每个任务大小差别很大，只限制数量可能导致内存爆炸。锁保护的状态也会更复杂：不仅有 `size`，还有 `bytes_in_queue`。生产者等待谓词变成“任务数未满足且字节数未超，或关闭”。

这个案例说明锁保护的是不变量集合。任务数、字节数、head/tail、closed 共同决定队列状态。拆开任何一个，都可能破坏背压语义。

10.33 锁章验收清单

本章完成后，项目里的每个锁都应能回答：保护哪些字段，临界区是否调用用户代码，是否可能同时持有多把锁，锁顺序是什么，等待谓词是什么，关闭时谁被唤醒，异常路径是否保持不变量，测试是否覆盖容量一、关闭、并发压力和重复关闭。若回答不出来，这把锁就是维护风险。

10.34 锁章节总结

锁的价值是让复杂不变量变得可证明，条件变量的价值是让线程在谓词不满足时睡眠，信号量的价值是表达资源额度。它们不是低级替代品，而是并发系统的基础工具。先用锁写正确 reference，再用测量决定是否拆锁、分片、读写锁、RCU 或无锁。把锁视为失败，只会诱导更难证明的错误代码。

10.35 同步原语决策表

选择同步原语时，先看状态形状。一个复杂不变量，用 `mutex`；等待某个谓词，用 `condition variable`；控制资源额度，用 `semaphore`；读多写少且读临界区较长，用读写锁或快照；只需要统计，用 `relaxed atomic`；单生产者单消费者队列，可以考虑 SPSC ring；多生产者多消费者且关闭语义复杂，先用 `mutex bounded queue`。选择同步原语不是按“高级程度”，而是按状态语义。

10.36 案例：锁内析构

C++ 中最后一个 `std::shared_ptr` 在锁内离开作用域，可能触发对象析构。析构函数如果写日志、释放大内存、关闭文件或调用回调，就把不可控工作放进锁内。高质量代码会在锁内把对象移动到局部变量，释放锁后再让局部变量析构。这个细节常常影响尾延迟。

锁内析构说明临界区不只包含显式语句，也包含对象生命周期副作用。审查锁内代码时，要看构造、析构、分配、回调和异常，不只看表面几行。

10.37 案例：等待谓词丢失

一个常见 bug 是消费者等待 `not_empty`，但关闭时只设置 `closed` 并通知 `not_full`。结果消费者如果正睡在 `not_empty`，就永远不醒。另一个 bug 是等待谓词只写 `size` 大于零，不包含 `closed`；关闭后消费者醒来，发现 `size` 仍为零，又睡回去。正确代码必须让每个等待者的谓词覆盖所有能让它退出等待的状态。

这个案例看似简单，却是线程池停不下来的常见根源。并发代码的退出路径和成功路径一样重要。

10.38 本章落到贯穿项目

Compute Systems Engine 当前的 `BoundedMpmcQueue` 已经有容量、head、tail、size、mutex 和 `not_full`，但它还不是完整线程池队列，因为它只有阻塞 push 和非阻塞 try pop，没有 `not_empty` 等待，也没有 close。作为教学阶段的最小模块可以接受，但后续运行时章节必须补全关闭语义和消费者等待，否则 worker 无法优雅阻塞和退出。

本章要求项目至少增加设计文档或测试计划：列出队列不变量，列出 push、try pop、未来 blocking pop 和 close 的状态转换，列出需要的测试。测试不能只 push 两个值再 pop 两个值，还要覆盖容量为一、队列满时生产者等待、队列空时消费者等待、close 唤醒等待者、多生产者多消费者压力、异常容量和析构时没有等待线程。

10.39 状态机比代码片段更重要

同步代码最怕只看局部片段。一个 push 函数看起来正确，一个 pop 函数看起来正确，合在一起仍可能在关闭、异常、等待和析构时出错。因此并发容器应先写状态机。以有界队列为例，状态至少包括 open empty、open partial、open full、closing nonempty、closed empty。每个状态下，push、try_push、pop、try_pop、close、wait_empty 的行为都要定义。

状态机能暴露隐藏问题。open full 状态下，阻塞 push 等待什么谓词？如果此时调用 close，生产者应返回失败还是继续等待？closing nonempty 状态下，消费者能否继续取出剩余任务？closed empty 状态下，pop 返回空值还是抛异常？这些问题不回答，代码只能靠偶然行为。一个高质量并发库的接口文档，应像协议一样说明状态转换。

状态机还要说明线性化点。对于 push，元素写入槽位并更新 tail 和 size 后，操作对消费者可见；对于 pop，元素被取出并更新 head 和 size 后，槽位重新可用；对于 close，closed_ 从 false 变为 true 的那一刻，后续生产者必须观察到关闭。线性化点不清楚，就无法判断并发调用的先后语义。

10.40 背压不是错误路径

有界队列满了，生产者等待，这不是程序变差，而是背压在工作。背压的目的，是让下游资源不足时，上游不要无限制地产生任务和占用内存。没有背压的系统，短时间 benchmark 可能看起来更快，因为它把工作全部塞进内存；一旦输入持续增长，就会出现内存膨胀、延迟暴涨和最终崩溃。有界队列用等待或拒绝把压力反馈给上游。

背压策略要和业务语义匹配。日志系统可以选择丢弃低优先级调试日志；任务运行时通常不能丢计算任务；网络服务可以对新请求返回繁忙；分布式 shuffle 可以暂停上游分片读取。队列只提供机制，业务决定策略。教材中如果只写“队列满则阻塞”，还不够完整；还要讨论阻塞会不会占住 compute worker，是否需要异步等待，是否需要超时，是否需要取消。

背压指标也要写入报告。队列满的次数、生产者等待总时长、最大队列长度、丢弃任务数、拒绝

请求数，这些指标能说明系统是在健康限流，还是下游长期跟不上。没有指标，开发者可能把背压误认为锁慢，然后盲目增加容量。容量变大只会把压力藏得更深，尾延迟和内存风险更高。

10.41 锁和异常传播

线程池中的任务可能抛异常。异常传播如果处理不好，会破坏锁保护的不变量。最基本原则是：任务执行不应发生在队列锁内，任务异常不应让未完成计数漏减，异常对象的发布要有同步关系。队列只负责交付任务，worker 从队列取出任务后释放锁，再执行任务。执行结束后，不管成功还是异常，都要更新任务状态和未完成计数，并通知等待者。

C++ 的 RAII 能帮助释放锁，但不能自动维护业务状态。若一个任务执行前增加了 in-flight 计数，异常路径必须减少；若 worker 把错误写入 shared state，等待者读取错误前要有同步；若异常对象包含动态分配资源，生命周期要覆盖读取者。并发异常不是“catch 一下”这么简单，它和状态机、通知和对象生命周期同时相关。

一个实用模式是把任务结果分成状态和载荷。状态可以是 Pending、Running、Succeeded、Failed、Cancelled；载荷是返回值或错误信息。worker 先写载荷，再在锁内或 release 语义下发布状态，然后通知等待者。等待者醒后检查状态，再读取载荷。这个模式和原子章节的对象发布一致，只是这里多了一层任务语义。

10.42 测试等待路径要制造极端条件

同步代码的测试不能只跑正常路径。容量为一的队列比容量很大的队列更容易暴露满和空之间的状态转换；一个 producer 一个 consumer 能测基本交替；多个 producer 一个慢 consumer 能测 `not_full`；一个 producer 多个 consumer 能测 `not_empty` 和惊群；关闭时有线程阻塞在 push 和 pop，能测 `notify_all` 是否完整。小容量、慢消费者、随机 yield、重复关闭和异常任务，是并发测试的基本工具。

测试还应覆盖析构边界。线程池析构时是否自动 shutdown？如果还有外部线程提交任务，行为是什么？析构能否在 worker 正在执行任务时等待？如果析构发生在某个任务持有业务锁时，会不会死锁？这些问题不一定都由队列解决，但队列和线程池接口必须定义。未定义行为可以存在，但必须明确禁止，而不是让用户猜。

压力测试不能替代不变量证明，但能发现实现错误。可以把队列操作与串行 reference 对比：多个线程随机 push 和 pop，同时记录成功操作日志，最终验证元素没有丢失、没有重复、容量不越界、关闭后没有新元素进入。对于不可确定顺序的并发队列，验证重点不是全局顺序完全一致，而是每个成功 push 最多被 pop 一次，关闭语义符合文档。

10.43 本章习题

习题与作业

习题一：为当前项目的 `BoundedMpmcQueue` 写不变量清单。逐行解释 `push` 和 `try_pop` 怎样保持这些不变量。指出当前接口缺少哪些线程池需要的语义。

习题二：把 `bounded queue` 设计成支持 `close`。不要求立刻写完整代码，但必须画出状态转换：`open nonempty`、`open empty`、`open full`、`closed nonempty`、`closed empty`。说明每个状态下 `push` 和 `pop` 应返回什么。

习题三：设计一个队列等待实验。多个生产者写入，消费者故意变慢，让队列经常满。记录生产者等待次数、最大队列长度和总吞吐。解释这是背压，不是单纯性能退化。

习题四：阅读 `glibc`、`musl` 或 `Linux futex` 文档和源码入口，写清 `mutex` 低竞争 `fast path` 和竞争 `slow path` 的差异。报告中必须说明哪些部分发生在用户态，哪些部分进入内核。

常见误区

不要把锁竞争问题简单理解成“换成无锁”。如果临界区保护的是复杂不变量，无锁实现可能更难证明正确，也可能在高竞争下 CAS 失败更多。

实验：实现一个 bounded queue。先用 mutex 和 condition variable 写阻塞 push/pop，再记录高生产者数量下的等待时间。解释锁保护的状态和等待谓词。

第 11 章

原子操作与内存序

原子操作提供不可分割的读写或读改写，并参与线程间同步。内存序定义操作之间的可见性和重排约束。理解原子操作，必须区分原子性、顺序性和生命周期。

11.1 原子不是普通变量

`std::atomic` 的 `load`、`store`、`fetch_add`、`compare_exchange` 等操作会生成特殊指令或特殊约束。它们可以避免数据竞争，但不自动保证算法正确。一个原子计数器容易写，一个无锁队列需要处理节点发布、竞争失败、ABA 和回收。

低竞争下，原子操作可能比 `mutex` 轻；高竞争下，多个线程反复修改同一 `cache line`，原子也会扩展性很差。判断依据仍然是共享写频率和一致性流量。

11.2 常见内存序

`memory_order_relaxed` 只保证原子性，不建立跨变量同步。计数器统计常用 `relaxed`，因为只关心最终数值。`memory_order_release` 发布写入，`memory_order_acquire` 获取发布的结果，二者配合可建立 `happens-before`。`memory_order_seq_cst` 提供更强的全局顺序，但成本和约束也更强。

选择内存序的原则是从语义出发，而不是从性能愿望出发。若一个线程写好对象后通过原子标志发布，写标志应 `release`，读标志应 `acquire`。若只是统计事件数量，`relaxed` 足够。

Listing 11.1 `release/acquire` 发布数据

```
1 struct Shared {
2     std::int32_t data = 0;
3     std::atomic<bool> ready = false;
4 };
5
6 void producer(Shared& shared) {
7     shared.data = 42;
8     shared.ready.store(true, std::memory_order_release);
9 }
10
11 std::int32_t consumer(Shared& shared) {
12     while (!shared.ready.load(std::memory_order_acquire)) {
13     }
14     return shared.data;
15 }
```

`release store` 之前的普通写，在 `acquire load` 看到该 `store` 后，对 `consumer` 可见。这里不能把 `ready` 简单改成普通 `bool`，也不能把 `acquire/release` 无脑改成 `relaxed`；否则程序缺少跨线程发布关系。

理解 `acquire/release` 时，不要把它想成“刷新所有缓存”这样粗糙的动作。更准确的教材模型是：`release` 把之前的写放在发布点之前，`acquire` 在看到这个发布后，让之后的读写不能越过获取点，并建立 `happens-before`。具体硬件怎样实现，取决于架构强弱和编译器约束。C++ 程序只应依赖语言层面的同步关系。

11.3 Relaxed 的正确用途

Relaxed 常用于统计计数、采样标志、引用计数的某些阶段和不参与对象发布的指标。它保证原子变量自身不会数据竞争，但不保证其他变量的可见性。用 relaxed 写高性能代码时，必须能说清“我只需要这个原子变量自己的数值，不需要借它同步别的数据”。

例如请求计数器可以用 relaxed，因为最终统计只关心总次数。对象指针发布不能只用 relaxed，因为读到指针的线程还需要看到对象内部字段已经初始化。

11.4 CAS 循环

Compare-and-swap 读取旧值，若仍等于期望值就写入新值。无锁算法常用 CAS 循环处理竞争。CAS 失败不表示错误，而是说明其他线程已经修改状态，需要重新读取并尝试。

CAS 循环必须考虑进度。低竞争时很快，高竞争时可能大量失败。复杂结构还要考虑 ABA：一个位置从 A 变 B 又变回 A，CAS 看到 A 可能误以为没有变化。解决方法包括版本号、hazard pointer、epoch 回收等。

CAS 循环还必须考虑循环体里是否做了不可重复副作用。失败重试意味着循环体可能执行多次，如果每次尝试都分配内存、写日志、修改外部状态或增加不可回滚计数，就会产生额外语义问题。常见写法是先准备候选新状态，在 CAS 成功后才执行对外可见副作用；失败时只更新局部期望值并重试。

11.5 CAS 的 weak 和 strong 版本

`compare_exchange_weak` 允许伪失败，即使当前值等于期望值也可能失败，因此通常放在循环里。`compare_exchange_strong` 不允许伪失败，更适合不在循环里的单次尝试。很多平台上二者生成代码可能接近，但语义差异必须理解。

CAS 还有一个容易忽视的行为：失败时会把当前实际值写回 `expected` 参数。这让循环可以基于新值继续尝试，但也容易让初学者误用 `expected`。写无锁代码时，要把成功内存序和失败内存序都写清楚。

11.6 内存屏障和硬件

不同架构的内存模型不同。x86-64 相对强，ARM 和 POWER 更弱。C++ 内存序提供跨平台抽象，编译器和硬件会根据目标架构生成必要约束。不要因为某段代码在 x86 上“看起来没问题”，就认为它是正确的并发程序。

11.7 原子与缓存一致性

原子读写通常会目标 cache line 进入独占修改路径。多个核心同时对一个原子变量做 `fetch_add`，实际上是在争夺同一条 cache line 的所有权。内存序越强，编译器和硬件重排空间越小；竞争越强，一致性流量越大。

这就是为什么高性能计数常用分片计数器。每个线程或每个核心写自己的本地计数，读取总数时再合并。它牺牲实时精确读取成本，换取写入热路径扩展性。

11.8 内存序选择流程

选择内存序可以按问题反推。第一，是否只需要原子变量自身不撕裂、不数据竞争。如果是统计计数，通常 relaxed 足够。第二，是否通过某个原子变量发布了其他普通数据。如果是，需要 `release/acquire` 或更强同步。第三，是否需要多个线程观察到一个全局一致顺序。如果确实需要，才考虑 `memory_order_seq_cst`。第四，是否可以用 mutex 表达更清晰的不变量。如果共享状态复杂，mutex 往往比一组难懂的原子更可靠。

内存序不是性能调参旋钮。把 `memory_order_seq_cst` 改成 relaxed 之前，必须写出被保留的同步关系和被放弃的顺序关系。若写不出来，说明程序设计还没清楚到可以安全降级。

11.9 x86 和 ARM 的差异

x86-64 的内存模型相对强，很多 acquire/release 在硬件指令上看起来很轻；ARM 等架构更弱，可能需要更显式的屏障。可移植 C++ 代码不能因为在 x86 上测试通过，就省略必要内存序。教材示例必须按 C++ 语义写正确，而不是按某台机器的偶然行为写正确。

11.10 三件事必须分开

学习原子操作时，最重要的是把三件事分开：原子性、可见性和顺序性。原子性回答“这个变量本身的读写会不会撕裂，会不会产生数据竞争”。可见性回答“一个线程写入的其他数据，另一个线程什么时候能看到”。顺序性回答“多个操作之间能否被编译器或硬件重排，其他线程能以什么顺序观察到”。很多错误来自把原子性误当成全部同步。

例如 `std::atomic<bool>ready` 可以原子地表示标志，但如果生产者用 relaxed 存储 `ready=true`，消费者用 relaxed 读取到 true 后再读普通字段，普通字段的初始化不一定通过这个标志建立可见性关系。标志本身没有数据竞争，不代表对象发布正确。反过来，一个 relaxed 计数器用于统计请求次数完全可以，因为读取计数不需要借它观察其他普通数据。

教材里的判断标准应是：这个原子变量是否只是自己的值有意义，还是它承担了发布其他状态的责任？只看自己的值，通常 relaxed 可以讨论；发布其他状态，就需要 release/acquire、mutex 或更强同步。

11.11 发布对象的完整故事

对象发布是内存序最常见的真实案例。生产者分配或准备一个对象，写入多个普通字段，然后把指针或 ready 标志交给消费者。消费者看到标志后读取对象字段。如果没有同步，消费者可能看到未初始化或部分初始化状态；即使在某台 x86 机器上看不出来，C++ 语义也不允许依赖这种偶然。

Listing 11.2 用原子指针发布只读对象

```

1 struct Config {
2     std::int32_t shard_count = 0;
3     std::int32_t queue_capacity = 0;
4 };
5
6 std::atomic<Config*> published_config{nullptr};
7
8 void publish(Config* config) {
9     config->shard_count = 16;
10    config->queue_capacity = 4096;
11    published_config.store(config, std::memory_order_release);
12 }
13
14 Config* acquire_config() {
15     Config* config = nullptr;
16     while (config == nullptr) {
17         config = published_config.load(std::memory_order_acquire);
18     }
19     return config;
20 }

```

release store 之前对 `config` 字段的写入，在 acquire load 读到同一个指针后，对消费者可见。这个例子还隐藏了生命周期问题：谁拥有 `Config`，何时释放，是否允许热更新？release/acquire 只解决发布时的可见性，不自动解决对象回收。无锁章节会继续处理这个问题。

11.12 消息传递和 happens-before

内存序最终要落到 happens-before 关系。若线程 A 的某些写入 happens-before 线程 B 的某些读取, B 就能按 C++ 规则观察到 A 发布的结果。mutex 的 unlock 和后续 lock 可以建立这种关系; release store 和读到它的 acquire load 也可以建立这种关系; 线程创建和 join 也有同步语义。

理解 happens-before 时, 不要把它想成“时间上先发生”。两个线程在物理时间上先后执行, 不一定建立 C++ 同步关系。生产者可能确实先写了 data, 再写 ready; 消费者也可能确实后读 ready, 再读 data。但如果 ready 使用 relaxed, 语言层面没有建立 data 的发布关系, 程序就缺少可移植保证。并发正确性看的是同步关系, 不是肉眼看到的执行顺序。

11.13 Release sequence 和读改写

C++ 内存模型里, release sequence 处理了一类常见场景: 一个 release 操作后面接着对同一个原子变量的一系列读改写操作, acquire 读到这个序列中的值时, 仍可能与最初 release 建立同步。这个规则让一些引用计数、队列状态和发布协议能正确表达, 但也容易让初学者迷糊。

本书不要求在入门阶段手写复杂 release sequence 算法, 但要知道两个原则。第一, 同步关系通常围绕同一个原子对象上的读写建立, 不能随便把一个原子变量的 release 借给另一个原子变量。第二, 读改写操作既读旧值又写新值, 它的成功和失败内存序都需要理解。CAS 成功时可能发布新状态, 失败时只是观察当前状态; 失败内存序不能比成功内存序更强。

11.14 CAS 循环的副作用边界

CAS 循环最容易漏讲的是副作用边界。因为 CAS 可能失败, 循环体可能执行很多次。若循环体里做了不可回滚副作用, 例如写日志、发送网络请求、增加外部计数、释放对象或修改另一个结构, 那么失败重试会让语义混乱。正确写法通常把“准备候选值”和“提交副作用”分开, 只有 CAS 成功后才执行对外可见副作用。

Listing 11.3 CAS 循环只在成功后提交外部副作用

```
1 bool try_set_max(std::atomic<std::int64_t>& target,
2                 std::int64_t candidate) {
3     std::int64_t observed = target.load(std::memory_order_relaxed);
4     while (observed < candidate) {
5         if (target.compare_exchange_weak(
6             observed,
7             candidate,
8             std::memory_order_release,
9             std::memory_order_relaxed)) {
10             return true;
11         }
12     }
13     return false;
14 }
```

这个函数在失败时, **observed** 会被更新成当前实际值, 下一轮基于新值继续判断。函数没有在循环内部写外部日志或修改其他共享结构, 因此失败重试不会产生额外语义。若调用者需要记录“最大值被更新”, 应在返回 true 后记录。

11.15 原子引用计数的双重语义

引用计数常用原子实现, 但它不是简单计数器。增加引用通常只需要保证计数本身原子, 减少引用并发现自己是最后一个引用时, 需要在销毁对象前看到其他线程对对象的修改。也就是说, 引用计数同时涉及原子性和对象生命周期。很多成熟库在 increment、decrement 和最后释放之间使用不同内存序, 就是因为不同阶段承担不同语义。

教材不应让读者以为“引用计数用 relaxed 就行”或“引用计数都用 `memory_order_seq_cst`

最安全”。更好的说法是：引用计数需要一套完整协议。获取引用时必须确保对象仍然活着；释放引用时如果不是最后一个，只是减少计数；如果是最后一个，必须和其他线程对对象的访问建立足够同步，然后才能析构。协议比单条原子指令重要。

11.16 Seq-cst 的价值和成本

`memory_order_seq_cst` 提供一个所有 seq-cst 操作参与的单一全局顺序。它让推理更接近直觉，适合初始实现、教学示例和确实需要全局顺序的算法。但它不是“让并发程序自动正确”的开关。若对象生命周期没处理，seq-cst 也不能防止 use-after-free；若不变量跨多个变量没有协议，seq-cst 原子也不能自动保持不变量。

性能上，seq-cst 可能限制编译器和硬件重排，在某些架构上需要更重屏障。x86 上某些 seq-cst load/store 也许看起来不重，但读改写和跨平台成本仍要考虑。工程建议是：先用清晰同步写正确，再在热点路径中根据证明降低到 acquire/release 或 relaxed。不要为了性能从一开始就写最弱内存序，也不要为了省脑把所有原子永远 seq-cst 后忽略结构设计。

11.17 原子和锁的组合

原子和 mutex 可以组合使用，但必须明确各自保护什么。常见模式是用原子做 fast path 标志，用 mutex 保护复杂慢路径状态。例如一个队列可以用原子 size 做近似指标，但真实 head、tail、buffer 仍由 mutex 保护。错误模式是同一份状态有时用锁保护，有时绕过锁用原子修改，导致不变量被拆碎。

如果一个变量参与锁保护的不变量，就不要随意在锁外用 relaxed 读它来做决策，除非这个决策允许过期和近似。例如队列长度指标可以近似展示；判断是否可以安全 pop 不能靠锁外近似 size。区分“指标”和“协议状态”非常重要。

11.18 内存序测试为什么困难

内存序 bug 很难通过普通测试证明不存在。它可能只在特定架构、编译器优化、核心数和时序下出现。压力测试能增加信心，但不能替代语义证明。写并发代码时，应先用 happens-before 关系证明，再用 ThreadSanitizer、压力测试和架构测试辅助验证。

手机或 x86 设备上跑过不代表 ARM 服务器、POWER 或未来编译器也正确。可移植 C++ 并发必须按语言模型写。教材示例因此宁愿稍保守，也不能依赖某个硬件“实际上不会乱序”的经验。

11.19 原子性的边界

原子操作首先保证的是单个原子对象上的读写不会产生数据竞争，并且读改写不会被撕裂。它不自动保证跨多个变量的不变量，也不自动发布普通变量，更不自动管理对象生命周期。把一个字段改成 `std::atomic`，只解决了这个字段本身的并发访问问题；如果这个字段只是一个更大状态机的一部分，状态机仍然可能错误。

例如队列的 `size` 使用原子，并不能让 `head`、`tail` 和 `buffer` 的关系安全。两个消费者可能都看到 `size` 大于零，然后读取同一个槽位。再例如一个指针使用原子发布，并不能说明指针指向对象何时释放。另一个线程 acquire 读到指针后，如果发布者随后释放对象，读者仍然可能悬垂。内存序解决可见性和顺序问题，不解决所有权问题。

因此，选择 atomic 之前要先回答三件事。第一，这个原子对象的线性化含义是什么，是计数、标志、指针、状态枚举还是队列索引。第二，它是否同步其他普通数据。第三，读取者拿到值后，对值关联的资源有什么生命周期保证。回答不清楚时，用 mutex 保护完整不变量通常更稳妥。

11.20 Relaxed 的正确用法

`memory_order_relaxed` 不是“随便乱序也无所谓”的开关，而是只需要原子性、不需要跨线程发布其他数据时的选择。请求总数、统计计数、采样次数、CAS 失败计数这类指标，通常可以 relaxed，因为读取者只关心数值最终大致准确或精确累加，不依赖它看到其他普通变量写入。

一个 relaxed 计数器可以保证每次 `fetch_add` 不丢，但不能保证“看到计数增加就一定看到对应请求对象已经初始化”。如果业务要用计数器作为队列元素可见的信号，就需要 release/acquire 或锁。很多并发 bug 来自把统计变量升级成协议变量，却没有同步语义。

Listing 11.4 Relaxed 适合不发布其他数据的统计计数

```

1 class RequestStats {
2 public:
3     void record_success() {
4         this->success_count_.fetch_add(1, std::memory_order_relaxed);
5     }
6
7     std::uint64_t success_count() const {
8         return this->success_count_.load(std::memory_order_relaxed);
9     }
10
11 private:
12     std::atomic<std::uint64_t> success_count_{0};
13 };

```

这个类的接口没有承诺读取计数时能看到某个请求的响应对象，也没有承诺计数和其他指标之间有一致快照。若监控页面同时读取 success、failure、timeout 三个 relaxed 计数，它们可能来自不同时间点。作为指标，这可以接受；作为协议判断，这不够。

11.21 发布对象的完整协议

release/acquire 常用来发布对象，但完整协议包含初始化、发布、获取、使用和回收五个阶段。初始化阶段由生产者独占对象，普通写入即可。发布阶段用 release store 把指针或状态写给消费者。获取阶段消费者用 acquire load 读到 release 写入的值。使用阶段消费者可以读取对象字段。回收阶段必须保证没有消费者仍在使对象，这一步不是 release/acquire 自动完成的。

如果对象只发布一次并且活到程序结束，协议很简单。如果对象会热更新，就需要版本、引用计数、RCU、shared pointer、epoch 或锁来管理旧对象。否则消费者可能刚 acquire 到旧指针，发布者就释放旧对象。不要把“可见”误认为“活着”。可见性和生命周期是两个不同问题。

```

publish protocol:
  producer initializes object fields
  producer release-stores pointer
  consumer acquire-loads pointer
  consumer reads fields
  reclamation waits until no consumer can hold old pointer

```

配置发布是学习这个边界的好例子。若配置对象数量很少，可以选择永不释放旧配置，用内存换简单性。若配置频繁更新，就需要引用计数或 RCU。若配置很大，又不能长时间保留旧版本，就要让读者理解回收成本会进入设计。

11.22 Acquire 和 Release 的方向

release 和 acquire 有方向。release 用在发布者把之前的普通写入推出去；acquire 用在消费者读到发布值后，把之后的普通读取拉进来。把消费者的 store 写成 release，不能让它看到生产者数据；把生产者的 load 写成 acquire，也不能发布数据。内存序要放在建立同步关系的那次读写上。

同步还要求 acquire 读到对应 release 或其 release sequence 中的值。如果消费者 acquire load 读到的是旧值，没有读到发布者的 release store，就没有通过这次操作建立同步。很多协议因此需要循环等待某个状态值，而不是只 load 一次。状态枚举比布尔值更清楚，因为它能表达 Empty、Writing、Ready、Closed 等阶段。

11.23 状态枚举比多个布尔值更稳

并发状态机如果用多个布尔值表达，很容易出现组合状态没有定义。例如 `ready=true`、`closed=true`、`failed=false` 到底代表什么？如果这些布尔值由不同线程更新，读者还可能看到中间组合。使用一个原子枚举表示状态，能让状态转换更集中，CAS 也更容易表达。

Listing 11.5 原子状态枚举表达提交阶段

```
1 enum class SlotState : std::int32_t {
2     Empty,
3     Writing,
4     Ready,
5     Closed,
6 };
7
8 struct SlotHeader {
9     std::atomic<SlotState> state{SlotState::Empty};
10 };
```

状态枚举不是万能的。若状态改变还伴随多个数据字段更新，仍然需要 `release/acquire` 或锁来发布这些字段。若状态之间有复杂不变量，可能要用 `mutex`。它的价值是减少非法组合，让线性化点更容易定位。

11.24 CAS 成功和失败的内存序

`compare_exchange_weak` 和 `compare_exchange_strong` 都有成功内存序和失败内存序。成功时，操作既读旧值又写新值；失败时，只读取当前值并把它写回 `expected` 参数。失败内存序不能是 `release`，因为失败没有发布新值；失败内存序也不能比成功更强。常见写法是成功使用 `acquire/release` 或 `acq_rel`，失败使用 `relaxed` 或 `acquire`，具体取决于失败路径是否要基于读取到的值观察其他数据。

CAS 循环还要处理 `weak` 可能伪失败。弱 CAS 允许即使值相等也失败，因此必须放在循环里。强 CAS 更适合不想处理伪失败的单次尝试，但在循环里弱 CAS 通常也可以。教材不应让读者机械背 API，而要理解：`expected` 是输入也是输出，失败后它变成实际观察值，下一轮判断要基于这个新值。

Listing 11.6 CAS 失败后 `expected` 被更新

```
1 bool mark_ready(std::atomic<SlotState>& state) {
2     SlotState expected = SlotState::Writing;
3     return state.compare_exchange_strong(
4         expected,
5         SlotState::Ready,
6         std::memory_order_release,
7         std::memory_order_relaxed);
8 }
```

如果返回 `false`，`expected` 不再一定是 `Writing`，而是当前实际状态。调用者可以用它决定是已经 `Ready`、已经 `Closed`，还是出现非法状态。很多 CAS 代码忽略失败后的 `expected`，错过了诊断信息。

11.25 Fetch-add 和热点

`fetch_add` 常被认为比 `mutex` 快，但在热点计数器上，它仍然会让同一 `cache line` 在多个核心之间迁移。原子读改写需要独占该 `cache line`；线程越多，迁移越频繁。低线程数下 `atomic counter` 很快，高线程数下可能成为扩展性瓶颈。这就是为什么 `sharded counter` 和 `per-thread partial` 如此

重要。

计数器有三种典型语义。第一，精确实时全局值，每次操作后都要读到最新总数，这最难扩展。第二，精确最终值，运行过程中不频繁读取，适合分片后最后合并。第三，近似指标，允许短时间误差，适合线程本地缓存、周期性汇总或采样。不要在需求只要第二或第三种时，实现第一种最昂贵语义。

11.26 Fence 的谨慎使用

C++ 还提供 atomic fence，但初学者应谨慎使用。很多 release/acquire 协议可以直接写在原子读写上，更容易审查。Fence 把顺序约束和具体原子对象分离，表达能力强，但也更容易写错。读代码的人必须追踪 fence 前后的普通读写和相关原子操作，证明它们组成同步关系。

工程建议是：除非你在实现底层库、移植成熟算法或有明确性能证据，否则优先使用带内存序的原子操作、mutex 或成熟并发容器。Fence 不是高级优化标志，而是更底层的工具。教材可以介绍它存在，但不应把它作为普通业务代码首选。

11.27 内存序和硬件屏障的关系

C++ 内存序是语言层面的契约，编译器再把它映射到目标架构的指令和屏障。x86 的内存模型较强，某些 acquire/release load/store 可能不需要额外硬件屏障；ARM 和 POWER 更弱，可能需要显式屏障或带 acquire/release 语义的指令。代码不能因为在 x86 上看起来正确，就省略 C++ 同步关系。

编译器也会根据内存序限制优化。普通变量在没有同步关系时，编译器可以重排、缓存或消除某些访问。原子操作同时约束编译器和硬件。学习内存序时，不能只画 CPU 执行顺序，也要记住编译器参与了程序变换。可移植并发的基础是语言模型，而不是某台机器的偶然表现。

11.28 ThreadSanitizer 能做什么

ThreadSanitizer 擅长发现数据竞争：一个变量被多个线程访问，至少一个写，并且没有 happens-before 关系。它能帮助发现忘记加锁、普通变量错误共享、发布协议缺失等问题。但它不能证明算法线性化正确，不能证明内存序最弱但足够，也不能发现所有 ABA 或生命周期协议错误。没有数据竞争的程序仍然可能逻辑错误。

因此测试组合应分层。TSan 用于发现明显数据竞争；压力测试用于增加时序覆盖；模型化小测试用于验证状态机；代码审查用于检查 happens-before 和生命周期；性能测试用于判断同步结构是否扩展。并发程序不能只靠一种测试工具。

11.29 把内存序写进注释

热点原子代码应该把内存序理由写在附近。注释不应重复“使用 release store”，而应说明“发布 buffer 槽位内容给 consumer”或“relaxed 因为该计数器不发布其他数据”。这样的注释能让后续维护者知道哪些语义不能随便改。

如果你无法写出一句清楚的理由，说明当前内存序选择还没有被理解。此时宁愿使用更保守的锁或 seq-cst，也不要写一个看似高性能但没人能证明的 acquire/release 组合。高性能并发代码的质量标准是可证明，不是看起来很底层。

11.30 发布订阅的完整示例

配置发布是 release/acquire 的经典案例。生产者构造新配置，填好普通字段，然后用 release store 发布指针。消费者用 acquire load 读到指针后，可以读取字段。这个例子看似简单，但完整实现还要处理空指针、版本、旧配置回收和多次更新。

Listing 11.7 配置发布的最小形态

```
1 struct RuntimeConfig {
2     std::int32_t worker_count = 0;
3     std::int32_t queue_capacity = 0;
```

```

4 };
5
6 class ConfigPublisher {
7 public:
8     void publish(RuntimeConfig* config) {
9         this->current_.store(config, std::memory_order_release);
10    }
11
12    RuntimeConfig* current() const {
13        return this->current_.load(std::memory_order_acquire);
14    }
15
16 private:
17     std::atomic<RuntimeConfig*> current_{nullptr};
18 };

```

这个类只负责可见性，不负责所有权。若发布的配置由外部保证活到程序结束，代码可以成立。若配置会替换和释放，就必须加回收协议。把所有权藏在调用者约定里可以作为教学简化，但生产代码要把它写进类型或文档。

11.31 版本化发布

发布配置时，指针之外常需要版本号。消费者读到配置后，可以记录 version；如果执行过程中需要确认配置没有变化，可以再次读取 version。版本号也方便日志和调试：看到某次请求使用配置 42，而不是只看到一个地址。版本号可以放在配置对象内部，也可以和指针一起发布。

如果指针和版本分成两个原子，就要小心一致性。消费者可能读到新指针旧版本或旧指针新版本。更稳的设计是把 version 放在不可变配置对象中，发布指针即发布完整对象；或者使用锁保护多字段快照；或者用原子 shared pointer 等更高层机制。多个原子变量不能自动组成一致快照。

11.32 引用计数案例

原子引用计数常见，但容易误讲。增加引用需要确保对象还活着；减少引用发现自己是最后一个引用时，需要在析构前看到其他线程对对象的修改。一个简单 shared ownership 协议通常把 control block 和对象生命周期绑定。引用计数的 `fetch_add`/`fetch_sub` 只是协议中的一部分。

若线程已经持有一个有效引用，复制引用时的计数增加可以使用较弱内存序，因为对象可见性来自已有引用的同步。释放引用时，如果不是最后一个，也只是减少计数；如果是最后一个，就要执行 `acquire` 语义或等价屏障，确保析构看到其他线程在释放前的写入。成熟 `std::shared_ptr` 实现非常细致，不建议读者随手自研。

教材要强调一点：引用计数解决的是生命周期，不自动解决对象内部并发访问。多个线程通过 shared pointer 同时修改对象字段，仍然需要 mutex、原子或不可变设计。很多人以为 shared pointer 让对象“线程安全”，这是错误的。

11.33 状态发布和数据发布分离

有些协议同时有数据和状态。生产者写 buffer，再把 state 改成 Ready；消费者看到 Ready 后读 buffer。这里 state 的 `release`/`acquire` 同步发布 buffer 内容。若生产者先把 state 设 Ready，再写 buffer，消费者可能读到未初始化数据。顺序必须写清。

Listing 11.8 槽位状态发布数据

```

1 enum class BufferState : std::int32_t {
2     Empty,
3     Ready,
4 };
5

```

```

6 struct BufferSlot {
7     std::array<std::int32_t, 16> values{};
8     std::atomic<BufferState> state{BufferState::Empty};
9 };
10
11 void publish_slot(BufferSlot& slot, std::span<const std::int32_t> input) {
12     for (std::size_t i = 0; i < input.size() && i < slot.values.size(); ++i) {
13         slot.values[i] = input[i];
14     }
15     slot.state.store(BufferState::Ready, std::memory_order_release);
16 }

```

消费者必须 acquire 读取 state，并且只有看到 Ready 后才能读 values。若消费者 relaxed 读取 state，就没有可移植同步关系。若 values 写入可能抛异常或失败，不能提前发布 Ready。状态发布点就是线性化点。

11.34 Relaxed 指标的快照问题

多个 relaxed 指标组成的快照不一致是常见现象。假设一个系统有 accepted、completed、failed 三个计数器。监控线程分别读取它们时，accepted 可能来自较新时间，completed 来自较旧时间，因此 completed 加 failed 可能短暂大于 accepted 或小于 accepted。这不一定是 bug，而是近似指标语义。

如果监控页面不能接受这种不一致，就需要加锁快照、sequence lock、双缓冲或周期性汇总。不要在每个指标上简单使用 seq-cst 后以为解决了跨变量一致性。seq-cst 给单个原子操作提供全局顺序，但多个变量之间的业务不变量仍需要协议。

11.35 Sequence lock 的思想

Seqlock 常用于读多写少、读者可重试的场景。写者先把序号变成奇数，修改数据，再把序号变成下一个偶数。读者先读序号，复制数据，再读序号；如果两次序号相同且为偶数，说明读到一致快照；否则重试。它让读者无锁，但读者可能在写频繁时反复重试，且读取的数据复制必须能安全进行。

Seqlock 不适合包含指针和复杂生命周期的数据，除非另有回收保护。读者在没有锁的情况下复制数据，如果写者释放了指针指向对象，读者可能悬垂。它适合简单 POD 快照，例如时间、统计摘要、路由版本小结构。这个例子再次说明：原子协议和对象生命周期必须一起设计。

11.36 原子等待和通知

C++20 提供 `atomic::wait` 和 `notify_one/notify_all`，可以在原子值变化时等待，某些场景下比条件变量更直接。它适合等待单个原子状态变化，例如状态从 Pending 到 Ready。它仍然需要谓词思维：wait 返回后要重新检查值，通知不是业务状态本身。复杂不变量仍然更适合 mutex 和 condition variable。

原子等待也有关闭和取消问题。若线程等待状态变成 Ready，但系统关闭时状态应变成 Closed 并 notify；若只通知 Ready，不通知 Closed，等待者可能悬挂。工具变了，状态机原则不变。

11.37 内存序选择流程

选择内存序可以按流程。第一，是否只是线程本地？若是，不需要原子。第二，是否保护多个字段不变量？若是，优先 mutex 或明确协议。第三，是否只是统计计数，不发布其他数据？可以考虑 relaxed。第四，是否发布普通数据给消费者？需要 release/acquire 或更高层同步。第五，是否需要所有线程看到单一全局顺序？才考虑 seq-cst。第六，是否涉及指针生命周期？必须设计回收，内存序本身不够。

这个流程能减少随意性。内存序不是越弱越专业，也不是越强越正确。正确选择来自语义需求。

11.38 架构差异的测试策略

如果项目要跨 x86 和 ARM，不能只在 x86 上测。x86 较强内存模型可能掩盖某些缺少 acquire/release 的 bug；ARM 更弱，错误更容易暴露。若没有多架构机器，可以使用 ThreadSanitizer、模型检查工具或至少按照 C++ 内存模型审查。手机 Termux 可能正好是 ARM 环境，适合跑并发压力测试，但工具权限可能有限。

报告中应写明架构。一个在 x86 上“跑了很多次没出错”的 relaxed 发布示例，不是正确证据。C++ 并发正确性以语言模型为准。

11.39 案例：SPSC 发布协议

SPSC ring buffer 中，生产者先写 buffer 槽位，再 release 更新 tail；消费者 acquire 读取 tail 后，才能读到槽位内容。Head 由消费者写，tail 由生产者写，单写者让协议简化。若把 tail store 改成 relaxed，消费者可能在语言模型上没有看到 buffer 写入。若先更新 tail 再写 buffer，消费者可能读到未完成数据。

这个案例适合训练内存序注释。Tail release 的理由是发布槽位内容；tail acquire 的理由是获取槽位内容；head release 的理由是发布槽位可复用；head acquire 的理由是生产者看到可用空间。每个内存序都能对应到状态转换，而不是凭感觉选择。

11.40 案例：近似指标发布

运行时指标常使用 relaxed 分片计数。Worker 本地统计任务数、空转次数、窃取次数；监控线程周期性读取并汇总。这里不要求读者看到精确同时刻快照，只要求趋势。因此 relaxed 或无锁读取加最终汇总可以接受。但如果指标用于触发限流或关闭，就必须重新定义同步。

同一个变量在不同用途下可能需要不同语义。作为监控，近似可以；作为协议，必须精确。设计时要防止“先当指标写，后来被业务拿去做决策”的语义漂移。

11.41 原子章验收清单

项目中每个原子变量都应写明：它表示什么状态，谁写谁读，是否发布其他数据，是否参与对象生命周期，选择的内存序理由是什么，失败路径如何处理，是否需要与其他变量组成快照。若只是统计，写明 relaxed 的理由；若是发布，写明 release/acquire 对应的数据；若涉及指针，写明回收协议。这个清单能显著降低并发维护风险。

11.42 原子章节总结

原子操作解决的是特定同步问题，不是并发正确性的万能答案。Relaxed 适合不发布数据的统计，release/acquire 适合发布和获取，seq-cst 适合需要简单全局顺序的场景。多个原子变量不会自动形成一致快照，原子指针不会自动管理生命周期。写原子代码时，必须能画出 happens-before、线性化点和回收边界。

11.43 内存序决策表

内存序选择可以压缩成一句话：先证明语义，再选择最弱足够顺序。只是计数，不发布数据，用 relaxed；发布对象字段，用 release/acquire；读改写既读又写，分别考虑成功和失败内存序；需要简单推理或教学，先用 seq-cst；涉及多个字段不变量，用锁；涉及指针生命周期，加回收协议。任何不能解释的内存序，都应该回到更简单同步。

11.44 案例：发布错误原因

如果一个任务失败，worker 写入错误对象，然后把状态设为 Failed。等待者看到 Failed 后要读取错误对象。这和发布配置完全一样：错误对象字段必须在 release store 状态之前写好，等待者必须 acquire load 状态后读错误。若错误对象是动态分配的，还要保证生命周期覆盖等待者读取。错误传播也是发布协议，不是简单设置一个布尔值。

这类案例在任务图和分布式 driver 中很常见。状态枚举和结果对象应绑定设计，避免状态已完成但结果不可见或已释放。

11.45 本章落到贯穿项目

Compute Systems Engine 在本章应增加三类原子实验。第一，统计计数器：mutex counter、atomic relaxed counter、sharded counter，用来区分原子性和扩展性。第二，发布实验：一个 producer 初始化配置后 release 发布，consumer acquire 获取，用来说明对象可见性。第三，CAS 实验：实现 `try_set_max` 或无锁最大值更新，记录 CAS 失败次数，说明竞争下重试成本。

这些实验都要写清语义。计数器 relaxed 的理由是“不发布其他数据”；配置发布 acquire/release 的理由是“读取者需要看到普通字段初始化”；CAS 的理由是“成功那一刻是线性化点”。如果写不出理由，就不应该选择内存序。

11.46 从锁版本迁移到原子版本

把锁版本改成原子版本，不能只把字段类型换成 `std::atomic`。锁版本通常保护一组不变量，原子版本必须重新定义线性化点和状态转移。一个 mutex 队列的 `head`、`tail`、`size` 和 `closed` 在同一把锁下更新；如果只把 `size` 改成原子，其他字段仍会被并发破坏。迁移第一步不是写 CAS，而是写出哪些状态可以独立，哪些状态必须一起变化。

较稳妥的迁移路线是分层。先保留锁版本作为 reference，写足正确性测试。然后引入只用于指标的 relaxed 原子，例如任务完成数和 CAS 失败数，这一步不改变协议。再实现一个边界明确的 SPSC ring，因为单生产者单消费者让 head 和 tail 各自只有一个写者，协议简单。最后才考虑 MPSC、MPMC 或无锁内存回收。这个路线看起来慢，但它让每一步都有可证明的语义和对照测试。

迁移还要考虑性能目标。低竞争 mutex 可能已经很快，原子版本反而因为 cache line 争夺、CAS 重试和复杂回收变慢。原子优化最适合共享状态小、线性化点清楚、生命周期可控、竞争形状明确的场景。若状态复杂、关闭语义复杂、需要等待和取消，mutex 加条件变量往往更清楚。高水平工程判断不是“无锁更高级”，而是知道什么时候不该无锁。

11.47 内存序代码审查

审查原子代码时，应逐个原子变量建立表格。第一列写变量含义，例如计数、状态、指针、索引、版本。第二列写写者和读者。第三列写它是否发布其他普通数据。第四列写内存序。第五列写失败路径和生命周期。若某个原子变量没有清晰含义，只是为了避免数据竞争而存在，通常说明设计还没完成。

看到 relaxed，要问它是否只关心自身数值。若 relaxed 之后读取普通对象字段，就要怀疑发布关系缺失。看到 release，要问哪个普通数据在它之前被发布。看到 acquire，要问它是否确实读到了对应 release 写入的值。看到 CAS，要问成功和失败内存序是否不同，失败后 expected 如何更新，循环体是否有不可回滚副作用。看到原子指针，要问对象何时释放，读者如何保证使用期间对象活着。

审查还要防止“强内存序掩盖设计问题”。把所有操作都写成 seq-cst，可能让小例子更容易运行，但它不会自动解决多变量不变量、丢失唤醒、对象回收和关闭协议。强顺序只能约束操作可见顺序，不能替你定义状态机。若设计不清楚，应该回到 mutex 或更高层同步，而不是用更强内存序硬撑。

11.48 线性化点和用户可见语义

并发对象对外应该像每个操作在某个瞬间生效，这个瞬间就是线性化点。mutex 版本通常把线性化点藏在临界区内部，因为临界区串行执行；原子版本必须显式找出来。无锁栈的 push 线性化点可能是 CAS 成功把新节点挂到 head；pop 的线性化点可能是 CAS 成功把 head 移到 next；SPSC ring 的 push 线性化点可能是 release 更新 tail。

线性化点不只是理论概念，它决定用户看到的行为。若 `try_pop` 返回空，表示在它的线性化点上队列为空，而不是整个调用期间都为空；若 `close` 成功，后续 `push` 应失败，但和 `close` 并发的 `push` 是否成功，要看线性化先后。文档写清这点，调用者才能理解边界。很多并发争议来自用户以为“调用先开始就先发生”，而实现按线性化点定义先后。

线性化点还影响测试。并发队列的输出顺序可能不唯一，不能用单一串行顺序硬比。测试应验证存在某个合法线性化顺序，或者验证更具体的不变量：元素不丢不重，关闭后没有新元素，容量不越界，已成功返回的操作符合文档。对于复杂对象，可以使用模型检查或小状态穷举，把线程交

错压到可分析范围内。

11.49 等待和原子的边界

原子操作适合表达状态变化，但等待是另一个问题。忙等会消耗 CPU；条件变量可以睡眠但需要 mutex；`atomic::wait` 可以围绕单个原子值睡眠；futex 是更底层的机制。选择等待方式时，要看等待时间、唤醒频率、状态复杂度和关闭语义。短等待可以自旋，长等待应睡眠，复杂谓词通常用条件变量更清楚。

自旋等待必须有退出策略。若等待对象的生产者可能被抢占，消费者一直自旋会浪费 CPU，甚至阻碍生产者被调度。常见策略是先短暂自旋，再 yield，再睡眠；或者让运行时知道这是阻塞等待，把 worker 释放给其他任务。等待策略和调度章节直接相连。原子状态本身很快，不代表围绕它等待的系统就快。

原子等待也要处理通知丢失的误解。和条件变量一样，通知不是业务状态。等待者应先读取状态，如果不满足再 wait；被唤醒后重新读取。关闭、取消、失败都应是状态枚举的一部分，并且对应通知。只通知成功状态，不通知关闭状态，会让等待者永久睡眠。工具从 condition variable 换成 atomic wait，谓词思维没有改变。

11.50 本章习题

习题与作业

习题一：给三个场景选择内存序并解释理由：请求计数器、配置对象发布、无锁最大值更新。每个场景都要说明是否同步其他普通数据，是否需要全局顺序，失败时会发生什么。

习题二：把一个错误的 relaxed 发布例子改成 release/acquire。要求画出 happens-before 关系，并说明对象生命周期仍然需要额外处理。

习题三：写一个 CAS 循环，要求循环体内没有不可回滚副作用。然后列出三种不应该放进 CAS 重试循环的副作用，并解释原因。

习题四：阅读项目中的 `AtomicCounter`。解释为什么当前 `fetch_add` 可以使用 relaxed，也说明它为什么不适合作为对象发布机制或队列同步机制。

实验：比较 mutex counter 和 atomic counter。在低线程数和高线程数下记录耗时。解释 relaxed 为什么适合这个计数器，也说明它为什么不能替代对象发布同步。

第 12 章

无锁数据结构

无锁数据结构的目标不是“没有同步”，而是在不使用互斥锁阻塞线程的情况下维护共享状态。它通常依赖原子操作、CAS、内存序和严格的对象生命周期管理。学习无锁结构，要先学习进度保证和失败模式。

12.1 进度保证

Blocking 算法可能因为一个线程持锁停顿而阻塞其他线程。Lock-free 算法保证系统整体持续前进，但单个线程可能反复失败。Wait-free 算法保证每个线程在有限步内完成，通常更难实现。Obstruction-free 只在无竞争时保证进展。

这些术语不是装饰。一个队列在压力下 CAS 反复失败，可能仍然 lock-free，但尾延迟很差。工程上需要结合竞争程度、临界区复杂度和可维护性选择方案。

进度保证要和系统目标匹配。低延迟交易路径可能愿意为 wait-free 付出巨大复杂度；普通后台队列通常 mutex 就足够；高吞吐日志管线可能只需要 SPSC ring buffer；线程池全局队列可能更适合分片队列加工作窃取。不要把 lock-free 当成荣誉标签，它只是进度性质的一种选择。

12.2 无锁栈和 ABA

Treiber stack 是经典无锁栈。push 读取 head，把新节点 next 指向旧 head，再 CAS head。pop 读取 head 和 next，再 CAS head 到 next。问题在于 ABA：head 可能从 A 变成 B 又变回 A，CAS 看见 A 成功，但 A 的生命周期可能已经变化。

解决 ABA 需要额外机制。版本标签把指针和计数器一起比较；hazard pointer 让线程声明自己正在访问的节点；epoch reclamation 延迟释放节点直到所有线程离开旧 epoch。无锁算法难点往往不在 CAS 本身，而在内存回收。

```
Treiber push shape:
load old head
new node next = old head
CAS head from old head to new node

Treiber pop shape:
load old head
load next from old head
CAS head from old head to next
```

这张形态图比直接贴完整代码更重要。读者必须先看出线性化点在哪里：push 和 pop 都在 CAS 成功那一刻生效。CAS 之前读到的状态只是一次尝试，失败后不能假装操作已经发生。

12.3 内存回收为什么困难

带锁结构中，线程持有锁时可以知道没有其他线程正在修改结构。无锁结构没有这个简单边界。一个线程从 head 读到节点指针后，可能还没来得及读取 next，另一个线程已经 pop 并释放了该节点。第一个线程随后访问这个指针，就会出现 use-after-free。

垃圾回收语言可以把节点释放推迟到不可达之后，C++ 程序必须自己设计回收协议。Hazard pointer 的思路是线程公开声明“我可能访问这个节点”，释放者扫描这些声明，确认没有线程持有后再释放。Epoch 的思路是把释放延迟到所有线程都离开旧时代之后。二者都有开销，也都有使用约束。

Hazard pointer 更精确，但每个线程需要发布自己正在保护的指针，释放者需要扫描 hazard 集

合。Epoch 回收批量化更强，读路径常常更轻，但如果某个线程长时间停留在旧 epoch，回收会延迟，内存占用可能增长。RCU 也是类似思想：读者进入读侧临界区，写者等待 grace period 后回收旧版本。不同机制的共同点是：释放不能只看“节点已经从结构中摘掉”，还要看“是否还有线程可能持有旧指针”。

12.4 ABA 的具体例子

假设栈顶是 A，线程 T1 读取 head=A，准备把 head 改成 A->next。此时 T2 pop A，又 pop B，然后把 A push 回栈顶。T1 再 CAS head，从 A 改成旧的 A->next。CAS 看到 head 仍然是 A，于是成功，但这个 A 已经不是 T1 当初理解的结构关系。版本标签通过让 head 变成“指针加版本号”，使 A 回来时版本不同，从而让 CAS 失败。

12.5 环形队列

有界环形队列适合固定容量、低分配、缓存友好的场景。单生产者单消费者队列可以用较简单的 head/tail 原子实现；多生产者多消费者队列需要为槽位维护序号或状态，避免多个线程同时写同一槽位。

环形队列的性能来自固定内存、连续布局 and 减少分配。但如果所有生产者争用同一个 tail，仍然会有原子热点。分片队列、每线程队列和工作窃取可以缓解热点。

12.6 SPSC、MPSC 和 MPMC

队列难度取决于生产者和消费者数量。SPSC 只有一个生产者和一个消费者，head 只被消费者写，tail 只被生产者写，可以用较轻的同步。MPSC 有多个生产者争用 tail，消费者独占 head。MPMC 两端都有竞争，通常需要更复杂的槽位序号或链表算法。

设计队列时先问清使用场景，不要直接上最复杂的 MPMC。日志线程、音频缓冲和管线阶段之间常常只需要 SPSC；线程池全局队列可能需要 MPMC；工作窃取运行时则常用每 worker deque 加窃取操作。

SPSC ring buffer 是学习无锁队列的最佳起点。生产者独占 tail，消费者独占 head，因此每个索引只有一个写者；双方通过 acquire/release 观察对方进度。MPSC 和 MPMC 复杂得多，因为多个线程会争用同一个端点，需要 CAS、槽位状态或序号。教材顺序应从 SPSC 到 MPSC 再到 MPMC，而不是一开始贴一个难以证明的万能队列。

12.7 线性化点

并发数据结构的线性化点，是操作在逻辑上生效的那一瞬间。带锁结构的线性化点通常在锁内某次状态修改；无锁结构的线性化点通常是成功 CAS。若无法指出线性化点，就很难证明结构正确。

例如无锁栈 push 的线性化点是 CAS head 成功。CAS 失败的尝试没有生效，必须重试。测试无锁结构时，不能只看最终数量，还要考虑每个操作是否能排列成某个合法串行顺序。

线性化点还帮助写测试。对于栈，可以记录每个 push 的值和每个 pop 的返回，检查是否存在某个合法串行顺序；对于队列，要检查 FIFO 语义是否在并发交错下仍成立；对于计数器，要检查每次成功自增是否唯一。压力测试只能增加信心，不能替代线性化论证。无锁结构没有明确线性化点，通常就不该进入生产代码。

12.8 何时不用无锁

如果共享状态复杂、竞争不高、临界区很短，mutex 可能更清晰也足够快。如果需要条件等待，锁加条件变量往往比忙等 CAS 更节能。无锁结构适合明确的高频热路径，且必须有严格测试、压力测试和代码审查。

12.9 工程验收标准

无锁结构的验收标准应比普通容器更高。第一，要有清晰的不变量、线性化点和进度保证说明。第二，要说明内存回收策略，不能留下 use-after-free 风险。第三，要有单线程 reference 行为测试、并发压力测试、边界容量测试和关闭语义测试。第四，要有性能对照，证明它在目标竞争模式下确

实优于 mutex 或分片锁。第五，要说明不适用场景，例如高竞争 CAS 风暴、需要阻塞等待或对象生命周期复杂。

如果这些条件做不到，就应该优先使用更简单的同步结构。复杂无锁代码的维护成本非常高，错误通常低概率且难复现。严肃工程里，简单正确经常比炫技更有价值。

12.10 SPSC ring buffer 完整推导

学习无锁队列应从 SPSC 开始，因为它的所有权最清楚：只有一个生产者写 tail，只有一个消费者写 head。生产者会读 head 判断是否有空间，消费者会读 tail 判断是否有元素。每个索引只有一个写者，这大幅降低了同步难度。SPSC 的关键不是“没有锁”，而是用所有权设计减少共享写。

环形队列保留一个空槽可以区分满和空。若 `next_tail==head`，队列满；若 `head==tail`，队列空。生产者写入元素后 release 发布新的 tail；消费者 acquire 读取 tail 后，能看到生产者写入的元素。消费者取出元素后 release 发布新的 head；生产者 acquire 读取 head 后，知道槽位已经可复用。对于简单整数元素，这个模型很直观；对于复杂对象，还要处理构造、析构和异常安全。

Listing 12.1 SPSC ring buffer 的核心实现

```

1 class SpscIntQueue {
2 public:
3     explicit SpscIntQueue(std::int32_t capacity)
4         : buffer_(static_cast<std::size_t>(capacity + 1), 0) {}
5
6     bool try_push(std::int32_t value) {
7         const std::uint64_t tail =
8             this->tail_.load(std::memory_order_relaxed);
9         const std::uint64_t next_tail = this->next(tail);
10        const std::uint64_t head =
11            this->head_.load(std::memory_order_acquire);
12        if (next_tail == head) {
13            return false;
14        }
15        this->buffer_[static_cast<std::size_t>(tail)] = value;
16        this->tail_.store(next_tail, std::memory_order_release);
17        return true;
18    }
19
20    std::optional<std::int32_t> try_pop() {
21        const std::uint64_t head =
22            this->head_.load(std::memory_order_relaxed);
23        const std::uint64_t tail =
24            this->tail_.load(std::memory_order_acquire);
25        if (head == tail) {
26            return std::nullopt;
27        }
28        const std::int32_t value =
29            this->buffer_[static_cast<std::size_t>(head)];
30        this->head_.store(this->next(head), std::memory_order_release);
31        return value;
32    }
33
34 private:
35     std::uint64_t next(std::uint64_t index) const {
36         const std::uint64_t size =
37             static_cast<std::uint64_t>(this->buffer_.size());

```

```

38     return (index + 1) % size;
39 }
40
41 std::vector<std::int32_t> buffer_;
42 alignas(64) std::atomic<std::uint64_t> head_{0};
43 alignas(64) std::atomic<std::uint64_t> tail_{0};
44 };

```

这段代码故意只支持 SPSC。若两个生产者同时调用 `try_push`，它们都会写 `tail` 和同一槽位，算法不再正确；若两个消费者同时 `try_pop`，它们都会写 `head`，也不正确。接口文档必须写清这个约束。无锁结构最忌讳把适用场景藏起来，让调用者误用。

`head` 和 `tail` 分开放置，是为了减少生产者和消费者频繁写同一 `cache line`。这里使用 `std::uint64_t` 是为了让索引范围清楚；容量参数仍需检查，生产代码还要处理容量溢出。教学代码展示核心同步，工程代码还要补足参数验证、移动禁用、析构约束和测试。

12.11 SPSC 的线性化点

SPSC `try_push` 的线性化点是 `release` 写 `tail`。写入 `buffer` 只是准备数据；`tail` 更新后，消费者 `acquire` 看到新 `tail`，元素才对消费者可见。`try_pop` 的线性化点可以看作读取元素并 `release` 更新 `head` 的组合：在返回元素前，它已经把槽位归还给生产者。更严格地说，对于整数槽位，消费者读取元素后操作结果已经确定，`head` 更新发布空间可用。

线性化点帮助分析失败路径。若 `try_push` 发现队列满返回 `false`，它没有写入 `buffer`，也没有更新 `tail`，操作没有生效。若 `try_pop` 发现空返回 `nullopt`，它没有更新 `head`。测试要覆盖这些失败路径，因为失败路径在有界队列中不是异常，而是正常背压。

12.12 从 SPSC 到 MPSC

MPSC 增加多个生产者。此时 `tail` 不再只有一个写者，生产者之间必须争用写入位置。常见做法是使用 `fetch_add` 或 `CAS` 为每个生产者预留槽位，然后再处理槽位可见性。问题立刻变复杂：生产者 A 预留了槽位但还没写完，生产者 B 写完后能否让消费者看到后面的槽位？消费者如何知道某个槽位的数据已经准备好？队列满时已经预留的位置如何处理？

这就是为什么 MPSC/MPMC 队列常为每个槽位维护序号或状态。`tail` 只表示预留进度不够，还要知道槽位是否已提交。队列算法难度不是来自环形数组本身，而是来自多写者之间的提交顺序、可见性和容量回收。教材应明确告诉读者：不要把 SPSC 代码简单加 `CAS` 就当 MPSC。

12.13 MPMC 队列的槽位序号直觉

一种经典 MPMC 有界队列会让每个槽位带一个 `sequence`。生产者根据 `tail` 找到槽位，检查 `sequence` 是否表示槽位空闲；成功 `CAS tail` 后写入数据，再更新 `sequence` 表示可读。消费者根据 `head` 找到槽位，检查 `sequence` 是否表示槽位可读；成功 `CAS head` 后读取数据，再更新 `sequence` 表示空闲。`sequence` 用于区分同一个数组槽位在不同轮次中的状态，避免只靠 `head/tail` 造成混淆。

这个模型比 SPSC 难很多。每个端点都有多个写者，`CAS` 失败是正常路径；槽位 `sequence` 也要选择正确内存序；容量必须是固定的或 `carefully` 管理；队列关闭和阻塞等待又是额外层次。生产系统通常会优先使用成熟库，或者先用 `mutex bounded queue`，只有在明确瓶颈和使用场景后才自研 MPMC。

12.14 ABA 和内存回收的关系

ABA 不只是“值变回去了”的小问题，它经常和内存回收连在一起。节点 A 被 `pop` 后释放，分配器可能把同一地址重新分配给新节点。另一个线程手里仍有旧指针，`CAS` 看到地址相同，以为结构没变，实际上对象生命周期已经换了。版本标签能区分同一地址的不同代；`hazard pointer` 和 `epoch` 能防止节点过早释放；垃圾回收语言则把释放延后到不可达之后。

C++ 无锁结构必须把“从数据结构中摘除”和“释放内存”分开。摘除只说明新操作不会再从结构入口到达该节点，不说明旧操作没有持有指针。回收协议就是证明旧操作不再持有指针的机制。

没有回收协议的无锁链表或栈，在轻测试下可能正常，在压力下可能 use-after-free。

12.15 Hazard pointer 的操作流程

Hazard pointer 的基本流程是：线程在读取共享指针后，把该指针发布到自己的 hazard 槽位；然后重新读取共享指针确认它没有变化；确认后才能安全解引用。删除节点的线程不能立即释放，而是把节点放入 retired 列表；当 retired 列表足够大时，扫描所有 hazard 槽位，把没有被任何线程保护的节点释放。

这套机制的成本很明确：读路径需要发布 hazard，释放路径需要扫描，线程数越多扫描成本越高。它的优点是精确，不需要等待所有线程离开全局 epoch。它适合指针访问较复杂、需要较及时回收的结构。教学时不一定要完整实现，但必须让读者知道无锁结构的真正难点在这里。

12.16 Epoch 回收的操作流程

Epoch 回收把时间分成阶段。线程进入无锁操作时宣布自己处于当前 epoch，退出时宣布离开。删除节点时不立即释放，而是记录删除发生的 epoch。只有当所有线程都已经离开旧 epoch，旧 epoch 中删除的节点才可以释放。它把单个节点的精确保护变成批量延迟回收。

Epoch 的读路径通常比 hazard pointer 更轻，但如果某个线程长时间不退出临界区，回收会被阻塞，内存占用可能增长。它适合操作短、线程行为可控的场景，不适合线程可能长期停顿在临界区的场景。系统设计必须考虑线程被抢占、阻塞、崩溃或取消时对回收的影响。

12.17 无锁不等于低尾延迟

lock-free 保证系统整体前进，不保证每个线程快速完成。高竞争 CAS 循环可能让某些线程反复失败，尾延迟很差。wait-free 才提供单线程有限步完成保证，但实现通常更复杂。对于服务系统，尾延迟可能比平均吞吐更重要，因此 lock-free 结构不一定比有锁结构更适合请求路径。

mutex 在高竞争下可能进入内核睡眠，平均吞吐下降但节省 CPU；lock-free CAS 在高竞争下可能烧满 CPU，不断失败重试。哪种更好取决于等待时间、核心预算、能耗、尾延迟和业务语义。教材不能把 lock-free 简化成“更高级所以更快”。

12.18 进展保证的层次

无锁数据结构讨论中，必须区分 blocking、obstruction-free、lock-free 和 wait-free。Blocking 结构可能因为某个线程持锁后停顿而阻塞其他线程。Obstruction-free 只保证一个线程在没有竞争时能完成。Lock-free 保证系统整体总有某个线程能前进，但单个线程可能一直失败。Wait-free 保证每个线程在有限步骤内完成。进展保证越强，实现通常越复杂，常数成本也可能更高。

很多工程代码把“没有 mutex”就叫 lock-free，这是不严谨的。如果算法里某个线程可以让所有线程永久等待，例如等待某个槽位状态从 Writing 变成 Ready，而写该槽位的线程崩溃后无人修复，那么它可能并不满足想象中的进展保证。进展保证要写清楚依赖条件：线程是否可能被取消，是否允许阻塞内存分配，是否依赖操作系统调度，是否在队列满时返回失败。

12.19 线性化点是正确性的核心

无锁结构必须能为每个成功操作指出线性化点，也就是这个操作在并发历史中看起来瞬间生效的位置。SPSC push 的线性化点可以是发布 tail；Treiber stack push 的线性化点通常是 CAS 成功更新 head；MPMC 队列生产者的线性化点可能是槽位 sequence 从写入中变成可读。没有线性化点，就无法说明并发操作等价于某个串行顺序。

失败操作也要有解释。队列满返回 false 的线性化点可以是观察到满状态的那次读取；队列空返回 null 的线性化点可以是观察到 head 等于 tail 的那次读取。但如果观察过程中状态变化了，就要证明返回仍然合法。许多无锁 bug 就藏在失败路径：成功路径看起来能工作，空、满、关闭、竞争失败和回收路径没有证明。

12.20 Treiber 栈作为最小反例

Treiber stack 是学习 CAS 的经典结构，但它也能展示无锁结构的陷阱。push 分配新节点，把新节点的 next 指向当前 head，然后 CAS head 为新节点。pop 读取 head 和 next，然后 CAS head 为 next。看起来很短，但一旦考虑 ABA 和内存回收，复杂度立刻上升。

Listing 12.2 Treiber 栈的教学骨架，不包含回收协议

```

1 struct StackNode {
2     std::int32_t value = 0;
3     StackNode* next = nullptr;
4 };
5
6 class LockFreeStack {
7 public:
8     void push(StackNode* node) {
9         StackNode* observed = this->head_.load(std::memory_order_relaxed);
10        do {
11            node->next = observed;
12        } while (!this->head_.compare_exchange_weak(
13            observed,
14            node,
15            std::memory_order_release,
16            std::memory_order_relaxed));
17    }
18
19 private:
20     std::atomic<StackNode*> head_{nullptr};
21 };

```

这段代码不能直接作为生产栈，因为 pop 需要解决节点何时释放。即使 push 只有十几行，完整数据结构也必须定义节点所有权。节点由调用者拥有、栈拥有、对象池拥有，还是回收系统拥有？pop 返回节点后，其他线程是否仍可能持有旧指针？这些问题比 CAS 本身更重要。

12.21 ABA 时间线

ABA 可以用时间线理解。线程 T1 读取 head 等于地址 A，并读取 A 的 next 等于 B。此时 T1 暂停。线程 T2 pop 掉 A，又 pop 掉 B，释放 A，分配器恰好把地址 A 分给新节点 C，再 push 到 head。T1 恢复后看到 head 仍然是地址 A，于是 CAS 成功，把 head 改成旧的 B。此时栈被破坏，因为 T1 以为 A 还是原来的节点，实际地址 A 的对象已经换了生命周期。

解决 ABA 的方法不是单一的。带版本标签的指针把地址和计数一起 CAS，同一地址再次出现时版本不同。Hazard pointer 防止 T2 在 T1 仍可能持有 A 时释放 A。Epoch 回收延迟释放，直到所有可能持有旧指针的线程离开旧 epoch。垃圾回收语言则把这个问题转移给运行时。C++ 需要显式选择一种策略。

12.22 带标签指针的直觉

带标签指针把指针和版本号组成一个逻辑值。每次 head 改变，版本号递增。这样即使指针地址从 A 变成 B 又回到 A，版本号也不同，CAS 不会误以为状态没变。实际实现要考虑平台是否支持双字 CAS、指针对齐是否能偷低位、版本溢出和 ABI 可移植性。教材阶段理解思想即可，不应鼓励读者随手写不可移植指针位操作。

Listing 12.3 标签指针的语义形状

```

1 struct TaggedIndex {

```



```

2     std::uint32_t index = 0;
3     std::uint32_t generation = 0;
4 };

```

用索引加 generation 比直接偷指针位更适合教学，也更接近很多对象池和 ECS 系统。数组槽位复用时 generation 增加，旧句柄即使 index 相同，也会因为 generation 不匹配而失效。这个思想贯穿本书：位置和对象生命周期不是同一件事，必须用版本把它们区分开。

12.23 Hazard pointer 的实现边界

Hazard pointer 要求每个线程拥有一个或多个 hazard 槽位。读取共享指针后，线程把指针写入自己的槽位，再重新读取共享指针确认没有变化。这个二次确认很重要：如果读取指针后还没发布 hazard，删除线程可能已经把节点放入 retired 列表并释放。只有发布并确认后，读者才能安全解引用。

释放线程不能每删除一个节点就扫描所有 hazard，因为扫描成本高。通常把删除节点放入 retired 列表，达到阈值后批量扫描。扫描时收集所有 hazard 指针，把没有出现在集合中的 retired 节点释放。阈值选择影响内存峰值和释放成本：阈值小，扫描频繁；阈值大，内存保留多。

Hazard pointer 还需要处理线程退出。如果一个线程退出前没有清空 hazard 槽位，其他线程可能永远不释放某些节点。生产实现要有线程注册、注销、槽位回收和异常安全。教学时可以用固定 worker 数简化，但必须在文字中说明简化假设。

12.24 Epoch 的暂停问题

Epoch 回收的优势是读路径轻，通常只需要进入和退出临界区时标记 epoch。但它害怕暂停线程。若一个线程进入 epoch 后被操作系统长时间抢占、阻塞在 I/O 或执行用户回调，所有在该 epoch 后退休的节点都可能无法释放。内存占用不断增长，系统可能因为一个暂停线程而退化。

因此 epoch 临界区必须短，不能在里面执行阻塞操作、分配大量内存、调用未知回调或等待锁。无锁结构常说“不要在临界区阻塞”，这里不仅是为了进展保证，也是为了回收。一个系统如果 worker 可能被长期暂停，hazard pointer 或引用计数可能更适合；如果操作非常短且线程数固定，epoch 更简单。

12.25 RCU 的读多写少模型

Read-Copy-Update 可以看成一种读多写少的发布和回收模型。读者进入 RCU read-side 临界区后，可以无锁读取当前指针；写者复制一份新结构，修改后发布新指针，然后等待所有旧读者离开 grace period，再释放旧结构。它适合路由表、配置表、只读索引、分片元数据等读频率高、更新频率低的场景。

RCU 的代价是写路径复制和延迟回收。若结构很大，复制成本高；若读者临界区很长，旧版本保留时间长；若更新频繁，多个旧版本会堆积。它不是通用替代 mutex 的工具。对本书项目而言，分布式路由表和配置快照可以用 RCU 思想解释：worker 读取稳定快照，driver 发布新版本，旧版本在没有读者后释放。

12.26 SPSC、MPSC、MPMC 的接口差异

队列名称必须包含生产者和消费者模型。SPSC 允许一个生产者和一个消费者，因此 head 只有消费者写，tail 只有生产者写，内存序和 cache line 布局都相对简单。MPSC 多个生产者争用 tail，消费者独占 head。SPMC 生产者独占 tail，多个消费者争用 head。MPMC 两端都有多个写者，最复杂。

接口也要体现差异。SPSC 可以在构造时绑定 producer 和 consumer，或者在文档中明确不可跨线程多端调用。MPSC 的 push 可能需要返回失败或阻塞，必须处理多个生产者之间的公平性。MPMC 的 try_push 和 try_pop 在高竞争下 CAS 失败很正常，调用者要能接受重试成本。把一个 SPSC 类型命名成 FastQueue 是危险的，因为调用者不知道约束。

12.27 有界和无界的取舍

有界无锁队列用固定数组，容量清楚，内存布局稳定，适合实时和高性能场景。队列满时返回失败或背压。无界队列通常需要动态分配节点，避免固定容量限制，但引入分配器、回收和内存峰值问题。很多系统宁愿选择有界队列，因为过载时应该背压，而不是无限分配直到 OOM。

有界队列的容量也不是小细节。容量太小，生产者频繁遇到满，吞吐下降；容量太大，缓存局部性变差，排队延迟增大，过载被隐藏更久。容量应来自吞吐、延迟和突发流量模型。异步 I/O 章和分布式 shuffle 章会继续使用这个思想：有界缓冲让过载显性化。

12.28 无锁结构里的内存分配

在 lock-free 操作内部调用通用分配器，可能破坏进展保证。分配器内部可能加锁，可能触发系统调用，可能因为内存压力阻塞。若算法声称 lock-free，但每次 push 都要进入可能阻塞的 allocator，那么整体语义要重新说明。成熟无锁队列常使用预分配节点池、有界数组或专门的非阻塞分配策略。

即使分配本身很快，释放也不能随意发生。pop 成功后节点从结构中摘除，但其他线程可能仍持有旧指针用于 CAS 验证。直接 delete 会制造 use-after-free。无锁结构的性能报告如果没有说明分配和回收，就只讲了一半。

12.29 测试无锁结构

无锁结构测试要覆盖三类性质。第一，功能正确：没有丢失、重复、越界、错误顺序。第二，并发安全：TSan 无数据竞争，压力测试在多线程、多核心、不同容量下稳定。第三，进展和资源：高竞争下不会无限自旋，失败重试次数可观测，retired 节点不会无限增长，关闭和析构不会悬挂。

测试数据应带唯一 ID。生产者生成连续 ID，消费者记录到位图或哈希集合，最终检查每个 ID 恰好出现一次。若队列只保证每生产者 FIFO，就为每个生产者单独检查 sequence；若队列不保证顺序，就不要要求全局 FIFO。内存回收测试应故意让消费者慢、生产者快、线程暂停和节点复用频繁，以增加 ABA 暴露机会。

```
lock-free queue test matrix:
capacity: 1, 2, 3, power of two, non power of two
producers: 1, 2, many
consumers: 1, 2, many
workload: burst, steady, producer fast, consumer fast
checks: no loss, no duplicate, declared order, no stuck shutdown
```

12.30 何时不要自研无锁结构

如果问题的主要成本是 I/O、序列化、哈希计算、远端网络或磁盘，队列是否无锁可能不重要。如果临界区保护复杂不变量，使用 mutex 可能更清楚。如果团队没有能力维护内存回收协议，自研无锁结构风险很高。如果已有成熟库满足需求，优先使用成熟库并写清接口保证。无锁结构适合明确的热点、清晰的生产者消费者模型、固定容量或明确回收策略，以及足够的测试预算。

本书仍然详细讲无锁，是为了让读者理解现代系统里的边界：原子不是魔法，线性化点要证明，ABA 要处理，回收要设计，进展保证要声明。理解这些以后，即使最终选择 mutex，也会是更成熟的选择。

12.31 MPMC 有界队列的序号模型

MPMC 有界队列常用每槽位 sequence 模型。数组容量固定，每个槽位有一个 sequence。生产者读取 tail，找到槽位，比较槽位 sequence 是否等于 tail，表示空闲；成功 CAS tail 后写数据，再把槽位 sequence 设置为 tail + 1，表示可读。消费者读取 head，找到槽位，比较 sequence 是否等于 head + 1，表示可读；成功 CAS head 后读数据，再把 sequence 设置为 head + capacity，表示下一轮空闲。

这个模型的难点在于同一个槽位会被循环复用。只用 Empty/Ready 两个状态不足以区分第几轮，sequence 把轮次编码进状态，避免旧状态被误认。它也展示了无锁结构的典型复杂性：tail/head

只是预留进度，槽位 `sequence` 才说明数据是否真正发布。生产者预留成功但尚未写完时，消费者不能读这个槽位。

教学中不必完整实现工业级 MPMC，但要让读者能画出状态。某个槽位从空闲到写入中到可读到读取后空闲，每次状态变化的发布点在哪里，哪个线程写 `sequence`，哪个线程读 `sequence`，失败时是否重试，队列满如何判断，都是正确性核心。

12.32 Work stealing deque 的边界竞争

工作窃取 deque 比普通队列更特殊。Owner 从 bottom push/pop，stealer 从 top steal。大多数时候 owner 独占 bottom，操作很快；只有当队列接近空，owner pop 和 stealer steal 可能争夺最后一个元素。这个边界需要 CAS 决定谁赢。若处理不当，可能同一个任务被执行两次或丢失。

Chase-Lev deque 的论文和实现值得阅读，但不要在没测试和理解的情况下照抄。它涉及 top、bottom、环形数组扩容、内存序和最后元素竞争。教学项目可以先用 mutex deque 验证工作窃取策略，再考虑无锁 deque。策略正确和数据结构无锁是两件事，分阶段实现更稳。

12.33 无锁和可阻塞边界

很多系统需要同时支持无锁 fast path 和阻塞等待。SPSC ring buffer 可以在 `try_push` 满时返回 false；上层若需要阻塞，就用 condition variable、semaphore 或事件机制等待空间。把阻塞语义直接塞进无锁结构，会增加复杂度。常见做法是底层提供非阻塞 try 操作，上层用有界队列协议处理等待、关闭和取消。

这种分层很重要。无锁结构负责快速状态转换，运行时负责背压和生命周期。若一个无锁队列没有关闭语义，上层必须定义关闭时如何唤醒等待者、如何拒绝新写、如何 drain 旧数据。不要以为无锁结构自动解决线程池退出问题。

12.34 内存回收的测试

无锁回收测试要专门设计。普通功能测试很难触发 ABA 和 use-after-free。可以使用小对象池，故意快速复用地址；使用少量节点，增加循环复用频率；插入 yield 或短 sleep，让线程在读取指针后暂停；开启 AddressSanitizer 或 ThreadSanitizer；统计 retired 列表长度，确保不会无限增长。

测试还要覆盖线程退出。一个线程持有 hazard pointer 后退出，系统是否清理槽位？一个线程进入 epoch 后长时间暂停，内存是否持续增长？对象池销毁时，是否还有未释放 retired 节点？这些问题在 happy path 中看不到，但生产中非常重要。

12.35 无锁结构和异常

无锁结构通常要求热路径不抛异常。若 push 过程中构造元素可能抛异常，槽位状态如何回滚？若 CAS 成功预留了位置，元素构造失败，消费者如何知道该槽位不可读且队列仍然一致？为了避免这类问题，有界队列常要求元素可无异常移动，或先在外部构造好对象，再把指针或索引放入队列；也可以使用预构造槽位和明确状态机。

C++ 泛型无锁容器因此很难写。支持任意 T、任意构造抛异常、任意析构副作用，会让协议复杂很多。教学示例使用 `std::int32_t` 或指针，不是偷懒，而是先隔离同步机制。生产容器要在模板约束和文档中写明要求。

12.36 无锁指标

无锁结构需要自己的指标。吞吐之外，应记录 CAS 尝试次数、CAS 失败次数、队列满次数、队列空次数、最大重试次数、retired 节点数量、回收扫描次数和平均扫描成本。没有这些指标，高竞争下的性能问题很难解释。一个 lock-free 队列吞吐高但 CAS 失败极多，可能浪费 CPU；一个 epoch 回收读路径快但 retired 节点堆积，可能隐藏内存风险。

指标也要避免成为共享热点。每个 worker 可以维护本地失败计数，最后汇总。若每次 CAS 失败都更新全局原子指标，指标本身会改变性能。观测性仍要遵守数据布局和同步原则。

12.37 成熟库评估清单

使用成熟无锁库也要评估。它支持 SPSC、MPSC 还是 MPMC？有界还是无界？内存回收策略是什么？元素类型要求是什么？是否支持阻塞等待、关闭、取消？是否保证 FIFO？是否有 ABA 保护？是否支持目标平台和编译器？许可证是否合适？测试和维护状态如何？

不要只因为库名里有 lockfree 就引入。若项目只需要单生产者单消费者，使用简单 SPSC ring buffer 可能更清楚；若需要复杂 MPMC 阻塞队列，成熟库可能比自研安全；若需要关闭和 drain，库的语义必须匹配。库选择同样是工程设计。

12.38 无锁学习路线

本章建议学习路线是：先用 mutex 队列理解不变量；再写 SPSC ring buffer 理解 acquire/release 和单写者；再学习 MPSC 预留和提交分离；再学习 MPMC sequence 槽位；再学习 ABA 和回收；最后阅读工作窃取 deque。不要跳过前面直接写 MPMC。每一步都要有测试和线性化点说明。

Compute Systems Engine 也应按这个路线演进。第一版使用 mutex bounded queue；第二版在单向管线中使用 SPSC；第三版只在明确瓶颈处评估 MPMC；工作窃取先用锁实现策略，再考虑无锁 deque。这样项目可以持续正确，而不是为了“高级”牺牲可维护性。

12.39 案例：日志管线中的 SPSC

日志管线可以用 SPSC 连接一个读取线程和一个解析线程。读取线程是唯一 producer，解析线程是唯一 consumer，固定 batch 槽位在 ring buffer 中循环使用。这个场景非常适合 SPSC，因为生产者和消费者模型清楚，容量固定，背压明确。若后续增加多个解析线程，SPSC 就不再适用，需要每个解析线程一个队列，或者改用 MPSC/MPMC。

这个案例强调接口约束。类型名和文档应写清 SingleProducerSingleConsumer。运行时如果需要动态扩展消费者，不能偷偷复用 SPSC。无锁结构的安全来自约束被遵守，而不是来自代码本身能抵抗所有误用。

12.40 案例：对象池中的 ABA

对象池会复用 slot，因此特别容易出现 ABA 形态。线程 A 持有 slot 5 的旧句柄；线程 B 释放 slot 5，又分配给新对象；线程 A 只看 index 仍然是 5，以为对象没变。Generation 可以解决这个问题。每次 slot 复用 generation 加一，句柄包含 index 和 generation，访问时检查两者匹配。

这个结构不一定是 lock-free，但它体现了 ABA 思想。ABA 不只属于无锁栈，也属于任何“位置复用但旧引用可能存在”的系统。分布式里的 epoch、防旧 leader，也是同一思想在更大尺度上的版本。

12.41 无锁章验收清单

项目若加入无锁结构，必须提供：适用生产者消费者模型，容量和满空语义，线性化点，内存序理由，关闭和取消如何处理，内存回收策略，ABA 防护，元素类型要求，压力测试矩阵，指标和 fallback。缺少其中任何一项，都不应把它作为默认基础设施。无锁结构进入系统的门槛应该高于 mutex 结构。

12.42 无锁章节总结

无锁结构的难点不在少写一个 mutex，而在进展保证、线性化点、内存序、ABA、回收、异常和误用边界。SPSC、MPSC、MPMC 是不同问题，不能互相冒充。工程上应优先把生产者消费者模型说清楚，把锁版本作为 reference，再在明确热点上引入无锁。无锁代码必须可证明、可测试、可回退。

12.43 无锁决策表

考虑无锁结构前，先问：生产者和消费者数量是否固定，容量是否可以有界，元素是否可无异常移动，是否需要阻塞等待，是否需要关闭和取消，节点是否动态分配，是否有回收协议，CAS 失败是否可接受，尾延迟是否重要。如果答案不清楚，优先使用锁结构。无锁是高要求工程工具，不是

默认优化按钮。

12.44 案例：无锁队列的关闭外置

一个 SPSC ring buffer 可以只提供 `try_push` 和 `try_pop`，关闭状态由外层管线管理。外层有一个原子 `closed` 标志和条件变量或 `eventfd` 唤醒等待者。这样 ring buffer 保持简单，上层负责生命周期。若把关闭、阻塞等待、取消和超时全部塞入 ring buffer，结构会迅速复杂。

分层设计让每个组件职责明确。底层 ring 负责无锁数据交换，上层 channel 负责阻塞和关闭，运行时负责任务生命周期。系统工程常常不是写一个万能类，而是把几个小语义组合起来。

12.45 本章落到贯穿项目

Compute Systems Engine 的无锁路线应分三步。第一步，实现并测试 SPSC ring buffer，只用于单生产者单消费者管线，例如一个 I/O 模拟线程向一个计算线程发送批次。第二步，比较 SPSC 和 mutex bounded queue 在目标场景下的吞吐和等待行为，确认收益来自减少锁和固定内存，而不是改变语义。第三步，再研究 MPMC 或工作窃取 deque，并在实现前写出不变量、线性化点、内存序和回收策略。

项目不应急着把所有队列替换成无锁队列。线程池全局队列最开始用 mutex 更容易证明关闭语义；SPSC 用于清晰管线；MPMC 用于明确的多生产者多消费者热点；work stealing deque 用于本地队列和窃取。不同结构服务不同场景。

12.46 本章习题

习题与作业

习题一：为本章 SPSC ring buffer 写出不变量、满空判断、push 线性化点和 pop 线性化点。说明为什么它不能被多个生产者同时调用。

习题二：把 SPSC ring buffer 扩展到支持容量检查和关闭语义。只要求设计状态机，不要求完整代码。说明关闭后 producer 和 consumer 分别应看到什么返回值。

习题三：写一份 ABA 场景时间线，至少包含两个线程、两个节点、一次 pop、一次释放和一次地址复用。然后分别说明版本标签、hazard pointer、epoch 如何缓解。

习题四：选择一个成熟无锁队列实现或论文，阅读其不变量和内存回收策略。报告必须说明它适用 SPSC、MPSC 还是 MPMC，不能只说“这是无锁队列”。

实验：阅读 Compute Systems Labs 的 bounded queue，写出 mutex 版本保护的不变量。再设计一个 SPSC ring buffer 的接口，说明哪些状态可以用 relaxed，哪些位置需要 acquire/release。

第零部分

并行算法与运行时

第 13 章

归约、扫描与分区

并行算法的核心是处理依赖。归约、扫描和分区是许多系统的基础操作：数据库聚合、日志统计、图计算、排序、渲染和科学计算都会用到它们。

13.1 归约

归约把多个输入合并成一个输出。并行归约的基本策略是局部归约加全局合并。它的性能取决于输入划分、局部性、合并成本和结果类型。整数加法简单，浮点加法要考虑精度和顺序，字符串拼接则可能有大量分配。

归约优化通常先避免共享写。每个线程写自己的 `partial`，最后串行或树形合并。树形合并可以减少合并深度，也更适合分布式系统。

归约的第一步是写出代数性质。操作是否有单位元，是否满足结合律，是否满足交换律，是否允许改变顺序。整数最小值、最大值、按位或通常容易并行；浮点求和需要说明误差模型；哈希合并需要说明冲突处理；对象聚合需要说明生命周期和分配。没有这些语义前提，谈并行归约是不完整的。

归约的第二步是写出内存布局。`partial` 结果应避免 `false sharing`，通常每个 worker 一个对齐槽位。若 `partial` 很大，可以按 NUMA 节点或缓存块组织，避免最后合并阶段变成单线程长尾。归约不是只有“线程本地加一下”这一种形式，它可以是线性合并、二叉树合并、分层合并或流水线合并。

13.2 扫描

扫描也叫 `prefix sum`，输出每个位置之前元素的累计结果。它比归约更难，因为每个输出都依赖前缀。并行扫描通常分两阶段：先对每个块做局部扫描，得到块总和；再扫描块总和；最后把块前缀加回块内结果。

扫描是很多算法的 `building block`，例如 `stream compaction`、并行分配输出位置、基数排序和图前沿构造。理解扫描，可以帮助读者把“有顺序依赖”的问题拆成局部和全局两层。

扫描比归约更能体现并行算法设计。表面上，每个输出都依赖前面所有输入，似乎完全串行；但把输入分块后，每个块内部可以先做局部扫描，块总和再做一次小规模扫描，最后把块前缀加回去。这个套路说明，很多“看起来串行”的依赖，可以拆成局部依赖和块间摘要。分布式系统里的很多计算也是这个形状：节点内先处理，节点间交换摘要，再回填结果。

13.3 分区

分区把元素按谓词或 `key` 分到不同区域。并行分区要先统计每个线程每个桶的数量，再计算全局偏移，最后写入输出。这与扫描密切相关。直接让多个线程 `push` 到共享 `vector`，会引入锁或原子热点。

分区的关键是先分配写入位置，再并行写入。以 `copy_if` 为例，朴素并行写法可能让所有线程对输出 `vector` 做 `push`，需要锁或原子。更好的写法是每个线程先统计自己保留多少元素，对这些计数做扫描得到每个线程的输出起点，然后每个线程写入自己的连续区间。这样写入没有冲突，输出也保持可解释顺序。

13.4 负载均衡

静态划分适合每个元素成本相近的场景。若元素成本差异大，静态划分会让某些线程早早结束，其他线程仍在工作。动态调度、任务队列和工作窃取可以改善负载均衡，但会增加调度开销。

负载均衡不是越动态越好。静态划分没有调度开销，局部性好，适合规则数组和均匀工作。动态队列能处理不均匀工作，但会增加同步和队列争用。工作窃取减少全局争用，但实现更复杂。设计时要估算任务成本方差：如果每个块成本接近，静态块划分更好；如果成本长尾明显，动态调度

才值得。

13.5 并行算法的正确性模板

本册要求每个并行算法都写出四个正确性问题。第一，串行语义是什么，reference 怎样实现。第二，并行切分后，每个 worker 负责哪些输入和输出。第三，局部结果怎样合并，合并顺序是否影响结果。第四，是否存在共享写、重复写、漏写或越界写。性能优化只能在这些问题的回答之后进行。

这套模板能防止“并行版本跑得快但其实错”。很多并行 bug 不会在小输入暴露，因为线程交错少、边界简单、缓存状态稳定。测试必须覆盖空输入、极小输入、刚好一个块、多个块、不能整除块大小、重复 key、倾斜 key 和高线程数。

13.6 从串行 reference 开始

并行算法一定要从串行 reference 开始。reference 不一定最快，但它必须简单、可读、边界清楚。所有优化版本先和 reference 比结果，再谈速度。没有 reference 的并行优化很危险，因为你很难判断一个高吞吐结果是否只是漏写、重复写或改变了语义。

以 `count_if` 为例，串行 reference 很短。

Listing 13.1 串行 count-if reference

```
1 std::int64_t count_at_least(std::span<const std::int32_t> values,
2                             std::int32_t threshold) {
3     std::int64_t count = 0;
4     for (const std::int32_t value : values) {
5         if (value >= threshold) {
6             ++count;
7         }
8     }
9     return count;
10 }
```

它的语义是：遍历每个输入元素，满足谓词则计数加一。并行版本不能改变谓词，不能漏掉元素，不能重复计数，不能因为线程数不同得到不同整数结果。这个 reference 也能用于测试空输入、负数、重复值、全部满足、全部不满足和随机输入。

13.7 静态切分的边界

静态切分把输入区间的按 worker 数分成若干连续块。连续块有两个优点：每个 worker 顺序扫描自己的数据，缓存和预取友好；输出 partial 数组按 worker 索引存放，容易合并。切分时要处理不能整除的情况，不能简单地让每个 worker 处理 `n/worker_count` 个元素后丢掉尾部。

Listing 13.2 连续区间切分公式

```
1 std::size_t block_begin(std::size_t size,
2                          std::int32_t worker,
3                          std::int32_t worker_count) {
4     return size * static_cast<std::size_t>(worker) /
5            static_cast<std::size_t>(worker_count);
6 }
7
8 std::size_t block_end(std::size_t size,
9                       std::int32_t worker,
10                      std::int32_t worker_count) {
11     return size * static_cast<std::size_t>(worker + 1) /
12            static_cast<std::size_t>(worker_count);
13 }
```



```
13 }
```

这个公式保证所有块覆盖 $[0, \text{size})$, 相邻块首尾相接, 不重叠也不漏元素。若 `worker_count > size`, 某些块可能为空, 因此实际 worker 数可以取 `min(worker_count, size)`, 或者允许空块但测试要覆盖。工程里还要检查 `worker_count` 必须为正。

13.8 并行 count-if: 从错误版本到正确版本

错误版本让所有线程更新一个全局原子计数器。它简单但共享写很重。正确的第一步是每个 worker 写自己的 partial。

Listing 13.3 并行 count-if 的 partial 版本

```
1 struct alignas(64) CountIfPartial {
2     std::int64_t count = 0;
3 };
4
5 void count_if_worker(std::span<const std::int32_t> values,
6                     std::int32_t threshold,
7                     CountIfPartial& partial) {
8     std::int64_t local = 0;
9     for (const std::int32_t value : values) {
10         if (value >= threshold) {
11             ++local;
12         }
13     }
14     partial.count = local;
15 }
```

这个 worker 函数只写自己的 partial, 不触碰共享计数器。主线程或合并任务最后扫描 partial 得到总数。这个结构的正确性来自区间划分: 每个输入元素属于且只属于一个 worker; 每个 worker 的 local 是该区间内满足谓词的数量; 合并是整数加法。性能收益来自减少共享写和保持连续扫描。

但 partial 版本也有边界。若输入很小, 线程创建和合并成本可能超过收益; 若谓词很复杂, 每个元素成本高, 动态调度可能比静态切分更适合; 若阈值和输入分布导致分支不可预测, 分支优化可能比并行更重要。并行不是第一反应, 正确的流程是先写 reference, 再测瓶颈, 再选择并行结构。

13.9 归约树和关键路径

线性合并 partial 很简单: 一个线程从头到尾加所有 partial。worker 数很少时足够; worker 数很多或 partial 很大时, 线性合并会成为串行尾巴。树形合并把 partial 两两合并, 关键路径从 $O(p)$ 变成 $O(\log p)$, 其中 p 是 partial 数。

树形合并也有代价。它需要额外任务或多轮同步; 若每个 partial 只是一个整数, 线性合并可能已经足够快; 若 partial 是大型直方图, 树形合并更有价值。不要把树形合并当成默认更好, 要看合并成本占比。

```
linear merge:
    (((p0 + p1) + p2) + p3) + p4 ...

tree merge:
    round 1: p0+p1, p2+p3, p4+p5, p6+p7
    round 2: s0+s1, s2+s3
    round 3: t0+t1
```

树形合并还为分布式 reduce 做准备。单机里 round 是线程池任务, 分布式里 round 是不同节

点之间的聚合阶段。理解关键路径后，读者才能解释为什么某些系统使用分层聚合，而不是把所有 partial 发到一个中心节点。

13.10 浮点归约的语义选择

浮点求和不能简单说“加法满足结合律”。串行从左到右、四累加器、树形合并、按 NUMA 节点合并都会产生不同舍入。工程系统必须选择语义：是否要求逐位一致，是否允许误差范围，是否使用确定性合并树，是否使用补偿求和，是否开启 fast-math。

逐位一致通常成本更高，因为合并顺序要固定，某些向量化和并行优化会受限。误差容忍则需要测试误差界，而不是只比较完全相等。科学计算、金融、渲染、机器学习统计的容忍度不同，不能用一种答案覆盖。

13.11 扫描的三阶段实现

扫描的三阶段可以写得很清楚。第一阶段，每个 worker 对自己的块做局部 scan，并记录块总和。第二阶段，对块总和做 scan，得到每个块之前的总偏移。第三阶段，每个 worker 把该偏移加到自己块内所有输出上。这个算法把一个长依赖链拆成多个短依赖链和一个小规模依赖链。

Listing 13.4 局部 scan 和回填偏移

```

1 std::int64_t local_inclusive_scan(std::span<const std::int32_t> input,
2                                 std::span<std::int64_t> output) {
3     const std::size_t count = std::min(input.size(), output.size());
4     std::int64_t prefix = 0;
5     for (std::size_t i = 0; i < count; ++i) {
6         prefix += input[i];
7         output[i] = prefix;
8     }
9     return prefix;
10 }
11
12 void add_block_offset(std::span<std::int64_t> output,
13                      std::int64_t offset) {
14     for (std::int64_t& value : output) {
15         value += offset;
16     }
17 }

```

如果使用 inclusive scan，输出位置 *i* 包含 input[0] 到 input[i]。如果使用 exclusive scan，输出位置 *i* 包含 input[0] 到 input[i-1]。两者都常用，但语义不同。教材和 API 必须在名称中写清 inclusive 或 exclusive，不能只叫 scan 后让调用者猜。

13.12 Scan 的工作量和空间

并行 scan 通常比串行 scan 做更多工作。它需要写输出、写 block sums、扫描 block sums、再回填。若输入很小，串行 scan 更快；若输入很大且多核可用，并行 scan 才有机会获益。scan 还需要额外存储 block sums，通常大小是 worker 数或 block 数，远小于输入，但仍要管理生命周期。

scan 也会受内存带宽限制。对每个元素，局部 scan 读输入写输出，回填又读写输出一次。若元素类型大或输出巨大，内存流量很可观。优化方向可能是把 scan 和后续写输出融合，例如 stream compaction 中，统计和写出阶段可以根据选择率和缓存做调整。算法分阶段清晰，但实现可以在不破坏语义的前提下融合。

13.13 Partition 的两级偏移

按 key 分区通常比 `copy_if` 更复杂，因为有多桶。每个 worker 先统计自己块中每个桶的数量，形成一个二维计数矩阵：worker 乘 bucket。然后对每个 bucket 的 worker counts 做 scan，得到每个 worker 在该 bucket 的写入偏移。最后每个 worker 再次遍历自己的输入，把元素写到对应 bucket 的对应区间。

这个过程避免了多个 worker 对同一 bucket 的共享 push。每个 worker 写入的输出区间是唯一的，不需要锁。代价是计数矩阵和两遍输入扫描。桶数很多时，计数矩阵可能很大；输入很大时，两遍扫描消耗带宽；key 分布倾斜时，某些 bucket 输出很大，后续处理仍会倾斜。分区解决写入冲突，不自动解决下游负载均衡。

```
parallel partition shape:
  pass 1: worker local counts per bucket
  scan:   compute global bucket offsets
  pass 2: worker writes to unique output ranges
```

这套结构是分布式 shuffle 的单机版本。pass 1 是 map 端计数，scan 是计算每个分区的输出位置，pass 2 是写分区文件或缓冲。分布式系统把 output range 换成远端节点或磁盘分区，核心思想不变。

13.14 稳定分区和非稳定分区

分区还要说明是否稳定。稳定分区保持相同桶内元素的原始相对顺序；非稳定分区不保证。稳定分区更容易推理和测试，但可能限制优化；非稳定分区可以使用更自由的交换和块内重排，适合只关心集合不关心顺序的场景。

并行稳定分区通常依赖每个 worker 处理连续块，并按 worker 顺序给每个 bucket 分配输出区间。这样同一 worker 内保持顺序，worker 间按原始区间顺序排列。若使用动态调度，稳定性就更难保证，因为任务完成顺序和输入顺序不同。稳定性不是免费属性，必须在接口里声明。

13.15 负载均衡和任务切分

归约和 scan 对均匀数组很友好，静态切分通常足够。分区和过滤则可能不均匀：某些块满足谓词的元素很多，写输出更重；某些 key 特别热，后续 bucket 处理更重。图算法和字符串解析更不均匀，元素成本可能差几个数量级。任务切分要考虑工作量方差。

一种策略是过分割：把输入切成比 worker 多得多的块，由线程池动态调度。这样负载更均衡，但任务调度成本和 partial 数增加。另一种策略是先采样估计工作量，再按估计切分。数据库和分布式系统常用采样来处理倾斜。没有一种策略永远最好，关键是通过指标观察 worker 空闲时间和长尾任务。

13.16 内存分配和输出所有权

并行算法最怕在热路径里动态扩容共享容器。多个线程对同一个 `std::vector push_back` 必须加锁，扩容还会移动数据。正确做法通常是先计算输出大小或每个块输出大小，再一次性分配输出，最后每个 worker 写自己的区间。scan 的重要性就在这里：它把“我要写到哪里”变成一个无冲突的确定偏移。

如果输出大小无法提前准确知道，可以使用 per-worker buffer，最后合并；也可以使用分块 allocator；或者使用有界队列把输出流式送给下游。每种方案都有代价：per-worker buffer 需要额外内存和合并；allocator 需要生命周期设计；队列引入背压和调度。输出所有权是并行算法设计的一部分，不是实现细节。

13.17 测试矩阵

归约、扫描和分区的测试要覆盖结构性边界。归约测试包括空输入、单位元、一个元素、多块、负数、溢出边界和浮点误差。扫描测试包括 inclusive/exclusive、块边界、最后不满块和输出长度不足。分区测试包括零个元素、全部同桶、每个桶均匀、极端倾斜、稳定性验证和输出容量。并发测试

还要在不同线程数下重复运行，捕捉隐藏竞态。

性能测试则要分离三个成本：读取输入、写输出、合并 partial。对于 count-if，读取输入和分支是主成本；对于 stream compaction，写输出和 scan 偏移也重要；对于 histogram，partial 内存占用和合并成本重要。报告不应只给总时间，而要解释哪一阶段主导。

13.18 从串行 reference 开始

并行算法的第一步不是开线程，而是写清串行 reference。Reference 的任务是定义语义：空输入返回什么，溢出如何处理，浮点误差如何比较，输出顺序是否稳定，异常输入是否允许。没有 reference，并行版本即使跑得快，也不知道是否正确。

以 count-if 为例，串行 reference 可以非常朴素。它不追求速度，不使用 SIMD，不提前分块，只表达每个元素被检查一次且满足谓词时计数加一。并行版本的正确性证明要回到它：输入被分成互不重叠的块，每个块使用同一谓词统计，partial 求和等于全局计数。这个证明比代码更重要，因为它让后续优化有基准。

Listing 13.5 串行 reference 定义 count-if 语义

```
1 std::int64_t count_positive_reference(std::span<const std::int32_t> values) {
2     std::int64_t count = 0;
3     for (const std::int32_t value : values) {
4         if (value > 0) {
5             ++count;
6         }
7     }
8     return count;
9 }
```

Reference 还可以故意保持简单类型和清晰边界。若输入长度可能超过 `std::int32_t`，计数使用 `std::int64_t`。若数据来自文件，reference 不负责 I/O，只处理已经解析好的数组。把语义层和 I/O 层分开，后续测试更稳定。

13.19 范围切分的数学边界

并行数组算法通常先把区间 $[0, n)$ 切成若干块。切分必须保证覆盖完整、互不重叠、不会越界、空输入正确、线程数大于元素数时仍正确。一个常见写法是根据 block index 计算 begin 和 end，最后一块自然处理余数。不要用容易溢出的 `i*block_size` 盲目计算，也不要假设 `n` 一定能整除 worker 数。

Listing 13.6 按块编号计算半开区间

```
1 struct Range {
2     std::size_t begin = 0;
3     std::size_t end = 0;
4 };
5
6 Range block_range(std::size_t total,
7                 std::size_t block_index,
8                 std::size_t block_count) {
9     const std::size_t begin = (total * block_index) / block_count;
10    const std::size_t end = (total * (block_index + 1)) / block_count;
11    return Range{begin, end};
12 }
```

这个写法让每个块大小相差不超过一，且不需要单独处理余数。但如果 `total*block_index` 可能超过 `std::size_t`，超大输入下仍要使用更稳的计算方式。教材阶段至少要让读者知道半开区

间和覆盖证明：第 k 块的 end 等于第 $k+1$ 块的 begin，第一块 begin 为零，最后一块 end 为 total。

13.20 单位元和结合性

归约要求一个二元操作和单位元。整数求和的单位元是零，乘法单位元是一，最小值的单位元可以是类型最大值，最大值的单位元可以是类型最小值。若没有正确单位元，空块就难处理。并行切分后，有些 worker 可能拿到空范围，因此算法不能只在“每块至少一个元素”的假设下成立。

结合性决定能否任意重排合并。整数加法在不溢出的数学整数上结合，但固定宽度整数溢出有语言和业务边界；浮点加法因为舍入不严格结合；字符串拼接结合但内存成本高；哈希合并可能依赖顺序。并行归约的 API 应说明操作是否必须结合，是否要求交换，是否保证确定性顺序。

13.21 确定性归约

并行归约常遇到一个现实需求：每次运行结果是否完全一样。整数和某些集合运算容易保持确定；浮点、哈希表迭代、动态调度和非稳定合并会造成结果波动。对机器学习训练日志、科学计算和金融报表，波动可能不能接受；对监控指标和近似统计，误差范围可能可以接受。

确定性归约可以使用固定切分和固定合并树。无论任务完成顺序如何，最终合并按 block id 顺序进行。这样牺牲一部分调度自由，换来可复现。另一个策略是使用补偿求和或更高精度 partial，降低误差但不一定逐位一致。教材应要求读者在报告中写明选择：追求吞吐、误差范围还是逐位复现。

```
deterministic reduce:
  split input into fixed block ids
  compute partial[block_id]
  merge partials by a fixed binary tree
  compare floating output by declared tolerance or exact rule
```

13.22 Histogram 的两级设计

Histogram 是 reduce、partition 和缓存布局的交汇点。朴素并行写法让所有线程对全局 bucket 做原子加法。若 bucket 少且输入大，热点非常明显。更好的写法是每个 worker 维护本地 histogram，最后合并。这样热路径写线程本地数组，避免全局原子争用。

Listing 13.7 线程本地 histogram 的核心形态

```
1 std::vector<std::uint64_t> histogram_local(
2     std::span<const std::uint32_t> keys,
3     std::uint32_t bucket_count) {
4     std::vector<std::uint64_t> buckets(bucket_count, 0);
5     for (const std::uint32_t key : keys) {
6         ++buckets[key % bucket_count];
7     }
8     return buckets;
9 }
```

这个版本只展示本地统计。完整并行版本要为每个 worker 准备一个 buckets 数组，再按 bucket 或按 worker 合并。bucket_count 很小时，本地数组很小，合并成本低；bucket_count 很大时，每个 worker 一份数组可能占用大量内存，而且合并会扫描很多零。稀疏 key 场景可能改用局部哈希表；密集小整数 key 场景用数组更快。选择数据结构要看 key 空间和实际非零数量。

13.23 Filter 和 Compaction 的推导

过滤输出满足谓词的元素，看起来像对 vector 做并发 push_back。朴素做法给输出 vector 加锁，每个命中元素都 push。这个做法正确但慢，因为锁进入每个命中元素的热路径。优化思路是把“我要写哪里”提前算出来。

第一步，每个 worker 统计自己块里命中的数量。第二步，对这些数量做 exclusive scan，得到每个 worker 的输出起始偏移。第三步，每个 worker 再扫描自己的输入，把命中元素写到自己的输出区间。这样每个元素最多读两遍，但写输出无锁且连续。若命中率很低，两遍读取可能仍划算；若谓词计算很重，可以在第一遍保存局部选择位图，第二遍避免重复计算。

```
stream compaction:
  count pass: local selected counts
  scan pass: worker output offsets
  write pass: each worker writes selected elements to owned range
```

这个推导要让读者看到优化从哪里来：不是凭空想到 scan，而是因为共享 push 的冲突需要被“所有权区间”替代。Scan 的作用就是把局部数量转换成全局唯一偏移。后续分布式 shuffle 也是同理：先知道每个 partition 有多少数据，再为每个 writer 分配输出位置。

13.24 Partition 的内存布局

多桶 partition 可以把输出组织成连续 bucket 区间，也可以每个 bucket 一个 vector。连续区间适合后续顺序扫描和一次性写文件，bucket vector 适合增量构建但容易产生多次分配。高性能实现常先统计 counts，计算 bucket offsets，分配一个大输出数组，再让 worker 写唯一范围。

如果需要稳定分区，worker 处理的输入区间顺序必须和输出区间顺序一致。对每个 bucket，worker 0 的元素排在 worker 1 前面，worker 内部保持原顺序。若不需要稳定性，可以在块内使用更快交换或局部重排。接口里声明稳定性很重要，因为稳定性会影响测试、性能和后续算法。

13.25 排序与分区的关系

分区可以看成粗粒度排序：只把元素按 bucket 聚到一起，不要求 bucket 内有序。排序则要求全局顺序。很多系统先 partition 再局部 sort，例如数据库按 hash 分区后每个 partition 内排序，或者分布式计算按 key 分区后 reduce 端排序。这样做能把全局大问题拆成多个小问题。

排序也能帮助 reduce。若输入按 key 排好，合并相同 key 只需要顺序扫描；若输入无序，需要哈希表聚合。排序成本是 $O(n \log n)$ ，哈希平均接近线性但内存随机访问多。小 key 空间可以用数组 histogram，稀疏大 key 用哈希，范围查询或外部合并用排序。并行算法教材应展示这种选择，而不是把所有聚合都写成一个 `unordered_map`。

13.26 并行算法和内存带宽

归约、scan、filter 和 partition 都可能受内存带宽限制。多线程能提高吞吐，直到内存控制器、LLC 或 NUMA 互连饱和。线程数继续增加，时间可能不降反升。报告中应画线程数扩展曲线，而不是只比较 1 线程和最大线程数。

算法阶段也要估算字节流量。Count-if 每个元素读一次，写 partial 很少；scan 读输入写输出，回填又读写输出；compaction 至少读输入两次或读输入加读位图；partition 读输入、写 counts、读写输出。若某个优化减少了计算却增加了内存流量，在带宽瓶颈下可能变慢。理解字节移动，是并行算法的基本功。

13.27 任务粒度和调度开销

并行算法要把工作拆成任务，但任务太小会被调度开销吞掉。每个任务提交、入队、唤醒、执行和记录统计都有成本。若一个任务只处理几十个整数，调度成本可能比计算成本大。任务太大又会负载不均，尤其是过滤、图遍历和字符串解析这类元素成本不均匀的工作。

工程上常用最小粒度阈值。例如每个任务至少处理数千或数万个元素，或者让每个任务目标运行几十微秒以上。这个阈值要测量，不同机器和任务不同。运行时可以自适应：开始使用较大块，发现长尾后切分剩余范围；或者过分割到 worker 数的若干倍，而不是无限细分。

13.28 错误传播和取消

并行算法不只处理成功路径。若某个 worker 在解析输入时发现错误，其他 worker 是否继续？已经写入的输出如何处理？partial 是否有效？线程池如何传播异常？取消后是否要等待正在运行的任务安全退出？这些问题决定算法能否进入真实系统。

一种简单策略是失败快速传播：共享一个原子错误标志，worker 在块边界检查；发现错误后停止提交新任务，等待已运行任务结束，然后丢弃输出。若输出写到外部文件，就必须写临时文件并在成功提交后重命名；失败时删除临时文件。分布式章节中的 attempt 提交协议，正是这个思想的跨机器版本。

13.29 源码阅读：并行 STL 和内核思想

C++ 标准库的并行算法接口展示了一个重要边界：用户提供操作，库决定执行策略。但并行 STL 的具体实现依赖标准库和后端，不能假设每个环境都真正多线程执行。阅读实现时，应关注执行策略如何切分范围、如何处理异常、如何限制任务粒度，以及哪些操作要求可结合或无副作用。

Linux 内核里也有类似思想，例如 per-cpu 计数、RCU 遍历和批量处理。它们不一定叫 reduce 或 scan，但思路一致：把高频写放到本地，低频合并；把读路径做轻，把更新延迟；把全局锁变成分层结构。源码阅读的目标不是照搬内核 API，而是识别这些并行结构在真实系统中的形态。

13.30 Segmented scan

普通 scan 在一个连续序列上计算前缀。Segmented scan 则把输入分成多个段，每段内部独立 scan。它适合按 key 分组后的前缀、稀疏矩阵行偏移、批量变长记录处理和图邻接范围处理。CSR 的 offsets 数组，本质上也和段边界有关。Segmented scan 让 scan 从“数组题”扩展到“结构化批数据”的位置分配工具。

实现 segmented scan 时，要处理段边界。若每个段很短，直接串行处理每段可能更快；若段很长，可以在段内并行；若段长度高度不均，长段会造成负载不均。可以先按段长度分类：短段批处理，长段拆任务。这个策略和图算法按顶点度分层类似。

13.31 Prefix sum 和内存分配器

内存分配器也会用到 scan 思想。假设每个 worker 需要输出不同数量的记录，先统计每个 worker 需要多少字节，对这些字节数做 prefix sum，就能一次性分配大缓冲，并给每个 worker 一个不重叠区间。这样比每个 worker 动态追加到共享 vector 更高效，也更容易恢复。

这种“先计数、再分配、再写入”的模式在本书会反复出现。Stream compaction 是元素级，partition 是 bucket 级，shuffle 文件是 partition 文件级，checkpoint manifest 是 block 级。理解 scan，就理解了许多无锁写输出的基础。

13.32 Radix partition 和 radix sort

Radix partition 按 key 的若干 bit 分桶。第一轮按低几位分桶，第二轮按更高几位分桶，最终可以得到 radix sort。它常用于整数 key、哈希分区和数据库 join。与比较排序不同，radix 方法依赖 key 表示和桶计数，核心步骤是 histogram、scan offsets、scatter。也就是说，它把本章的三个核心模式组合在一起。

Radix partition 的性能取决于每轮处理多少 bit。每轮 bit 多，桶数多，counts 矩阵大，cache 压力高；每轮 bit 少，轮数多，读写次数增加。选择 radix bits 是 cache、TLB、内存带宽和任务粒度的综合问题。教材不需要给固定答案，但要让读者知道这是可测参数。

13.33 并行 stable sort 的代价

排序经常被当作库函数，但并行排序有复杂成本。它需要划分、局部排序、合并或采样分区。稳定排序还要保持相等 key 的原始顺序，可能需要额外内存或更保守合并。若后续只需要按 bucket 分组，不一定需要完整排序；若后续需要 top-k，也不一定需要全排序。算法目标要匹配业务需求。

在 Compute Systems Engine 中，如果日志统计只需要每个 event 的计数，排序全部记录通常是过度；如果输出要按 event id 排序，最终对聚合结果排序即可，不必排序所有输入。这个例子提

醒读者：先问输出语义，再选内核。

13.34 并行哈希表的困难

并行哈希聚合看似简单：多个线程往哈希表里更新 key。困难在于冲突、扩容、内存分配和 cache line 争用。全局哈希表加锁简单但竞争大；每个 bucket 一把锁降低冲突但锁数量多；线程本地哈希表最后合并避免热路径共享，但内存占用高；无锁哈希表实现复杂，删除和扩容尤其难。

因此很多系统先做线程本地聚合，再合并。合并阶段可以按 key 分区，把不同 key 交给不同 worker，减少冲突。若 key 空间倾斜，热点 key 仍会造成长尾，需要特殊处理。并行哈希表不是一行 `unordered_map` 加 mutex 能解决的。

13.35 负载估计和采样

Partition 和 reduce 的负载均衡可以通过采样改善。先抽样估计 key 分布或记录成本，再决定分区边界或热点 key 列表。数据库查询优化、分布式排序和 shuffle 都常用采样。采样有成本，也可能不准，尤其是输入分布随时间变化或热点很稀有时。

采样结果应进入报告。若系统根据样本决定把 key 42 视为热点，就要记录样本大小、估计频率和实际频率。否则热点处理失败时无法判断是采样不准、实现错误还是输入变化。性能系统里的“智能策略”都需要可观测证据。

13.36 并行算法的可组合性

单个内核正确，不代表组合后正确。Filter 后的输出顺序会影响后续 stable partition；partition 后的 bucket 顺序会影响 deterministic reduce；top-k 的 tie-break 会影响最终输出；scan 的 exclusive/inclusive 选择会影响写偏移。组合系统要把每个内核的语义写成合同。

Compute Systems Engine 的 kernel suite 应为每个内核记录：是否保持输入顺序，是否确定性，是否允许误差，是否可能丢弃，是否改变 key 表示，是否需要 materialization。这样流水线组合时，语义冲突能早发现。

13.37 案例：错误行压缩

日志解析中，错误行通常占少数。我们希望把错误行的行号和 offset 写到错误列表，用于最后报告。朴素并行做法是每个 worker 发现错误就加锁 push 到全局 vector。更好的做法是 stream compaction。第一遍每个 worker 统计错误行数；scan 得到每个 worker 的错误输出偏移；第二遍写入错误行元数据。这样错误列表稳定、无锁、可复现。

如果错误率很低，两遍扫描似乎浪费。另一种折中是每个 worker 使用本地 vector 收集错误，最后按 worker 顺序合并。本地 vector 会动态分配，但错误少时成本可接受。选择两遍 compaction 还是本地 vector，取决于错误率、稳定顺序要求和内存分配成本。教材案例要展示这种权衡，而不是只有一种答案。

13.38 案例：输出 offset 分配

Shuffle 文件写入也需要 offset 分配。每个 map worker 对每个 reduce partition 产生不同大小的 block。若希望把同一 partition 的 blocks 写入一个连续文件，可以先统计每个 block 大小，对 sizes 做 prefix sum，得到每个 block 在文件中的 offset。然后每个 worker 可以写自己的区间。这个过程和数组 compaction 完全同构，只是单位从元素变成字节。

这种设计避免了多个 worker 共享文件 append offset 的锁竞争，也让 manifest 可以记录每个 block 的位置和大小。代价是需要先知道大小，可能要先编码到临时缓冲或先估算。若大小无法提前知道，可以先写每 worker 临时文件，最后合并。算法选择受 I/O 和内存约束影响。

13.39 案例：并行去重

去重可以用 sort 或 hash。Sort-based 去重先排序，再扫描相邻相等元素；hash-based 去重边读边插入集合。并行去重时，sort 路线可以分区排序再合并，hash 路线可以按 key partition 到不同 worker，每个 worker 独占一部分 key 空间。若直接让所有线程更新一个全局 set，锁竞争和分配会

很重。

去重语义也要明确。保留第一个出现，还是任意一个？输出是否保持原顺序？若保留第一个出现，并行 partition 会破坏顺序，需要额外位置比较。若只关心集合，算法可以更自由。很多算法题默认语义简单，系统工程必须把这些边界写清。

13.40 并行算法的内存上限

并行算法常用额外数组：partial、counts、offsets、selection、temporary output。线程数越多，partial 越多；bucket 越多，counts 矩阵越大；输入越大，selection 可能接近输入大小。内存上限必须进入设计。一个算法吞吐高但需要 10 倍输入内存，可能无法处理大数据。

报告中应列出额外空间复杂度和实际字节。例如 partition counts 是 `worker_count` 乘 `bucket_count` 乘 8 字节；selection vector 最坏是 `input_count` 乘 4 字节；per-worker hash map 取决于本地 unique key 数。写出这些数字，读者才能判断算法是否适合目标机器。

13.41 并行算法章节总结

Reduce、scan 和 partition 是并行数据处理的三根支柱。Reduce 消除共享写，scan 分配唯一输出位置，partition 把数据按后续处理需求重排。它们组合出 histogram、compaction、radix partition、shuffle 和分布式 reduce。掌握这些模式，比记住某个线程库 API 更重要。每个模式都要同时说明语义、内存上限、确定性和负载均衡。

13.42 并行算法决策表

设计并行算法时，可以先填决策表。输出大小是否已知？若已知，可以预分配并按区间写；若未知，先 count/scan 或使用 per-worker buffer。输出顺序是否稳定？若稳定，切分和合并必须保序；若不稳定，可以更自由重排。操作是否结合？若结合，可以树形合并；若不结合，需要固定顺序。key 是否倾斜？若倾斜，要考虑热点拆分。内存预算是否允许 per-worker state？若不允许，可能要分批处理。

这张表能把算法选择从“凭经验”变成“按约束推导”。同样是 filter，稳定输出和非稳定输出不同；同样是 histogram，稠密 key 和稀疏 key 不同；同样是 reduce，整数精确和浮点误差不同。并行算法的难点在约束，而不只是线程 API。

13.43 案例：分批 partition

当输入太大，counts 矩阵或输出缓冲无法一次容纳时，可以分批 partition。每批独立统计、scan、写临时 block，最后再合并 block。这样降低峰值内存，但增加多批调度和最终合并。分批处理还更容易 checkpoint，因为每个 batch 可以作为恢复单位。代价是全局稳定顺序更难，需要记录 batch 顺序和 bucket 内顺序。

分批 partition 是从单机算法走向分布式 shuffle 的中间形态。分布式系统本质上就是把超大输入拆成可调度、可恢复的批次。

13.44 案例：并行算法和容错

单机并行算法通常默认进程不崩溃，但大规模计算必须考虑容错。Reduce partial 可以重算，scan offsets 可以重算，partition block 可以根据 input split 重写。若每个阶段的输入输出边界清楚，失败后就能从最近边界恢复；若多个阶段完全融合，失败后只能从更早位置重做。算法分阶段不仅是性能问题，也是恢复问题。

这解释了为什么分布式系统喜欢 stage 边界和 materialization。它牺牲一些 I/O，换来可恢复性。单机高性能和分布式容错之间常有张力，系统设计要明确哪条路径更重要。

13.45 案例：确定性输出

并行算法常因为调度顺序不同产生不同输出顺序。若业务只关心集合，这可以接受；若输出文件要用于缓存、测试或审计，确定性很重要。确定性输出通常要求固定 partition 顺序、固定 worker block 顺序、固定 tie-break 和固定合并树。它可能牺牲一些性能，但能显著提高可测试性和可恢复

性。

日志统计最终输出可以按 event type 升序，这样无论 map task 如何调度，输出文件都稳定。Top-k 可以按 score 降序、key 升序打破平局。Manifest 可以按 task id 和 partition 排序。确定性是大型系统调试的朋友。

13.46 本章落到贯穿项目

Compute Systems Engine 在本章应增加三个模块。第一，parallel count-if：作为 reduce 的简单案例，展示全局原子反例和 partial 版本。第二，parallel scan：实现局部 scan、block sums scan、回填，并提供 inclusive/exclusive 两个接口或至少明确选择一个。第三，stream compaction：用 count、scan、write 三阶段把满足谓词的元素写到输出。

这三个模块会成为后续分布式 shuffle 的前置训练。count-if 训练局部统计，scan 训练偏移分配，compaction 训练无冲突写输出。分布式章节会把“写输出区间”升级成“写目标分区”，把“block sums”升级成“map 端摘要”。

13.47 从串行语义推导并行形态

并行算法不应从“开几个线程”开始，而应从串行语义开始。以 filter 为例，串行语义是按输入顺序检查每个元素，保留满足谓词的元素，并保持相对顺序。这个语义里有三个信息：谓词独立，输出数量未知，输出顺序稳定。谓词独立说明检查可以并行；输出数量未知说明不能让所有线程直接写同一个当前位置；顺序稳定说明每个元素的输出位置必须等于它之前满足谓词的元素数量。这样自然推出 count、scan、write 三阶段。

如果业务不要求稳定顺序，算法形态会改变。每个 worker 可以写本地 buffer，最后任意合并；甚至可以按 bucket 输出，不保留原顺序。这样减少 scan 成本，但输出语义变弱。若业务只需要集合，非稳定输出可以接受；若输出要用于测试、审计或和原记录对齐，稳定输出更重要。并行优化的第一步永远是明确哪些串行语义必须保留，哪些可以放松。

Reduce 也一样。串行求和有固定顺序；并行求和和改变合并树。整数加法在不溢出的数学模型下结合，结果一致；浮点加法不严格结合，结果可能变化；字符串拼接结合但顺序重要；带副作用的操作根本不能随意并行 reduce。教材讲 reduce 时不能只写公式，还要让读者检查操作是否结合、是否交换、是否有单位元、是否要求确定性顺序。

13.48 并行写输出的三种模式

并行算法最大的工程问题之一，是多个 worker 怎样写输出。第一种模式是预先切分固定区间，例如 map 每个输入元素写一个输出元素，worker 按输入区间写输出区间，没有冲突。第二种模式是 count/scan/write，例如 filter、partition、错误行压缩，先计算每个 worker 或每个 bucket 的输出数量，再分配不重叠区间。第三种模式是本地 buffer 后合并，适合输出数量未知但稳定性要求较弱，或者输出很稀疏的场景。

全局锁 append 是最直观但通常最差的模式。它把所有输出写入串行化，还会让输出顺序依赖调度。全局原子 `fetch_add` 分配位置比锁轻，但仍然有共享热点，而且稳定顺序不一定满足。若只需要任意顺序，它可能可用；若需要输入顺序，它不够。工程报告要说明为什么选择某种写出模式，而不是只说“避免锁”。

写输出还要考虑内存分配。预先分配需要知道输出上界或精确数量；本地 buffer 需要处理扩容；临时文件需要处理 I/O 和失败清理；mmap 输出需要处理页错误和持久化。单机算法讲清这些细节，后续分布式 shuffle 就更容易理解，因为 shuffle 只是把输出区间从内存数组扩展到磁盘文件和网络分区。

13.49 可复现性和测试 oracle

并行算法的测试 oracle 可以是串行 reference，但要先定义输出是否要求完全一致。稳定 filter、确定性 sort、按 key 排序的聚合，适合和串行结果逐元素比较。非稳定 partition、任意顺序去重、近似 top-k，可能只能比较集合、计数或误差范围。测试必须匹配语义，不能用过强 oracle 错杀合理实现，也不能用过弱 oracle 放过错误。

可复现性在大型系统里很有价值。输出顺序固定，失败重跑更容易比较；stage manifest 固定，

缓存命中更稳定；日志行号稳定，排查更容易。为了可复现性，系统可能选择固定合并顺序、固定 worker 分区、固定随机 seed 和固定 tie-break。这些选择会牺牲一点调度自由，但换来测试和运维上的确定性。教材应让读者知道，性能不是唯一目标。

当可复现性和吞吐冲突时，要按业务分类。离线训练数据生成、账务报表、审计日志通常更重视确定性；在线推荐候选召回可能更重视吞吐并接受顺序变化；科学计算可能重视数值误差可控。并行算法的“正确”不是抽象的，它来自业务语义和可验证性要求。

13.50 本章习题

习题与作业

习题一：为 parallel count-if 写完整正确性证明。证明每个输入元素被统计一次且只统计一次，partial 合并结果等于串行 reference。

习题二：设计 inclusive scan 和 exclusive scan 的 API。给出输入 [3,1,4] 的两个输出，并说明哪个更适合分配输出偏移。

习题三：为 stream compaction 写三阶段伪代码。要求说明每阶段读写哪些数组、是否并行、是否有共享写、需要哪些临时空间。

习题四：设计一个倾斜 key 的 partition 测试。要求解释为什么写入区间没有冲突，但下游 reduce 仍可能负载不均。

习题五：比较线性合并和树形合并。给出 partial 数为 8 时的合并步骤，并说明什么时候树形合并不值得。

实验：在 Compute Systems Labs 中扩展 parallel `count_if`。先让每个线程本地计数，再合并。随后实现输出压缩：先统计、扫描偏移，再写输出数组。

第 14 章

任务运行时与工作窃取

线程是执行资源，任务是工作单位。把线程和任务分离，是构建可扩展运行时的关键。线程池、任务队列、工作窃取和任务图，是现代并行运行时的核心结构。

14.1 线程池

线程池创建固定数量或可控数量的工作线程，任务提交到队列，由工作线程取出执行。这样避免频繁创建线程，也便于控制并发度。线程池接口看似简单，难点在关闭、异常传播、任务等待、队列阻塞和避免死锁。

如果线程池里的任务再等待同一个线程池中的子任务，而工作线程数量不足，就可能发生饥饿。运行时设计必须考虑任务依赖，而不只是一个全局队列。

线程池的最小语义包括三件事：提交任务是否可能失败，关闭后队列中任务是否继续执行，调用 `wait` 或析构时是否等待所有任务完成。没有这些语义，线程池只是一个会运行函数的对象，不是可靠运行时。教材实现应先把这些边界写清，再谈性能。

14.2 Future 和等待依赖

Future 把“将来会完成的结果”变成对象。它适合表达异步任务，但如果 worker 线程在执行任务时同步等待另一个 future，就可能形成线程池内阻塞。更好的方式是依赖表达给运行时，让运行时在依赖满足后再调度后续任务，而不是占着 worker 等。

这也是任务图运行时的价值：等待不再消耗工作线程，依赖边决定任务何时 ready。构建系统和数据流引擎都大量依赖这种模型。

Future 的危险是把异步伪装成同步。用户看到 `future.get()` 很方便，但如果在 worker 线程里调用它，就可能把 worker 变成等待者。运行时可以通过 work helping 缓解：等待时主动执行其他 ready 任务；也可以要求任务不能阻塞等待，只能注册 continuation。两种设计都要明确写进接口语义。

14.3 任务粒度

任务太大，负载不均；任务太小，调度成本高。合理粒度通常需要根据输入规模、核心数、缓存块和调度开销选择。递归拆分算法常设 cutoff，小于某个规模就顺序执行。

任务粒度可以通过一个简单不等式判断：每个任务的计算时间应显著大于一次提交、入队、出队和唤醒的固定成本。若任务只做几十条指令，调度成本会吞掉收益；若任务运行几百毫秒，负载不均会造成长尾。实践中常把任务切到缓存友好的块大小，再用 benchmark 调整 cutoff。

14.4 工作窃取

工作窃取让每个线程拥有本地双端队列。线程优先从本地取任务，空闲时从其他线程窃取任务。这减少全局队列争用，并改善缓存局部性。常见策略是 owner 从底部 push/pop, stealer 从顶部 steal。

工作窃取的正确性和性能都依赖队列设计。Chase-Lev deque 是经典方案，但实现复杂，涉及原子、内存序和边界竞争。教学时先理解模型，再进入源码。

工作窃取还有一个重要性质：正常情况下 worker 访问自己的本地 deque，只有空闲时才访问别人的 deque。这把高频操作留在本地，低频窃取才跨线程同步。相比所有 worker 抢一个全局队列，它减少了共享热点。这个结构和 NUMA、分布式本地优先原则一致：先消费近处工作，再去远处拿工作。

14.5 本地性和窃取方向

工作窃取通常让 owner 按 LIFO 处理本地任务，让 stealer 从另一端 FIFO 窃取较老任务。LIFO 有利于缓存局部性，因为新产生的子任务往往访问相近数据；窃取较老任务则倾向拿到较大工作块，减少窃取频率。这个设计体现了运行时对局部性和负载均衡的折中。

14.6 任务图

很多计算不是简单队列，而是任务之间有依赖。任务图把节点表示任务，边表示依赖。运行时在依赖满足后调度任务。构建系统、数据处理引擎和图计算框架都大量使用任务图思想。

任务图需要维护 ready 计数。每个任务记录还有多少前驱未完成；当前驱完成时，后继计数减一；计数变为零，任务进入 ready 队列。这个模型避免 worker 阻塞等待依赖。它的难点在于任务失败、取消、异常传播和图生命周期：一个任务失败后，后继任务是跳过、取消还是继续运行，必须在运行时语义里说明。

14.7 取消和关闭

运行时必须定义关闭语义。提交中的任务是否接受，队列里的任务是否执行完，正在运行的任务是否可取消，异常如何传播，这些都要明确。没有关闭语义的线程池很容易在程序退出时丢任务、死锁或访问已销毁对象。

14.8 运行时观测指标

任务运行时不能只测总时间。还要观察队列长度、每个 worker 执行任务数、空闲时间、窃取次数、任务平均耗时、最长任务、等待依赖数量和关闭耗时。没有这些指标，负载不均和调度开销很难定位。一个任务图慢，可能是关键路径太长，不是 worker 不够；一个线程池慢，可能是任务太小，不是锁太慢。

这些指标也会指导分布式计算。单机里 worker 空闲，对应集群里 executor 空闲；单机里队列积压，对应集群里 stage backlog；单机里任务倾斜，对应分布式里的热点 key。运行时思维是从多线程走向分布式框架的桥。

14.9 最小线程池的状态机

线程池最小实现通常包含任务队列、worker 列表、停止标志、未完成任务计数和条件变量。状态机比代码更重要。open 状态允许 submit；shutting down 状态拒绝新任务但继续执行队列中任务；stopped 状态表示 worker 已退出。若没有这些状态，析构、等待和异常路径会变得模糊。

Listing 14.1 线程池状态的概念模型

```
1 enum class RuntimeState : std::int32_t {
2     Open,
3     ShuttingDown,
4     Stopped,
5 };
6
7 struct RuntimeCounters {
8     std::uint64_t submitted = 0;
9     std::uint64_t completed = 0;
10    std::uint64_t failed = 0;
11 };
```

submit 在 Open 状态下把任务加入队列并通知 worker；在 ShuttingDown 或 Stopped 状态下应失败或抛出明确异常。**shutdown** 把状态从 Open 改成 ShuttingDown，并唤醒所有 worker；worker 在队列为空且状态不是 Open 时退出；最后一个 worker 退出后，线程池进入 Stopped。这个流程必须能重复调用而不破坏状态，即 shutdown 应尽量幂等。

未完成任务计数用于 **wait**。提交任务时增加，任务完成或失败时减少，计数变为零时通知等待

者。异常不能让计数泄漏，否则 `wait` 会永远阻塞。生产级线程池还需要保存异常并传给 `future` 或错误回调；教学实现至少要保证异常不会让 `worker` 线程静默终止后破坏计数。

14.10 任务对象和生命周期

任务不是单纯的 `std::function<void()>`。它可能携带输入范围、优先级、placement、依赖计数、取消标志、异常存储和指标字段。使用 `std::function` 便于教学，但可能分配内存、类型擦除成本较高，也不方便表达任务元数据。高性能运行时通常会使用自定义 `task object`、对象池或 intrusive queue。

任务生命周期至少有五个阶段：构造、提交、ready、running、completed。任务图还会有 waiting 状态，因为依赖未满足。取消和失败会引入 canceled、failed 等状态。每个状态允许哪些操作必须定义清楚。例如 running 任务是否能被取消？completed 任务的结果保存多久？shutdown 后 waiting 任务怎么处理？这些语义比 worker 循环更难。

14.11 全局队列线程池的 baseline

全局队列线程池是最好的 baseline。它容易实现，容易证明关闭语义，适合作为 correctness reference。它的缺点也清楚：所有 worker 竞争同一个队列锁；任务入队出队集中；任务局部性弱；负载高时队列可能成为瓶颈。正因为缺点明确，它才适合作为后续 work stealing 的对照。

教材实现应先写全局队列版本，并记录指标：每个 worker 执行任务数、队列最大长度、worker 空转次数、submit 等待时间。若这些指标显示全局队列不是瓶颈，就不应急着实现复杂 work stealing。若 worker 大量争用队列锁、队列长度波动大、任务很小且频繁，才有证据推进到本地队列。

14.12 工作窃取 deque 的所有权

工作窃取 deque 的核心是所有权不对称。owner 线程高频 push/pop 本地端，stealer 低频从另一端 steal。这个不对称让常见路径更轻，但边界竞争很复杂，尤其是 deque 里只剩一个任务时，owner pop 和 stealer steal 可能竞争同一个元素。Chase-Lev 算法之所以复杂，正是为了解决这些边界和内存序。

学习时可以先用 mutex 保护每个 worker deque，实现工作窃取策略，再用测量判断策略收益。等策略正确后，再研究无锁 deque。这样分层能避免把“调度策略是否有效”和“无锁 deque 是否正确”混在一起。运行时工程最怕同时引入新策略和新同步算法，出了问题很难定位。

14.13 关键路径和并行度

任务图的性能由两个量共同决定：总工作量和关键路径长度。总工作量是所有任务耗时之和，关键路径是依赖图中最长路径。线程数再多，也不能把关键路径压短到依赖允许之外。一个任务图如果关键路径很长，增加 worker 只会让更多线程空闲；如果总工作量大且依赖宽，worker 才有足够 ready 任务可执行。

因此任务图优化不只是让 worker 更多，还要改变依赖结构。把串行合并改成树形合并，可以缩短关键路径；把过大的任务拆分，可以增加并行度；把过小任务合并，可以降低调度开销；把热点 key 单独处理，可以减少长尾。任务图分析把算法结构和运行时调度连接起来。

14.14 Continuation 代替阻塞等待

阻塞等待会占住 worker，continuation 则把“前置任务完成后做什么”注册给运行时。前置任务完成时，运行时把后续任务加入 ready 队列。这样等待不占线程，依赖由计数器和队列表达。异步 I/O、构建系统和数据流引擎都大量使用 continuation 思想。

Continuation 的代价是控制流不如同步代码直观，异常传播和取消更复杂。结构化并发试图把异步任务组织成有作用域的树，让父任务能等待子任务完成并统一处理取消。第二册后续不需要完整实现现代 coroutine runtime，但要让读者理解：高性能运行时的很多复杂性来自“不要让线程闲等，但还要保持语义清楚”。

14.15 背压进入运行时

任务队列不能无限增长。无限队列把生产速度和消费速度的差异藏进内存，最终可能造成延迟爆炸或 OOM。有界队列把容量作为系统契约：生产者太快时必须等待、失败、丢弃、降级或转移。背压不是性能失败，而是系统保护机制。

线程池 `submit` 如果使用有界队列，应定义队列满时语义。阻塞 `submit` 会把背压传给调用者；`try submit` 返回 `false` 让调用者决定；带超时 `submit` 可以避免永久等待；丢弃策略适合可丢指标，不适合必须执行的任务。每种策略都影响上层业务。运行时不能只说“队列满了就等”，要说明谁等、等多久、能否取消。

14.16 运行时指标的低开销实现

指标本身不能成为瓶颈。每个任务完成时更新全局原子计数器，在高频小任务场景下可能形成共享写热点。更好的做法是每个 worker 写自己的 `WorkerStats`，周期性汇总。队列长度可以在队列锁内更新最大值，也可以采样。窃取次数由 `stealer` 本地记录，最后合并。指标精度和成本要匹配用途。

若指标用于调度决策，例如根据队列长度做负载均衡，就需要更实时、更一致；若指标只是实验报告，可以延迟汇总。不要把观测性和控制路径混为一谈。实验指标可以近似，协议状态不能近似。

14.17 本章落到贯穿项目

Compute Systems Engine 的运行时路线应分四步。第一步，全局队列线程池：实现 `submit`、`wait`、`shutdown`，队列用 `mutex` 和条件变量，保证关闭不丢任务。第二步，任务指标：记录每个 worker 执行任务数、空转次数、队列最大长度和任务耗时摘要。第三步，任务图：实现 `ready count`，支持归约中的树形合并和并行扫描。第四步，工作窃取：先用 `mutex deque` 验证调度策略，再决定是否实现无锁 `deque`。

每一步都要保留前一步作为对照。全局队列版本是 `work stealing` 的 `reference`；顺序执行是任务图的 `reference`；固定任务粒度是动态调度的 `reference`。没有 `reference`，运行时一旦出错就很难判断是调度策略、同步结构还是任务语义的问题。

14.18 本章习题

习题与作业

习题一：为一个最小线程池写状态机。状态至少包括 `Open`、`ShuttingDown`、`Stopped`。列出每个状态下 `submit`、`wait`、`shutdown` 和 `worker loop` 的行为。

习题二：设计线程池指标结构，要求避免所有 worker 高频写同一个 `cache line`。说明哪些指标需要精确，哪些可以近似或延迟汇总。

习题三：把并行归约的最后合并从线性合并改成树形合并。画出任务图，计算关键路径长度如何变化，说明任务数增加带来的调度成本。

习题四：设计一个工作窃取实验。先用 `mutex deque` 实现策略，不使用无锁 `deque`。比较全局队列和本地队列在任务粒度不同、负载不均不同情况下的 worker 空闲时间和窃取次数。

14.19 线程池接口的最小完整性

一个线程池要进入项目，至少要有四类接口：提交任务、等待完成、关闭运行时、查询指标。只有 `submit` 而没有 `wait`，调用者不知道任务何时全部结束；只有 `shutdown` 而没有明确拒绝新任务语义，析构时可能丢任务；没有指标，性能问题无法定位。接口越小越好，但不能缺少生命周期。

Listing 14.2 线程池接口的教学形态

```
1 class ThreadPool {
2 public:
3     explicit ThreadPool(std::int32_t worker_count);
```

```

4     ~ThreadPool();
5
6     bool submit(std::function<void()> task);
7     void wait_idle();
8     void shutdown();
9     [[nodiscard]] RuntimeCounters counters() const;
10
11 private:
12     void worker_loop(std::int32_t worker_id);
13 };

```

这只是接口草图，不是最终性能实现。它暴露了几个必须讨论的问题：`submit` 返回 `false` 表示线程池关闭；`wait_idle` 等待已提交任务完成但不关闭线程池；`shutdown` 拒绝新任务并让 worker 退出；析构必须安全调用 `shutdown`。若 `task` 抛异常，worker 不能直接终止而让未完成计数泄漏，至少要捕获并记录 `failed`。

14.20 未完成计数和条件变量

线程池 `wait_idle` 常用未完成任务计数。提交成功时计数加一，任务执行完毕后计数减一，计数为零时通知等待者。这个计数必须和任务队列状态一起受锁保护，或者用原子和条件变量 *carefully* 组合。若任务抛异常、被取消或线程池关闭时漏减，等待者会永远阻塞。

等待谓词可以写成“未完成任务数为零”。这里要注意：队列为空不等于任务完成，因为 worker 可能已经取出任务正在执行；worker 空闲也不等于没有任务，因为提交者可能正在入队。未完成计数把 `queued` 和 `running` 任务都算进去，才适合 `wait`。

14.21 异常传播策略

线程池任务的异常不能悄悄消失。教学版可以记录失败次数，并把异常转成日志；`future` 版应把异常保存在 `future` 中，让调用者 `get` 时重新抛出；任务图版要决定一个节点失败后后继节点是取消、跳过还是继续。异常策略是运行时语义的一部分。

如果项目暂时不实现 `future`，可以要求提交的任务内部处理异常，线程池只捕获所有逃逸异常并记录 `failed`。这个策略简单，但调用者无法知道哪个任务失败。后续如果要支持可靠计算，必须引入 `task id`、结果对象或错误回调。

14.22 任务图 `ready count` 实现

任务图的核心是 `ready count`。每个节点记录还未完成的前驱数量。初始为零的节点进入 `ready` 队列；一个节点完成后，遍历后继，把它们的 `ready count` 减一；某个后继减到零时，进入 `ready` 队列。这个模型把等待变成状态变化，而不是让 worker 阻塞。

Listing 14.3 任务图节点的核心字段

```

1 struct TaskNode {
2     std::uint32_t id = 0;
3     std::atomic<std::int32_t> remaining_dependencies{0};
4     std::vector<std::uint32_t> successors;
5 };

```

这里 `successors` 是 `vector`，适合教学。大型运行时会更关心内存布局，可能用连续边数组压缩图。`ready count` 必须能处理并发完成：多个前驱可能同时减少同一个后继计数，因此通常需要原子。减到零的那一次线程负责把任务入队。

14.23 任务图的生命周期

任务图不能在还有任务运行时被销毁。最简单方式是图对象由运行时持有，直到所有节点完成或取消。若任务捕获图中数据引用，生命周期更要清楚。C++ 中常见错误是提交 lambda 捕获局部引用，函数返回后 worker 才执行，导致悬垂引用。教学代码应优先捕获值或使用共享状态对象，并明确所有权。

任务图还要处理失败。如果一个前驱失败，后继是否仍然执行？对于构建系统，依赖失败通常取消后继；对于日志统计，某个 map task 失败可以重试，后继 reduce 等待最终成功 attempt；对于 best-effort 指标，失败可能被忽略。运行时不应替业务猜语义，但必须提供表达方式。

14.24 工作窃取和 NUMA

普通工作窃取从任意 victim 偷任务，NUMA 感知工作窃取会先在本节点内偷，再跨节点偷。这样可以优先保持 cache 和内存本地性。跨节点 steal 不是禁止，而是作为负载不均时的后备。这个策略和分布式数据本地调度一致：先本地，后远程。

实验可以构造两类任务。第一类任务只做 CPU 计算，数据很小，跨节点窃取影响小；第二类任务访问大数据块，数据按 NUMA 节点 first-touch 放置，跨节点窃取会访问远端内存。比较两类任务能说明“负载均衡”和“数据本地性”之间的张力。

14.25 任务粒度自动调节

固定 cutoff 很难适应所有机器。运行时可以记录任务平均耗时、队列长度和 worker 空闲时间，动态调整拆分粒度。若任务太短且队列争用高，就增大 chunk；若 worker 空闲多且长尾明显，就减小 chunk 或启用 work stealing。自动调节要谨慎，避免频繁震荡。

教学项目可以先不做自动调节，但 benchmark 应输出足够指标，让读者手动选择 cutoff。最终目标不是让运行时神秘地“自动变快”，而是让调整有证据。

14.26 本章项目验收清单

线程池进入 Compute Systems Engine 前，至少要通过这些验收：提交 N 个任务全部执行一次；任务抛异常不会杀死 worker；wait_idle 在任务完成后返回；shutdown 后 submit 失败；析构不会遗留 joinable thread；多线程提交不会破坏队列；指标能统计 submitted、completed、failed。任务图进入项目前，还要测试依赖顺序、并发前驱、失败取消和空图。

性能验收则包括：全局队列在小任务下的队列锁争用，任务粒度曲线，每个 worker 执行任务数分布，worker 空闲时间和关闭耗时。没有这些数字，工作窃取是否必要无从判断。

14.27 本章扩展习题

习题与作业

- 习题五：为线程池设计异常策略。比较“吞掉异常只计数”、“future 保存异常”、“错误回调”三种方案的语义和适用场景。
- 习题六：设计任务图内存布局。比较每个节点一个 vector successors 和统一边数组两种方案的局部性、分配次数和修改难度。
- 习题七：构造一个 worker 捕获局部引用导致悬垂的例子，并改成安全的所有权模型。说明为什么并发代码里 lambda 捕获尤其危险。
- 习题八：设计 NUMA 感知 work stealing 策略。写出本地偷取、跨节点偷取、victim 选择和指标记录方案。

14.28 Future 等待的死锁风险

线程池里最危险的接口之一是 worker 内部等待 future。假设线程池只有四个 worker，四个任务都在运行，并且每个任务都等待同一个池中尚未开始的子任务。队列里有子任务，但没有空闲 worker 执行它们，系统就死锁。这个问题不是 future 本身的错，而是阻塞等待和固定 worker 数组合后的结果。

解决思路有几种。第一，禁止 worker 内部阻塞等待同池任务，改用 continuation。第二，允许 worker 等待时帮助执行队列里的其他任务，类似 work-first 或 help-first 调度。第三，把可能阻塞的任务放到独立线程池，避免占满计算 worker。第四，使用结构化并发，让父任务的等待由运行时管理，而不是任意阻塞线程。哪种策略合适，取决于运行时复杂度和业务语义。

```
pool deadlock shape:
worker 0 runs parent A and waits for child A1
worker 1 runs parent B and waits for child B1
worker 2 runs parent C and waits for child C1
worker 3 runs parent D and waits for child D1
child tasks are queued but no worker is free
```

教材要让读者形成一个原则：线程是执行资源，不要轻易拿执行资源去等待同一资源池里的未来工作。等待可以发生，但必须被运行时理解，或者发生在不会饿死被等待任务的地方。

14.29 Work-first 和 help-first

工作窃取运行时常讨论两种调度风格。Work-first 倾向于让当前 worker 继续执行新生成的子任务，把 continuation 留给别人偷；help-first 倾向于把子任务放入队列，当前 worker 继续执行 continuation。二者会影响 cache 局部性、栈深度、窃取频率和关键路径执行。经典 Cilk 风格偏 work-first，因为它让本地深度优先执行，减少调度开销，窃取者拿到较大的 continuation。

教学项目不必完整实现这些理论，但要理解任务提交顺序不是无关紧要。若每个任务递归拆分两个子任务，然后当前线程等待两个子任务，容易制造过多任务和等待。更好的方式是当前线程直接处理其中一部分，把另一部分提交给运行时，最后合并结果。这样能减少任务数，并保持本地性。

Listing 14.4 分治任务中保留一半本地执行

```
1 std::int64_t reduce_range(std::span<const std::int32_t> values,
2                           ThreadPool& pool,
3                           std::size_t cutoff) {
4     if (values.size() <= cutoff) {
5         return sequential_sum(values);
6     }
7
8     const std::size_t mid = values.size() / 2;
9     std::future<std::int64_t> right =
10         pool.submit_value([right_values = values.subspan(mid)]() {
11             return sequential_sum(right_values);
12         });
13     const std::int64_t left = reduce_range(values.first(mid), pool, cutoff);
14     return left + right.get();
15 }
```

这段代码展示思想，但也暴露等待风险：如果 submit_value 把任务放进同一个固定池，get 可能阻塞 worker。生产实现应让等待者帮助执行，或者使用任务图 continuation。教材写代码时要明确这是讨论调度形态，不是最终线程池 API。

14.30 任务优先级和饥饿

运行时常需要优先级：用户请求比后台压缩重要，控制面任务比数据面批处理重要，提交协议比指标刷新重要。优先级队列能让高优先级任务先执行，但也可能让低优先级任务长期饥饿。若低优先级任务负责释放资源或推进 checkpoint，饥饿会反过来影响高优先级任务。

一种常见策略是多队列加配额。高优先级队列可以获得更多调度机会，但运行时周期性执行低优先级任务。另一种策略是 aging：任务等待越久，优先级逐渐提高。还有一种策略是资源隔离：不

同优先级使用不同线程池或不同队列容量，避免后台任务占满前台资源。优先级不是简单排序，而是调度公平性和业务目标的表达。

14.31 阻塞任务和计算任务分池

计算线程池适合 CPU-bound 任务，阻塞 I/O 任务适合异步 I/O 或独立阻塞池。把大量阻塞任务提交到计算池，会让 worker 被占住，CPU 计算任务无处执行。反过来，把 CPU 密集任务提交到 I/O completion 线程，也会延迟完成回调，破坏异步系统。运行时设计应区分任务类型。

Compute Systems Engine 可以定义两个执行域。Compute domain 运行归约、scan、partition、hash 聚合等 CPU 任务；I/O domain 负责文件读取、网络模拟和完成回调。两者通过有界队列连接，I/O 太快时对 compute 背压，compute 太慢时 I/O 暂停读取。第 15 章会继续展开这个管线。

14.32 本地队列的缓存行为

本地队列不仅减少锁竞争，也改善缓存行为。worker 刚生成的子任务通常访问相邻数据或共享上下文，放入本地队列后由同一 worker 继续执行，cache 命中更高。窃取者从另一端偷较老任务，通常粒度更大，减少窃取频率。这个设计把“常见路径本地执行，少见路径跨 worker 平衡”写进数据结构。

但本地队列也会让负载短时间不均。某个 worker 本地队列很长，其他 worker 空闲，需要窃取策略及时发现。victim 选择可以随机，可以轮询，也可以优先同 NUMA 节点。随机选择实现简单，避免所有窃取者同时冲向同一 victim；拓扑感知选择更复杂，但对大机器更好。实验要记录 steal attempts、successful steals 和 empty steals。

14.33 任务对象池和运行时局部性

任务对象频繁分配会成为隐藏成本。每次提交 `std::function` 可能分配闭包对象，任务图节点可能分散在堆上，worker 访问队列时不断追指针。运行时可以使用任务对象池或 arena，把同一批任务连续存放。这样既减少分配器开销，也改善遍历和调试。

对象池必须和任务生命周期一致。任务完成后，结果是否还被 future 持有？异常对象是否还要读取？指标是否还要汇总？如果任务对象一完成就回收到池里，而 future 还保存指向任务的指针，就会悬垂。高性能运行时常把任务执行状态和用户可见结果分离：任务对象可以复用，结果对象由 future 或 promise 管理。

14.34 运行时的内存屏障位置

线程池初版用 mutex 和 condition variable，内存可见性由锁提供。工作窃取和无锁 ready 队列会暴露内存序问题。任务生产者必须先写完任务对象，再把它发布到队列；worker 从队列取到任务后，必须看到任务字段。这个发布关系通常由锁、release/acquire 或队列内部协议提供。不要在队列外再用一个普通布尔标志表示任务 ready。

任务完成也需要发布。若 worker 写结果后把状态设为 completed，等待者读取 completed 后要看到结果字段。future/promise 已经封装这些同步；自研结果对象就必须自己建立 happens-before。运行时不是只调度函数，它还负责结果可见性。

14.35 取消传播

取消不是强杀线程。安全取消通常是协作式：运行时设置取消标志，任务在边界检查并尽快退出。边界可以是处理完一个块、完成一次 I/O、写完一个输出批次。取消标志可以是原子布尔，但如果它还发布错误对象或原因，就需要更强同步或锁。

任务图取消要传播给后继节点。若某个节点失败，依赖它的节点可能永远不 ready，因此运行时要把它们标记为 canceled，并减少全图未完成计数。取消还要处理已经在队列中但未执行的任务。最简单策略是 worker 取出任务后检查状态，若 canceled 就跳过并完成计数。更复杂策略是从队列中删除任务，但删除本身可能昂贵。

14.36 运行时和调试可视化

任务运行时适合可视化。一个时间线图可以显示每个 worker 何时执行任务、何时空闲、何时窃取、何时等待 I/O。一个任务图可以显示关键路径和长尾任务。一个队列图可以显示队列长度随时间变化。没有图，运行时问题很难凭日志理解。

即使不做完整 GUI，也可以输出 CSV 或 JSON trace。字段包括 timestamp、worker id、task id、event type、queue size、steal victim、duration。后续可以用脚本画图。教学项目中的 trace 不需要高性能到生产级，但要让读者学会从执行历史看调度问题。

14.37 运行时源码阅读入口

源码阅读可以从三个成熟系统切入。第一，Cilk 或 oneTBB 的任务调度，关注工作窃取和任务粒度。第二，LLVM/MLIR 或编译器构建系统里的任务图，关注依赖和 worklist。第三，Linux scheduler 的 CFS 和 NUMA balancing，关注线程不是由用户态运行时独占控制，操作系统调度会影响 worker。阅读时不要试图一次读完全部源码，而是围绕一个问题：为什么 worker 空闲，为什么任务迁移，为什么等待时间高。

把用户态运行时和内核调度联系起来很重要。用户态线程池以为自己有 N 个 worker，但操作系统可能把它们迁移到不同 CPU，可能和其他进程共享核心，可能因为 I/O 睡眠唤醒改变亲和性。性能报告应记录线程绑定、CPU 利用和上下文切换，否则运行时结论可能不稳定。

14.38 结构化并发

结构化并发的核心是让任务生命周期有作用域。父作用域启动子任务，离开作用域前必须等待子任务完成或取消。这样资源不会在调用栈之外漂浮，取消和错误能沿任务树传播。相比随意提交 detached task，结构化并发更容易保证关闭和异常安全。

在线程池里，可以用 task group 表达这个思想。Task group 记录未完成子任务数、取消标志和错误状态。提交到 group 的任务完成后减少计数；某个任务失败后设置错误并请求取消；调用者等待 group 结束。这样 wait 的对象不再是整个线程池，而是一个有边界的任务集合。

结构化并发也有成本。任务必须属于某个作用域，不能随意长期存在；后台服务型任务需要单独生命周期管理；跨作用域依赖要谨慎。它不是所有场景的万能答案，但对批处理计算、并行算法和测试非常有帮助。

14.39 Work helping

Work helping 用于避免 worker 阻塞等待时浪费执行资源。若 worker 等待某个 task group 完成，它可以继续从队列取任务执行，帮助推进系统，而不是睡眠。这样能缓解父任务等待子任务导致的线程池耗尽。Work helping 要小心重入和栈增长：等待路径执行了其他任务，其他任务可能又等待更多任务。

一种简单策略是只允许在等待同一 task group 时帮助执行该 group 的 ready 任务，避免任意重入。另一种策略是全局帮助，吞吐更好但控制流更复杂。教学项目可以先禁止 worker 内等待，再在运行时章节作为扩展设计 work helping。关键是接口必须写清：wait 在 worker 内调用时会做什么。

14.40 任务图内存布局

任务图节点如果每个都单独分配，执行时会产生大量指针追踪和分配器开销。更高效的布局是把节点数组和边数组连续存放。每个节点记录 successor 起始偏移和数量，successors 全部放在一个 `std::vector<std::uint32_t>` 中。这类似 CSR 图结构。任务图本身就是有向图，完全可以借鉴图布局。

Listing 14.5 任务图的 CSR 式布局

```
1 struct TaskGraphNode {
2     std::uint32_t first_successor = 0;
3     std::uint32_t successor_count = 0;
4     std::atomic<std::int32_t> remaining_dependencies{0};
```



```

5 };
6
7 struct TaskGraphStorage {
8     std::vector<TaskGraphNode> nodes;
9     std::vector<std::uint32_t> successors;
10 };

```

这个布局适合构图后执行多次或执行一次但节点很多的场景。若任务图动态变化频繁，连续边数组修改成本高，可能需要增量结构。再次说明：数据布局选择要看生命周期和修改模式。

14.41 任务窃取的粒度

窃取太频繁会增加跨 worker 同步和 cache 迁移；窃取太少会负载不均。一个常见策略是偷较大的任务或半个队列，而不是每次偷一个极小任务。分治算法中，窃取较老的 continuation 往往能拿到更大工作块；本地 worker 继续处理新生成的小任务，保持 cache 局部性。

实验要记录 empty steals。空窃取多，说明 victim 选择或粒度不合适；成功窃取少但 worker 空闲多，说明任务不足或本地队列不暴露；成功窃取多但性能差，可能窃取成本太高或破坏局部性。Work stealing 的好坏不能只看总时间。

14.42 任务取消的检查点

协作式取消需要任务在安全点检查取消标志。安全点通常是块边界、循环批次边界、I/O 完成边界或输出提交前。不能在任意指令强行取消 C++ 任务，否则可能破坏对象不变量、锁状态和资源释放。取消检查太频繁会增加开销，太少会延迟响应。

长任务应拆成可取消块。例如解析大文件时，每处理一批记录检查一次取消；归约大数组时，每处理一个 chunk 检查；shuffle 发送时，每个 block 边界检查。取消后要定义已经产生的 partial 是否丢弃、是否提交、是否需要清理。取消不是简单设置 bool。

14.43 运行时错误分类

任务运行时的错误至少分为用户任务异常、运行时内部错误、取消、资源不足和关闭拒绝。用户任务异常应归还给提交者或 task group；内部错误可能需要停止运行时；取消是预期控制流；资源不足可能触发背压；关闭拒绝说明提交太晚。把所有错误都记成 failed，会让上层无法做正确处理。

错误分类也影响指标。failed tasks、canceled tasks、rejected submissions、dropped low-priority tasks 应分开计数。否则看见失败数上升，不知道是业务 bug、取消策略还是过载保护。

14.44 运行时 trace 格式

Trace 事件应简单稳定。可以包含 event type、timestamp、worker id、task id、queue id、duration、extra value。事件类型包括 submit、start、finish、fail、cancel、steal_attempt、steal_success、wait_begin、wait_end、queue_full。输出 CSV 足够教学使用，后续可转 JSON。

Trace 采样也要考虑成本。每个任务都记录详细事件适合调试小规模；大规模时可以采样或只记录慢任务。运行时应允许关闭 trace，只保留轻量 counters。观测性的开销要被测量，不能默认忽略。

14.45 运行时和分布式 driver 的对应

线程池运行时和分布式 driver 结构相似。线程池调度本地任务，driver 调度远端 task attempt；线程池 ready queue 对应分布式 pending task 队列；worker stats 对应节点指标；task group wait 对应 stage completion；异常传播对应 task failure；work stealing 对应任务重新分配。理解本地运行时后，分布式调度不再是全新概念，而是把执行实体从线程换成节点，把队列从内存结构换成 RPC 和持久化元数据。

这种对应关系能帮助项目设计。Compute Systems Engine 可以先在本机线程池里实现 task id、attempt、状态和指标，再把同样字段用于分布式模拟。字段稳定，尺度变化，系统更容易演进。

14.46 运行时章节总结

运行时把算法的并行性变成实际执行。它需要队列、worker、任务状态、依赖、等待、取消、异常、指标和关闭协议。工作窃取解决负载不均，但也带来本地性和同步问题；任务图解决依赖等待，但也带来生命周期和错误传播问题。一个好的运行时不是只会跑函数，而是能解释任务从提交到完成的每个状态。

14.47 运行时决策表

选择运行时结构时，可以先判断任务关系。任务独立且均匀，固定线程池加静态切分足够；任务独立但不均匀，需要动态队列或 work stealing；任务有依赖，需要任务图；任务会阻塞 I/O，需要分离 I/O 池或异步；任务需要优先级，需要多队列或调度策略；任务需要取消，需要 task group 和取消检查点。不要一开始就上最复杂运行时。

运行时还要和算法协同。算法切分太细，运行时开销大；算法切分太粗，work stealing 也救不了长尾；算法隐藏依赖，任务图无法调度；算法把大对象复制进 task，队列会变慢。运行时不是算法外部的黑盒，而是算法实现的一部分。

14.48 案例：慢任务隔离

如果某些任务可能非常慢，例如解析超长行、处理热点 key 或写慢磁盘，把它们和普通短任务放同一个 FIFO 队列，可能拖慢整体。运行时可以检测任务耗时，把慢任务标记出来，后续单独调度或拆分。也可以为不同任务类型设置队列，避免慢任务占满所有 worker。

慢任务隔离需要指标支持。没有 task duration 和类型标签，运行时无法知道谁慢。这个案例再次说明观测性是调度策略的前提。

14.49 案例：Speculation 的本地版本

分布式系统有 speculative execution，本地运行时也可以有类似思想。若某个任务长时间未完成，运行时可以把剩余可拆分范围重新切成更小任务，让其他 worker 帮忙。但这只适用于可拆分、幂等、输出可去重的任务。普通函数任务不能随便复制执行，否则副作用会重复。

这个案例连接运行时和分布式 attempt。Speculation 的前提永远是：重复执行不会产生重复外部效果，或者提交协议能去重。没有这个前提，speculation 是错误放大器。

实验：设计一个线程池接口，列出 submit、shutdown、wait 的语义。不要急着实现完整工作窃取，先实现固定线程数和全局队列，并写出关闭时不丢任务的测试。

第 15 章

I/O、异步与背压

计算系统不只做 CPU 运算。磁盘、网络、管道、日志和用户请求都会引入 I/O。I/O 的延迟和吞吐特征与 CPU 完全不同，因此运行时必须区分计算等待和 I/O 等待。

15.1 同步 I/O 和异步 I/O

同步 I/O 调用会让当前线程等待结果。少量线程处理少量 I/O 时简单可靠；大量并发连接或慢设备场景下，同步等待会浪费线程。异步 I/O 把提交和完成分离，让线程可以处理其他工作。

事件循环、回调、future、协程和 `io_uring` 都是组织异步 I/O 的方式。选择哪种方式，要看系统需要的并发数、延迟要求、代码复杂度和平台支持。

异步 I/O 的核心不是“换一种语法”，而是把等待从工作线程上移走。同步调用让当前线程停在系统调用或库调用里；异步模型把请求提交出去，完成时再处理结果。这样可以用较少线程管理大量等待。但异步也会引入状态机、取消、超时、错误传播和资源生命周期问题。若并发度很低，同步 I/O 可能更简单可靠。

15.2 背压

背压是系统告诉上游“我处理不过来”的机制。没有背压，队列会无限增长，内存耗尽，延迟失控。背压可以通过 bounded queue、令牌桶、限流、拒绝请求、降级和批处理实现。

并发系统里 bounded queue 的意义不只是节省内存，更是把过载变成可观察、可控制的状态。阻塞、返回错误或丢弃，都比默默无限排队更可管理。

背压必须一路传到真正的生产者。只在中间队列限长，但上游仍然无限接收请求，过载会换一个地方堆积。一个服务端如果 worker 队列满了，可以选择拒绝新请求、降低读取速度、返回重试、丢弃低优先级任务或触发降级。选择哪种方式取决于业务语义，但不能没有选择。

15.3 批处理

批处理用一次调度或一次 I/O 处理多个元素，摊薄固定成本。网络发送、日志写入、数据库提交和向量计算都常用批处理。批太小浪费固定成本，批太大增加延迟和缓存压力。

批处理要控制两个阈值：数量阈值和时间阈值。只按数量凑批，在低流量时可能让请求等太久；只按时间刷批，在高流量时可能批太小。很多系统使用“达到 N 条或等待 T 时间就发送”的策略。批处理还要考虑失败语义：一批里部分成功时如何重试，是否会重复执行，是否需要幂等 ID。

15.4 CPU 与 I/O 的边界

I/O 密集任务不应占满计算线程池。常见设计是分离 I/O 线程、计算线程和后台维护线程，或者使用异步运行时统一调度但明确资源限制。否则慢 I/O 会阻塞计算任务，计算任务也会延迟 I/O 完成处理。

15.5 队列是系统的压力表

队列长度、入队速率、出队速率和等待时间，是理解系统压力的基本指标。队列持续增长说明下游处理能力不足或上游没有背压；队列经常为空说明下游可能在等上游；队列长度锯齿状变化可能是批处理或周期性调度造成的。只看 CPU 使用率可能误判，因为系统可能 CPU 很空但卡在 I/O，也可能 CPU 很满但队列仍然增长。

有界队列实验要记录三种结果：生产者是否阻塞，消费者是否空闲，丢弃或拒绝是否发生。这样才能判断背压策略是否真的控制住过载。

15.6 I/O 和 CPU 的成本模型不同

CPU 计算的基本问题是怎样让核心持续退休有用指令，I/O 的基本问题是怎样管理等待、排队和设备吞吐。一次磁盘读取、网络请求或管道写入可能比一条 CPU 指令慢很多数量级。若用 CPU-bound 的思维处理 I/O，很容易开太多线程、阻塞 worker、隐藏队列积压，最后系统看起来 CPU 不满但请求延迟很高。

I/O 还有两个重要特征。第一，吞吐和延迟经常需要批处理折中。单条写入延迟低但吞吐差，大批写入吞吐高但单条等待更久。第二，错误和取消是常态。磁盘可能返回短读，网络可能超时，远端可能重置连接，写入可能部分成功。高质量运行时不能只写成功路径。

Compute Systems Engine 后续的分布式 shuffle 会大量使用 I/O 思维。map 端产生中间数据，缓冲、批量写入、按分区发送、等待 reduce 端接收、处理重试和背压。如果单机阶段没有建立 I/O 模型，分布式阶段就会写成“把数据发过去”这种空话。

15.7 同步阻塞的适用边界

同步阻塞 I/O 并不是低级。对于简单工具、并发度低的批处理程序、启动脚本或一次性文件读取，同步 I/O 清晰可靠。问题出在大量连接或大量慢设备等待时，每个等待都占用一个线程。线程有栈、调度状态、上下文切换成本和内存占用；阻塞线程太多，系统会被调度和内存压力拖垮。

因此选择同步还是异步，不是看语法潮流，而是看并发等待数量和生命周期复杂度。若程序同时等待几十个文件读取，同步线程池可能足够；若服务同时维持几十万连接，事件循环或异步 I/O 更合适。若任务既有 CPU 密集计算又有慢 I/O，至少要把计算线程池和 I/O 等待分开，避免慢 I/O 占满计算 worker。

15.8 事件循环的核心

事件循环把“等待多个事件”集中到少量线程中。它通常维护一组已注册的 fd、定时器和任务队列，等待内核通知哪些事件 ready，然后执行对应回调。Linux 上常见机制包括 `epoll` 和 `io_uring`。事件循环不等于更快的 CPU，它只是避免每个等待都占用一个线程。

事件循环的难点是回调不能长时间占用 loop 线程。若在事件循环线程里做 CPU 密集解析、压缩或排序，其他 I/O 完成事件会被延迟处理。常见架构是事件循环负责接收和发送，CPU 工作提交到计算线程池，完成后再把结果交回事件循环。这样系统同时需要 I/O 队列、计算队列和背压策略。

15.9 io_uring 的直觉

`io_uring` 的核心直觉是提交队列和完成队列。用户态把 I/O 请求放入提交队列，内核处理后把结果放入完成队列。它减少某些系统调用和数据结构转换成本，也支持更丰富的异步操作。但它不是魔法：设备仍有真实延迟和带宽，队列仍可能满，请求仍可能失败，完成处理仍需要 CPU。

学习 `io_uring` 时，不要一开始追求所有高级特性。先理解三件事：提交多少 in-flight 请求，完成事件如何被消费，失败和取消如何传播。过多 in-flight 请求会增加队列延迟和内存占用；完成处理太慢会让完成队列积压；取消语义不清会导致请求完成后访问已释放对象。

15.10 超时、取消和生命周期

异步系统必须处理超时和取消。一个请求提交出去后，上层可能已经超时返回，用户对象可能准备释放，但底层 I/O 仍可能稍后完成。若完成回调访问已销毁对象，就会出现 use-after-free。正确设计通常需要请求上下文对象、引用计数或所有权转移，确保完成路径和取消路径共享同一生命周期协议。

超时也不是简单地“过了时间就删掉”。如果底层操作不能立即取消，超时只表示上层不再等待，底层完成事件仍要被回收和丢弃。若请求已经发送到远端，远端可能仍执行，重试可能造成重复提交。因此超时、取消和幂等会在分布式章节重新出现。单机异步 I/O 是理解分布式失败语义的前置训练。

15.11 背压的几种策略

背压不是只有阻塞。常见策略包括阻塞等待、立即失败、带超时等待、丢弃低优先级任务、降级处理、批量合并、采样、令牌桶限流和动态降低读取速度。每种策略都有业务语义。日志采样可以丢一部分低优先级日志；支付请求不能随便丢；监控指标可以合并；用户请求可能返回 429 或重试建议。

有界队列是背压的最小机制。队列满时，生产者必须面对现实：等、失败、丢弃或降级。无限队列只是把这个选择推迟到 OOM 或尾延迟爆炸。第二册里的 bounded queue 不只是并发练习，它是流控思想的基础。

Listing 15.1 带超时的提交语义草图

```
1 enum class SubmitResult : std::int32_t {
2     Accepted,
3     TimedOut,
4     Closed,
5 };
6
7 struct SubmitPolicy {
8     std::int32_t timeout_milliseconds = 0;
9     bool allow_drop = false;
10 };
```

这个片段没有实现队列，而是表达接口语义。调用者需要知道任务是否被接受、是否超时、队列是否关闭。若接口只返回 void，调用者无法区分提交成功和任务丢失。系统工程里，返回值设计就是背压协议的一部分。

15.12 批处理的延迟预算

批处理必须有延迟预算。假设日志写入器每满 4096 条刷一次磁盘，如果流量很低，最后几条日志可能等很久。因此通常设置两个阈值：数量达到 N 或时间达到 T 就刷。N 控制吞吐，T 控制尾延迟。选择 N 和 T 要看业务：监控指标可以延迟几秒，交互请求响应不能等太久。

批处理还要考虑内存和缓存。批太大，可能超过缓存，处理时局部性下降；批太小，系统调用和队列成本占比高。网络批处理还要考虑 MTU、拥塞控制和远端处理能力。数据库提交还要考虑事务语义和 fsync 成本。没有一个“最佳批大小”，只有工作负载下的实验矩阵。

15.13 异步流水线

一个典型异步流水线可以分成读取、解析、计算、写出四段。读取是 I/O 密集，解析可能 CPU 密集，计算可能需要线程池，写出又是 I/O 密集。段与段之间用有界队列连接。每个队列的长度、入队速率和出队速率告诉我们瓶颈在哪。

read stage	->	parse queue	->	compute workers	->	write queue	->	output stage
I/O		bounded		CPU		bounded		I/O

若 parse queue 持续增长，说明解析或计算跟不上读取；若 write queue 持续增长，说明输出慢；若所有队列为空而 CPU 很空，说明上游读取慢；若 CPU 满且队列也增长，说明计算是瓶颈。队列指标让系统不再是黑箱。

15.14 优先级和公平性

真实系统通常有不同优先级任务。高优先级请求、后台压缩、指标上报、日志写入不应完全同等对待。单个 FIFO 队列可能让大量低优先级任务阻塞高优先级请求。可以使用多队列、优先级队列、配额、公平调度或令牌桶隔离。

优先级也会带来饥饿风险。若高优先级持续不断，低优先级任务可能永远不执行。公平性策略

要说明最低保障、老化机制或后台窗口。系统设计经常是在延迟、吞吐、公平和实现复杂度之间取舍。

15.15 I/O 错误和重试

I/O 错误必须分类。临时错误可以重试，例如网络超时、EAGAIN、远端暂时不可用；永久错误不应盲目重试，例如权限错误、格式错误、目标不存在。重试要有预算、退避和幂等保障。无限立即重试会放大故障，形成重试风暴。

写入操作尤其要小心部分成功。一次批量写可能只写了一部分；一次远程请求可能已经被对方执行但响应丢失；一次文件写入可能进入 page cache 但还没持久化。系统必须定义提交点：什么时候认为任务完成，什么时候可以重试，重复执行是否安全。分布式章节会把这套语义扩展到幂等 ID 和检查点。

15.16 Linux 观察入口

Linux I/O 观察可以从几个层次开始。系统调用层可用 `strace` 观察 `read/write/epoll/io_uring` 相关调用，但它会带来明显开销。性能层可以用 `perf` 看 CPU 热点和调度，使用 `iostat`、`pidstat` 或 `ss` 观察设备和网络状态。内核源码阅读可以从 VFS、page cache、block layer、`epoll` 或 `io_uring` 子系统选择一个窄问题进入。

不要写“Linux I/O 很复杂”这种泛泛结论。应围绕现象阅读：为什么 `mmap` 首次访问触发 page fault，为什么 `epoll` 能等待多个 fd，为什么写文件返回不等于持久化，为什么 `io_uring` 有提交队列和完成队列。读源码要和用户态实验绑定。

15.17 本章落到贯穿项目

Compute Systems Engine 在本章应实现一个本机异步模拟，不必立刻依赖真实网络。可以用一个生产者按固定速率生成批次，一个解析阶段消耗 CPU，一个写出阶段按可配置延迟模拟 I/O。阶段之间使用 bounded queue，记录队列长度、阻塞次数、丢弃次数、批大小、端到端延迟和每阶段吞吐。

然后加入三种背压策略：阻塞生产者、try submit 失败、批量合并。再加入超时和取消：请求超过延迟预算后标记过期，但底层完成事件仍要被消费。这个实验会直接服务分布式 shuffle：网络发送、reduce 端接收、重试和检查点都可以从这里演化出来。

15.18 本章习题

习题与作业

习题一：设计一个三阶段流水线：读取、计算、写出。为每个阶段写出资源类型、队列容量、背压策略、指标和失败语义。

习题二：比较阻塞 submit、try submit 和 timed submit。分别说明调用者能感知哪些状态，适合哪些业务，不适合哪些业务。

习题三：设计批处理策略。给定数量阈值 N 和时间阈值 T，说明高流量和低流量下的行为，解释吞吐和尾延迟如何权衡。

习题四：写一份 `io_uring` 学习报告提纲。只要求解释提交队列、完成队列、in-flight 请求和取消语义，不要求完整实现。

习题五：构造一个重试风暴场景。说明没有退避和预算时系统怎样恶化，再设计带预算、退避和幂等 ID 的重试策略。

15.19 完成队列和回调线程

异步系统通常有完成队列。I/O 提交后，完成事件进入 completion queue，某个线程负责取出事件并执行后续处理。这个线程不应该做重 CPU 工作，否则完成队列会积压，导致已经完成的 I/O 迟迟不被用户态处理。常见做法是完成线程只解析结果、更新状态，然后把 CPU 工作提交到计算池。

完成事件必须携带请求上下文 ID，而不是直接裸指针访问业务对象。若请求已经超时，上下文

可能处于 canceled 状态；完成线程应安全地把结果丢弃或记录，而不是继续执行已过期业务逻辑。这里的状态机通常包括 submitted、completed、timed out、canceled、reaped。reaped 表示底层完成事件已经被消费，资源可以释放。

15.20 短读、短写和 partial completion

I/O API 经常允许短读和短写。读取文件或 socket 时，返回的字节数可能小于请求字节数；写入也是如此。正确代码必须循环直到满足条件、遇到 EOF、出错或超时。若把一次 read/write 返回当成完整消息，就会在压力和边界情况下出错。

协议通常需要 framing：先读固定长度 header，再根据长度读 body；或者使用分隔符；或者使用固定大小消息。异步实现中，一个逻辑消息可能对应多次底层完成事件。请求状态机要记录已经读写多少字节、剩余多少字节、下一步做什么。I/O 编程的难点经常不在系统调用，而在状态机完整性。

15.21 限流器和令牌桶

背压可以由队列容量触发，也可以由限流器主动控制。令牌桶是一种常见模型：系统按固定速率产生 token，请求需要消耗 token 才能发送；桶容量允许短时间突发；长期速率受 token 生成速度限制。它适合控制出站 RPC、日志写入、后台压缩和重试。

令牌桶的参数要有业务含义。生成速率对应可持续吞吐，桶容量对应允许突发，等待策略决定超出速率时阻塞还是失败。若重试也共用同一个令牌桶，就能防止重试风暴；若重试绕过限流，就会在故障时放大流量。

15.22 队列容量的选择

队列容量不是越大越好。容量太小，短暂抖动就会阻塞上游，吞吐可能下降；容量太大，过载时延迟积压很深，失败反馈太晚，还可能耗尽内存。容量应根据服务时间、突发大小、延迟预算和内存预算选择。

一个实用方法是先估算：如果消费者每秒处理 10000 个任务，允许吸收 100 ms 突发，则队列容量至少约 1000；如果端到端延迟预算只有 50 ms，队列排队就不能无限增长。测量时要记录队列等待时间，而不只是队列长度。长度相同，处理速率不同，等待时间也不同。

15.23 优雅关闭和 drain

I/O 系统关闭时要定义 drain 语义。立即关闭会丢弃队列中尚未处理的请求；drain 关闭会拒绝新请求，但继续处理已接受请求；带超时 drain 会在预算内尽力完成，超时后取消或丢弃。不同业务选择不同语义。日志系统可能允许丢低优先级日志，数据库提交不应随便丢。

线程池、bounded queue 和异步 I/O 都需要关闭协议。关闭不是析构时把标志设为 true 就结束，还要唤醒等待者、停止接受新请求、等待 running 请求、消费完成队列、释放资源并报告未完成请求。若完成队列没有 drain，底层稍后完成可能访问已经销毁的上下文。

15.24 本章项目验收

Compute Systems Engine 的 I/O 模拟模块应通过这些验收：生产速度大于消费速度时队列不会无限增长；阻塞策略能降低生产速度；try 策略能报告失败；批处理策略能降低固定成本但不超过延迟预算；超时请求的完成事件仍被安全回收；关闭时不丢已接受且必须完成的任务；指标能显示队列长度、等待时间、批大小、超时数和丢弃数。

这些验收看起来像工程细节，但它们决定系统能否在过载和退出时保持可控。很多线上事故不是算法算错，而是队列无界、关闭不完整、重试没有预算、完成事件访问已释放对象。

15.25 文件 I/O 的提交语义

文件写入有多层语义。应用把数据复制到用户态缓冲，只说明用户态内存有数据；调用 `write` 成功，通常说明数据进入内核 page cache 或设备队列，不一定持久化；调用 `fsync` 成功，才更接近“崩溃后可恢复”，但还要考虑文件系统、目录项和设备缓存。若用临时文件加 `rename` 提交，还

要考虑 rename 后 fsync 目录，确保目录项持久化。

这对 checkpoint 和 manifest 非常关键。一个作业写完 manifest 后就宣布成功，但 manifest 没有 fsync，机器掉电后可能丢失。或者输出文件已经 rename，但目录项没持久化，恢复时看不到文件。教学项目可以在手机上简化持久化实验，但正文必须说明真实可靠提交需要明确持久化边界。

```

durable output shape:
  write temporary output
  fsync temporary output
  rename temporary output to final name
  fsync parent directory
  record manifest commit

```

不同文件系统和存储服务语义不同。对象存储可能没有 POSIX rename，NFS 语义也可能不同。本书不展开存储系统细节，但要求读者不要把“写了文件”误认为“已经安全提交”。

15.26 Page cache 对 benchmark 的影响

文件读取 benchmark 很容易测到 page cache。第一次读大文件可能来自磁盘，第二次读同一文件可能来自内存。若报告没有说明冷 cache 还是热 cache，结果无法解释。清 page cache 需要权限，也会影响系统其他进程；普通用户可以通过读取不同文件、增大数据集或明确报告热缓存来处理。

写入 benchmark 也类似。写入 page cache 很快，后台写回稍后发生。若只测 write 返回时间，可能低估真实持久化成本；若测 fsync，成本会集中暴露。日志系统通常用批量 fsync 或 group commit，在吞吐和延迟之间取舍。第 17 和 18 章的 checkpoint 提交，需要继承这里的语义。

15.27 epoll 的 ready 语义

epoll 通知的是 fd ready，而不是业务消息完成。Socket 可读表示读一次不会阻塞，但不保证完整应用消息已经到达；可写表示发送缓冲有空间，但不保证整个响应能一次写完。应用仍然要处理短读短写和消息 framing。事件循环只是等待机制，不替你实现协议状态机。

边沿触发和水平触发也有差异。水平触发下，只要状态仍 ready，事件会继续出现；边沿触发下，状态从 not ready 变 ready 时通知一次，应用必须读写到 EAGAIN，否则可能遗漏后续机会。边沿触发可以减少重复通知，但更容易写错。教学阶段建议先理解水平触发，再研究边沿触发。

15.28 网络缓冲和 backpressure

网络发送通常先进入用户态缓冲、内核 socket buffer、网卡队列，再到网络。send 成功不表示对端应用已经处理，只表示本端接受了数据。若对端处理慢，接收窗口缩小，本端发送缓冲会填满，最终 send 阻塞或返回 EAGAIN。这是 TCP 层面的背压。

应用层还需要自己的背压。TCP 只能说明字节流压力，不知道业务优先级、请求幂等、队列容量和服务端 CPU。应用可以根据 reduce partition 队列长度返回 busy，可以根据请求 deadline 拒绝，可以按优先级丢弃低价值消息。网络背压和应用背压要配合，而不是互相替代。

15.29 零拷贝的生命周期问题

零拷贝听起来总是好事，但它把缓冲区生命周期变长。普通发送可以把数据复制到内核缓冲后复用用户内存；零拷贝发送可能要求用户缓冲在发送完成前保持不变。若应用过早复用或释放缓冲，就会发送错误数据或崩溃。完成通知必须告诉应用何时可以回收缓冲。

这和异步 I/O 的请求上下文相同：提交出去的东西，在完成前不能随便销毁。对象池、引用计数、completion queue 和取消状态机都要配套。没有生命周期协议的零拷贝优化，会把一个性能问题变成正确性问题。

15.30 Group commit

Group commit 把多个提交合并到一次持久化操作中。数据库日志、消息队列和检查点系统都常用这个思想。单个请求每次 fsync，延迟高且吞吐低；多个请求共享一次 fsync，可以大幅提高吞吐，

但每个请求可能等待批次凑齐。数量阈值和时间阈值决定延迟和吞吐。

Group commit 还要处理失败。如果批次 fsync 失败，批内所有请求都失败还是部分失败？如果 fsync 成功但响应发送失败，客户端重试时如何返回旧结果？如果进程在 fsync 后、响应前崩溃，恢复时是否能根据日志判断请求已提交？这些问题直接连接分布式幂等表和提交点。

15.31 异步错误传播

同步函数可以通过返回值或异常报告错误，异步系统的错误可能在提交之后很久才出现。提交成功只表示请求被接受，不表示操作成功。调用者需要一个 completion、future、回调或事件来接收最终结果。若接口只返回 accepted，却没有完成路径，错误会丢失。

异步错误还可能发生在取消之后。上层已经超时，底层返回失败或成功，完成路径仍要消费事件并更新状态。状态机应区分 user-visible result 和 internal cleanup。对用户来说请求已超时；对系统来说底层资源还没 reaped。混淆这两者会导致内存泄漏或 use-after-free。

15.32 队列的水位线

除了满和空，队列还可以设置高水位线和低水位线。超过高水位线时上游减速或停止；降低到低水位线时恢复。水位线可以避免在满和不满之间频繁抖动。网络协议、日志系统和消息队列都常用类似机制。

水位线选择要结合处理速率和突发。高水位线太接近容量，反馈太晚；太低，吞吐可能受限。低水位线太接近高水位线，系统频繁开关；太低，恢复太慢。实验可以画队列长度随时间变化，观察是否出现锯齿、振荡或持续增长。

15.33 优先级丢弃

有些数据可以丢，有些不能丢。监控采样、调试日志、推荐候选可以在过载时丢弃或降级；支付请求、提交协议和检查点不能随意丢。队列满时，如果只用 FIFO，低价值任务可能占住空间，高价值任务被拒绝。优先级丢弃或多队列隔离可以改善这种情况。

但丢弃必须可观测。系统要记录丢弃数量、原因、优先级和影响范围。静默丢弃会让上层误以为系统正常。对于可丢数据，也要定义最大丢弃比例或降级策略，避免过载时完全失明。

15.34 背压传播链

背压必须沿数据流传播到真正的源头。假设 write queue 满了，compute worker 被阻塞；compute worker 阻塞后 parse queue 满；parse queue 满后 read stage 停止读取；read stage 停止读取后 socket receive buffer 满；最终 TCP 窗口缩小，上游发送变慢。这是一条健康的传播链。若中间某处使用无限队列，背压就断了，内存会积压在那里。

分布式 shuffle 也一样。Reduce 队列满，应反馈给 map 端；map 端降低发送或合并更多；如果 map 端仍无限读取输入并缓存 shuffle，内存会爆。背压链条要在设计图里画出来，不能只在某个队列上写容量。

15.35 超时和 Deadline 的区别

Timeout 是从当前操作开始计算的一段时间，deadline 是整个请求的截止时间。异步流水线更适合传 deadline，因为请求经过多个阶段，每个阶段都应知道剩余预算。若每个阶段都设置固定 timeout，端到端延迟可能超过用户预算。Deadline 还能帮助队列做决策：已经没有足够时间完成的任务，可以提前拒绝或降级。

Deadline 也要进入重试策略。一个请求剩余 5 ms，却启动一个预期 50 ms 的重试，通常没有意义。重试库应检查剩余预算、幂等性和重试次数。分布式系统中，deadline 还要考虑时钟差异；本机流水线中，单调时钟即可。

15.36 I/O 模拟器的参数

本章项目的 I/O 模拟器应有可调参数：生产速率、消费速率、队列容量、批大小、批时间阈值、错误率、短写概率、超时预算、重试次数和是否允许丢弃。通过改变这些参数，读者可以观察队列从

稳定到过载的过程。模拟器不是为了假装真实设备，而是为了训练状态机和背压思维。

每次实验要记录参数。否则“队列爆了”没有意义。记录后可以复现：在生产速率两倍于消费速率、队列容量 1024、批大小 64、错误率 1% 时，阻塞策略如何表现，try 策略丢多少，批处理如何影响延迟。

15.37 I/O 章节和分布式章节的连接

本章所有概念都会在分布式章节放大。短写对应消息分片，completion queue 对应 RPC 响应，deadline 对应跨服务预算，令牌桶对应重试限流，group commit 对应提交日志，manifest fsync 对应 checkpoint，背压链对应 shuffle 流控。学 I/O 不是为了记住某个系统调用，而是为了理解等待、完成、提交和过载这些通用结构。

15.38 端到端流水线案例

以日志统计为例，端到端流水线可以这样设计。Read stage 从文件读取 batch，Parse stage 把文本转成列式热字段，Compute stage 做本地聚合，Shuffle stage 按 partition 发送 partial，Write stage 写 manifest 或最终输出。每两个 stage 之间都有有界队列。每个 batch 都有 batch id、输入 offset、记录数、deadline 和错误计数。

这个流水线里，任何阶段慢都会反映到队列。Parse 慢，read-to-parse 队列增长；Compute 慢，parse-to-compute 队列增长；Shuffle 慢，compute 输出缓冲增长；Write 慢，提交延迟上升。通过这些队列，系统可以定位瓶颈，而不是只看到总时间变长。队列也是背压传播路径，防止 read stage 无限读取文件占用内存。

Listing 15.2 流水线 batch 元数据

```
1 struct PipelineBatch {
2     std::uint64_t batch_id = 0;
3     std::uint64_t begin_offset = 0;
4     std::uint32_t record_count = 0;
5     std::uint32_t error_count = 0;
6 };
```

Batch 元数据应和数据分开。热字段列可能很大，元数据用于调度、指标和恢复。后续分布式版本中，batch id 可以映射到 task id 或 shuffle block id。

15.39 可靠写入的状态机

写输出可以分成 preparing、writing、flushing、committing、committed、failed。Preparing 创建临时路径和缓冲；writing 写数据；flushing 执行必要持久化；committing 发布 manifest 或 rename；committed 对外可见；failed 清理临时状态或保留诊断。把这些状态写清，恢复时才知道半成品如何处理。

如果进程在 writing 中崩溃，临时文件可能不完整，应丢弃或重写；在 flushing 后、committing 前崩溃，数据可能持久但未对外宣布，恢复时应根据 manifest 决定是否采用；在 committing 后响应丢失，重试应返回已提交。这个状态机和分布式 task attempt 提交完全一致，只是资源从网络输出变成文件。

15.40 读放大和写放大

I/O 优化要关注读放大和写放大。读取一条记录可能需要读入整页、解压整个块或读取索引；写一条记录可能导致写一个 page、一个日志段或一个压缩块。若每条小记录单独写文件，元数据和 fsync 成本会非常高；批量写能降低写放大。若每次查询都读完整原始日志，只为了统计一个字段，读放大很高；列式存储和索引可以降低读放大。

读写放大和 cache line 放大是同一思想在存储层的版本。硬件按 cache line 搬，文件系统按页和块搬，网络按包和缓冲搬。系统设计要让每次搬动包含足够有用信息。

15.41 异步管线的内存预算

异步流水线中，每个队列容量乘以 batch 大小，就是内存预算的一部分。若 read queue 容量 128，每个 batch 4 MiB，只这一段就可能占 512 MiB。多个阶段叠加后，内存峰值很快上升。队列容量不能只按任务数量设置，还要按字节设置。

因此实际系统常同时限制 item count 和 byte count。一个 batch 很大时，即使 item count 不高，也可能触发背压。Shuffle 阶段尤其需要按字节限流，因为不同 partition 的 block 大小差别很大。第 18 章的 map in-flight shuffle bytes 就来自这个原则。

15.42 完成回调的幂等

异步完成回调可能被重复触发吗？理想 API 不应重复，但上层重试、取消和状态恢复可能让同一逻辑请求出现多个完成路径。回调内部应先检查请求状态，只有从 submitted 转到 completed 的那一次执行用户可见副作用。后到的完成、取消或超时只做清理或记录重复。

这可以用 CAS 状态转换表达。状态从 Submitted 到 Completed 成功的线程负责发布结果；若状态已经 TimedOut，完成线程只回收底层资源；若状态已经 Canceled，同样不执行业务回调。这个小状态机和分布式 exactly-once 提交同构。

15.43 I/O 失败的可恢复性

不是所有 I/O 失败都可恢复。临时 EAGAIN、超时、连接重置可以重试；磁盘满、权限错误、校验失败、格式错误可能需要停止或人工处理。系统应把错误分为 retryable、fatal、corrupt、backpressure。Corrupt 错误尤其重要：如果读取 block 校验失败，盲目重试同一损坏文件可能无用，需要重新生成或从副本恢复。

错误分类进入 metrics。Retryable 错误升高可能说明下游过载；fatal 错误说明配置或权限；corrupt 错误说明存储或写入协议；backpressure 说明容量保护生效。没有分类，错误数只是噪声。

15.44 本章附加实验设计

附加实验一：模拟 writer 每批写入后是否 fsync。比较只 write、每批 fsync、每 N 批 group fsync 的吞吐和延迟，并说明持久化语义差异。附加实验二：模拟队列按 item count 限制和按 byte count 限制，构造少量大 batch 和大量小 batch，观察哪种策略能控制内存。附加实验三：模拟 completion 在 timeout 后到达，验证资源能被 reaped 且业务回调不会重复执行。

这些实验会让 I/O 章和分布式章真正连接起来。可靠计算系统的难点不是一次成功写入，而是在超时、失败、重复和关闭时仍保持状态机正确。

15.45 案例：限速读取

当下游处理慢时，读取阶段不应继续把文件读入内存。限速读取可以根据 parse queue 或 compute queue 的水位线决定是否暂停。若队列超过高水位线，read stage 暂停或减小 batch；低于低水位线后恢复。这样背压从 compute 传回 read。限速读取比读完再排队更稳定。

这个案例在本机文件上也成立。即使文件读取很快，如果解析和聚合慢，无限制读取会占用内存。真实网络输入更明显：停止读取 socket 会让 TCP 接收窗口缩小，把背压传给发送端。系统应尽量让背压接近源头，而不是在中间堆积。

15.46 案例：写出线程和计算线程分离

Reduce 完成后要写输出。若 worker 在线程池里直接执行写文件和 fsync，它会占住计算 worker，其他 reduce 任务无法运行。更好的设计是计算 worker 生成 OutputBlock，提交给 write stage；write stage 有自己的并发度和队列。写慢时，write queue 背压 reduce；reduce 再背压 shuffle；最终 map 降速。

分离后也要处理提交结果。Write stage 成功后通知 driver commit，失败后通知 driver 重试或失败作业。Compute worker 不能假设提交立即成功。这个结构把 I/O 等待从计算池中剥离，但增加了异步状态机。

15.47 案例：超时后的输出

一个写出请求可能在上层超时后仍然成功落盘。如果上层已经重试并提交了另一个 attempt，迟到的成功不能覆盖最终输出。解决方法是写出路径只写 attempt 临时文件，最终 manifest 由 driver 决定。迟到 attempt 即使写成功，也不会进入 manifest；清理线程稍后删除它。I/O 超时和分布式 attempt 去重在这里完全重合。

这个案例说明为什么写出路径不能直接写最终文件。直接写最终文件没有给 driver 选择获胜 attempt 的机会，也无法安全处理迟到完成。临时输出和 manifest 提交是可靠计算的基本结构。

15.48 端到端延迟分解

流水线报告应把端到端延迟分解为队列等待和服务时间。一个 batch 的延迟可以拆成 read service、parse queue wait、parse service、compute queue wait、compute service、write queue wait、write service、commit wait。若只看总延迟，无法知道瓶颈在哪。若队列等待占比高，说明下游容量不足或背压；若 service 时间高，说明该阶段本身慢。

Trace 可以记录每个 batch 的阶段时间。聚合后可以看中位数和尾部。很多系统平均吞吐正常，但少数 batch 因为写出 fsync 或热点 key 尾部很长。端到端分解能帮助定位尾延迟。

15.49 I/O 背压验收清单

本章最终验收应包含：队列有 item 和 byte 两种容量；生产者在高水位线暂停；低水位线恢复；写出使用临时文件和提交点；completion 迟到不会重复业务回调；超时请求最终 reaped；错误分类进入指标；关闭支持 drain 和立即取消两种策略；trace 能显示每个 batch 的等待和服务时间。达到这些要求，I/O 模块才足以支撑第 18 章的分布式计算。

15.50 I/O 章节总结

I/O 系统的核心是等待和完成。同步阻塞简单但占线程，异步减少线程等待但增加状态机；批处理提高吞吐但增加延迟；有界队列暴露过载；deadline 和取消控制生命周期；临时文件和 manifest 定义提交。I/O 代码的质量不在于用了哪个 API，而在于是否能在短读、短写、超时、取消、关闭和失败时保持状态正确。

15.51 I/O 决策表

设计 I/O 路径时，先问并发等待数量是否大，是否需要异步；数据是否必须持久化，是否需要 fsync 和 rename；是否允许丢弃，队列满时阻塞还是失败；是否有 deadline，超时后底层完成如何 reaped；是否需要批处理，数量阈值和时间阈值是多少；是否按 item 和 byte 双重限流；错误是否分类；关闭是 drain 还是立即取消。把这些答案写清，I/O 状态机才不会在边界条件下崩溃。

15.52 案例：检查点写入调度

Checkpoint 写入可能和正常计算争用磁盘。若每个 stage 结束都立即写大 checkpoint，作业延迟会出现尖峰；若后台慢慢写，又要处理写未完成时进程崩溃。可以把 checkpoint 分成元数据和大数据：关键 manifest 小而同步提交，大块数据在 stage 输出时已经写好并校验。这样 checkpoint 提交只发布指针和摘要，减少关键路径 I/O。

这个案例展示了控制面和数据面分离在 I/O 层面的意义。真正必须同步持久化的是“哪些数据有效”的小元数据，大数据可以提前流式写入临时位置。第 18 章 manifest 正是这种结构。

15.53 案例：日志丢弃策略

系统过载时，调试日志可以丢，但错误报告和提交日志不能丢。可以为日志设置优先级队列：critical 日志阻塞或落盘，debug 日志在队列满时丢弃并增加 drop counter。这样过载时系统仍保留关键证据，不会被低价值日志拖垮。

丢弃策略必须透明。Drop counter 要进入指标，报告中要说明哪些日志可能丢。静默丢弃会让故障排查失去证据。背压策略本质上是业务价值排序。

15.54 案例：异步写入和 backpressure 闭环

假设 reduce worker 生成输出 block, write stage 写磁盘。如果 write queue 超过高水位, reduce worker 停止接收新的 shuffle block; reduce 队列满后 map worker 降低发送; map 输出缓冲满后 input reader 暂停读取。这个闭环把磁盘压力传到输入源。若任何一段使用无限队列, 闭环断开, 内存会在那段堆积。

设计背压时, 要画出这条闭环, 并标出每段的容量和策略。哪个阶段阻塞, 哪个阶段返回 busy, 哪个阶段丢弃, 哪个阶段降级, 都要明确。背压不是单个队列属性, 而是整条数据流的协议。

15.55 案例：超时预算贯穿流水线

一个 batch 从读取到提交可能有总预算。Read stage 用掉一部分, parse queue 等待用掉一部分, compute 用掉一部分, write fsync 又用掉一部分。如果每段只看自己的 timeout, 端到端可能超预算。更好的方式是 batch 携带 deadline, 每个阶段根据剩余时间决定继续、降级还是失败。

Deadline 也帮助背压。队列里等待太久的 batch, 即使后续执行也可能超过预算, 可以提前丢弃或标记失败, 释放资源给仍有机会成功的任务。这个策略不适合必须完成的批处理, 但适合交互请求和有时效性的日志分析。时间语义是调度的一部分。

15.56 本章扩展习题

习题与作业

习题六：设计 completion queue 的请求状态机。状态至少包含 submitted、completed、timed out、canceled、reaped。说明每个状态允许哪些转移。

习题七：写一个短读短写处理流程。要求支持 header/body 消息, 记录已处理字节数, 并说明错误和 EOF 如何处理。

习题八：为重试请求设计共享令牌桶。说明正常请求和重试请求是否共用 token, 为什么。

习题九：设计 shutdown drain 策略。比较立即关闭、drain 关闭、带超时 drain 三种语义, 以及它们适合的业务。

实验：用 bounded queue 模拟生产者和消费者。让生产速度高于消费速度, 观察队列大小和阻塞行为。解释这为什么是背压, 而不是简单的性能问题。

第零部分

分布式计算：把单机原则扩展到集群

第 16 章

分布式计算模型

分布式计算把计算扩展到多台机器，但同时引入网络通信和故障。共享内存消失后，线程之间的读写变成消息；锁和原子不再直接可用；时间、顺序和失败都变得不可靠。

16.1 网络不是共享内存

单机内存访问通常以纳秒计，网络往返通常以微秒到毫秒计。单机 cache miss 已经很贵，跨网络请求更贵。分布式系统要尽量减少同步往返，使用批处理、流水线、数据本地性和异步通信。

网络还可能丢包、重排、延迟抖动或断开。应用层必须处理超时、重试、重复请求和部分失败。一个请求超时，不代表对方没有执行；重试可能造成重复副作用，因此幂等性非常重要。

16.2 RPC 的成本

RPC 把远程调用包装成类似本地函数调用的形式，但它不是本地函数。参数要序列化，消息要进入内核或用户态网络栈，经过网卡和交换机，到达对端后再反序列化和调度执行。一次 RPC 的固定成本远高于一次函数调用，因此分布式系统要避免细粒度频繁 RPC。

批处理、流水线和异步调用是常见优化。批处理减少每条消息固定开销，流水线隐藏往返延迟，异步调用避免线程阻塞。但它们都会增加错误处理复杂度：部分成功、乱序返回、超时重试和取消都要定义清楚。

RPC 还会隐藏背压问题。本地函数调用如果慢，调用者自然等在当前线程；远程调用如果被异步提交，调用者可能继续制造更多请求，直到队列、连接池或对端服务被压垮。因此 RPC 客户端必须有并发上限、超时、重试预算和排队策略。没有这些边界，异步 RPC 只是把阻塞变成不可控积压。

16.3 时间和顺序

不同机器没有完美同步时钟。日志时间戳可以帮助观察，但不能单独作为一致性依据。分布式系统常用逻辑时钟、版本号、任期号和日志索引表达顺序关系。

顺序要区分因果顺序和物理时间顺序。请求 A 触发请求 B，B 因果上发生在 A 之后；但两个机器的墙钟时间可能显示 B 的时间戳更早。协议通常不依赖墙钟证明因果，而是使用请求 ID、版本号、日志索引、任期或向量时钟等逻辑信息。日志时间戳主要用于观测和排查，不应被误用成强一致证据。

16.4 幂等性

幂等操作执行一次和执行多次效果相同。分布式系统中，客户端超时后常常不知道服务器是否执行了请求。若请求不是幂等的，重试可能造成重复扣款、重复创建或重复写入。常见做法是给请求分配唯一 ID，服务器记录已处理 ID，并在重复请求时返回同一结果。

幂等设计要落到存储结构。只在客户端生成请求 ID 不够，服务器还要把请求 ID、执行结果和业务状态放在同一个原子提交边界里。否则服务器可能已经修改业务状态，但还没记录请求 ID 就崩溃，重试后仍然重复执行。工程上常用去重表、条件写、事务日志或状态机日志解决这个问题。

16.5 故障模型

机器可能宕机，进程可能重启，网络可能分区，磁盘可能慢，消息可能重复。多数工程系统采用 crash-stop 或 crash-recovery 模型，不处理任意恶意错误。明确故障模型，是协议设计的第一步。

16.6 数据本地性

分布式计算中，移动计算到数据附近通常比移动大量数据更划算。MapReduce、Spark、分布式数据库和对象存储都围绕数据分片、任务调度和网络 shuffle 做权衡。

数据本地性是 NUMA 原则在集群尺度上的版本。单机里远端 NUMA 访问慢，集群里跨机拉取数据更慢；单机里先在线程本地 partial 再合并，集群里先在 map 端 combine 再 shuffle；单机里热点 cache line 会拖慢所有核心，集群里热点 key 会拖慢整个 stage。前面章节的思想在这里不是类比游戏，而是同一原则换了成本单位。

16.7 CAP 的正确理解

CAP 不是说系统只能在一致性、可用性、分区容忍性里任选两个。网络分区是分布式系统必须面对的现实；在分区发生时，系统需要在继续响应请求和保持强一致之间做选择。正常情况下，系统仍然要同时追求低延迟、高可用和足够一致。工程讨论要具体到故障场景，而不是只背缩写。

16.8 超时、重试和重试风暴

超时不是失败证明，只是本地等待时间超过阈值。请求可能在网络上，可能在对端队列里，可能已经执行但响应丢失，也可能确实失败。重试可以提高成功率，也可能放大过载。如果一个服务已经慢了，所有客户端同时重试，会制造重试风暴，让系统更慢。

健康的重试策略通常包括指数退避、抖动、最大重试次数、总超时预算、幂等保证和熔断。重试预算尤其重要：一个上游请求如果调用十个下游，每个下游都重试三次，总请求量可能迅速膨胀。分布式系统的性能问题常常来自这些控制路径，而不是用户函数本身。

核心思想

分布式系统的性能问题往往是正确性问题的影子。重试、复制、超时、共识和恢复都会改变吞吐和延迟。

16.9 从共享内存到消息

单机多线程里，线程通过共享内存通信。即使共享内存有 cache line、一致性协议和内存序成本，程序仍然可以用一个地址表达共同状态。分布式系统里，这个前提消失了。机器 A 不能直接读取机器 B 的内存；它只能发送消息，请求 B 执行某个操作或返回某份数据。共享变量变成消息，锁变成协议，原子操作变成带版本号或日志索引的提交。

这个变化非常根本。单机中一个 mutex 临界区通常在纳秒到微秒级，跨网络的一次协调可能是微秒到毫秒级，还会遇到排队、丢包、重试和对端故障。单机中线程崩溃通常意味着进程整体失败，分布式中一个节点失败后其他节点可能继续运行。单机中内存序由语言和硬件定义，分布式中消息顺序、重复和超时都要由协议处理。

因此分布式系统不能把远程调用当成本地函数。RPC 库可以隐藏序列化细节，但不能隐藏失败语义。调用者必须问：请求是否到达，对端是否执行，响应是否丢失，重试是否安全，重复请求如何处理，超时后资源如何回收。若这些问题没有答案，系统在正常路径看起来能跑，在故障路径会变得不可控。

16.10 消息的生命周期

一条消息从客户端到服务器，至少经过序列化、排队、发送、网络传输、接收、反序列化、调度、执行、生成响应、响应传输和客户端处理。每一步都可能排队，每一步都可能失败。性能报告如果只写“RPC 延迟 3 ms”，还不够；要拆出客户端排队、网络 RTT、服务端队列、执行时间和响应排队。

消息还需要身份。没有 request id，客户端无法把响应和请求可靠对应；没有 attempt id，服务端无法区分第一次请求和重试；没有 trace id，观测系统无法串起跨节点路径；没有 partition id，shuffle 系统无法知道数据应写到哪里。系统工程中，消息头不是形式，它承载正确性和观测性。

Listing 16.1 本机模拟用的消息元数据

```
1 struct MessageHeader {
2     std::uint64_t request_id = 0;
3     std::uint32_t source_worker = 0;
4     std::uint32_t target_partition = 0;
5     std::uint32_t attempt = 0;
```



```

6 };
7
8 struct ShuffleMessage {
9     MessageHeader header;
10    std::vector<std::int32_t> keys;
11    std::vector<std::int64_t> values;
12 };

```

这段代码不是网络协议实现，但它展示了分布式消息需要的基本边界。request id 用于幂等和去重，source worker 用于观测和调试，target partition 用于路由，attempt 用于重试分析。真实系统还会有校验、版本、压缩格式、认证信息和时间戳。

16.11 延迟、吞吐和 in-flight 请求

分布式系统里，吞吐往往依赖足够的 in-flight 请求。一个请求的往返延迟可能很高，如果客户端每次只发一个请求，吞吐会被 RTT 限制。流水线允许多个请求同时在网络中飞行，提高吞吐。但 in-flight 太多会导致队列积压、内存占用上升、对端过载和尾延迟恶化。

这和 CPU 的内存级并行度很像：核心可以同时挂起多个 cache miss 来隐藏延迟，但 miss 太多会占满 load buffer；客户端可以同时发多个 RPC 来隐藏 RTT，但请求太多会占满队列和连接。系统要有并发窗口，窗口大小要由延迟、吞吐、错误率和对端能力共同决定。

一个简单模型是 Little's Law：在稳定系统中，平均并发数约等于吞吐乘以平均延迟。若希望每秒处理 10000 个请求，平均延迟 10 ms，则系统中大约有 100 个请求在飞行。这个模型不能替代测量，但能帮助你判断一个并发上限是否荒谬。

16.12 超时不是取消

客户端超时只表示客户端不再等待，不表示服务器停止执行。服务器可能已经完成业务状态修改，只是响应丢失；也可能还在队列中等待；也可能稍后才执行。若客户端超时时立即重试，而操作不是幂等的，就会重复副作用。若客户端释放请求上下文，但底层完成回调稍后到达，就可能访问已释放对象。

因此超时要配合取消和幂等。取消是尽力而为，不一定成功；幂等保证重复执行的外部效果可控；请求上下文保证完成路径无论成功、失败、超时还是取消，都能安全回收。异步 I/O 章节里的生命周期问题，在分布式 RPC 中更严重，因为远端是否执行不是本地进程能完全控制的。

16.13 幂等提交点

幂等性不是“重试时先查一下”这么简单。真正的幂等需要把请求 ID、业务状态修改和响应结果放在同一个提交边界里。考虑转账、创建订单或写检查点：如果服务器先修改业务状态，随后记录 request id 前崩溃，重试时看不到旧 request id，就可能重复执行；如果先记录 request id，但业务状态没修改就崩溃，重试时又可能误以为已经成功。

一种常见设计是把请求处理当作状态机事务：检查 request id 是否已处理；若已处理，返回保存的结果；若未处理，执行业务修改，同时写入 request id 和结果；提交后再返回。这个过程可以用数据库事务、共识日志或本地 WAL 实现。关键是提交点必须清楚。

16.14 消息顺序和重复

网络可能重排消息。客户端发送 A 再发送 B，不代表服务器一定先处理 A。即使 TCP 在单连接内保持字节顺序，应用层多连接、多队列、多线程处理仍可能改变完成顺序。协议如果依赖顺序，就必须显式携带序号、版本或日志索引。

消息还可能重复。客户端重试会重复发送，网络代理可能重放，服务端超时后客户端又发下一次 attempt。服务端必须决定重复消息如何处理：直接丢弃、返回旧结果、合并、覆盖还是报错。没有 request id 和幂等表，重复消息通常无法安全处理。

16.15 故障检测的模糊性

分布式系统无法可靠地区分“节点死了”和“节点很慢”。心跳超时可能因为节点崩溃、网络拥塞、GC 暂停、磁盘卡顿、CPU 被抢占或调度延迟。把慢节点误判为死节点可能导致重复调度和资源浪费；把死节点长期当成活节点会拖慢恢复。故障检测本质上是基于超时的怀疑，不是数学证明。

工程上通常使用多次心跳、租约、任期、健康检查、慢节点隔离和人工可观测指标。对计算任务而言，可以把慢任务 speculative retry 到其他节点，但要处理重复输出；对存储副本而言，不能轻易让旧 leader 继续写，需要任期和共识保证。故障检测和一致性协议紧密相连。

16.16 部分失败和降级

单机程序常常整体成功或整体失败，分布式系统经常部分失败。一个查询请求可能需要访问十个分片，其中九个成功、一个超时。系统要定义结果：失败整个请求，返回部分结果，使用缓存旧值，还是降级为近似结果。这个选择取决于业务语义。

搜索结果可以返回部分分片并标记结果不完整；计费系统不能忽略一个失败分片；监控系统可以延迟补齐；推荐系统可以使用缓存。分布式教材不能只讲协议，还要让读者看到业务语义如何决定失败处理。

16.17 背压和重试预算

分布式背压要跨节点传播。服务端队列满时，应让客户端降低发送速率、拒绝低优先级请求或返回明确错误。若服务端已经过载，客户端仍按固定策略重试，只会放大故障。健康的客户端要有最大并发、总超时、重试次数、指数退避、抖动和熔断。

重试预算可以按上游请求计算。例如一个用户请求最多允许产生 20 次下游调用尝试，超过就停止重试并返回错误。这样可以防止多层服务各自重试导致请求量指数膨胀。预算还应被观测：报告中要记录原始请求数、重试请求数、超时数、最终失败数和重试原因。

16.18 数据本地性和调度

分布式计算里，调度任务时要考虑数据位置。若输入分片在节点 A，本地执行可以避免网络读取；若节点 A 忙或失败，可以远程读取或复制数据，但要付出网络成本。数据本地性不是绝对规则，而是调度决策的一项成本。

这和 NUMA 类似。单机里，线程靠近内存节点更快；集群里，任务靠近数据分片更快。若本地节点排队很长，远程执行可能更快；若远程网络拥塞，本地等待更好。调度器需要把数据位置、队列长度、节点健康、资源类型和任务优先级一起考虑。

16.19 本机多进程模拟

本册不要求读者立刻搭建真实集群。可以先用本机多进程或多线程模拟分布式系统：每个 worker 有自己的输入分片和消息队列，driver 负责分配任务，网络用有界队列和随机延迟模拟，故障用随机丢弃、超时和重复消息模拟。这样可以在一台机器上训练协议思维。

模拟不能证明真实网络性能，但能验证语义：请求 ID 是否去重，重试是否幂等，队列是否背压，检查点是否能恢复，重复输出是否被提交协议挡住。先在本机把语义做清楚，再迁移到真实网络环境，风险更低。

16.20 RPC 不是函数调用

远程调用看起来像函数调用，但语义完全不同。本地函数调用要么进入函数体，要么在调用前就失败；调用栈、对象生命周期和异常传播都在同一进程内。RPC 经过序列化、发送队列、内核网络栈、网卡、交换机、对端内核、服务端队列、业务线程、响应路径。任何一段都可能排队、失败、重复、超时或部分完成。把 RPC 当成本地函数，是分布式系统设计里最常见的错误之一。

RPC 的调用者必须接受一个事实：超时后不知道远端到底发生了什么。远端可能没有收到请求，可能收到但还没执行，可能已经执行并提交，可能响应丢失，可能响应正在返回。这个不确定性决定了幂等、请求 ID、重试预算和提交点不是附加功能，而是 RPC 的基本语义。只要系统允许超时重试，就必须回答重复请求如何处理。

服务端也不能把 RPC 当成本地函数。请求进入服务端后，首先进入接收队列，再由某个 worker 处理。若业务处理很慢，接收队列会增长；若队列无限，内存会增长；若队列有限，客户端会遇到拒绝或超时。服务端处理完后，响应也可能在发送队列中等待。一次 RPC 的延迟不是业务函数耗时，而是排队、执行、序列化和网络共同组成的端到端路径。

16.21 消息边界和协议版本

分布式系统中，消息必须有明确边界。TCP 是字节流，不保留应用层消息边界；一次 `write` 不等于对端一次 `read`。应用协议需要 framing，例如固定长度头部加变长 body，或者长度前缀，或者分隔符。没有 framing，服务端无法可靠知道一条消息在哪里结束，下一条消息在哪里开始。

消息还需要版本。系统升级时，客户端和服务端可能不是同一版本。若消息结构只按当前代码隐式解析，一次滚动升级就可能造成字段错读。稳健协议会在 header 中放 protocol version、message type、flags、payload length 和 checksum。新增字段要有默认值，删除字段要经历兼容期。分布式系统的接口不是函数签名，而是长期存在的协议合同。

Listing 16.2 消息头部的语义字段

```
1 struct WireHeader {
2     std::uint16_t protocol_version = 1;
3     std::uint16_t message_type = 0;
4     std::uint32_t flags = 0;
5     std::uint64_t request_id = 0;
6     std::uint32_t payload_bytes = 0;
7     std::uint32_t checksum = 0;
8 };
```

这段代码只是表达字段，不是跨平台二进制协议的完整实现。真实协议还要定义字节序、对齐、字段编码、字符串格式和兼容规则。不要直接把 C++ 结构体内存写到网络里，因为 padding、大小端、编译器布局和 ABI 都可能不同。协议格式要按字节定义，而不是按当前进程的内存布局定义。

16.22 序列化的成本和边界

序列化不是简单的“把对象转成字节”。它会消耗 CPU、分配内存、访问字段、复制数据、计算校验，可能还要压缩。对象布局如果适合本地计算，不一定适合网络传输；网络格式如果紧凑，也不一定适合 CPU 直接计算。高性能系统常在内部布局和 wire format 之间做显式转换。

零拷贝也不是免费。要真正避免拷贝，需要缓冲区生命周期跨越发送完成，不能在发送返回后立即复用；需要处理分片、对齐和 scatter-gather；还要保证应用不能在网卡尚未发送完成时修改内容。许多系统先用清晰的复制式协议做正确，再对热点路径引入缓冲池和批量发送。不要在协议语义没清楚时追求零拷贝。

序列化还影响故障恢复。检查点文件、shuffle manifest、任务提交表都需要稳定格式。若格式依赖当前类定义，代码升级后旧检查点可能无法读取。一个工程化系统应为持久化数据定义版本和迁移策略。分布式计算不是只在网络上传消息，还要把状态写到磁盘，并在失败后读回来。

16.23 逻辑时间和因果关系

分布式系统不能依赖全局精确时钟来判断所有事件顺序。机器之间时钟会漂移，NTP 会校正，进程会暂停，网络延迟不稳定。物理时间适合做超时、租约和观测，但协议正确性常需要逻辑时间。最简单的逻辑时间是单调递增版本号、epoch、term 或日志索引。它们不表示真实时间，而表示协议中的先后和所有权。

例如 driver 每次发布新的路由表版本，就把 epoch 加一。Worker 收到旧 epoch 的任务，可以拒绝并要求刷新。Raft 的 term 也是类似思想：旧任期的 leader 即使还活着，也不能继续以旧身份提交新写。逻辑时间让系统能区分“我看到的状态”和“当前合法状态”。

向量时钟能表达更细的因果关系，但成本更高。很多系统不需要完整向量时钟，只需要针对某个资源的版本号。教材阶段要让读者理解：当两个事件分布在不同机器上时，肉眼时间先后不一定

可靠；协议需要自己的顺序标记。

16.24 请求 ID 的设计

请求 ID 不是随机数装饰，而是去重、追踪、幂等和调试的核心。一个请求 ID 至少应在一次逻辑操作内稳定，重试 attempt 不应改变逻辑 request id。否则服务端无法知道两次到达的请求其实是同一个操作的重复。Attempt id 可以单独增加，用于观测和超时分析。

请求 ID 的生成要考虑冲突范围。单机自增 ID 在单个进程内简单，但跨进程和重启后需要节点 ID、时间段或持久化计数。随机 ID 冲突概率低，但调试时不如结构化 ID 直观。雪花类 ID 把时间、节点和序列号组合起来，但依赖时钟行为。教学项目可以使用 driver 分配的 `std::uint64_t` 递增 ID，因为它在本机模拟里足够清楚；真实系统要写明冲突域和重启策略。

Listing 16.3 逻辑请求和 attempt 分离

```
1 struct RequestKey {
2     std::uint64_t request_id = 0;
3     std::uint32_t attempt = 0;
4 };
```

服务端去重表通常按 `request_id` 存储逻辑结果，而不是按 attempt 存储。新 attempt 到达时，如果 `request_id` 已提交，就返回旧结果或忽略重复副作用。Attempt 只用于记录“这是第几次尝试”，不改变业务语义。

16.25 幂等表的容量和回收

幂等表不能无限增长。服务端记录所有 request id，内存迟早耗尽。系统必须定义 request id 的有效期、客户端重试窗口和服务端保留时间。若客户端可能在一天后重试，服务端至少要保留一天；若服务端只保留一分钟，客户端一分钟后重试就可能重复执行。幂等语义是客户端和服务端共同的合同。

回收幂等表时要小心。按时间删除最简单，但依赖时钟和请求延迟分布。按提交日志截断更稳：如果所有客户端都保证不会重试某个 checkpoint 之前的请求，服务端可以删除旧记录。对于批处理任务，幂等表可以和 task attempt manifest 绑定：一个 stage 完成并 checkpoint 后，旧 attempt 去重记录可以随 checkpoint 压缩。

幂等表里保存什么也要权衡。只保存 request id 可以防止重复执行，但重复请求无法返回原响应；保存 request id 和响应结果，可以让客户端收到同样结果，但占用更多空间；保存结果摘要可以减少空间，但可能不能满足业务。设计时要从调用者语义出发。

16.26 错误分类

分布式错误不能只返回“失败”。至少要区分可重试错误、不可重试错误、过载错误、版本错误、权限错误和未知错误。可重试错误包括临时网络失败、服务端 busy、leader 切换中。不可重试错误包括请求格式错误、权限不足、资源不存在。版本错误说明客户端路由表或协议版本过旧，需要刷新而不是盲目重试。

错误分类直接影响重试策略。对不可重试错误反复重试，只会浪费资源；对 busy 错误立即高频重试，会加剧过载；对版本错误不刷新路由，会一直打到错误节点。返回码是背压和控制面的接口，不只是调试信息。

Listing 16.4 错误分类的教学形态

```
1 enum class RpcStatus : std::int32_t {
2     Ok,
3     Retryable,
4     Busy,
5     StaleRoute,
6     InvalidRequest,
```



```

7     PermissionDenied,
8     Unknown,
9 };

```

真实协议还会有更细错误码和错误详情，但分类必须稳定。客户端重试库通常只看分类，日志和排查再看详细原因。把所有错误都归为 Unknown，会让客户端无法做正确行为。

16.27 超时预算的分解

一个用户请求的总超时预算应被分解到各个下游调用，而不是每一层都重新给自己一个完整超时。假设用户请求总预算 1000 ms，服务 A 调服务 B，B 调 C。如果每层都设置 1000 ms，最坏情况下总延迟会远超用户预算。更好的做法是把剩余 deadline 随请求传递，下游根据剩余时间决定是否执行、降级或立即失败。

Deadline 比 timeout 更适合跨层传播。Timeout 是相对时间，每一层都要重新计算；deadline 是绝对截止点，传递后各层可以看当前剩余时间。时钟漂移会影响跨机器 deadline，因此通常还要保守处理，但它仍比无预算重试更清楚。批处理系统也有类似概念：一个 stage 的时间预算应来自整作业 SLA，而不是每个 task 自己无限重试。

16.28 重试、退避和抖动

重试要慢下来，而不是立刻把同一个请求再打一遍。指数退避让连续失败后的等待时间增长，抖动让大量客户端不要在同一时刻重试。没有抖动时，所有客户端可能在服务恢复的一瞬间同时重试，造成第二波过载。重试预算限制一次逻辑请求最多产生多少 attempt，避免多层服务重试相乘。

重试还要和幂等绑定。读请求通常更容易重试，写请求必须有 request id 和提交语义。若写请求不是幂等的，客户端在超时后可能只能查询状态，而不是直接重试。教材要让读者看到：重试是正确性功能，不是简单可靠性增强。

16.29 慢节点和尾延迟

分布式系统的整体延迟常被最慢分片决定。一个查询要访问十个分片，九个分片很快，一个分片慢，整体就慢。慢节点可能没有完全失败，只是磁盘抖动、CPU 被抢占、GC、网络拥塞或热点 key。故障检测如果只判断死活，就看不见慢节点。

处理慢节点的方法包括 speculative execution、请求 hedging、负载隔离、热点迁移和降级。Speculation 会启动重复 attempt，最先完成者获胜；hedging 会在等待一段时间后给另一个副本发同样读请求。这些方法能降低尾延迟，但增加资源消耗和重复副作用风险。没有幂等提交和重试预算，speculation 会让系统更危险。

16.30 队列和排队论直觉

分布式系统里很多延迟来自排队。服务时间接近满载时，队列等待会快速上升。即使平均处理能力略高于平均到达率，突发流量也会造成队列。背压和限流的目的，就是在队列失控前让上游看到压力。

Little's Law 给出一个粗略关系：系统内平均在途请求数等于吞吐乘以平均延迟。若 in-flight 请求数持续增加而吞吐没有增加，说明延迟正在上升。若队列长度上升但 CPU 很空，可能是 I/O 或锁等待；若 CPU 满且队列上升，可能是计算能力不足；若网络出站队列上升，可能是下游慢或带宽瓶颈。队列是分布式系统最重要的压力表。

16.31 消息总线的本机实现

本机模拟可以用一个 MessageBus 对象表示网络。它不需要真实 socket，先用多个有界队列表达节点之间的通道。每条消息带 source、target、request id、attempt、payload bytes、deadline 和 status。MessageBus 可以注入延迟、丢弃、重复和乱序。这样一台手机或一台开发机就能训练分布式协议。

Listing 16.5 本机消息的核心字段

```

1 struct LocalMessage {
2     std::uint32_t source_worker = 0;
3     std::uint32_t target_worker = 0;
4     std::uint64_t request_id = 0;
5     std::uint32_t attempt = 0;
6     std::uint32_t payload_bytes = 0;
7     RpcStatus status = RpcStatus::Ok;
8 };

```

模拟器要把故障做成可重复。使用固定 seed，记录每次丢包、重复、延迟和重排事件。测试失败后，可以用同一个 seed 重放。没有可重复性，分布式 bug 会非常难查。

16.32 实验报告的故障矩阵

分布式实验报告不能只写正常路径。至少要列出故障矩阵：请求丢失、响应丢失、服务端忙、服务端执行后崩溃、客户端超时后重试、重复消息到达、旧路由请求、慢节点、队列满、driver 重启。每一项都要写预期行为：重试、去重、返回旧结果、刷新路由、背压、失败还是恢复。

这张矩阵会迫使设计暴露缺口。例如如果响应丢失后客户端重试，而服务端没有保存响应结果，客户端可能无法知道第一次是否成功；如果执行后崩溃但提交点没持久化，恢复后可能重复执行；如果旧路由请求没有版本检查，迁移期间可能写到旧分片。故障矩阵不是测试附录，而是分布式设计本身。

16.33 连接管理

真实 RPC 系统还要管理连接。每个请求新建 TCP 连接成本高，通常会复用连接池；连接池又要处理最大连接数、空闲超时、健康检查、连接重置和负载均衡。一个连接上的请求可能按顺序发送，也可能多路复用。若一个连接出现队头阻塞，后续请求会被拖慢；若连接太多，内核和远端都会有压力。

连接池的背压语义也要定义。连接都忙时，新请求是等待、失败、还是新建连接？等待是否消耗用户 deadline？连接错误时，正在飞行的请求如何标记？这些问题和线程池、有界队列非常相似。连接池不是网络库细节，而是分布式系统的执行资源管理。

16.34 服务发现和负载均衡

客户端需要知道请求发给哪些服务实例。服务发现可以来自静态配置、DNS、注册中心或控制面。发现结果通常会缓存，因此要处理实例上下线、健康状态和版本变化。负载均衡策略可以是轮询、随机、最少连接、按延迟加权、按 locality 选择。策略不同，故障时行为也不同。

负载均衡不能只看请求数量。一个实例可能处理轻请求很多，另一个处理重请求少；一个实例延迟高可能是过载，也可能是网络远；一个实例刚启动缓存冷，直接大量流量打过去会抖动。工程系统常使用慢启动、健康检查、熔断和 outlier detection。教材项目可以简化为 driver 分配 worker，但应让读者知道真实服务发现是控制面的一部分。

16.35 熔断和隔离

熔断器在下游持续失败或延迟过高时，暂时停止向它发送请求，快速失败或降级。这样可以避免线程、连接和队列被坏下游拖住，也给下游恢复时间。熔断不是为了隐藏错误，而是保护系统。熔断状态通常包括 closed、open、half-open：正常发送、快速失败、试探恢复。

隔离则是把不同下游或不同优先级请求放进不同资源池。若所有下游共享一个线程池，某个慢下游可能耗尽所有 worker，影响无关请求。隔离可以用独立队列、独立连接池、独立线程池或配额。代价是资源利用率可能下降，但故障影响范围更可控。

16.36 幂等性和业务键

请求 ID 可以由系统生成，也可以来自业务键。创建订单时，客户端可以提供 client request id；写日志时，可以使用文件分片和 offset；提交 map output 时，可以使用 job id、stage id、task id。业务键更容易跨重启恢复，因为它和逻辑操作绑定。系统生成的随机 ID 如果只在内存里保存，重启后可能丢失去重语义。

幂等键还要定义作用域。一个 request id 在单个用户内唯一，还是全系统唯一？一个 task id 在一个 job 内唯一，还是跨 job 唯一？作用域不清会导致误去重或漏去重。教学项目应使用结构化 key，例如 job id、stage id、task id、attempt。这样日志和 manifest 更容易读。

16.37 时钟和单调时间

超时测量应使用单调时钟，而不是 wall clock。系统时间可能被 NTP 调整，向前或向后跳；单调时钟适合计算 elapsed time。分布式系统传 deadline 时，跨机器 wall clock 差异仍要考虑，但本机队列和 I/O 超时时应使用单调时间。日志时间戳和超时计时是不同用途。

租约和跨机器 deadline 更复杂。若依赖物理时间做安全判断，就要给时钟漂移和暂停留余量。很多协议避免直接依赖绝对时间来保证一致性，而使用 term、epoch 和 quorum。时间可以优化性能，不应轻易成为唯一正确性依据。

16.38 有序、因果和重放

消息重放是测试分布式系统的重要方法。记录一段消息历史，包括发送、延迟、丢弃、重复和处理结果，然后在本机重放。若系统确定性足够，重放能复现 bug。为了支持重放，消息处理应尽量由输入消息和当前状态决定，避免隐藏随机数和真实时间直接影响核心语义。随机故障应使用记录的 seed。

因果关系也可以在 trace 中表达。一个 retry attempt 由哪次 timeout 触发，一个 reduce commit 由哪些 shuffle block 组成，一个 checkpoint 覆盖到哪个 stage，这些都应有 ID 关联。没有因果关联的日志，只是一堆时间戳，出了问题很难追。

16.39 分布式观测性

分布式观测性至少包含 metrics、logs 和 traces。Metrics 聚合数量和延迟，适合看趋势；logs 记录离散事件和错误，适合排查细节；traces 串起一次请求或一个 task attempt 的路径，适合分析长尾。三者需要共同的 ID：request id、job id、stage id、task id、attempt、partition。没有 ID，无法把 driver、worker、shuffle 和 reduce 的事件连起来。

指标也要按维度分解。只看全局平均延迟，会掩盖某个 partition 长尾；只看总重试数，无法知道哪个 worker 或哪类错误导致；只看总吞吐，无法发现队列越来越长。分布式系统的指标设计要服务故障定位，而不是只服务漂亮仪表盘。

16.40 本章新增实验

本章可以增加三个小实验。第一，连接池模拟：限制最大 in-flight 连接数，观察过载时等待和失败。第二，熔断模拟：让一个 worker 持续超时，客户端在多次失败后暂时停止发送，观察重试量是否下降。第三，trace 关联：为每条消息记录 request id 和 parent event id，重放一次响应丢失导致的重试。

这些实验不需要真实网络，但能训练分布式语义。它们也会给第 18 章的 driver 和 message bus 提供基础。

16.41 多租户和公平性

分布式计算系统常同时运行多个 job。若所有 job 共用 worker 和 shuffle 队列，大 job 可能占满资源，小 job 长时间等待。公平调度需要定义资源份额、优先级和抢占策略。最简单的策略是每个 job 有独立队列，driver 按轮询或权重取任务；更复杂策略会考虑输入大小、deadline、用户配额和历史用量。

公平性和吞吐有时冲突。让大 job 连续占用资源，吞吐可能最高；穿插小 job，平均响应更好但

缓存和调度成本增加。教学项目可以先支持单 job，但设计字段时保留 job id，就是为了未来多租户和公平调度。没有 job id，所有指标和去重都会混在一起。

16.42 背压和公平的交汇

多个上游共享一个下游时，背压还要公平。若 reduce partition 队列满，是否阻塞所有 map worker，还是只阻塞发送该 partition 的 worker？若某个 job 产生大量热点 key，是否影响其他 job 的正常 partition？系统可以为每个 job 或 partition 设置独立队列和配额，防止一个热点拖垮全局。

这和单机线程池里的优先级队列类似。资源隔离降低故障传播，但可能降低资源利用率。工程设计要写清目标：批处理吞吐、交互延迟、公平性还是隔离。不同目标会得到不同调度器。

16.43 数据本地性和复制选择

若输入分片有多个副本，调度器可以选择负载较低且距离较近的副本。只看本地性可能把任务都打到一个繁忙节点，只看负载可能产生大量远程读取。调度器需要一个成本函数，把数据位置、队列长度、节点健康、网络成本和优先级结合起来。教学项目可以用简单规则：优先本地，若本地队列超过阈值则远程。

这个规则和 NUMA 调度完全相似：优先本地内存，过载时跨节点。第二册反复建立这种跨尺度类比，是为了让读者面对新系统时能迁移思维。

16.44 协议演进和兼容测试

分布式协议一旦持久化或跨进程，就需要兼容测试。新增字段要有默认值，旧节点收到新字段能忽略，新节点收到旧消息能补默认值。删除字段要经历过渡期。消息版本、feature flags 和最小兼容版本都要有策略。否则滚动升级时，集群可能出现部分新旧节点互相不能通信。

教学项目可以简单要求“同一版本运行”，但如果写成教材，就要说明这是简化。第三册 AI 推理引擎如果加入模型格式和算子版本，也会遇到同样问题。协议演进是长期系统的基本能力。

16.45 混沌测试和演练

故障注入可以从单元测试扩展到混沌测试：在长时间运行中随机让 worker 变慢、丢消息、重启 driver、损坏临时 block、让队列变满。混沌测试的目标不是制造混乱，而是验证系统在预期故障集合内保持可控。每次注入都应记录事件，失败后能回放。

混沌测试必须有安全边界。不要在没有隔离的真实数据上随意注入；不要注入系统无法承受的未知破坏；不要只看系统没崩就算通过，还要检查结果正确、重试预算、队列长度和恢复时间。可靠性测试同样需要 oracle。

16.46 分布式模型的知识地图

到本章结束，读者应能把分布式系统拆成：消息、时间、故障、身份、路由、幂等、背压、调度和观测。消息回答数据如何传；时间回答何时放弃；故障回答什么可能坏；身份回答这是哪个请求；路由回答发给谁；幂等回答重复怎么办；背压回答过载怎么办；调度回答谁执行；观测回答如何知道发生了什么。后两章会把这些部件组合成分片复制和计算引擎。

16.47 分布式模型章节总结

分布式系统的困难不是网络 API，而是不确定性。消息可能丢、重复、乱序；节点可能慢、停、恢复；客户端超时不代表服务端停止；重试可能放大故障；旧路由和旧 leader 可能继续发送请求。解决这些问题需要请求身份、版本、deadline、幂等表、背压、故障矩阵和 trace。把这些基础模型建好，后面的复制和计算工程才有可靠语义。

16.48 分布式模型决策表

设计一个 RPC 或消息协议时，先填决策表。请求是否幂等？若不是，超时后不能盲目重试。是否有 request id？若没有，服务端无法去重。是否有 deadline？若没有，多层调用会放大延迟。是否区分 busy 和 fatal？若没有，客户端无法正确退避。是否有路由版本？若没有，迁移期间旧请求难以

拒绝。是否有 trace id? 若没有, 故障后无法串联路径。

这张表适用于任何分布式调用, 也适用于本机异步消息。只要执行和完成分离, 就需要身份、时间、状态和错误语义。

16.49 消息协议审查表

每个消息类型都应审查: 谁发送, 谁接收, 是否允许重复, 是否有响应, 超时后发送方做什么, 接收方是否幂等, payload 是否有版本和长度, 错误码是否分类, 是否进入 trace, 是否影响提交状态。审查后会发现很多消息只是指标, 可以丢; 有些消息是提交, 必须持久化; 有些消息是控制面, 必须带 epoch。消息不是字节包, 而是系统状态转换。

16.50 本章落到贯穿项目

Compute Systems Engine 的分布式模拟从三个对象开始: driver、worker、message bus。driver 生成任务和 request id; worker 读取本地分片并执行 map/reduce; message bus 用有界队列模拟网络, 支持随机延迟、超时、重复和丢弃。每条消息携带 request id、attempt、source、target partition 和 payload size。

第一版只做日志计数: worker 本地统计 key, 再把 partial 发给 reduce partition。第二版加入重试: message bus 随机丢响应, driver 重试同一 request id, reduce 端必须去重。第三版加入背压: reduce 队列满时返回 busy 或延迟, map 端降低发送速率。第四版加入检查点: 每个 partition 记录已提交 request id 和结果。

16.51 本章习题

习题与作业

习题一: 画一次 RPC 的完整生命周期, 从客户端提交到服务端响应返回。标出每个排队点、失败点和需要记录的指标。

习题二: 设计一个幂等请求表。说明 request id、业务状态、响应结果如何在同一个提交边界内更新。给出崩溃发生在提交前和提交后的恢复行为。

习题三: 构造一个重试风暴。假设服务端延迟升高, 客户端超时后立即重试。计算请求量如何放大, 并设计重试预算、退避和抖动策略。

习题四: 用本机多线程模拟消息乱序和重复。要求每条消息携带 request id 和 attempt, 服务端能识别重复请求并返回同一结果。

习题五: 比较数据本地调度和远程调度。给出本地排队长、远程网络快以及本地网络快、远程排队短两种场景, 说明调度器应如何取舍。

实验: 在本机用多个进程模拟客户端和服务端。人为加入随机延迟和超时, 观察重试如何造成重复请求。设计一个幂等请求 ID 机制。

第 17 章

分片、复制与共识

分布式系统通过分片扩展容量，通过复制提高可用性，通过共识在故障中维护关键状态一致。三者共同构成大多数存储和计算平台的基础。

17.1 分片

分片把数据按 key、范围或哈希分到不同节点。好的分片要负载均衡、支持扩容、减少跨分片事务。热点 key 会破坏均衡，需要拆分、缓存、限流或特殊路由。

分片后的查询可能需要 scatter-gather：向多个分片发请求，再合并结果。它类似并行归约，但网络延迟和节点故障让合并更复杂。

分片策略要匹配访问模式。哈希分片适合点查和均匀 key，范围分片适合范围扫描和排序，但容易出现热点区间。虚拟分片把逻辑分片数量做得比物理节点多，便于迁移和均衡。热点 key 不能只靠增加节点解决，因为所有请求仍然打到同一个逻辑位置；需要缓存、拆 key、限流、异步化或改变业务模型。

17.2 再分片

系统运行一段时间后，数据量和热点会变化，需要再分片。再分片要迁移数据、更新路由、处理迁移期间的读写，并保证失败后可恢复。简单哈希分片在节点数变化时可能移动大量 key，一致性哈希或虚拟分片可以减少迁移范围。

再分片最难的是迁移期间的一致性。客户端可能按旧路由写，控制面可能已经发布新路由，数据可能正在复制。常见做法是让分片有状态机：准备迁移、双写或转发、追平增量、切换路由、清理旧副本。每一步都要能失败重试。没有状态机的再分片脚本，往往在中途失败后留下难以解释的双写和漏写。

17.3 复制

复制保存多个副本，提高可用性和读吞吐。同步复制延迟高但数据更安全，异步复制延迟低但可能丢失最新写入。主从复制简单，multi-leader 和 leaderless 复制更复杂，需要处理冲突。

复制协议必须说明写入何时算成功。如果 leader 写本地日志就返回，延迟低但 leader 崩溃可能丢数据；如果多数副本确认后返回，延迟高但故障恢复更稳；如果所有副本确认后返回，可用性和尾延迟都受最慢副本影响。工程系统通常让不同数据选择不同级别：元数据强一些，缓存和统计弱一些。

17.4 读写仲裁

Quorum 系统常用 W 个写副本和 R 个读副本满足 $R+W>N$ ，从而让读集合和写集合有交集。这个模型帮助理解一致性和可用性权衡。 W 大提高写安全性但增加写延迟， R 大提高读新鲜度但增加读延迟。实际系统还要处理慢副本、hinted handoff、read repair 和版本冲突。

17.5 一致性

强一致性让读者看到最新写入，但通常需要更多同步。最终一致性允许副本暂时不同，换取可用性和低延迟。选择一致性模型要看业务语义：计费、库存和元数据通常更偏强一致；缓存、推荐和日志统计可以接受延迟收敛。

17.6 共识

Raft 和 Paxos 这类共识协议用于在故障中让多个节点就日志顺序达成一致。共识不是让所有操作都变快，而是让关键状态在故障中仍然可恢复。共识路径应尽量小而清晰，不应把所有大数据流

量都压进共识日志。

共识适合维护小而关键的顺序状态，例如元数据、配置、leader 选举、分片归属和事务提交点。把大对象、热数据流和高频统计全部放进共识日志，通常会让系统吞吐受限。高性能分布式系统常把控制面和数据面分开：控制面用共识保证关键决策一致，数据面用分片、复制、批处理和校验保证吞吐。

17.7 Leader、任期和日志

以 Raft 的直觉看，系统在一个任期内选出 leader，客户端写入先进入 leader 日志，leader 复制给 follower，多数确认后提交。任期号防止旧 leader 在网络恢复后继续破坏新状态，日志索引和任期共同描述顺序。理解这些概念，能帮助读者读懂很多分布式存储的元数据路径。

17.8 读路径和租约

写路径需要顺序，读路径也要定义新鲜度。强一致读可能需要联系 leader 或多数副本，延迟较高；从 follower 读延迟低，但可能读到旧数据；租约读通过时间约束减少通信，但依赖时钟和租约安全边界。选择读路径时要问：业务是否允许旧读，旧到什么程度，故障切换时如何避免旧 leader 继续服务强一致读。

17.9 复制和备份

复制和备份经常被混为一谈。复制解决的是在线可用性：某个副本不可用时，系统还能从其他副本继续服务。备份解决的是历史恢复：数据被误删、程序 bug 写坏、勒索软件破坏或运维误操作后，系统能回到过去某个正确版本。复制通常追求低延迟和快速同步，备份通常追求隔离、保留周期和可验证恢复。一个系统可以有三个副本，仍然必须有备份。

常见误区

不要把复制等同于备份。复制会快速传播错误删除和错误写入；备份强调历史恢复。两者解决的问题不同。

17.10 分片是分布式版的数据布局

单机数据布局决定 cache line、TLB 和 NUMA 成本；分布式分片决定网络、负载和故障边界。把 key 放在哪个分片，不只是存储问题，也是计算调度问题。一个 group-by 如果按 key 分片，reduce 端可以把同一个 key 的 partial 聚到一起；一个范围查询如果按范围分片，可以少访问无关分片；一个热点 key 如果没有特殊处理，会压垮单个分片。

分片策略应从访问模式出发。哈希分片让点查和均匀聚合比较平衡，但范围扫描需要访问多个分片。范围分片支持顺序扫描和范围查询，但时间戳、递增 ID 这类 key 容易把新写入集中到尾部分片。一致性哈希适合节点增减时减少迁移，但范围查询不友好。没有通用最佳分片，只有和 workload 匹配的分片。

分片还定义故障半径。一个分片不可用，影响它负责的数据；一个节点承载多个分片，节点故障影响这些分片；一个热点分片过载，会拖慢访问它的请求。虚拟分片把逻辑分片数量做得比物理节点多，便于迁移、均衡和限流。它类似单机里把工作拆成比线程更多的任务块，让调度器有调整空间。

17.11 路由表和版本

客户端或 coordinator 需要知道 key 属于哪个分片、分片当前在哪些节点、谁是 leader、读写应该走哪个副本。这些信息组成路由表。路由表必须有版本，否则迁移和故障切换时，旧路由和新路由会混在一起。请求中通常携带路由版本或 epoch，服务端发现版本过旧时返回重定向或刷新提示。

路由表更新是控制面问题。控制面要保证“某个分片当前由谁负责”这个决定一致；数据面负责具体读写和数据传输。把控制面和数据面分开，是高性能分布式系统的重要设计。控制面通常用共识维护小而关键的元数据，数据面用分片、复制和批处理处理大量数据。

17.12 再分片状态机

再分片不能只靠一次复制命令。一个稳健再分片通常有状态机。准备阶段创建目标副本和迁移任务；复制阶段把历史数据搬到新位置；追增量阶段处理迁移期间的新写入；切换阶段发布新路由；清理阶段删除旧副本。每个阶段都要能重试，且重试不能重复破坏状态。

迁移期间读写策略有几种。可以让旧分片继续服务，目标追增量；可以让旧分片把新写转发到目标；可以短暂停写切换；也可以使用双写。每种策略都有代价。双写要处理一边成功一边失败；转发增加延迟；停写影响可用性；追增量需要日志或变更流。工程选择要看业务写入量、可用性要求和实现复杂度。

17.13 热点 key 的处理

热点 key 是分片系统的常见敌人。哈希分片只能把不同 key 分散，不能拆开同一个 key。若某个 celebrity user、热门商品或全局计数器承载大量请求，它仍会集中到一个分片。解决方法包括缓存、读写分离、key 拆分、局部聚合、限流、异步化和业务重构。

以日志统计为例，某个 event key 特别热。map 端可以先按 worker 做本地 partial，再把同一个热 key 拆成多个 subkey 发给多个 reduce 任务，最后再做二级合并。这和单机里把全局计数器改成 sharded counter 是同一个思想。热点 key 是分布式版的共享写热点。

17.14 复制的提交点

复制协议的核心是提交点。客户端写入何时算成功？如果 leader 写入内存就返回，崩溃后可能丢失；如果写入本地 WAL 后返回，单机持久性好一些但 leader 丢失时仍可能没有副本；如果多数副本持久化后返回，故障恢复更强但延迟更高。不同数据需要不同提交强度。

提交点还影响读路径。若写只有 leader 本地提交，follower 读可能很旧；若多数提交，强一致读可以通过 leader 或带任期的读协议实现；若异步复制，读扩展性好但新鲜度弱。教材要让读者看到：复制不是“多存几份”这么简单，写成功、读新鲜和故障恢复都由提交协议决定。

17.15 WAL 和状态机

很多复制系统使用 write-ahead log。先把操作写入日志，再应用到状态机。日志提供顺序和恢复依据：崩溃后可以重放已提交日志，未提交日志根据协议处理。状态机复制的思想是所有副本以同样顺序执行同样操作，就得到同样状态。

这个模型适合小而确定的操作。若操作包含读取本地时间、随机数、外部系统调用或不确定迭代顺序，就会破坏状态机一致性。因此共识日志通常记录已经决定好的命令和参数，执行必须确定。大数据流量一般不直接放进共识日志，而是把元数据、提交点和路由放进去。

17.16 Quorum 的边界

Quorum 模型用 R 、 W 、 N 解释读写集合交集，但真实系统比公式复杂。慢副本会拖高尾延迟，失败副本需要 hinted handoff 或补数据，读到多个版本需要冲突解决，时钟偏差会影响最后写入胜出的策略。 $R + W > N$ 只说明集合有交集，不自动处理冲突、删除、版本向量和故障恢复。

读修复是一种常见机制：读请求发现副本版本不一致，返回最新值的同时修复旧副本。它能逐步收敛，但把修复成本放到读路径。反熵任务则在后台比较和修复副本，减少读路径成本但收敛更慢。选择哪种方式，要看读延迟、数据新鲜度和后台资源。

17.17 一致性模型要面向业务

一致性模型不是越强越好。强一致简化上层推理，但需要更多同步和更高延迟；最终一致提高可用性和低延迟，但上层要处理旧读和冲突。业务语义决定选择。库存扣减、元数据路由、权限变更通常需要强一些；日志统计、推荐特征、缓存可以弱一些。

还要区分不同读保证。读己之写保证用户写完自己能读到；单调读保证同一客户端不会读到时间倒退的数据；前缀一致保证不会看到日志后面的操作却看不到前面的操作。很多系统不需要全局线性一致，但需要某些会话保证。精确说出需要哪种保证，比泛泛说“要一致”更有工程意义。

17.18 Raft 的工程直觉

Raft 可以从三个问题理解：谁能接受写入，日志顺序如何确定，故障后如何避免旧 leader 继续写。任期号回答领导权的新旧，日志索引回答操作顺序，多数派确认回答提交安全。leader 接收客户端命令，追加日志，复制给 follower，多数确认后提交，再应用到状态机。

Raft 的价值不是速度，而是在故障和网络分区中保持一条一致的日志。它的成本也很明确：写入通常要经过 leader 和多数副本，跨机 RTT 进入关键路径。因此工程上要把 Raft 用在小而关键的控制面，例如分片归属、配置、任务提交点和元数据，不要把所有大数据 shuffle 记录都塞进 Raft。

17.19 Leader 租约和读

强一致读如果每次都走多数派，延迟较高。leader 租约是一种优化：在租约有效期内，leader 相信自己仍是合法 leader，可以本地服务某些读。但租约依赖时间边界，必须考虑时钟漂移、网络延迟和暂停。若旧 leader 在租约误判下继续服务强一致读，就会返回过期数据。

因此租约读不是“读很快”的简单技巧，而是一套时间假设和安全边界。若系统无法可靠控制时钟和暂停，应该使用更保守的读协议。对教材来说，关键是让读者知道读路径也需要一致性设计，不是只有写路径才重要。

17.20 控制面和数据面

控制面维护少量关键决策：分片在哪里，谁是 leader，任务属于哪个 stage，哪些输出已经提交，哪些节点健康。数据面处理大量数据：读取文件、执行 map、shuffle、写中间块、reduce 输出。控制面需要强一致和可恢复，数据面需要吞吐和背压。把二者混在一起会导致系统又慢又难恢复。

Compute Systems Engine 的分布式模拟可以用一个 driver 作为简化控制面：它分配 partition，记录 task attempt，维护提交表。worker 数据面负责读输入、统计 key、发送 shuffle 消息。后续如果扩展，可以把 driver 状态写入日志或复制，但不必把每条日志记录都写入控制面共识。

17.21 备份、快照和恢复演练

复制不能替代备份，备份也不是只把文件复制到另一个目录。备份要有快照时间点、保留策略、隔离权限、校验和恢复演练。没有恢复演练的备份只是心理安慰。分布式系统中，备份还要保证多个分片之间的一致性时间点，或者记录足够日志让恢复后能达到一致状态。

计算系统也需要类似思想。长时间作业的 checkpoint 就是计算状态的备份。checkpoint 要能恢复，不只是写文件；恢复要验证输入 offset、算子状态、输出提交点和去重表是否一致。第 18 章会把它放进日志统计系统里。

17.22 分片是布局问题

分片可以看成数据布局在集群尺度上的版本。单机上，我们关心数组元素怎样落到 cache line 和 NUMA 节点；集群里，我们关心 key、文件块、任务和副本怎样落到机器。布局决定访问成本、负载均衡、故障域和扩容方式。一个错误分片策略会让后续复制、调度和缓存都很痛苦。

哈希分片把 key 均匀打散，适合点查和均匀 key，但不适合范围扫描，也不能自动处理热点 key。范围分片保持 key 顺序，适合范围查询和顺序扫描，但递增 key 容易打到最后一个分片，范围边界也可能因数据分布变化而倾斜。一致性哈希减少节点增减时迁移的数据量，适合缓存和某些存储系统，但虚拟节点数量、负载权重和热点仍要处理。没有一种分片策略能同时解决所有问题。

教学时要把访问模式列出来。点查、范围查、批量 scan、按时间写入、按用户聚合、按事件类型统计，它们需要的分片方式不同。日志统计系统如果按文件分片读取，map 阶段局部性好；shuffle 阶段按 key 分区，reduce 聚合好。一个作业里可能同时存在多种分片：输入分片、shuffle 分区、输出分区和检查点分区。每一种都要有理由。

17.23 路由缓存和失效

客户端通常会缓存路由表，否则每次请求都问控制面，控制面会成为瓶颈。缓存带来失效问题：分片迁移、leader 切换或节点故障后，客户端可能继续把请求发到旧位置。服务端收到旧路由请求时，不应静默执行，而应返回 stale route 或 redirect，让客户端刷新。

路由缓存要配版本。请求中带 route epoch，服务端比较自己的 epoch。若请求版本旧，返回新版本提示；若请求版本未来，说明客户端和服务端状态不一致，也不能盲目执行。版本检查是迁移期间避免双写和旧写的重要保护。

控制面还要考虑推送和拉取。推送能让客户端更快更新，但需要维护连接和处理丢通知；拉取简单，但更新延迟高。很多系统结合二者：错误时拉取刷新，后台周期性拉取，控制面也可以推送重大变更。教材项目可以先用错误触发刷新，保证语义清晰。

17.24 再分片期间的一致性

再分片最难的是迁移期间的新写入。历史数据复制只是第一步；复制期间旧分片仍可能收到写入。如果没有增量日志或双写，目标分片复制完历史数据后仍然落后。切换路由时，还要防止旧客户端继续写旧分片。一个完整再分片状态机必须同时处理历史复制、增量同步、路由切换和旧请求拒绝。

一种清晰策略是以日志位置为边界。准备阶段记录迁移开始位置；目标复制开始位置之前的历史数据；随后追赶从开始位置到切换位置的增量；切换时控制面发布新 epoch；旧分片拒绝旧 epoch 之后的写或转发到新分片；确认没有旧写后清理。这个策略需要每个分片有可重放日志或变更流。

另一种策略是短暂停写。它实现简单：停止旧分片写入，复制剩余数据，切换路由，再恢复写入。代价是可用性下降。对低写入量、维护窗口或教学系统，停写可能可以接受；对高可用服务，必须使用更复杂的在线迁移。工程选择要写清业务允许的停顿时间。

17.25 热点 key 的层次化处理

热点 key 不能只靠扩容解决，因为同一个 key 无论多少节点都可能落到一个分片。处理热点要分层。读热点可以用缓存、副本读、只读快照和 CDN；写热点可以用分片计数、局部聚合、异步批处理和业务限流；混合读写热点可能需要改变业务模型，例如把全局排行榜做成分桶近似，再周期性合并。

日志统计中的热点 key 是理想例子。若所有 map worker 都把同一个 key 发到同一个 reduce partition，reduce 长尾会很明显。可以把热 key 拆成 **key, shard**，让多个 reduce worker 分别合并，再做二级 reduce。这个方案会增加最终合并阶段，但能平摊第一阶段压力。它和单机 sharded counter 完全同构。

```
hot key split:
normal key: partition = hash(key) % reduce_count
hot key:    partition = hash(key, local_shard) % hot_reduce_count
final:      merge all shards of the hot key
```

关键是热 key 检测和正确性。热 key 可以通过采样、历史统计或运行时指标发现。拆分后，最终输出必须把所有子 key 合回原 key，且重复 attempt 不能重复计数。热点处理不是单纯性能优化，它改变了数据流图。

17.26 复制拓扑

复制不一定是简单一主多从。常见拓扑包括 leader-follower、链式复制、quorum 复制、多主复制和无主复制。Leader-follower 写路径清楚，冲突少，但 leader 是写入瓶颈。链式复制按固定顺序传递写入，尾节点确认，吞吐和延迟有不同权衡。Quorum 复制用多数或读写集合交集提高容错，但冲突处理复杂。多主复制提高可用性，但冲突解决困难。

教材不需要把每种协议细节都展开成论文，但要让读者理解选择维度：写延迟、读延迟、故障恢复、冲突处理、运维复杂度和业务一致性。对控制面元数据，leader-follower 加共识通常更合适；对缓存或日志指标，最终一致复制可能足够；对银行账本，弱冲突解决不可接受。

17.27 日志复制和快照压缩

状态机复制如果只保留无限日志，存储会增长，恢复也会越来越慢。快照用于把某个日志索引之前的状态压缩成状态文件，之后恢复时先加载快照，再重放快照之后的日志。快照必须和日志索

引绑定，不能只写一个状态文件却不知道它覆盖到哪里。

快照也有一致性边界。写快照时，状态可能仍在变化。可以暂停应用日志写快照，也可以使用 copy-on-write 或增量快照。快照写完后还要校验，确保恢复能读。很多系统的故障不是因为没有写日志，而是从未演练过快照恢复，真正崩溃后发现快照损坏或版本不兼容。

Compute Systems Engine 的简化版本可以把 driver 元数据写成 checkpoint：包含路由 epoch、task 状态、已提交 attempt 和 shuffle manifest。每次 stage 完成后写 checkpoint，恢复时读 checkpoint 并校验文件存在和摘要匹配。这个训练比空谈“做 checkpoint”更实用。

17.28 Quorum 的读写路径

用 N 表示副本数， W 表示写入需要确认的副本数， R 表示读取需要查询的副本数。如果 $R + W > N$ ，读写集合至少有交集，读有机会看到最新写。但“有机会”不等于自动返回正确值。读路径需要版本比较，可能读到多个版本；写路径需要处理并发写；删除需要 tombstone；故障恢复需要补数据。

例如 $N=3$, $W=2$, $R=2$ 。写入成功到两个副本后返回。读取两个副本，如果其中一个有新版本，另一个旧版本，客户端或协调者要选择新版本，并可能发起读修复。若两个并发写写到不同副本集合，版本冲突需要解决。若用最后写入时间胜出，时钟偏差可能丢数据。Quorum 是框架，不是完整数据库。

17.29 一致性模型的例子

线性一致要求每个操作看起来在调用和返回之间某个瞬间生效，并且所有客户端看到同一个全局顺序。它最容易给上层推理，但成本高。顺序一致要求所有操作有某个全局顺序且每个客户端程序顺序保持，但不要求实时顺序。因果一致保留因果相关操作顺序，不强制无关操作顺序。最终一致保证没有新写时副本最终收敛。

业务常常需要的是特定保证。用户修改密码后，自己必须读到新密码，这是读己之写；用户连续刷新页面，不应看到版本倒退，这是单调读；日志统计可以最终一致，但审计记录不能丢。教材要训练读者说具体保证，而不是笼统说“强一致”或“弱一致”。

17.30 租约的时间假设

租约允许某个节点在一段时间内拥有权利，例如 leader 读、本地缓存写入或分片所有权。租约依赖时间，因此必须处理时钟漂移、进程暂停和网络延迟。安全租约通常要求保守的时间边界：授予方认为租约到期之前，持有方必须更早停止使用；持有方不能因为本地时钟慢而延长权利。

进程暂停是租约的敌人。一个节点拿到 10 秒租约后暂停 30 秒，恢复时如果只看本地状态，可能以为自己仍有权利。正确系统要在使用租约前检查当前时间和任期，或者使用更保守的共识读。租约能优化读路径，但不能成为绕过一致性的借口。

17.31 控制面数据的规模

控制面适合维护少量关键元数据，不适合承载大数据流。把每条 shuffle 记录都写进共识日志，会让控制面被数据面拖垮。正确做法是控制面记录 stage、partition、attempt、manifest、提交点和路由，数据面传输大批量数据文件或消息。控制面决定“哪些数据有效”，数据面负责“搬运数据”。

这个边界和单机运行时类似。线程池控制面维护任务状态，任务函数处理大数组；队列控制面维护 head/tail，buffer 承载数据；分布式 driver 控制面维护 manifest，worker 数据面写中间块。系统设计的很多层次都在重复这个分离。

17.32 备份和复制的区别

复制用于可用性和读扩展，备份用于灾难恢复和人为错误恢复。若用户误删数据，复制会迅速把删除同步到所有副本；只有独立备份能恢复旧状态。若软件 bug 写坏数据，复制也会复制坏数据。备份必须隔离权限、保留历史版本、定期校验和演练恢复。

快照还要考虑一致点。多分片系统如果分别备份每个分片，恢复时可能得到不同时间点的状态。可以使用全局 checkpoint barrier，也可以记录每个分片的日志位置，恢复后重放到一致点。计算作业的 checkpoint 同样如此：输入 offset、算子状态和输出提交表必须对应同一个逻辑时间。

17.33 恢复演练流程

恢复演练应写成流程，而不是口头承诺。第一步，选择一个备份或 checkpoint。第二步，在隔离环境恢复元数据和数据文件。第三步，运行校验：行数、校验和、manifest、去重表、路由 epoch。第四步，执行一组只读查询或模拟任务，确认业务语义。第五步，记录恢复耗时和失败点。没有演练，就不知道 RTO 和 RPO 是否真实。

Compute Systems Engine 可以模拟 driver 重启：保存 checkpoint 后删除内存状态，重新加载 checkpoint，继续运行未完成任务。测试要覆盖 checkpoint 前崩溃、checkpoint 后响应丢失、重复 attempt 到达和 manifest 文件缺失。这样恢复不再是抽象概念。

17.34 成员变更

复制组不是永远固定。节点会扩容、缩容、维护、故障替换。成员变更比普通写入更危险，因为它改变 quorum 的定义。若旧配置和新配置没有安全交叠，可能出现两个不同多数派，各自认为自己能提交，造成脑裂。成熟共识协议会把成员变更作为日志中的配置变更，并使用联合共识或分阶段切换，确保新旧配置之间有交集。

教学阶段不必实现完整成员变更，但必须理解控制面不能随意改副本列表。一个 driver 如果把 partition 从 worker A 移到 worker B，要确保旧 worker 不再接受新 epoch 写入，新 worker 已经拥有必要状态，客户端路由刷新。成员变更是再分片、故障恢复和复制的一部分。

17.35 脑裂和 fencing

脑裂指两个或多个节点同时认为自己拥有同一资源的写权限。网络分区、旧 leader 暂停后恢复、租约误判都可能造成脑裂。解决脑裂需要 fencing：让旧持有者即使还活着，也无法继续对资源产生有效写入。常见 fencing token 是单调递增 epoch、term 或 lease version。下游资源只接受最新 token 的写入。

例如一个 worker 持有 partition epoch 5 的写权限。控制面故障切换后，把 partition 交给另一个 worker，epoch 变成 6。旧 worker 恢复后仍尝试提交 epoch 5 输出，driver 或存储层必须拒绝。没有 fencing，旧 worker 的迟到写会覆盖新结果。Epoch 不只是日志字段，它是防旧写的安全边界。

17.36 Leader 读和线性一致

Leader 处理写入并不自动意味着 leader 本地读一定线性一致。若 leader 已经失去领导权但还不知道，它可能返回过期读。解决方法包括读前确认自己仍拥有 quorum、使用 lease 且满足时间假设、或把读也走日志。不同方法在延迟和安全性之间取舍。

很多系统提供多种读：强一致读、leader 本地读、follower stale read、快照读。业务应选择所需语义。配置和权限通常需要强一些；监控和推荐可以接受旧读；日志统计作业读取 checkpoint manifest 时要确保 manifest 已提交。教材要训练读者把读语义说清，而不是只关注写复制。

17.37 冲突解决

无主或多主复制会遇到并发写冲突。冲突解决可以是最后写入胜出、版本向量、CRDT、业务合并或人工处理。最后写入胜出简单但可能丢数据，且依赖时钟；版本向量能检测并发但元数据更大；CRDT 适合某些可合并数据类型，例如集合、计数器和寄存器变体；业务合并需要理解语义。

日志统计中的计数器是可合并的，多个副本的增量可以求和；订单状态不是简单可合并，两个并发状态变更可能冲突。数据类型决定复制策略。不要把最终一致当成一个开关，它需要冲突语义。

17.38 删除和 Tombstone

复制系统中，删除不能简单地从某个副本移除数据。若一个副本离线时错过删除，恢复后可能把旧数据重新传播回来。Tombstone 用一个删除标记表示“这个 key 已被删除”，让其他副本也能学习删除。Tombstone 需要保留一段时间，直到确认旧副本不会再带回归值。

Tombstone 的代价是空间和查询成本。保留太短，旧数据复活；保留太长，存储膨胀。后台 compaction 会清理安全 tombstone，但必须知道所有副本或日志位置已经越过删除点。这个细节说明复制系统的“删除”比单机 map erase 复杂得多。

17.39 反熵和 Merkle tree

副本之间可能因为故障、丢包或离线而不一致。反熵任务在后台比较副本并修复差异。直接比较所有数据成本高，Merkle tree 可以把数据分块计算哈希，先比较上层哈希，不同再下钻到具体范围。它适合大规模 key-value 副本修复。

反熵是最终一致系统的重要补充。读修复只修复被读到的 key，冷数据可能长期不一致；反熵后台扫描能逐步收敛，但消耗 I/O 和网络。调度反熵时要避免影响前台请求。Compute Systems Engine 的简化恢复可以用 manifest checksum 类似思想：先比较摘要，再决定是否重读 block。

17.40 检查点和日志截断

复制日志不能无限增长。生成快照后，可以截断快照索引之前的日志，但前提是所有需要恢复的节点都已经拥有快照或不再需要旧日志。若 follower 落后太多，leader 可能发送快照而不是从很旧日志补起。日志截断是存储空间和恢复能力的平衡。

计算作业也有类似问题。Stage 完成并 checkpoint 后，是否可以删除中间 shuffle 文件？如果后续 stage 失败需要回滚，删除太早会导致无法恢复；保留太久占用磁盘。系统要定义保留策略：保留到下一个 checkpoint 成功，保留到作业完成，还是按时间清理。清理也是协议的一部分。

17.41 配置变更的审计

控制面配置变更更可审计。谁把 partition 从 A 迁到 B，路由 epoch 多少，何时生效，旧副本何时清理，是否完成校验，都应记录。没有审计，出现数据错乱时无法追踪。分布式系统的操作日志和业务日志同样重要。

本书项目可以把 driver 的每次状态变更写入简单事件日志：assign partition、start attempt、commit attempt、publish manifest、checkpoint、recover。这个日志用于调试和恢复演练，不需要一开始就高性能。先有可见性，再谈优化。

17.42 一致性选择表

为业务选择一致性时，可以列一个表：数据类型、读写比例、是否允许旧读、是否允许冲突、丢失风险、延迟要求、恢复要求。比如路由表：小、读多写少、不允许旧写、需要强一致；日志指标：大、写多读少、允许短暂旧读、可合并；输出 manifest：小、提交关键、不允许重复、需要持久化。表格化思考能避免一句“用强一致”覆盖所有数据。

Compute Systems Engine 的不同状态也应有不同保证。任务状态和 manifest 需要强提交；worker 运行指标可以最终一致或近似；日志统计中间 partial 可以重复生成但最终提交要去重；调试 trace 可以丢弃低优先级事件。把所有状态都放进同一个强一致路径，会让系统过慢；把所有状态都弱一致，会让结果不可信。

17.43 分区元数据的数据结构

分区元数据至少包含 partition id、key range 或 hash range、leader、followers、epoch、状态和迁移目标。状态可以是 Serving、MigratingOut、MigratingIn、ReadOnly、Offline。请求处理前先检查 partition 是否 serving 且 epoch 匹配。迁移期间旧 partition 可以进入 ReadOnly 或 Forwarding，避免继续接受旧写。

Listing 17.1 分区元数据的教学形态

```
1 enum class PartitionState : std::int32_t {
2     Serving,
3     MigratingOut,
4     MigratingIn,
5     ReadOnly,
6     Offline,
7 };
8
9 struct PartitionMetadata {
```

```

10     std::uint32_t partition_id = 0;
11     std::uint64_t epoch = 0;
12     std::uint32_t leader_worker = 0;
13     PartitionState state = PartitionState::Offline;
14 };

```

这段结构没有包含完整副本列表和 key range，但展示了关键字段。Epoch 和 state 是防旧请求的基础。若请求只带 partition id，不带 epoch，服务端很难区分旧路由请求和当前合法请求。

17.44 迁移校验

再分片或副本迁移完成后，需要校验。校验可以比较记录数、字节数、checksum、Merkle tree 或抽样查询。没有校验就切换路由，可能把不完整副本变成主副本。校验成本可以很高，因此系统常在后台增量校验，但切换关键状态前至少要有基本一致性检查。

Compute Systems Engine 的迁移简化为 shuffle manifest 校验。Map 输出 block 写完后记录 checksum，reduce 读取前校验；checkpoint 恢复时再次校验。这个小机制训练的是同一思想：状态切换前证明数据完整。

17.45 Read repair 的成本归属

读修复把副本修复放到读请求路径中。读者发现旧副本后，返回结果同时发修复写。这样冷数据可能长期不修复，热数据很快收敛。缺点是读路径延迟和写放大增加。若前台延迟敏感，读修复要异步或限流；若一致性要求高，可能需要同步修复或强读。

反熵把修复放后台，前台读更稳定，但收敛慢。两者可以同时使用。教材要让读者看到：一致性维护成本总要付，要么前台付，要么后台付，要么接受更长不一致窗口。

17.46 租约和缓存失效

客户端缓存路由表或配置时，可以用租约限制缓存有效期。租约过期后必须刷新。租约能减少控制面查询，但引入时间假设。另一种方式是版本检查：客户端可以长期缓存，但每个请求带版本，服务端发现旧版本就拒绝。版本检查不依赖客户端准时过期，更稳，但每次请求都要服务端判断。

很多系统结合二者：正常情况下按租约周期刷新，异常时根据 stale route 错误立即刷新。这样控制面负载低，迁移时也能快速纠错。Compute Systems Engine 的 route epoch 就是版本检查的简化形式。

17.47 副本滞后指标

复制系统应记录副本滞后。日志复制可以用 last log index 差距，异步复制可以用字节或时间，计算 checkpoint 可以用 stage 或 task 数。没有滞后指标，读旧副本是否安全无法判断，故障切换时也不知道候选副本差多少。副本滞后还影响备份和清理：落后副本可能需要旧日志，不能过早截断。

指标要按 partition 维度记录。全局平均滞后小，不代表每个分区都健康。一个热点 partition 的 follower 落后，故障时就会影响关键数据。分片系统的一切指标都应带 partition 维度。

17.48 共识的使用边界

共识很强，但不应滥用。适合放进共识的是少量关键决策：leader、路由、配置、任务提交、manifest 指针。不适合放进共识的是大批数据记录、每条日志原文、每个中间 partial。把所有东西都放进共识会让系统吞吐被最慢副本和日志复制限制。

正确模式是：共识决定哪份数据有效，数据本身走高吞吐路径。比如 driver 通过强一致状态记录某个 shuffle block 已提交，block 内容存储在文件或对象存储中；控制面记录 block 的 checksum 和路径，数据面负责传输和读取。这种控制面/数据面分离是大型系统的基本结构。

17.49 操作手册

分片复制系统需要操作手册。节点下线前如何迁移分片，节点故障后如何提升副本，路由错误如何回滚，checkpoint 损坏如何恢复，热点 key 如何拆分，旧副本何时清理。没有操作手册，系统只能在正常路径运行，一遇到故障就靠临场判断。教材项目可以写简化 runbook，训练工程思维。

Runbook 应包含前置条件、执行步骤、验证指标、回滚方案和风险。比如“迁移 partition 7”：确认目标 worker 健康，创建迁移任务，复制数据，校验 checksum，发布 epoch，观察 stale route 错误下降，清理旧副本。这个流程比单独讲算法更接近真实工程。

17.50 分片复制章节总结

分片是集群尺度的数据布局，复制是失败下保持服务和数据的机制，共识是少量关键决策的安全顺序。分片要考虑访问模式和热点，复制要定义提交点和读语义，共识要限制在控制面，恢复要靠快照、日志、校验和演练。一个系统能否可靠，不取决于是否“有副本”，而取决于旧写是否被挡住、提交点是否清楚、恢复是否演练过。

17.51 分片复制决策表

选择分片和复制策略时，先问：查询是点查还是范围，写入是否递增，是否有热点 key，是否允许旧读，写成功后允许丢失吗，故障切换能等待多久，副本是否跨故障域，删除如何传播，备份如何恢复。每个答案都会改变设计。哈希分片、范围分片、同步复制、异步复制、quorum、Raft、租约、读修复都不是孤立选项，而是这些约束的组合结果。

17.52 复制协议审查表

每个复制协议都应审查写路径、读路径、故障路径和恢复路径。写路径：写到哪里算成功，是否写 WAL，是否多数确认。读路径：读 leader 还是 follower，是否可能旧读，如何保证读己之写。故障路径：旧 leader 如何被 fencing，慢副本如何处理，成员变更如何保证 quorum 交叠。恢复路径：快照和日志如何重放，tombstone 保留多久，备份如何演练。缺任何一项，复制都不完整。

17.53 本章落到贯穿项目

Compute Systems Engine 在本章应加入分片元数据和提交表。分片元数据描述 key 到 partition 的映射、partition 到 worker 的映射和版本号。提交表记录 task attempt、request id、输出文件或结果摘要。第一版可以由 driver 单进程维护，后续再讨论复制和持久化。

项目还要实现热点 key 处理实验。构造一个 key 分布高度倾斜的输入，比较普通 hash partition、map-side combine、热 key 拆分和二级合并。这个实验能把单机 sharded counter、分布式 partition、shuffle 和 reduce 长尾串起来。

17.54 本章习题

习题与作业

习题一：为一个 key-value 系统选择分片策略。分别讨论点查、范围查询、递增 key 写入和热点 key 下哈希分片与范围分片的表现。

习题二：设计再分片状态机。至少包含准备、复制、追增量、切换、清理五个状态。说明每个状态失败后如何重试。

习题三：比较三种写提交点：leader 内存返回、本地 WAL 返回、多数副本返回。分别说明延迟、数据丢失风险和读路径影响。

习题四：用 Raft 直觉解释任期号、日志索引和多数提交分别解决什么问题。不要写协议伪代码，重点写工程意义。

习题五：给日志统计 shuffle 设计热点 key 拆分方案。说明如何拆、如何二级合并、如何保持幂等提交。

实验：设计一个三副本 key-value 存储的写入流程。分别画出同步多数写、异步主从写和读修复流程，并说明各自的延迟和失败行为。

第 18 章

分布式计算工程实践

分布式计算工程把算法、运行时、存储、网络 and 观测性结合起来。真正的系统不仅要在正常路径跑得快，还要在机器慢、网络抖动、任务失败和数据倾斜时保持可诊断、可恢复。

18.1 Shuffle

Shuffle 把数据按 key 重新分布，是很多分布式计算框架的核心成本。它包含序列化、压缩、网络传输、落盘、排序、合并和反序列化。一个 group-by 或 join 的成本，常常主要来自 shuffle，而不是用户函数。

优化 shuffle 的方向包括减少数据量、提前聚合、压缩、分区均衡、避免热点 key、使用本地磁盘顺序写和控制并发拉取。它是分布式版的数据布局问题。

Shuffle 的基本流程可以拆成 map 端分区、局部排序或缓冲、落盘、网络拉取、reduce 端合并。每一步都有独立瓶颈：序列化消耗 CPU，压缩节省网络但增加计算，落盘受磁盘带宽影响，拉取并发过高会压垮对端，key 倾斜会让少数 reduce 任务变成长尾。优化 shuffle 不能只调一个参数，要先确认慢在哪一步。

18.2 检查点和容错

长任务必须考虑失败。检查点保存中间状态，让失败后从最近点恢复，而不是从头开始。检查点太频繁会拖慢正常路径，太稀疏会增加恢复成本。流式计算还要处理 exactly-once 或 at-least-once 语义。

检查点设计要说明一致性边界。批处理任务可以在 stage 边界保存输出，失败后重跑某个分区；流式任务需要同时保存输入 offset、算子状态和输出提交位置。若只保存算子状态，不保存已经输出到哪里，恢复后可能重复输出；若只保存 offset，不保存状态，聚合结果会丢失。所谓 exactly-once 往往不是“消息真的只执行一次”，而是通过幂等提交、事务输出或去重让外部效果等价于一次。

18.3 观测性

没有观测性，分布式系统无法维护。日志、指标、trace、profile 和事件审计要能回答：请求经过哪些节点，在哪里等待，重试了几次，队列多长，哪个分片慢，哪个任务倾斜，哪次部署引入变化。

观测性要围绕问题设计。延迟问题需要端到端 trace 和每段耗时；吞吐问题需要输入速率、处理速率和队列长度；倾斜问题需要按 key、分片、worker 的分布；可靠性问题需要错误码、重试、超时和恢复路径；资源问题需要 CPU、内存、磁盘、网络 and GC 或分配器指标。只打印一行“任务失败”没有工程价值。

18.4 从单机原则到集群原则

第二册前面讲的原则在分布式系统中仍然成立。局部性对应数据本地调度；锁竞争对应热点 key；false sharing 对应多个任务争用同一分片；背压对应限流和队列控制；归约对应 map-side combine 和树形聚合；缓存一致性对应复制一致性和失效策略。系统规模变大，原则没有消失，只是成本单位从 cache line 变成网络消息和磁盘块。

18.5 一个贯穿工程案例

考虑一个日志统计系统：输入日志按文件分片放在多台机器上，每个 worker 读取本地分片，解析出 key，先做本地 hash 聚合，再把 partial 按 key 分区 shuffle 到 reduce worker，最后合并并写出结果。这个系统把第二册几乎所有原则串在一起。本地解析要考虑 CPU 和内存布局，本地聚合要避免锁热点，shuffle 要处理网络和背压，reduce 要处理 key 倾斜，输出要保证幂等，失败后要能从检查点恢复。

如果某个 key 特别热，单机里它像一个热点锁，分布式里它像一个热点分片。解决方法也相似：先把热点拆成多个局部 partial，再在后续阶段合并。若网络拉取过多，像单机任务队列无背压一样，系统会排队爆炸。若 worker 失败后重跑，输出必须用任务 ID 和分区 ID 保证幂等提交。这样的案例比孤立讲术语更能训练系统思维。

核心思想

分布式计算不是单机计算的替代，而是单机计算原则在通信、故障和复制约束下的扩展。

18.6 贯穿项目的分布式形态

Compute Systems Engine 在第二册最后要变成一个本机可运行的分布式计算模拟。它不追求替代 Spark 或 Flink，而是把前面章节的核心原则压进一个可测、可失败、可恢复的小系统。输入是一批日志记录，map worker 读取本地分片并解析 key，map 端先做本地聚合，shuffle 把 partial 按 key 分发给 reduce partition，reduce worker 合并结果，driver 负责任务分配、attempt、检查点和最终提交。

这个项目的价值在于贯穿。顺序 reference 来自第一批归约；数据布局来自热字段和冷字段分离；SIMD 和 map 内核用于解析和过滤；scan 和 partition 用于分配 shuffle 输出；bounded queue 和背压用于阶段之间限流；线程池和任务图用于本机执行；幂等和 request id 用于重试；检查点用于恢复；观测性用于解释慢在哪里。它不是另起炉灶，而是把整本书串起来。

18.7 数据格式和 reference

任何分布式项目都要先有可验证 reference。日志记录可以简化成三列：timestamp、user id、event id。reference 单线程读取全部输入，按 event id 计数，输出排序后的计数结果。分布式模拟的最终输出必须和 reference 一致。只有 reference 清楚，后面的重试、分区、合并和检查点才有正确性锚点。

Listing 18.1 日志记录和聚合结果的简化类型

```
1 struct LogRecord {
2     std::uint64_t timestamp = 0;
3     std::uint64_t user_id = 0;
4     std::int32_t event_id = 0;
5 };
6
7 struct EventCount {
8     std::int32_t event_id = 0;
9     std::int64_t count = 0;
10 };
```

这里用明确位宽类型，是为了让文件格式、网络消息和内存布局都可解释。真实系统还会处理字符串、变长字段和 schema 版本，但教学项目先用固定字段，能把注意力放在计算和系统语义上。

18.8 Map 端本地聚合

朴素 shuffle 会把每条日志都发送到 reduce 端。若一亿条日志中只有一万个 event id，这样会浪费大量网络和序列化成本。map-side combine 先在每个 worker 本地对 event id 计数，再发送 partial。这样发送量从“记录数”下降到“本地出现过的 key 数”。这是 reduce 原则在网络尺度上的应用。

本地聚合可以用哈希表，也可以在 event id 范围小且密集时用数组。哈希表适合稀疏 key，但有随机访问、分配和 rehash 成本；数组适合密集小 key，连续且容易向量化，但 key 空间大时浪费内存。教材项目可以同时保留两种策略，让读者根据 key 分布选择。

18.9 Shuffle 文件和网络缓冲

真实 shuffle 常把 map 输出按 partition 写成多个缓冲或文件。每个 reduce partition 拉取属于自己的数据。即使本机模拟，也可以按 partition 组织缓冲：map worker 产生 `partition_id->`

partials, message bus 按 partition 投递给 reduce worker。这样读者能看到 shuffle 的核心结构, 而不用先搭真实网络。

Shuffle 的成本包括序列化、压缩、缓冲、落盘、发送、接收和合并。每一步都可以成为瓶颈。压缩减少网络字节但增加 CPU; 落盘提高容错但增加 I/O; 内存缓冲快但可能 OOM; 并发拉取提高吞吐但可能压垮 map 端。优化前必须先记录每一步的字节数和耗时。

18.10 Reduce 端合并和热点 key

Reduce 端收到多个 map worker 的 partial 后, 按 key 合并。若 key 分布均匀, 每个 reduce partition 工作量接近; 若某个 key 极热, 它所在 partition 会成为长尾。热点 key 的处理通常要二级聚合: 先把热 key 拆成多个子分区, 让多个 reduce worker 合并 partial, 再把这些结果做最后合并。

这和单机 sharded counter 完全对应。全局 hot key 就像全局原子计数器; map-side combine 是线程本地 partial; 热 key 拆分是 sharded counter; 最终合并是读取总数。第二册前面的并发原则在这里直接变成分布式算法。

18.11 任务 attempt 和幂等输出

分布式任务可能失败重试, 因此一个逻辑 task 可能有多个 attempt。输出提交必须按逻辑 task 去重, 不能让两个 attempt 都提交成功。常见做法是每个 task 输出到 attempt 临时位置, 完成后由 driver 或 commit protocol 原子地把某个 attempt 标记为成功。其他 attempt 即使晚完成, 也不能覆盖已提交结果。

Listing 18.2 任务提交元数据

```
1 enum class TaskState : std::int32_t {
2     Pending,
3     Running,
4     Committed,
5     Failed,
6 };
7
8 struct TaskAttempt {
9     std::uint64_t task_id = 0;
10    std::uint32_t attempt = 0;
11    TaskState state = TaskState::Pending;
12 };
```

这个模型同样适用于本机模拟。worker 完成后发送 commit 请求, driver 检查 task id 是否已经 committed。若没有, 则提交该 attempt; 若已经提交, 则丢弃重复结果并记录 duplicate attempt。这个机制训练 exactly-once 外部效果的基本思想。

18.12 检查点的内容

检查点不能只保存“程序运行到第几步”。对于日志统计系统, 检查点至少应包含输入分片进度、已完成 map task、shuffle 输出位置、reduce partition 状态、已提交输出和请求去重表。缺少任何一项, 恢复后都可能漏处理或重复输出。

检查点频率要权衡。每处理一条记录就 checkpoint, 正常路径太慢; 只在最后 checkpoint, 失败恢复成本太高。批处理可以在 stage 边界 checkpoint, 例如 map 输出全部完成后记录 shuffle manifest, reduce 输出提交后记录最终结果。流式系统则需要周期性 checkpoint, 并把输入 offset、状态和输出提交绑定。

18.13 Exactly-once 的工程含义

Exactly-once 常被误解成“每条消息真的只执行一次”。在真实系统中, 消息可能重复, 任务可能重跑, 函数可能执行多次。工程上追求的是外部效果等价于一次: 重复执行被幂等提交、去重表

或事务输出吸收。理解这点非常重要，否则读者会在网络不可靠时追求不可能的语义。

对于日志统计，若同一个 map attempt 输出被 reduce 合并两次，计数会翻倍。解决方法是 reduce 端按 `task_id, attempt` 或 logical map id 去重，或者 driver 只让一个 attempt 的输出进入 manifest。提交协议要定义清楚哪些输出属于最终结果，哪些只是临时文件。

18.14 调度和数据本地性

Driver 调度 map task 时，应优先把任务放到拥有输入分片的 worker；若本地 worker 忙或失败，可以远程调度。Reduce task 则按 partition 分配，尽量让同一 partition 的数据由同一个 worker 合并，减少跨节点状态移动。若某个 reduce partition 太大，可以拆分或引入二级 reduce。

调度还要考虑 speculative execution。若某个 task 明显慢，可以启动另一个 attempt。第一个完成并提交的 attempt 获胜，另一个 attempt 的结果丢弃。Speculation 能处理慢节点，但会增加资源消耗和重复输出风险。没有幂等提交，不应开启 speculation。

18.15 背压贯穿 shuffle

Shuffle 最容易出现过载。Map 端产生数据太快，reduce 端处理不过来，网络缓冲和队列增长。正确做法是让 reduce 端容量反馈到 map 端：每个 reduce partition 有接收队列容量，队列满时 map 端暂停、降低发送速率或批量合并。若 map 端无限发送，最终只是把 OOM 从一个节点转移到另一个节点。

背压指标包括每个 partition 的队列长度、入站字节率、处理字节率、拒绝或 busy 次数、map 端等待时间和端到端延迟。没有这些指标，shuffle 慢时只能猜。第 15 章的有界队列实验在这里变成跨 worker 流控。

18.16 观测性设计

一个可维护的分布式计算系统至少要回答七个问题：哪个 stage 慢，哪个 task 慢，哪个 worker 慢，哪个 partition 数据最多，哪个 key 最热，重试发生在哪里，输出提交是否重复。指标要按 stage、worker、partition、task attempt 和 key 分布聚合。

日志要包含 task id、attempt、partition、request id 和错误码。Trace 要能从 driver 调度追到 worker 执行、shuffle 发送、reduce 合并和输出提交。Profile 要能分辨 CPU 解析、hash 聚合、序列化、压缩、网络等待和磁盘写入。观测性不是最后加的，它决定系统是否能被调试。

18.17 故障注入

没有故障注入的分布式系统测试是不完整的。项目应主动注入：随机丢消息、延迟消息、重复消息、让 worker 中途失败、让 reduce 队列变慢、让某个 key 极热、让磁盘写入失败、让 commit 响应丢失。每个故障都应有预期行为：重试、去重、背压、恢复或明确失败。

故障注入要可重复。使用固定随机种子，记录注入事件，失败时能重放。否则低概率 bug 很难定位。系统工程的测试不只看 happy path，更要让失败路径成为日常。

18.18 本机模拟实现路线

第一阶段，只实现串行 reference 和单进程多 worker 模拟。输入切成分片，worker 本地聚合，driver 合并所有结果。第二阶段，加入 shuffle partition，每个 map worker 把 partial 发到 reduce partition 队列。第三阶段，加入 request id 和 attempt，支持重复消息去重。第四阶段，加入 checkpoint manifest，失败后从 manifest 恢复。第五阶段，加入热点 key 拆分和背压。

每阶段都保留测试。第一阶段测试结果正确；第二阶段测试 partition 后结果仍一致；第三阶段测试重复消息不重复计数；第四阶段测试崩溃恢复；第五阶段测试热点和过载下系统不会无限排队。这样项目逐步长大，不会变成一次性堆代码。

18.19 端到端 reference

分布式计算项目必须先有端到端 reference。对日志统计来说，reference 可以是单线程读取所有记录，解析出 key，然后用一个 map 聚合计数，最后按 key 排序输出。它不追求性能，只定义语

义：哪些行有效，解析失败如何处理，key 如何规范化，计数类型是什么，输出顺序如何确定。后续 map-side combine、shuffle、reduce、热点拆分和重试都要和它对比。

Reference 还要固定输入格式。不要一边开发分布式执行器，一边让输入格式隐式变化。可以定义每行包含 timestamp、user id、event type 和 value；解析失败的行进入错误计数；event type 作为聚合 key；value 使用 `std::int64_t` 累加。若后续加入更多字段，也要版本化。计算系统项目不是只练线程和网络，也要练数据合同。

Listing 18.3 日志记录的教学格式

```
1 struct LogRecord {
2     std::uint64_t timestamp = 0;
3     std::uint64_t user_id = 0;
4     std::uint32_t event_type = 0;
5     std::int64_t value = 0;
6 };
```

这段结构体用于内存中的解析结果，不等于文件格式。文件格式可以是文本或二进制，但解析后进入稳定内存结构。所有优化版本都以同样的 `LogRecord` 或等价列式布局作为输入，避免把解析差异误当成执行器差异。

18.20 输入分片和 offset

按文件分片时，不能随意从字节中间开始解析一行。若每个 worker 负责文件字节范围，分片边界要调整到行边界：非零 begin 先跳到下一行起点，end 可以读到当前行结束。每个分片记录文件名、begin offset、end offset 和分片 ID。这样失败恢复时可以重新读取同一范围。

如果输入是二进制块，分片边界要按记录格式对齐。若记录变长，就需要索引或块目录。不要把“把文件切成 N 份”说得过于简单。分片正确性决定每条记录是否恰处理好一次。漏一行和重复一行都会让最终计数错误。

Listing 18.4 输入分片元数据

```
1 struct InputSplit {
2     std::uint64_t split_id = 0;
3     std::uint64_t file_id = 0;
4     std::uint64_t begin_offset = 0;
5     std::uint64_t end_offset = 0;
6 };
```

`split_id` 是逻辑任务 ID 的基础。即使 attempt 重试，`split_id` 不变。输出提交时按 `split_id` 去重，而不是按 attempt 去重。这个原则贯穿整个项目。

18.21 Map-side combine 的内存设计

Map worker 不应每读一条记录就发一条网络消息。它应在本地按 key 聚合，形成 partial map，再按 reduce partition 输出。这样减少网络消息数量和字节数，也把热点 key 的一部分压力留在本地。Map-side combine 是分布式版的线程本地 partial。

本地聚合的数据结构要根据 key 空间选择。event type 如果是小整数，可以使用 vector buckets；如果 key 是字符串或大整数，可能使用哈希表；如果输入已经按 key 排序，可以顺序合并。教材项目可以从小整数 key 开始，后续扩展到字符串 key。无论使用哪种结构，都要记录内存峰值和 key 数量，否则大输入可能让 map 端内存失控。

Listing 18.5 小整数 key 的本地聚合

```
1 std::vector<std::int64_t> combine_events(
2     std::span<const LogRecord> records,
```

```

3     std::uint32_t event_type_count) {
4     std::vector<std::int64_t> partial(event_type_count, 0);
5     for (const LogRecord& record : records) {
6         partial[record.event_type] += record.value;
7     }
8     return partial;
9 }

```

生产代码要检查 `event_type` 越界，并处理解析错误。教学片段突出 combine 思想。局部聚合后的数据再按 partition 打包，发送给 reduce。

18.22 Shuffle 文件和消息

Shuffle 可以走内存消息，也可以写本地文件再由 reduce 拉取。内存消息简单，适合本机模拟；文件 shuffle 更接近大规模批处理，因为中间数据可能超过内存，也需要失败恢复。无论哪种方式，都要有 manifest 描述哪些 map 输出存在、属于哪个 partition、来自哪个 attempt、大小和校验是什么。

Manifest 是控制面，shuffle 数据是数据面。Reduce 不应扫描目录猜哪些文件有效，而应读取 driver 提交的 manifest。这样重复 attempt 产生的临时文件不会被误读；失败 attempt 的残留文件也不会进入结果。控制面决定有效数据集合，是 exactly-once 外部效果的关键。

Listing 18.6 Shuffle manifest 条目

```

1 struct ShuffleBlock {
2     std::uint64_t map_task_id = 0;
3     std::uint32_t attempt = 0;
4     std::uint32_t reduce_partition = 0;
5     std::uint64_t byte_size = 0;
6     std::uint64_t checksum = 0;
7 };

```

真实系统还会记录文件路径、压缩格式、记录数、写入时间和副本位置。教学项目至少要有 map task、attempt、partition、大小和 checksum。恢复时，driver 读取 manifest 后校验每个 block 是否存在且摘要匹配。

18.23 Reduce 端去重

Reduce 端不能简单地把收到的每条 partial 都加进去。若 map attempt 因响应丢失而重试，reduce 可能收到两个内容相同的 partial。正确策略是按逻辑 map task 去重，或者只接受 manifest 中被 driver 选中的 attempt。若 reduce 直接接收推送消息，也要维护已处理 request id 或 map task id 集合。

去重的粒度要慎重。按 attempt 去重只能防止同一 attempt 重复发送，不能防止不同 attempt 的同一逻辑 task 重复合并。按 map task id 去重能保证一个逻辑输入分片只贡献一次，但要处理 attempt A 部分发送后失败、attempt B 重试成功的情况。更稳的做法是 attempt 输出先写临时 block，driver 只在完整成功后把某个 attempt 加入 manifest，reduce 根据 manifest 拉取。

18.24 输出提交协议

最终输出也需要提交协议。Reduce task 不应直接写最终文件名，因为失败重试可能留下半文件，多个 attempt 可能互相覆盖。常见做法是每个 attempt 写到临时路径，写完并校验后向 driver 提交。Driver 检查该 reduce partition 是否已经提交；若未提交，原子发布该 attempt 的输出；若已提交，拒绝重复 attempt 并清理临时文件。

在本地文件系统里，可以用 rename 模拟原子发布。跨对象存储或分布式文件系统时，rename 语义可能不同，需要专门 commit protocol。教材项目可以先用本地目录，但必须在文字中说明提交

点：只有 driver manifest 记录的输出才是有效最终结果，临时文件不是。

18.25 Checkpoint 内容清单

本项目 checkpoint 至少包含六类内容。第一，输入 splits 和每个 split 的任务状态。第二，map attempt 和已提交 shuffle block manifest。第三，reduce partition 的状态和已提交输出。第四，路由或 partition 版本。第五，请求去重表或其可恢复摘要。第六，作业配置和输入数据摘要。缺少作业配置，恢复后可能用不同 reduce 数重新解释旧 shuffle；缺少去重表，重复消息可能再次生效。

Checkpoint 还要有版本。项目代码升级后，旧 checkpoint 如何读取？教学阶段可以要求版本不兼容时拒绝恢复，并提示重新运行。成熟系统需要迁移工具。不要把 checkpoint 当作临时 dump，它是持久化协议。

18.26 故障注入驱动开发

分布式项目应从一开始就有故障注入，而不是最后补测试。每个阶段都加入一个最小故障。Map 阶段加入任务失败重试；shuffle 阶段加入重复消息；manifest 阶段加入响应丢失；reduce 阶段加入慢 partition；checkpoint 阶段加入 driver 重启。这样协议随功能一起成长。

故障注入要可配置。可以在 MessageBus 中按 request id 决定是否丢弃，也可以在 worker 执行到某个 split 时故意失败。随机故障用于压力测试，确定性故障用于单元测试。单元测试应能精准复现“attempt A 写完但提交响应丢失，attempt B 后完成”这种场景。

18.27 热点 key 实验设计

热点 key 实验不能只说“构造倾斜输入”。要定义分布。例如 90% 的记录使用 event type 1，其余 10% 均匀分布到其他 key。运行普通 hash partition，记录每个 reduce partition 的输入字节、记录数、处理时间和队列等待。然后启用 map-side combine，再启用热 key 拆分和二级 reduce，对比长尾是否下降。

还要记录网络流量。Map-side combine 可能大幅减少消息数；热 key 拆分可能增加最终二级合并流量，但降低第一阶段长尾。报告要解释总吞吐、尾延迟和资源消耗之间的取舍。不要只写“热点拆分更快”，要说明在哪个阶段更快，代价是什么。

18.28 数据倾斜和任务倾斜

数据多不一定任务慢，任务慢也不一定数据多。某些 key 处理逻辑复杂，某些记录解析成本高，某些分区落在慢节点，某些节点 I/O 慢。观测性要区分数据倾斜和执行倾斜。每个 task 记录输入记录数、输入字节、输出字节、CPU 时间、排队时间、重试次数和所在 worker。只有这样，慢任务才能归因。

调度策略也不同。数据倾斜可以通过重新分区、热点拆分和二级聚合处理；执行倾斜可能需要 speculative execution、节点隔离或资源调整；I/O 倾斜可能需要数据本地性和预取；解析倾斜可能需要改变输入格式。系统工程不能把所有慢都归为“节点慢”。

18.29 资源隔离

分布式计算作业里，map、shuffle、reduce、checkpoint 和指标上报共享 CPU、内存、磁盘和网络。若 shuffle 发送占满网络，checkpoint 可能超时；若 reduce 哈希表占满内存，map 输出队列可能 OOM；若指标日志太多，磁盘写入干扰输出提交。资源隔离要明确每个阶段的上限。

本机模拟可以用简单参数表达：每个 reduce partition 最大队列字节数，map 端最大 in-flight shuffle 字节数，每个 worker 最大本地聚合内存，checkpoint 写入频率和并发数。达到上限时触发背压，而不是继续申请内存。资源上限是系统稳定性的组成部分。

18.30 压缩和编码

Shuffle 数据常需要压缩。压缩减少网络和磁盘字节，增加 CPU。若 reduce 端是网络瓶颈，压缩有益；若 map 端 CPU 已满，压缩可能变慢。编码也类似：小整数 key 可以用定长数组，稀疏 key 可以用 varint 或字典，字符串 key 可以先 intern 成 ID。压缩格式要进入 manifest，否则恢复和跨

版本读取会出错。

实验应比较无压缩、轻量压缩和更强压缩。指标包括 map CPU 时间、shuffle 字节、reduce 解压时间、端到端时间和内存峰值。不要把压缩当作固定开关，要让读者看到它是 CPU 与 I/O 的成本交换。

18.31 本机到多机的迁移边界

本机模拟验证语义，但不能证明真实多机性能。迁移到多机时，会新增真实网络延迟、连接管理、序列化成本、内核缓冲、TLS、时钟漂移、节点故障和部署问题。模拟中一个队列 push 可能微秒级，真实 RPC 可能毫秒级；模拟中进程共享时钟，真实机器时钟不一致；模拟中磁盘路径可靠，真实对象存储有不同一致性语义。

因此项目文档要把“已验证”和“未验证”分开。已验证：去重、manifest、提交协议、背压状态机、故障恢复逻辑。未验证：真实网络吞吐、跨机时钟、对象存储 commit、节点安全、部署滚动升级。成熟工程知道自己的证据边界。

18.32 代码组织建议

Compute Systems Engine 应按责任分模块。reference 模块只做串行正确性。input 模块处理分片和解析。map 模块做本地聚合。shuffle 模块处理 partition、消息、manifest 和背压。reduce 模块合并 partial。driver 模块维护任务状态、attempt 和 checkpoint。fault 模块提供故障注入。metrics 模块记录指标。不要把所有逻辑写进一个 main 函数，否则后续实验无法扩展。

接口要围绕数据合同，而不是围绕演示脚本。Map worker 接收 InputSplit，输出 ShuffleBlock；Reduce worker 接收 partition id 和 manifest blocks，输出 OutputBlock；Driver 只看 task id、attempt 和状态，不关心内部聚合代码。模块边界越清楚，故障注入和测试越容易。

18.33 验收测试清单

本项目最终至少要有这些测试。串行 reference 与分布式模拟结果一致。输入分片覆盖每条记录一次。Map attempt 重试不会重复进入 manifest。Reduce 重复接收同一 block 不会重复计数。Driver 重启后能从 checkpoint 恢复。旧 route epoch 的消息被拒绝。Reduce 队列满时 map 端背压。热点 key 拆分后最终结果仍合并成原 key。输出提交只接受一个 attempt。损坏 block 校验失败并触发重试或作业失败。

性能测试至少包括均匀输入、热点输入、大 key 空间、小 key 空间、慢 reduce、丢消息、重复消息和 checkpoint 频繁写入。每个测试都要有正确性 oracle。性能数字没有 correctness oracle，意义不大。

18.34 项目报告结构

综合实验报告建议按固定结构。第一，语义：输入格式、输出格式、错误行处理、计数类型。第二，系统结构：driver、worker、message bus、manifest、checkpoint。第三，正确性：reference 对比、去重、提交点、恢复。第四，性能：各 stage 时间、队列长度、shuffle 字节、热点 key、重试次数。第五，故障注入：列出注入事件和系统行为。第六，限制：哪些只在本机模拟验证，哪些需要真实 Linux 多进程或多机验证。

这样的报告训练的是系统工程表达能力。面试或工作中，真正有价值的不是说“我写了一个分布式计算框架”，而是能清楚说明语义、瓶颈、失败模式和证据边界。

18.35 项目目录结构

贯穿项目应该有清晰目录，而不是所有代码堆在一个文件中。建议结构包括 `include/` 放公共接口，`src/input/` 放分片和解析，`src/kernels/` 放 map、filter、histogram、top-k，`src/runtime/` 放线程池、队列和任务图，`src/shuffle/` 放 partition、manifest 和消息，`src/driver/` 放 task 状态和 checkpoint，`src/fault/` 放故障注入，`tests/` 放单元和集成测试，`benchmarks/` 放性能实验。

目录结构体现责任边界。Input 模块不应知道线程池内部锁；kernel 不应直接写 checkpoint；driver 不应解析每行日志；shuffle 不应决定业务 key 的含义。边界清楚，测试才能独立，后续第

三册 AI 推理引擎也能复用 runtime、benchmark 和 I/O 模块。

18.36 核心类型清单

项目可以先定义少量稳定类型。JobId、StageId、TaskId、AttemptId、PartitionId 使用明确位宽整数，避免随手用 `int` 或 `longlong`。InputSplit 描述输入范围，MapOutput 描述 map 产生的 shuffle block，ReduceOutput 描述最终输出，TaskState 描述任务状态，Checkpoint 描述可恢复状态。这些类型是系统语言。

Listing 18.7 项目 ID 类型的形态

```

1 struct JobId {
2     std::uint64_t value = 0;
3 };
4
5 struct TaskId {
6     std::uint64_t value = 0;
7 };
8
9 struct AttemptId {
10     std::uint32_t value = 0;
11 };
12
13 struct PartitionId {
14     std::uint32_t value = 0;
15 };

```

强类型 ID 可以防止把 partition id 误传成 worker id。C++ 中可以进一步定义比较和哈希。教学阶段即使只用简单 struct，也比裸整数更能表达语义。

18.37 Driver 状态机

Driver 是控制面。它维护 job 状态、stage 状态、task attempt、manifest 和 checkpoint。一个 task 可以处于 pending、running、committed、failed、canceled。Attempt 是 task 的一次执行尝试，可以成功、失败或超时。Task committed 表示某个 attempt 的输出被接受，其他 attempt 的输出无效。

Driver 的核心操作包括 schedule task、record attempt start、commit attempt、fail attempt、checkpoint、recover。每个操作都要幂等。比如 commit attempt 请求可能因为响应丢失被重复发送，driver 再次收到同一 attempt commit，应返回已经提交或重复，而不是再次添加 manifest。幂等不是只在 worker 端需要，控制面同样需要。

18.38 Worker 执行模型

Worker 从 driver 获取 task，读取 input split，执行 map-side combine，写 shuffle block，发送 commit。Worker 本身不决定最终输出是否有效，它只产生 attempt output。Worker 可以失败、超时或被取消。Worker 重启后，不应依赖内存中旧状态决定提交；有效性由 driver manifest 和 checkpoint 决定。

Worker 也要有本地资源限制：最大本地聚合内存、最大 in-flight shuffle 字节、最大输出临时文件数、最大并行 task 数。没有限制，单个 worker 可能因为热点 key 或慢 reduce OOM。资源限制触发背压或任务失败，driver 再调度。

18.39 MessageBus 接口

本机 MessageBus 可以先提供 send 和 poll completion。send 接受 LocalMessage，可能返回 accepted、busy、closed；poll 返回已完成消息或超时。故障注入在 MessageBus 内部进行：延迟、丢

弃、重复、乱序。这样 worker 和 driver 不直接调用随机故障逻辑，测试可以统一控制。

MessageBus 还应记录指标：发送消息数、丢弃数、重复数、busy 次数、队列最大长度、平均延迟和超时数。分布式系统的网络层必须可观测，否则上层只会看到任务慢，不知道是网络、reduce 还是 driver。

18.40 Manifest 格式

Manifest 是恢复的核心。它能回答：哪些 map task 已提交，哪个 attempt 获胜，产生了哪些 shuffle block，每个 block 属于哪个 reduce partition，大小和校验是多少，文件路径或消息范围在哪里。Reduce 阶段只读取 manifest 中的 block，不能读取临时目录中所有文件。

Manifest 还要版本化。第一版可以是简单文本或 JSON，但字段要稳定；后续若改格式，version 用于拒绝或迁移。Manifest 写入要使用临时文件加提交步骤，避免半写文件被当成有效。恢复时先校验 manifest，再校验 block。若 block 缺失或 checksum 不匹配，系统应重跑对应 map task 或报告不可恢复。

18.41 Checkpoint 恢复算法

恢复算法可以写成明确步骤。第一，加载最新 checkpoint 文件并校验版本和 checksum。第二，恢复 driver 的 job、stage、task 状态。第三，读取 manifest，确认已提交 outputs 存在。第四，把 running 但未 committed 的 task 改回 pending，因为旧 attempt 是否还活着不可依赖。第五，重新调度 pending task。第六，重复收到旧 attempt commit 时，根据 epoch 和 task state 拒绝或去重。

这个算法说明恢复不是“读回内存结构”这么简单。运行中的 task 在崩溃后状态不确定，必须回到可重试状态；已经 committed 的 task 不能重复；临时文件需要清理但不能误删已提交文件。恢复算法应有单元测试。

18.42 故障注入表

项目可以维护一张故障注入表。按阶段列：input read failure、parse error、map worker crash、shuffle message drop、shuffle duplicate、reduce busy、commit response lost、checkpoint write fail、driver restart。每项写注入方式、预期行为、需要的测试和指标。表驱动能防止只测自己想到的 happy path。

例如 commit response lost：worker 写出 block 并向 driver commit，driver 已记录 manifest，但响应被丢。Worker 超时后重试 commit。预期：driver 返回 already committed，manifest 不重复，reduce 只读取一次。这个测试能验证幂等提交。

18.43 性能里程碑

项目性能里程碑应逐步建立。第一，串行 reference 正确。第二，单机多 worker 比串行在大输入上有加速。第三，map-side combine 减少 shuffle bytes。第四，热点 key 拆分降低 reduce 长尾。第五，背压下队列不无限增长。第六，故障注入下正确性保持。每个里程碑都有指标，不要求一开始达到生产性能。

里程碑顺序很重要。不要在 reference 还不稳定时优化 shuffle；不要在 manifest 去重没做好时开启 speculation；不要在指标缺失时调复杂调度。系统工程需要可验证台阶。

18.44 实验输入集

至少准备四类输入。第一，均匀 key，小规模，用于快速 correctness。第二，均匀 key，大规模，用于吞吐和带宽。第三，热点 key，用于倾斜和二级聚合。第四，含错误行和边界值，用于解析和错误处理。每类输入都要有固定 seed 和摘要。测试不能只靠随机生成后立即丢弃 seed。

输入还应覆盖空文件、单行文件、没有换行结尾、超长行、非法字段、event_type 越界、value 溢出边界。系统教材的质量体现在这些边界上。只处理理想 CSV，不是真正的数据处理系统。

18.45 结果校验

结果校验不只是比较输出文件文本。可以先把输出解析成 key 到 value 的 map，再和 reference 比较。若输出顺序声明确定，再比较顺序；若输出顺序不保证，就不要因为顺序不同判错。错误行计数、重复请求计数和丢弃低优先级日志也要进入校验。校验规则要和语义合同一致。

浮点或近似算法需要误差校验。日志计数通常是整数精确，适合完全相等。若未来 AI 推理引擎加入浮点算子，校验策略会不同。第二册先用整数聚合训练 exactly-once 和确定性，是有意降低数值复杂度，让系统语义更清楚。

18.46 从本机线程到多进程

本机多线程模拟共享地址空间，bug 和真实多进程不同。下一步可以把 worker 做成多进程，通过 pipe、Unix domain socket 或本地 TCP 通信。多进程能训练序列化、进程崩溃、文件持久化和真实内核队列。代价是实现复杂度增加。项目可以先把 MessageBus 接口抽象好，后续替换传输层。

多进程版本要处理进程启动、退出、信号、日志收集和临时目录清理。Worker 崩溃后，driver 要发现并重试任务。共享内存不再可直接传递对象指针，消息必须序列化。这是从“模拟分布式”走向“本机分布式”的关键一步。

18.47 安全和权限边界

即使是教学项目，也要意识到权限边界。Worker 不应随意写任意路径，输出目录应由 driver 分配；消息解析要检查长度和版本，避免越界；临时文件名要避免冲突；恢复时不要信任损坏 manifest。真实系统还需要认证、授权和加密，本册不展开，但接口应留出 metadata 和错误码空间。

安全不是第三册或生产阶段才考虑。很多数据损坏来自缺少边界检查和信任内部输入。分布式系统里，“内部消息”也可能因为 bug、旧版本或损坏而非法。

18.48 第三册的接口预留

虽然 AI 推理放在第三册，第二册项目可以预留计算内核接口。今天的 map/reduce kernel 处理日志记录，未来可以处理 tensor block；今天的 thread pool 执行 histogram，未来可以执行 GEMM tile；今天的 manifest 记录 shuffle block，未来可以记录模型分片或量化权重缓存。接口应围绕任务、数据块、运行时、指标和检查点，而不是写死日志业务。

这也是为什么第二册先把多核、高性能、并发、I/O 和分布式计算讲透。第三册的 CPU 推理引擎不是凭空出现，它会复用数据布局、SIMD、线程池、背压、checkpoint 和观测性。第二册越扎实，第三册越能深入算子和模型，而不是返工基础设施。

18.49 最小可运行版本

最小可运行版本应非常克制。一个命令读取固定输入文件，使用串行 reference 输出结果；另一个命令使用多 worker 模拟，输出同样结果。此时不需要真实网络、复杂 checkpoint 或热 key 拆分。目标是建立输入格式、输出格式、校验器和 benchmark harness。没有这个最小版本，后面所有功能都缺少落脚点。

最小版本也要有错误处理。输入文件不存在、格式错误、event type 越界、value 溢出、输出路径不可写，都要返回明确错误。教材项目不能只在理想输入上成功。系统质量从第一个版本就开始积累。

18.50 第二阶段：单机并行

第二阶段加入线程池或固定 worker。输入 splits 被分配给 worker，每个 worker 本地 combine，driver 合并 partial。这个阶段重点是任务切分、partial 布局、worker stats、正确性对比和扩展曲线。还不需要 shuffle，因为所有 partial 可以在进程内合并。

实验应比较串行、每次创建线程、固定线程池三种版本。输入小的时候，串行可能最快；输入大时，多 worker 有收益；任务太细时，线程池调度成本上升。这个阶段验证第二册前半部分：数据布局、线程、NUMA、测量和并行 reduce。

18.51 第三阶段：本机 shuffle

第三阶段加入 shuffle partition。Map worker 不再直接把 partial 交给 driver，而是按 reduce partition 写入队列或 block。Reduce worker 从 partition 队列读取并合并。此时出现背压、partition 队列长度、热点 key 和 reduce 长尾。所有通信仍在本机内存中，但数据流已经像分布式系统。

这个阶段要加入 manifest 概念，即使 block 还在内存中。Manifest 记录哪些 map task 的输出属于哪个 partition。Reduce 只读 manifest 中有效 block。这样第四阶段加入失败重试时，不需要重写架构。

18.52 第四阶段：故障和重试

第四阶段加入 attempt、request id 和故障注入。Map task 可能失败重试，shuffle message 可能重复，commit response 可能丢失。Driver 只接受一个 attempt 提交，manifest 去重。Reduce 端不能重复合并同一逻辑 map task。这个阶段验证 exactly-once 外部效果。

测试要使用确定性故障。比如 task 3 的第一次 attempt 在写完 block 后 commit response 丢失；第二次 attempt 重试；最终 manifest 只能有一个 task 3 输出。再比如 reduce 队列第一次返回 busy，map 端退避后重试。每个故障都有明确预期。

18.53 第五阶段：Checkpoint

第五阶段加入 checkpoint 和恢复。Driver 在 stage 边界写 checkpoint，包含 task 状态和 manifest。测试中强制 driver 在 checkpoint 后重启，恢复后继续运行。Running task 回到 pending，committed task 保持 committed，临时文件清理，重复 commit 被拒绝。

Checkpoint 阶段要非常保守。不要同时引入复杂压缩和真实网络。先让恢复语义正确，再优化写入成本。恢复测试应成为常规测试，不是偶尔手工演练。

18.54 第六阶段：热点和优化

只有在正确性、重试和恢复稳定后，才加入热点 key 拆分、压缩、拓扑调度、work stealing 和多进程传输。此时每个优化都有 reference 和故障测试保护。热点 key 拆分要保证最终合并回原 key；压缩要保证 checksum 覆盖压缩数据或解压后数据；多进程传输要保证消息 framing 和版本。

优化阶段要保留开关。能关闭热点拆分回到普通 hash partition，能关闭压缩，能关闭 speculation。开关让对照实验和故障定位更容易。生产系统也常需要 feature flag 控制风险。

18.55 接口示例：MapTask

MapTask 输入是 InputSplit 和作业配置，输出是若干 ShuffleBlock。它不直接写最终结果。它可以返回 MapTaskResult，包含 attempt id、block 列表、记录数、错误数和指标。Driver 决定是否提交这个结果。

Listing 18.8 MapTaskResult 的接口形态

```
1 struct MapTaskResult {
2     TaskId task_id;
3     AttemptId attempt_id;
4     std::vector<ShuffleBlock> blocks;
5     std::uint64_t records_read = 0;
6     std::uint64_t records_rejected = 0;
7 };
```

这个接口把执行结果和提交分开。Worker 产生 result，不代表 result 生效。Driver commit 才改变控制面状态。这个分离是分布式计算正确性的核心。

18.56 接口示例: ReduceTask

ReduceTask 输入是 partition id 和 manifest 中属于该 partition 的 block 列表, 输出是临时 OutputBlock。Reduce 不应扫描所有 map 输出, 也不应接受 manifest 之外的 block。这样 repeated attempt 和临时文件不会污染最终结果。

ReduceTask 还要处理输入 block 缺失或损坏。若 block 缺失, 返回 retryable 或 corrupt, 让 driver 重新调度对应 map task 或失败作业。若直接跳过, 最终计数会少。错误处理必须进入输出语义。

18.57 指标示例

项目指标可以分五组。Input: 读取字节、解析记录、错误行。Map: 本地聚合 key 数、输出 block 数、map 耗时。Shuffle: 发送字节、队列等待、busy 次数、重复消息。Reduce: 输入 block 数、合并 key 数、热点 key 时间、输出字节。Control: attempt 数、重试数、commit 延迟、checkpoint 耗时、恢复耗时。

每组指标都要带 job、stage、task、partition 或 worker 维度。全局总数用于概览, 维度指标用于定位。指标命名稳定后, 实验报告和图表才能复用。

18.58 作业失败策略

不是所有错误都无限重试。每个 task 应有最大 attempt 数; 每个 job 应有总重试预算; 某些错误如格式不兼容、输出路径权限错误应立即失败; 某些错误如 reduce busy 可退避重试; 某些错误如 block checksum mismatch 可能重跑 map。失败策略要写成表, 而不是散落在 if 语句里。

作业失败后要清理资源: 停止提交新任务, 取消 pending task, 等待 running task 到安全点, 保留或删除临时文件, 写失败报告。失败报告应包含错误分类、失败 task、attempt 历史和可恢复建议。一个只返回 false 的作业系统很难调试。

18.59 实验报告中的图

本书不能在 EPUB 中依赖图片, 但报告可以用文本图、CSV 和外部脚本生成图。关键图包括线程扩展曲线、队列长度时间线、每 partition 输入大小、每 worker 任务数、重试次数、热点 key 分布、checkpoint 耗时。即使最终书里用文字描述, 项目输出应保留数据, 方便读者自己画。

图的价值是发现形状。一个 partition 明显比其他大, 说明倾斜; 队列长度持续上升, 说明背压不足; worker 任务数差异大, 说明调度不均; checkpoint 耗时周期性尖峰, 说明持久化影响。数字表和图要互相印证。

18.60 测试分层

项目测试应分四层。第一层是纯函数单元测试, 例如 parse line、hash partition、top-k merge、checksum。第二层是并发结构测试, 例如 bounded queue、thread pool、SPSC ring。第三层是 stage 集成测试, 例如 map stage、shuffle stage、reduce stage。第四层是端到端故障测试, 例如 driver restart、duplicate commit、hot key split。分层测试能快速定位问题。

每层测试的速度和覆盖不同。纯函数测试应该很多且快; 并发结构测试需要重复和 sanitizer; stage 测试需要临时文件和较小输入; 端到端故障测试较慢但覆盖协议。不要只写端到端测试, 失败时很难定位; 也不要只写单元测试, 协议组合 bug 会漏掉。

18.61 属性测试

日志统计适合属性测试。随机生成一批记录, reference 和分布式模拟结果应相同; 打乱输入顺序后, 如果语义是按 key 求和, 结果应相同; 把输入分成不同 split, 结果应相同; 重复发送同一个 committed block, 结果不应改变; 删除一个未提交临时 block, 结果不应改变; 删除一个已提交 block, 恢复应失败或重跑。属性测试比手写几个例子更能覆盖边界。

属性测试要记录 seed。失败时输出 seed、输入摘要和最小化后的反例。这样复杂系统 bug 才能复现。随机测试如果不能复现, 只会增加噪声。

18.62 性能测试分层

性能测试也要分层。Kernel benchmark 测 map、histogram、top-k；runtime benchmark 测队列和线程池；stage benchmark 测 map stage 和 reduce stage；end-to-end benchmark 测完整作业。每层回答不同问题。End-to-end 慢了，不知道具体瓶颈；kernel 快了，不代表系统快。分层性能数据才能指导优化。

每次优化要说明影响哪一层。比如 AoSoA 改动影响 kernel；work stealing 影响 runtime；map-side combine 影响 stage 和 end-to-end；manifest fsync 优化影响 commit 阶段。若优化目标不清，性能报告就会散。

18.63 文档和学习产物

贯穿项目还应产出文档：设计文档、状态机、故障矩阵、性能报告、源码阅读笔记和实验记录。代码只是学习的一部分。系统工程需要把设计讲清楚，让别人能审查和复现。每章结束时，项目文档应补充相应部分：数据布局章补 batch layout，锁章补队列不变量，I/O 章补背压链，分布式章补 manifest 和恢复。

这些文档不应长篇空话，而应包含具体字段、状态、返回值和测试。比如“BoundedQueue close 语义”比“队列是线程安全的”有用得多；“commit response lost 测试”比“支持故障恢复”有用得多。

18.64 最终验收标准

第二册阶段的最终项目验收可以定义为：在固定 seed 输入下，串行 reference、多线程本机模拟、带 shuffle 模拟输出一致；在注入重复消息、响应丢失和 worker 失败时输出仍一致；在热点 key 输入下能输出倾斜指标并可启用用热点拆分；在生产速度超过消费速度时队列不无限增长；checkpoint 后 driver 重启能恢复；所有结果文件或 manifest 有 checksum；性能报告包含线程扩展、队列、partition 和重试指标。

这个验收不要求商业级分布式系统，但要求语义完整、证据完整、失败路径可控。达到这个标准，读者已经具备进入第三册 CPU 推理引擎的系统基础。

18.65 学习成果清单

完成第二册项目后，读者应该能交付几类成果。第一，能写出一个正确的串行 reference，并用它校验优化版本。第二，能把数据按 batch 和列式布局组织，说明冷热字段和生命周期。第三，能实现有界队列和线程池，说明关闭和等待语义。第四，能使用 reduce、scan、partition 构造无共享写的并行内核。第五，能设计本机 shuffle、manifest 和幂等提交。第六，能写故障注入测试并解释恢复行为。第七，能用指标和 trace 分析瓶颈。

这些成果比某个单独优化技巧更重要。第三册的 AI 推理引擎会需要矩阵内核、量化数据布局、线程池调度、模型文件加载、缓存和错误处理。如果第二册的成果扎实，第三册就能把精力放在模型和算子本身。

18.66 本册收束

第二册没有直接进入 AI，是因为现代 CPU 推理离不开本册知识。一个量化模型的推理引擎需要理解 cache、SIMD、线程池、NUMA、I/O、任务图和观测性；一个可靠本地服务还需要背压、超时和恢复。先把计算系统讲清楚，再讲 AI 算子开发，学习路径会更稳。

到这里，第二册形成闭环：硬件执行模型解释单核成本，内存层级解释数据移动，多线程和同步解释共享状态，并行算法解释任务切分，I/O 和背压解释等待，分布式计算解释故障和提交，贯穿项目把它们组合成一个可运行系统。后续第三册只是在这个系统上换一种更复杂的计算负载。

18.67 项目章节总结

本章把全书原则落成一个工程：日志统计计算引擎。它从 reference 开始，用数据布局改善热路径，用线程池组织执行，用 reduce/scan/partition 构造并行内核，用 I/O 管线和背压控制等待，用 manifest 和 checkpoint 定义提交与恢复，用故障注入验证幂等。这个项目不是商业产品，却覆盖现

代计算系统最重要的骨架。读者真正完成它，就不再只是知道概念，而是能把概念组合成系统。

18.68 项目决策表

推进项目时，每加一个功能都要填决策表。功能改变了哪个语义？影响哪个状态机？需要哪些新字段？失败时如何恢复？是否影响 reference？需要哪些新测试？需要哪些指标？性能瓶颈预期迁移到哪里？是否能开关回退？例如加入热点 key 拆分，就要新增 hot key 识别、subkey、二级 reduce、结果合并、重复 attempt 去重和倾斜指标。没有决策表，功能很容易堆成不可维护的复杂度。

18.69 面向第三册的过渡任务

第二册结束后，可以做一个过渡任务：把日志统计里的 batch、thread pool、benchmark、manifest 保留下来，把计算内核替换成一个简单 tensor elementwise 算子，例如 $y=a*x+b$ 。这个任务不涉及模型推理，却能验证第二册基础设施是否通用。如果基础设施只能处理日志而不能处理 tensor batch，说明抽象边界还不够好。第三册会在这个过渡上继续走向量化算子、矩阵乘、量化和本地推理引擎。

18.70 第二册项目风险清单

项目最大的风险不是性能不够，而是边界不清。输入格式边界不清，会导致解析和恢复不一致；任务 ID 边界不清，会导致去重失败；输出提交边界不清，会导致重复或丢失；队列容量边界不清，会导致 OOM；线程池关闭边界不清，会导致悬挂；checkpoint 边界不清，会导致恢复错误。每个风险都对应本册一个主题。

因此项目评审应先看风险清单，再看性能数字。一个快但无法恢复的系统，不是高性能系统；一个吞吐高但队列无界的系统，只是把故障推迟；一个无锁但没有回收协议的队列，是潜在崩溃点。质量要求高，就必须把这些边界写进设计。

18.71 第二册项目交付目录

交付时可以包含四类文件。源码目录包含模块化实现；测试目录包含单元、并发、集成和故障测试；结果目录包含 benchmark CSV 和报告；文档目录包含状态机、故障矩阵、布局报告和恢复演练。书中的文字只是指导，项目产物才是读者真正掌握的证据。

这也符合用户要求的多书仓库结构：每本书有自己的 source、labs、materials、reports 和 results。后续第三册可以复用第二册的 labs 基础设施，但输出到独立书目录，避免版本混乱。

18.72 项目审查表

项目审查可以按模块进行。Input 模块：split 是否覆盖每条记录一次，错误行如何处理。Kernel 模块：reference 是否存在，边界测试是否完整。Runtime 模块：队列是否有界，shutdown 是否唤醒所有等待者。Shuffle 模块：block 是否有 checksum，partition 是否可恢复。Driver 模块：attempt 是否幂等，manifest 是否只提交一次。Checkpoint 模块：恢复是否把 running task 变回 pending。Metrics 模块：每个指标是否有维度和成本控制。

审查表让项目保持高质量。没有审查，代码越写越多，语义却越来越不清楚。第二册的目标不是堆功能，而是让每个功能都有边界、测试和证据。

18.73 本册之后的复盘问题

完成第二册后，建议读者复盘十个问题：哪一次优化改变了瓶颈？哪一个 bug 来自生命周期不清？哪一个测试暴露了并发问题？哪一个指标最有助于定位？哪一个队列需要背压？哪一个数据结构布局收益最大？哪一个失败注入最有价值？哪一个模块最难恢复？哪一处抽象边界需要调整？哪些基础设施能复用到第三册？这些问题能把项目经验固化下来。

18.74 从实验到教材的写法

项目报告可以反过来成为教材内容。每个章节都应包含一个问题、一个朴素版本、一个瓶颈或失败、一个改进版本、一个验证方法和一个边界说明。比如队列章节：朴素无界队列会积压，改成有界队列后出现背压，验证队列长度和生产者等待，边界是业务必须选择阻塞或失败。这样的写法比

直接给最终答案更适合教学。

第二册后续继续扩写时，也应保持这种写法：从问题出发，解释为什么朴素方案不够，推导改进，给出代码或状态机，最后说明测试和限制。高质量教材不是堆知识点，而是让读者跟着推理过程建立判断力。

18.75 第二册最终定位

第二册定位为现代计算系统和高性能并发的基础册。它不是 AI 册，也不是单纯操作系统册，而是把硬件、操作系统、运行时、算法、I/O 和分布式计算连接起来。它的目标读者是已经会写 C++ 和基础算法，但想把程序写成可靠、高性能、可测量系统的人。第三册 AI 推理引擎会在这个基础上继续，而不是替代它。

18.76 完整作业演练

一次完整作业演练可以这样写。准备输入：一份均匀日志、一份热点日志、一份包含错误行的日志。先运行串行 reference，保存结果摘要。再运行多 worker 本机版本，比较结果。随后开启 shuffle，记录每个 partition 的 block 数和字节数。再注入一次 commit response lost，验证 manifest 不重复。再让一个 reduce partition 慢下来，观察背压链。最后写 checkpoint，重启 driver，继续完成作业。

演练报告要把每一步的预期和实际写清。不是只说“测试通过”，而是说明哪个机制被验证：reference 验证语义，多 worker 验证并行正确性，shuffle 验证分区，commit 丢失验证幂等，慢 reduce 验证背压，driver 重启验证恢复。这样的演练才符合本书的质量要求。

18.77 项目扩展边界

第二册项目还可以扩展，但要控制边界。可以加入多进程 worker，可以加入真实 socket，可以加入压缩，可以加入更复杂 key 类型，可以加入简化 Raft 控制面。但每次扩展都要保持 reference、manifest、故障测试和性能报告。不要一次加入多项，否则问题难以定位。

扩展的判断标准是：它是否服务本书主线，是否能被测试，是否能被解释。如果只是为了功能看起来多，而没有语义和证据，就不应加入教材主体。高质量教材宁愿少而深，也不要多而散。

18.78 本章落到全书

到这里，第二册的主线闭环了。核心流水线解释单线程为什么慢；缓存、TLB 和 NUMA 解释数据移动；测量章提供证据；数据布局 and SIMD 提供单核内核；锁、原子和无锁结构提供同步基础；线程池和任务图组织单机并行；I/O 和背压处理等待；分布式模型、分片复制和本章工程实践把这些原则扩展到集群。后续第三册 AI 推理引擎，会在这个基础上讨论算子、模型、量化和 CPU 推理优化。

18.79 本章习题

习题与作业

习题一：为日志统计系统写端到端数据流图。要求包含输入分片、map-side combine、shuffle partition、reduce merge、输出提交和 checkpoint。

习题二：设计 shuffle manifest。说明它记录哪些 map 输出、partition、attempt、文件位置或消息范围，以及恢复时如何使用。

习题三：构造热点 key 输入。比较普通 hash partition、map-side combine、热 key 拆分和二级 reduce。说明每种方案的网络流量和 reduce 长尾变化。

习题四：设计 task attempt 提交协议。要求处理 attempt A 成功但响应丢失、attempt B 后完成、driver 重启三种情况。

习题五：为本机模拟系统设计故障注入表。至少包含丢消息、重复消息、慢 worker、慢 reduce、提交失败和 checkpoint 损坏。每个故障写出预期恢复行为。

综合实验:设计一个小型分布式日志统计系统。输入按文件分片,worker 本地统计,shuffle 按 key 聚合,driver 合并结果。写清数据格式、失败重试、幂等输出、指标和瓶颈。

附录 A

实验路线

第二册的实验围绕 Compute Systems Labs 展开。第一组实验是硬件与测量基础，包括 `lscpu` 拓扑记录、缓存层级观察、TLB 与缺页计数、`perf stat`、顺序归约和并行归约。第二组实验是数据布局，包括 AoS、SoA、false sharing、顺序访问、随机访问和预取效果。第三组实验是单机高性能计算，包括多累加器、SIMD 自动向量化、Roofline 分析和典型内核模式。

第四组实验是多线程同步，包括 mutex counter、atomic counter、bounded queue、条件变量、信号量和线程亲和性。第五组实验是无锁结构，包括 SPSC ring buffer、CAS 循环、ABA 现象、版本标签和内存回收方案分析。第六组实验是并行算法，包括 reduce、scan、partition、thread-local histogram 和任务粒度比较。第七组实验是运行时，包括线程池、任务队列、工作窃取模型、异步 I/O 和背压。第八组实验是分布式计算，包括本机多进程 RPC 模拟、超时重试、幂等请求、分片、复制和 shuffle。

每个实验必须记录环境、输入、命令、输出、正确性检查和结论。性能数据没有上下文就不能进入正文。若实验受手机、Termux 或虚拟化环境限制，也要明确记录限制。

附录 B

源码阅读索引

本附录保存后续源码专题的入口。

Linux 内核源码用于理解调度、NUMA、页表、透明大页、perf 事件、futex、epoll、内存分配和网络栈。阅读时不要从全局搜索函数名开始，而要围绕问题进入：线程为什么迁移、缺页为什么发生、futex 怎样把用户态锁和内核等待连接起来。

glibc 和 musl 用于阅读 pthread、mutex、condition variable、malloc 和系统调用封装。重点观察用户态 fast path 和进入内核 slow path 的分界。

Intel TBB、oneTBB 或 LLVM OpenMP runtime 用于阅读任务调度、线程池、工作窃取和并行算法运行时。阅读时关注任务粒度、队列结构、窃取策略和 shutdown 语义。

Folly、Abseil 和 Boost.Lockfree 用于阅读并发容器、原子封装、hazard pointer、RCU 风格回收和高性能队列。阅读时先搞清进度保证和内存回收，不要只看 CAS。

Redis、RocksDB、Kafka 和 Spark 用于阅读分布式计算和存储工程：事件循环、后台线程、LSM compaction、分区、复制、日志、shuffle、背压和故障恢复。阅读时抓数据流和失败路径，不要只列模块名。

术语表

算术强度 操作数与数据移动字节数的比值，用来判断一个内核更可能受计算吞吐还是内存带宽限制。

Roofline 用计算峰值、内存带宽和算术强度估算性能上界的模型。

SIMD 一条指令处理多个数据 lane 的执行方式，是现代 CPU 单核吞吐的重要来源。

乱序执行 CPU 在保持单线程可观察结果不变的前提下，提前执行已经准备好的指令以隐藏延迟。

分支预测 CPU 在分支结果确定前预测执行路径，预测失败会清空错误路径上的工作。

cache line 缓存一致性和缓存填充的基本粒度，常见大小为 64 字节。

TLB 地址翻译缓存，用于缓存虚拟页到物理页的映射。

NUMA 非统一内存访问结构，不同核心访问不同内存节点的成本不同。

false sharing 多个线程写不同变量，但变量落在同一 cache line 上，导致一致性流量和扩展性下降。

内存序 原子操作对可见性和重排施加的约束，例如 relaxed、acquire、release 和 `memory_order_seq_cst`。

lock-free 一类进度保证，表示系统整体能持续前进，但不保证每个线程有限步完成。

背压 下游处理不过来时向上游施加的流量控制机制。

共识 分布式系统中多个节点在故障下对日志顺序或状态达成一致的协议问题。

索引

instruction

add, 13